Constructing and Evaluating Weak Memory Models

by

Sizhuo Zhang

B.E., Tsinghua University (2013)
S.M., Massachusetts Institute of Technology (2016)

Submitted to the Department of Electrical Engineering and Computer Science
in partial fulfillment of the requirements for the degree of

Doctor of Philosophy

at the

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

June 2019

© Massachusetts Institute of Technology 2019. All rights reserved.

Author .....
Department of Electrical Engineering and Computer Science
May 23, 2019

Certified by .....
Arvind
Johnson Professor of Computer Science and Engineering
Thesis Supervisor

Accepted by .....
Leslie A. Kolodziejski
Professor of Electrical Engineering and Computer Science
Chair, Department Committee on Graduate Students

Constructing and Evaluating Weak Memory Models

by

Sizhuo Zhang

Submitted to the Department of Electrical Engineering and Computer Science
on May 23, 2019, in partial fulfillment of the
requirements for the degree of
Doctor of Philosophy

Abstract

A memory model for an instruction set architecture (ISA) specifies all the legal multithreaded-program behaviors, and consequently constrains processor implementations. Weak memory models are a consequence of the desire of architects to preserve the flexibility of implementing optimizations that are used in uniprocessors, while building a shared-memory multiprocessor. Commercial weak memory models like ARM and POWER are extremely complicated: it has taken over a decade to formalize their definitions. These formalization efforts are mostly empirical—they try to capture empirically observed behaviors in commercial processors—and do not provide any insights into the reasons for the complications in weak-memory-model definitions.

This thesis takes a constructive approach to study weak memory models. We first construct a base model for weak memory models by considering how a multiprocessor is formed by connecting uniprocessors to a shared memory system. We try to minimize the constraints in the base model as long as the model enforces single-threaded correctness and matches the common assumptions made in multithreaded programs. With the base model, we can show not only the differences among different weak memory models, but also the implications of these differences, e.g., more definitional complexity or more implementation flexibility or failures to match programming assumptions. The construction of the base model also reveals that allowing load-store reordering (i.e., a younger store is executed before an older load) is the source of definitional complexity of weak memory models. We construct a new weak memory model *WMM* that disallows load-store reordering, and consequently, has a much simpler definition. We show that WMM has almost the same performance as existing weak memory models.

To evaluate the performance/power/area (PPA) of weak memory models versus that of strong memory models like TSO, we build an out-of-order superscalar cache-coherent multiprocessor. Our evaluation considers out-of-order multiprocessors of small sizes and benchmark programs written using portable multithreaded libraries and compiler built-ins. We find that the PPA of an optimized TSO implementation can match the PPA of implementations of weak memory models. These results provide

a key insight that load execution in TSO processors can be as aggressive as, or even more aggressive than, that in weak-memory-model processors. Based on this insight, we further conjecture that weak memory models cannot provide better performance than TSO in case of high-performance out-of-order processors. However, whether weak memory models have advantages over TSO in case of energy-efficient in-order processors or embedded microcontrollers remains an open question.

Thesis Supervisor: Arvind
Title: Johnson Professor of Computer Science and Engineering

Acknowledgments

I want to first thank my advisor, Prof. Arvind, for his guidance throughout my graduate study. He is always patient and supportive, and willing to devote a whole afternoon to discussing technical details. I am also inspired by his constant enthusiasm in asking new questions and finding out simple and systematic solutions to complex problems. His particular way of thinking also influenced me deeply.

I also want to thank my thesis committee members, Prof. Daniel Sanchez and Prof. Joel Emer, for their help and feedback on my research. Although Daniel and Joel are not my advisors, they have been providing me with all kinds of help and advice ever since I entered MIT. Their help has broadened my horizons on the field of computer architecture.

I would like to thank other CSAIL faculty that I had opportunities to interact with. I thank Prof. Martin Rinard for the advice on writing introductions. Besides, his sense of humor can always relieve the pressure of paper deadlines. I also want to thank Prof. Srini Devadas and Prof. Adam Chlipala for introducing new research areas to me.

I am thankful to all the members of the Computation Structures Group (CSG), both past and present. I want to thank Muralidaran Vijayaraghavan, Andrew Wright, Thomas Bourgeat, Shuotao Xu, Sang-Woo Jun, Ming Liu, Chanwoo Chung, Joonwon Choi, Xiangyao Yu, Guowei Zhang, Po-An Tsai, and Mark Jeffrey for all the conversations, discussions, and collaborations. In particular, Murali brought me to the field of memory models, i.e., the topic of this thesis. Thanks to Asif Khan, Richard Uhler, Abhinav Agarwal, and Nirav Dave for giving me advice during my first year at MIT. I am thankful to Jamey Hicks for providing tools that make FPGAs much easier to use. Without his tools and technical supports, it is impossible to complete the work in this thesis. I want to thank Derek Chiou and Daniel Rosenband for hosting me for summer internship and helping me get industrial experience.

I also want to thank all my friends inside and outside MIT. Their support makes my life much better in these years.

I am particularly thankful to my parents, Xuewen Zhang and Limin Chen, and my girlfriend, Siyu Hou. Without their love and support throughout these years, it is impossible for me to complete the graduate study. In addition, without Siyu's reminder that I need to graduate at some day, this thesis cannot be completed by this time.

Finally I would like to thank my grandfather, Jifang Zhang, who is a role model of hardworking and striving. My grandfather grew up in poverty in a rural area in the south of China, but he managed to get a job in Beijing, the capital city of China, by paying much more efforts than others. His life story constantly inspires me to overcome difficulties and strive for higher goals.

Contents

1 Introduction **21**

1.1 A Common Base Model for Weak Memory Models 23

1.2 A New Weak Memory Model with a Simpler Definition 24

1.3 Designing Processors for Evaluating Memory Models 26

1.4 Evaluation of WMM versus TSO 27

1.5 Thesis Organization 29

2 Background and Related Work **31**

2.1 Formal Definitions of Memory Models 31

2.1.1 Operational Definition of SC 32

2.1.2 Axiomatic Definition of SC 32

2.2 Fence Instructions 34

2.3 Litmus Tests 34

2.4 Atomic versus Non-Atomic Memory 36

2.4.1 Atomic Memory 36

2.4.2 Non-Atomic Memory 36

2.4.3 Litmus Tests for Memory Atomicity 38

2.4.4 Atomic and Non-Atomic Memory Models 39

2.5 Problems with Existing Memory Models 40

2.5.1 SC for Data-Race-Free (DRF) 40

2.5.2 Release Consistency (RC) 40

2.5.3 RMO and Alpha 41
2.5.4 ARM 43
2.6 Other Related Memory Models 43
2.7 Difficulties of Using Simulators to Evaluate Memory Models 44
2.8 Open-Source Processor Designs 45

3 GAM: a Common Base Model for Weak Memory Models 49
3.1 Intuitive Construction of GAM 50
3.1.1 Out-of-Order Uniprocessor (OOO^U) 50
3.1.2 Constraints in OOO^U 52
3.1.3 Extending Constraints to Multiprocessors 54
3.1.4 Constraints Required for Programming 57
3.1.5 To Order or Not to Order: Same-Address Loads 61
3.2 Formal Definitions of GAM 65
3.2.1 Axiomatic Definition of GAM 65
3.2.2 An Operational Definition of GAM 67
3.2.3 Proof of the Equivalence of the Axiomatic and Operational Definitions of GAM 71
3.3 Performance Evaluation 78
3.3.1 Methodology 78
3.3.2 Results and Analysis 80
3.4 Summary 82

4 WMM: a New Weak Memory Model with a Simpler Definition 85
4.1 Defintional Complexity of GAM 85
4.1.1 Complexity in the Operational Definition of GAM 85
4.1.2 Complexity in the Axiomatic Definition of GAM 87
4.2 WMM Model 88
4.2.1 Operational Definitions with I^2E 89
4.2.2 Operational Definition of WMM 92

4.2.3 Axiomatic Definition of WMM . . . 97
4.2.4 Proof of the Equivalence of the Axiomatic and Operational Definitions of WMM . . . 98
4.3 Comparing WMM and GAM . . . 105
4.3.1 Bridging the Operational Definitions of WMM and GAM . . . 105
4.3.2 Same-Address Load-Load Ordering . . . 117
4.3.3 Fence Ordering . . . 119
4.4 WMM Implementation . . . 119
4.4.1 Write-Back Cache Hierarchy (CCM) . . . 120
4.4.2 Out-of-Order Processor (OOO) . . . 121
4.5 Performance Evaluation . . . 123
4.5.1 Methodology . . . 123
4.5.2 Results and Analysis . . . 125
4.6 Summary . . . 130

5 RiscyOO: a Modular Out-of-Order Processor Design 133
5.1 Composable Modular Design (CMD) Framework . . . 136
5.1.1 Race between Microarchitectural Events . . . 136
5.1.2 Maintaining Atomicity in CMD . . . 138
5.1.3 Expressing CMD in Hardware Description Languages (HDLs) 140
5.1.4 CMD Design Flow . . . 140
5.1.5 Modular Refinement in CMD . . . 141
5.2 Out-of-Order Core of RiscyOO . . . 141
5.2.1 Interfaces of Salient Modules . . . 143
5.2.2 Connecting Modules Together . . . 151
5.2.3 Module Implementations . . . 153
5.3 Cache-Coherent Memory System of RiscyOO . . . 155
5.3.1 L2 Cache . . . 155
5.4 Evaluation of RiscyOO . . . 158
5.4.1 Methodology . . . 158

5.4.2 Effects of TLB microarchitectural optimizations 160
5.4.3 Comparison with the in-order Rocket processor 162
5.4.4 Comparison with commercial ARM processors 163
5.4.5 Comparison with the academic OOO processor BOOM 164
5.5 ASIC Synthesis 165
5.6 Summary 166

6 Evaluation of WMM versus TSO **167**
6.1 Methodology 169
6.1.1 Benchmarks 169
6.1.2 Processor Configurations 172
6.1.3 Memory-Model Implementations 173
6.1.4 Energy Analysis 180
6.2 Results of Single-threaded Evaluation 182
6.2.1 Performance Analysis 182
6.2.2 Energy Analysis 185
6.3 Results of Multithreaded Evaluation: PARSEC Benchmark Suite . . 186
6.3.1 Performance Analysis 186
6.3.2 Energy Analysis 188
6.4 Results of Multithreaded Evaluation: GAP Benchmark Suite 190
6.4.1 Performance Analysis 190
6.4.2 Energy Analysis 198
6.5 ASIC Synthesis 199
6.6 Summary 201

7 Conclusion **205**
7.1 Contributions on Weak Memory Models 205
7.2 Future Work on Evaluating Weak Memory Models and TSO 207
7.2.1 High-Performance Out-of-Order processors 207
7.2.2 Energy-Efficient In-Order Processors 208
7.2.3 Embedded Microcontrollers 209

7.3 Future Work on High-Level Language Models 210
7.4 Other Contributions and Future Work 210

List of Figures

2-1 SC abstract machine 32

2-2 Dekker algorithm 32

2-3 Axioms of SC 33

2-4 Litmus tests for instruction reordering 35

2-5 Examples of non-atomic memory systems 37

2-6 Litmus tests for non-atomic memory 39

2-7 RMO dependency order 42

2-8 OOTA 42

3-1 Structure of OOO^{U} 51

3-2 Constraints on execution orders in OOO^{U} 53

3-3 Store forwarding 53

3-4 Load speculation 53

3-5 Constraints for load values in OOO^{MP} 55

3-6 Additional constraints in OOO^{MP} 56

3-7 Additional constraints for fences 58

3-8 Litmus tests of data-dependency ordering 59

3-9 Litmus tests for same-address loads 63

3-10 Axioms of GAM 67

3-11 Abstract machine of GAM 68

3-12 Rules to operate the GAM abstract machine (part 1 of 2) 69

3-13 Rules to operate the GAM abstract machine (part 2 of 2) 70

3-14 Relative Performance (uPC) improvement (in percentage) of ARM, GAM0, and Alpha* over GAM 80
3-15 Number of kills caused by same-address load-load orderings per thousand uOPs in GAM 81
3-16 Number of stalls caused by same-address load-load orderings per thousand uOPs in GAM and ARM 82
3-17 Number of load-to-load forwardings per thousand uOPs in Alpha* . . 82
3-18 Reduced number of L1 load misses per thousand uOPs for Alpha* over GAM 83

4-1 Behavior caused by load-store reordering 86
4-2 I^2E abstract machine of TSO 91
4-3 Operations of the TSO abstract machine 93
4-4 PSO background rule 93
4-5 I^2E abstract machine of WMM 94
4-6 Rules to operate the WMM abstract machine 95
4-7 MP+Ctrl: litmus test for control-dependnecy ordering 96
4-8 MP+Mem: litmus test for potential-memory-dependency ordering . . 96
4-9 Operations on the GAM^{VP} abstract machine (part 1 of 2: rules same as GAM) 107
4-10 Rules to operate the GAM^{VP} abstract machine (part 2 of 2: rules different from GAM) 108
4-11 Loads for the same address with an intervening store for the same address in between 118
4-12 CCM+OOO: implementation of WMM 120
4-13 Performance (uPC) of WMM-SB20 and WMM-SB10 normalized to that of WMM-SB42 126
4-14 Renaming stall cycles due to a full store buffer in WMM in configurations SB42, SQ20 and SQ10. Stall cycles are normalized to the execution time of WMM-SB42. 127

4-15 Reduction (in percentage) for the EARLY policy over the LATE policy on the time that a store lives in the store buffer in the WMM processor with different store-buffer sizes 127

4-16 Relative performance improvement (in percentage) of GAM over WMM in configurations SB42, SB20 and SB10 128

4-17 Reduced renaming stall cycles caused by full stores buffers for GAM over WMM. Reduced cycles are normalized to the execution time of WMM-SB42. 129

4-18 Reduction (in percentage) for GAM over WMM on the time that a store lives in the store buffer in configurations SB42, SB20 and SB10, respectively 130

5-1 Race between microarchitectural events Rename and RegWrite in an OOO processor 137

5-2 Pseudo codes for the interfaces of IQ and RDYB and the atomic rules of Rename and RegWrite 139

5-3 Top-level moduels and rules of the OOO core. Modules are represented by rectangles, while rules are represented by clouds. The core contains four execution pipelines: two for ALU and branch instructions, one for memory instructions, and one for floating point and complex integer instructions (e.g., multiplication). Only two pipelines are shown here for simplicity. 142

5-4 In-order pipeline of the Fetch module 144

5-5 Rules for LSQ and Store Buffer 152

5-6 Internal states and rules of LSQ 154

5-7 RiscyOO multiprocessor 156

5-8 Microarchitecture of the L2 cache. Modules are represented by blocks, while rules are represented by clouds. All the rules access the MSHR module; arrows pointing to MSHR are not shown to avoid cluttering. Uncached loads from TLBs are also not shown for simplicity; they are handled in a similar way as L1 requests. 156

5-9 Performance of RiscyOO-T$^+$ normalized to RiscyOO-B. Higher is better. 162

5-10 Number of L1 D TLB misses, L2 TLB misses, branch mispredictions, L1 D misses and L2 misses per thousand instructions of RiscyOO-T$^+$ 162

5-11 Performance of RiscyOO-C$^-$, Rocket-10, and Rocket-120 normalized to RiscyOO-T$^+$. Higher is better. 163

5-12 Performance of A57 and Denver normalized to RiscyOO-T$^+$. Higher is better. 164

5-13 IPCs of BOOM and RiscyOO-T$^+$R$^+$ (*BOOM results are taken from [77]*) 165

6-1 Number of atomic instructions and fences per thousand user-level instructions in PARSEC and GAP benchmarks on a 4-core WMM multiprocessor 171

6-2 Execution time of WMM-SI64, TSO-Base and TSO-SP in SPEC benchmarks. Numbers are normalized to the execution time of WMM-Base. 183

6-3 Number of loads being killed by cache evictions per thousand instructions in TSO-Base and TSO-SP in SPEC benchmarks 183

6-4 Load-to-use latency in SPEC benchmarks 184

6-5 Cycles that SQ is full in SPEC benchmarks. The numbers are normalized to the execution time of WMM-Base. 184

6-6 Number of DRAM accesses per thousand instructions in SPEC benchmarks 185

6-7 Number of bytes per instruction transferred between cores and L2 in SPEC benchmarks 186

6-8 Execution time of WMM-SI64, TSO-Base and TSO-SP in PARSEC benchmarks. Numbers are normalized to the execution time of WMM-Base. 187

6-9 Execution time of WMM-SI64, WMM-SI256 and WMM-SI1024 in PARSEC benchmarks. Numbers are normalized to the execution time of WMM-Base. 187

6-10 Number of loads being killed by cache evictions per thousand instructions in TSO-Base in PARSEC benchmarks 188

6-11 Cycles that SQ is full in PARSEC benchmarks. The numbers are normalized to the execution time of WMM-Base. 188

6-12 Number of mis-speculative loads per thousand instructions in PARSEC benchmarks 189

6-13 Number of DRAM accesses per thousand instructions in PARSEC benchmarks 189

6-14 Number of bytes per instruction transferred between cores and L2 in PARSEC benchmarks 190

6-15 Breakdown of Number of bytes per instruction transferred between cores and L2 in PARSEC benchmarks 191

6-16 Number of bytes per instruction transferred for upgrade requests and responses between cores and L2 in WMM-Base and WMM-SI processors in PARSEC benchmarks 192

6-17 Number of bytes per instruction transferred between cores and L2 in WMM-Base and WMM-SI processors in PARSEC benchmarks 192

6-18 Execution time of WMM-SI64, TSO-Base and TSO-SP in GAP benchmarks. Numbers are normalized to the execution time of WMM-Base. 193

6-19 Execution time of WMM-SI64, WMM-SI256 and WMM-SI1024 in GAP benchmarks. Numbers are normalized to the execution time of WMM-Base. 193

6-20 Number of Reconcile fences per thousand instructions in WMM-Base and WMM-SI64, and the number of full fences (including atomics) per thousand instructions in TSO-Base and TSO-SP 194
6-21 Number of loads being killed by cache evictions per thousand instructions in TSO-Base and TSO-SP in GAP benchmarks 195
6-22 Number of system instructions per thousand instructions in GAP benchmarks 196
6-23 Number of system calls per thousand instructions in GAP benchmarks 196
6-24 Number of Reconcile fences per thousand instructions in WMM-Base and WMM-Relax, and the number of full fences (including atomics) per thousand instructions in TSO-Base and TSO-SP 197
6-25 Execution time of WMM-Relax, TSO-Base and TSO-SP in GAP benchmarks. Numbers are normalized to the execution time of WMM-Base. 198
6-26 Number of mis-speculative loads per thousand instructions in GAP benchmarks 198
6-27 Number of DRAM accesses per thousand instructions in GAP benchmarks 199
6-28 Number of bytes per instruction transferred between cores and L2 in GAP benchmarks 200
6-29 Number of bytes per instruction transferred between cores and L2 in WMM-Base and WMM-SI processors in GAP benchmarks 200
6-30 Normalized area of each processors. Numbers are normalized to the area of WMM-Base. 201

List of Tables

3.1 Processor parameters 79

4.1 Truth table for $\mathsf{order}_{\mathsf{wmm}}(X, Y)$ 97
4.2 Different store-buffer sizes used in the evaluation 124
4.3 Different recycle policies of store-queue entries 125

5.1 RiscyOO-B configuration of our RISC-V OOO uniprocessor 161
5.2 Processors to compare against 161
5.3 Variants of the RiscyOO-B configuration 161
5.4 ASIC synthesis results 165

6.1 Measurement parameters of GAP benchamrks (adapted from [35, Table 1]) 170
6.2 Baseline configuration of uniprocessors 173
6.3 Baseline configuration of 4-core multiprocessors 173

Chapter 1

Introduction

A memory model for an instruction set architecture (ISA) is the specification of all legal multithreaded-program behaviors. If the hardware implementation conforms to the memory model, software remains compatible. The definition of a memory model must be specified precisely. Any ambiguity in the memory-model specification can make the task of proving the correctness of multithreaded programs and hardware implementations untenable. A precise definition of a memory model can be given *axiomatically* or *operationally*. An *axiomatic* definition is a set of axioms that any legal program behaviors must satisfy. An *operational* definition is an abstract machine that can be operated by a set of rules, and legal program behaviors are those that can be produced by running the program on the abstract machine. It is highly desirable to have equivalent axiomatic and operational definitions for a memory model.

While strong memory models like Sequential Consistency (SC) [81] and Total Store Order (TSO) [135, 109, 127, 126] are well understood, weak memory models, which are driven by hardware implementations, have much more complicated definitions. Although software programmers never asked for such complexity, they have to deal with the behaviors that arise as a consequence of weak memory models in important commercial machines like ARM [30] and POWER [70]. Many of the complications and features of high-level languages (e.g., C++11) arise because of the need to generate efficient code for ARM and POWER, the two major ISAs that have weak memory models [72]. It should be noted that even if a C++ programmer is writing software for

x86 machines with the TSO memory model, the programmer still needs to deal with the complications in the C++ language model that are caused by the weak memory models of ARM and POWER.

In spite of the broad impact of weak memory models, some weak memory models are not defined precisely in the ISA manuals, i.e., not in the form of axioms or abstract machines. For example, the memory model in the POWER ISA manual [70] is described informally as the reorderings of events, and an event refers to performing an instruction with respect to a processor. While reorderings of events capture some properties of memory models, it is unclear how to determine the value of each load, which is the most important information in program behaviors, given the orderings of events. The lack of precise definitions for weak memory models has triggered a series of studies on weak memory models over the last decade [24, 26, 22, 94, 27, 125, 124, 25, 55, 114]. These previous studies have taken an *empirical* approach—starting with an existing machine, the developers of the memory model attempt to come up with an axiomatic or operational definition that matches the observable behavior of the machine. However, we observe that this approach has drowned researchers in the subtly different observed behaviors on commercial machines without providing any insights into the reasons for the complications in weak-memory-model definitions. For example, Sarkar et al. [125] published an operational definition for POWER memory model in 2011, and Mador-Haim et al. [94] published an axiomatic definition that was proven to match the operational definition in 2012. However, in 2014, Alglave et al. [27] showed that the original operational definition, as well as the corresponding axiomatic definition, ruled out a newly observed behavior on POWER machines. As another instance, in 2016, Flur et al. [55] gave an operational definition for ARM memory model, with no corresponding axiomatic definition. One year later, ARM released a revision in their ISA manual explicitly forbidding behaviors allowed by Flur's definition [30], and this resulted in another proposed ARM memory model [114]. Clearly, formalizing weak memory models empirically is error-prone and challenging.

This thesis takes a different, *constructive* approach to study weak memory models, and makes the following contributions:

1. Construction of a common base model for weak memory models.

2. Construction of a weak memory model that has a much simpler definition but almost the same performance as existing weak memory models.

3. RiscyOO, a modular design of a cache-coherent superscalar out-of-order (OOO) multiprocessor that can be adapted to implement different memory models.

4. A quantitative evaluation of weak memory models versus TSO using RiscyOO.

1.1 A Common Base Model for Weak Memory Models

It is important to find a common base model for weak memory models because, as we have just discussed, even experts cannot agree on the precise definitions of different weak models, or the differences between them. Having a common base model can help us understand the nature of weak memory models, and, in particular, the features and optimizations in hardware implementations that add to the complexity of weak memory models.

It should be noted that hardware optimizations are all transparent in uniprocessors, i.e., a uniprocessor always appears to execute instructions in order and one at a time. However, in the multiprocessor setting, some of these optimizations can generate behaviors that cannot be explained by executing instructions in order on each processor and one at a time, i.e., the behaviors are not sequentially consistent. Architects hope that weak memory models can admit those behaviors, so multiprocessor implementations that keep using these optimizations are still legal. Therefore, we construct the common base model for weak memory models with the explicit goal of admitting all the behaviors generated by the uniprocessor optimizations. That is, we assume that a multiprocessor is formed by connecting uniprocessors to a shared memory system, and then derive the minimal constraints that all processors must obey. We show that there are still choices left regarding same-address load-load orderings

and regarding dependent-load orderings. Each of these choices results in a slightly different memory model. Not surprisingly, ARM, Alpha [16] and RMO [142] differ in these choices. Some of these choices add complexity to the memory-model definition, or fail to match the common assumptions in multithreaded programs. After carefully evaluating the choices, we have derived the *General Atomic Memory Model* (GAM) [150], i.e., a common base model. We believe this insight can help architects choose a memory model before implementation and avoid spending countless hours reverse engineering the model supported by an ISA.

It should be noted that we do not consider *non-atomic* memory systems (see Section 2.4) when constructing GAM. This is because most memory models (except POWER) do not support non-atomic memory. Although we have pointed out the different choices that can be made in different memory models, the definition of GAM is just for one specific choice and is not parameterized by the choices.

1.2 A New Weak Memory Model with a Simpler Definition

During the construction of the common base model, GAM, we discovered that allowing load-store reorderings (i.e., a younger store is executed before an older load) is a major source of complexity of both operational and axiomatic definitions of weak memory models. Here we explain briefly why load-store reorderings complicate memory-model definitions; more details will be given in Section 4.1.

In case of operational definitions of weak memory models, load-store reorderings allow a load to see the effect of a future store in the same processor. To generate such behaviors, the abstract machine in the operational definition must be able to model partial and out-of-order execution of instructions.

The axiomatic definitions of weak memory models must forbid the so-called out-of-thin-air (OOTA) behaviors [40]. In an OOTA behavior, a load can get a value that should never be generated in the system. Such behaviors can be admitted by

axiomatic definitions which do not forbid cycles of dependencies. OOTA behaviors must be forbidden by the memory-model definition, because they can never be generated in any existing or reasonable hardware implementations and they make formal analysis of program semantics almost impossible. To rule out OOTA behaviors in the presence of load-store reorderings, the axioms need to define various dependencies among instructions, including data dependencies, address dependencies and control dependencies. It is non-trivial to specify these dependencies correctly, as we will show during the construction of GAM [150].

We notice that most processors commit instructions in order and do not issue stores to memory until the stores are committed, so many processor implementations do not reorder a younger store with an older load. Furthermore, we show by simulation that allowing load-store reordering in aggressive out-of-order implementations does not lead to performance improvements (Section 4.5). In order to derive a new weak memory model, which has a much simpler definition, we can simply disallow load-store reorderings. After doing so, the axiomatic definition can forbid OOTA behaviors without defining dependencies. This leads to a new memory model, *WMM* [147], which has a much simpler axiomatic definition than GAM. Since the definition does not track dependencies, WMM also allows the reordering of dependent instructions. WMM also has a much simpler operational definition than GAM. Instead of modeling out-of-order execution, the abstract machine of WMM has the property of *Instantaneous Instruction Execution* (I^2E), the property shared by the abstract machines of SC and TSO (see Sections 2.1.1 and 4.2.1). In the I^2E abstract machine of WMM, each processor executes instructions in order and instantaneously. The processors are connected to a *monolithic memory* that executes loads and stores instantaneously. There are buffers between the processors and the monolithic memory to model indirectly the effects of instruction reorderings. We have also proved the equivalence between the axiomatic and operational definitions of WMM.

It should be noted that *the I^2E abstract machine is purely for definitional purposes, and it does not preclude out-of-order implementations.* In particular, we will show in Section 4.3 how the I^2E abstract machine of WMM simulates the effects of out-of-

order execution.

1.3 Designing Processors for Evaluating Memory Models

It turns out that evaluating the performance of memory models can be as difficult as defining memory models. This is because memory models affect the microarchitecture, and performance depends on the timing of the synchronizations between processors. It is very difficult to have a fast simulator that models accurately both the microarchitecture details and the timing of synchronizations (see Section 2.7).

Our approach is to build processors for different memory models, and evaluate the performance of the processor prototypes on FPGAs. To reduce the development effort, we do not want to design each processor from scratch. Instead, we would like to reuse codes and modules as much as possible across processors of different memory models. That is, we first design one processor for one specific memory model from scratch, and then make changes to a limited number of modules (e.g., the load-store queue and caches) to adapt the design to other memory models. Therefore, we need a *modular* design methodology so that the changed modules can be composed easily with other modules.

We found that existing processor design methodologies cannot meet our requirement, so we developed the *Composable Modular Design* (CMD) framework to achieve modularity and composability. In CMD, (1) The interface methods of modules provide instantaneous accesses and perform atomic updates to the state elements inside the module; (2) Every interface method is *guarded*, i.e., it cannot be applied unless it is ready; and (3) Modules are composed together by *atomic rules* which call interface methods of different modules. A rule either successfully updates the state of all the called modules or it does nothing. Using CMD, we designed *RiscyOO* [151], a superscalar out-of-order cache-coherent multiprocessor, as our evaluation platform. The processor uses the open-source RISC-V instruction set [10], has been proto-

typed on the AWS F1 FPGA [1], and can boot Linux. Our evaluation (Section 5.4) shows that RiscyOO can easily outperform in-order processors (e.g., Rocket [5]) and matches state-of-the-art academic OOO processors (e.g., BOOM [77]), though it is not as highly optimized as commercial processors (e.g., ARM Cortex-A57 and NVIDIA Denver [41]).

1.4 Evaluation of WMM versus TSO

The question on the performance comparison between weak memory models and TSO is extremely difficult to answer. While ARM and POWER have weak models, Intel, which has dominated the high-performance CPU market for decades, adheres to TSO. There is a large number of architecture papers [59, 115, 65, 61, 45, 143, 38, 132, 86, 63, 145, 116, 51, 54, 50, 80, 103, 146, 117] arguing that implementations of strong memory models can be made as fast as those of weak models. It is unlikely that we will reach consensus on this question in the short term, especially because of the entrenched interests of different companies. Nevertheless, we would like to present our perspective on this question.

To narrow down the breadth of this study, we choose WMM as the representative weak memory model because of its simpler definition. For TSO implementations, we do not consider out-of-window speculation [45, 143, 38], i.e., speculative techniques that require checkpointing the whole processor state. For WMM, we have two flavors of implementations: one uses the conventional MESI coherence protocol as the TSO implementation does, while the other uses a self-invalidation coherence protocol, which cannot be used in any TSO implementations.

Besides performance, we also compare the energy efficiency of WMM and TSO by looking at the number of energy-consuming events like DRAM accesses and network traffic. By applying standard ASIC synthesis flow on the RTL codes of the WMM and TSO implementations, we can also compare the area of different processors. Our evaluation considers out-of-order multiprocessors of small sizes and benchmark programs written using portable multithreaded libraries and compiler built-ins. The

evaluation results show that the performance/power/area (PPA) of TSO can match that of WMM.

Based on these results, we further conjecture that weak memory models cannot provide better performance than TSO in case of high-performance out-of-order processors. *The key insight is that load execution in TSO processors can be as aggressive as, or even more aggressive than, that in weak-memory-model processors.* In a TSO out-of-order processor, a load can start execution speculatively as soon as it knows its load address, regardless of the states of older instructions. In spite of the aggressive speculation in TSO, the checking logic for detecting speculation failures is still very simple because of the simple definition of TSO. In contrast, load execution in a weak-memory model processor may be stalled by older fence instructions, and superfluous fences can make the performance of weak memory models worse than TSO (Section 6.4). It is possible to also execute loads speculatively in the presence of older fences in a weak-memory-model implementation, but the checking logic to detect precisely the violation of the memory ordering required by the weak memory model will be more complicated than that in TSO. This is because the definition of weak memory models is much more complicated than TSO. Even if we make the effort to implement speculative execution of loads over fences in weak-memory-model hardware and minimize the insertion of fences in software, the aggressiveness of load execution in weak memory models will just be the same as, but not more than, that in TSO. Given the same aggressiveness of speculative load execution, the performance difference between weak memory models and TSO depends on the rate of speculation failures, which could in theory be reduced by having hardware predictors (on whether speculation should be turned on or off) or software hints (which suggest hardware to turn off speculation).

The only performance bottleneck we notice for TSO is store execution, because TSO keeps store-store ordering. However, our evaluation shows that the store-execution overhead in TSO can be mitigated effectively using store prefetch (Section 6.2).

As a result, if the goal is to achieve high performance, then we believe weak

memory models do not provide any benefits over TSO. However, this thesis does not address whether weak memory models have advantages over TSO in case of in-order processors or embedded microcontrollers.

1.5 Thesis Organization

Chapter 2 introduces the background on memory models and related work. Chapter 3 constructs the common base model, GAM. Chapter 4 identifies the source of complexity in GAM, and presents WMM, a simpler weak memory model. Chapter 5 details the design of RiscyOO, the processor used for performance evaluation of memory models. Chapter 6 evaluates the performance of WMM versus TSO. Chapter 7 offers conclusions.

Chapter 2

Background and Related Work

In this chapter, we review the background on memory models and processor implementations. Section 2.1 explains axiomatic and operational definitions in more details. Section 2.2 introduces fence instructions to restore sequential consistency. Section 2.3 introduces the concept of litmus tests, which we will be using throughout the thesis to show properties of memory models. Section 2.4 classifies memory models into two categories according to the atomicity of memory, one of the most important properties of memory models. Section 2.5 reviews existing memory models, and illustrates the subtleties of memory models by showing their individual problems. Section 2.6 covers other related memory models. Section 2.7 describes the difficulties of using simulators to evaluate memory models. Section 2.8 reviews open-source processors, and contrasts our CMD framework with existing processor-design frameworks.

2.1 Formal Definitions of Memory Models

A memory model can be defined formally using an *axiomatic* definition or an *operational* definition. An *axiomatic* definition is a set of axioms that any legal program behaviors must satisfy. An *operational* definition is an abstract machine that can be operated by a set of rules, and legal program behaviors are those that can be produced by running the program on the abstract machine. We use Sequential Consistency (SC) [81] as an example to explain operational and axiomatic definitions in

more details.

2.1.1 Operational Definition of SC

Figure 2-1 shows the abstract machine of SC, in which all the processors are connected directly to a *monolithic memory* that processes load and store requests instantaneously. The operation of this machine is simple: in one step we pick any processor to execute the next instruction on that processor *atomically*. That is, if the instruction is a reg-to-reg (i.e., ALU computation) or branch instruction, it just modifies the local register states of the processor; if it is a load, it reads from the monolithic memory instantaneously and updates the register state; and if it is a store, it updates the monolithic memory instantaneously and increments the PC. It should be noted that no two processors can execute instructions in the same step.

As an example, consider the Dekker algorithm in Figure 2-2 (all memory locations are initialized as 0). If we operate the abstract machine by executing instructions in the order of $I_1 \rightarrow I_2 \rightarrow I_3 \rightarrow I_4$, then we get the legal SC behavior $r_1 = 0$ and $r_2 = 1$. However, no operation of the machine can produce $r_1 = r_2 = 0$, which is forbidden by SC.

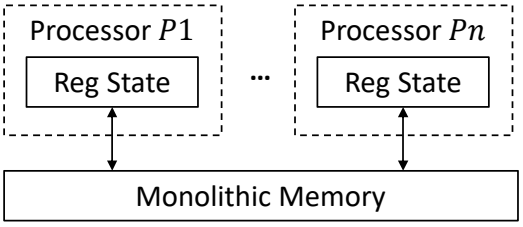


Figure 2-1: SC abstract machine

Proc. P1	Proc. P2
I_1 : St $[a]$ 1	I_3 : St $[b]$ 1
$I_2 : r_1 =$ Ld $[b]$	$I_4 : r_2 =$ Ld $[a]$
SC allows $\langle r_1 = 1, r_2 = 1\rangle$, $\langle r_1 = 0, r_2 = 1\rangle$ and $\langle r_1 = 1, r_2 = 0\rangle$, but forbids $\langle r_1 = 0, r_2 = 0\rangle$.	

Figure 2-2: Dekker algorithm

2.1.2 Axiomatic Definition of SC

Before giving the axioms that program behaviors allowed by SC must satisfy, we first need to define what is a program behavior in the axiomatic setting. For all the axiomatic definitions in this thesis, a program behavior is characterized by the following three relations:

- Program order $<_{po}$: The local ordering of instructions executed on a single processor according to program logic.

- Global memory order $<_{mo}$: A total order of all memory instructions from all processors, which reflects the real execution order of memory instructions.

- Read-from relation $\xrightarrow{rf}$: The relation that identifies the store that each load reads (i.e., store $\xrightarrow{rf}$ load).

The program behavior represented by $\langle <_{po}, <_{mo}, \xrightarrow{rf} \rangle$ will be allowed by a memory model if it satisfies all the axioms of the memory model.

It should be noted that $<_{mo}$ and $\xrightarrow{rf}$ cannot be observed directly from the program result. The program result can only tell us which instructions have been executed (i.e., $<_{po}$) and the value of each load (but not which store supplies the value). To determine if certain load values of a program is allowed by the memory model, we need to come up with relations $<_{mo}$ and $\xrightarrow{rf}$ that satisfy the axioms of the memory model. The need to find relations that are not directly observable is a common drawback of axiomatic definitions compared to operational definitions which can run the program directly on the abstract machine to produce answers.

Figure 2-3 shows the axioms of SC. Axiom $\text{InstOrder}_{\text{SC}}$ says that the local order between every pair of memory instructions (I_1 and I_2) must be preserved in the global order, i.e., no rerodering in SC. Axiom $\text{LoadValue}_{\text{SC}}$ specifies the value of each load: a load can read only the youngest store among the stores older than the load in $<_{mo}$. Notation $\max_{<mo}\{\text{set of stores}\}$ returns the youngest one among the set of stores according to $<_{mo}$.

Axiom $\text{InstOrder}_{\text{SC}}$ (preserved instruction ordering):

$$I_1 <_{po} I_2 \;\Rightarrow\; I_1 <_{mo} I_2$$

Axiom $\text{LoadValue}_{\text{SC}}$ (the value of a load):

$$\mathsf{St}\ [a]\ v \xrightarrow{rf} \mathsf{Ld}\ [a] \;\Rightarrow \mathsf{St}\ [a]\ v = \max_{<mo} \left\{ \mathsf{St}\ [a]\ v' \;\middle|\; \mathsf{St}\ [a]\ v' <_{mo} \mathsf{Ld}\ [a] \right\}$$

Figure 2-3: Axioms of SC

It should be noted that axiomatic definitions do not give a procedure to produce legal program behaviors. They can only check if a behavior is legal or not. In contrast, operational definitions can generate all legal program behaviors by running the program on the abstract machines. Therefore, axiomatic and operational definitions are complementary to each other, and it is highly desirable to have equivalent axiomatic and operational definitions for a memory model.

2.2 Fence Instructions

If an ISA has a memory model weaker than SC, then it must provide *fence instructions* as a mean to ensure that multithreaded programs behave sequentially consistent. For example, the memory model of ARM is weaker than SC, and the program in Figure 2-2 will occasionally give the non-SC result $r_1 = r_2 = 0$ in an ARM machine. To forbid this behavior, one needs to insert an ARM DMB fence between the store and the load in each processor.

The types and semantics of fence instructions vary across memory models and ISAs. We defer the discussion of the formal definitions of fence instructions to Chapters 3 and 4 when we introduce the memory models we have constructed. In this chapter, we informally use FenceXY to represent a fence instruction, which stalls instructions of type Y younger than the fence from being issued to memory until instructions of type X older than the fence complete their memory accesses. For example, FenceLS is a load-store fence. Any store instructions older than the fence (in the same processor) cannot be issued to memory until all load instructions older than the fence (in the same processor) have loaded their values.

2.3 Litmus Tests

In the rest of the thesis, we will use litmus tests like Figure 2-4a to show the properties of memory models or to differentiate two memory models. A litmus test is a program snippet, and we focus on whether a specific behavior of this program is allowed by

each memory model. In all litmus test, it is assumed that the initial value of every memory location is 0.

As an example, Figure 2-4 shows several litmus tests for instruction reorderings (FenceLL and FenceSS are load-load fence and store-store fence, respectively).

Test SB (Figure 2-4a): A TSO machine can execute I_2 and I_4 while I_1 and I_3 are buffered in the store buffers. The resulting behavior is as if the store and the load were reordered on each processor.

Test MP+FenceLL (Figure 2-4b): In an Alpha machine, I_1 and I_2 may be drained from the store buffer of P1 out of order. This is as if P1 reordered the two stores.

Test MP+FenceSS (Figure 2-4c): In an Alpha machine, I_4 and I_5 in the ROB of P2 may be executed out of order. This is as if P2 reordered the two loads.

Test LB (Figure 2-4d): An Alpha machine may enter a store into the memory before all older instructions have been committed. This is as if the load and the store were reordered on each processor.

Proc. P1	Proc. P2
I_1 : St $[a]$ 1	I_3 : St $[b]$ 1
I_2 : r_1 = Ld $[b]$	I_4 : r_2 = Ld $[a]$
SC forbids, but TSO allows: $r_1 = 0, r_2 = 0$	

(a) SB: test for store-load reordering (same as Figure 2-2)

Proc. P1	Proc. P2
I_1 : St $[a]$ 1	I_3 : r_1 = Ld $[b]$
I_2 : St $[b]$ 1	I_4 : FenceLL
	I_5 : r_2 = Ld $[a]$
TSO forbids, but Alpha, RMO and ARM allow: $r_1 = 1, r_2 = 0$	

(b) MP+FenceLL: test for store-store reorderings

Proc. P1	Proc. P2
I_1 : St $[a]$ 1	I_4 : r_1 = Ld $[b]$
I_2 : FenceSS	I_5 : r_2 = Ld $[a]$
I_3 : St $[b]$ 1	
TSO forbids, but Alpha, RMO and ARM allow: $r_1 = 1, r_2 = 0$	

(c) MP+FenceSS: test for load-load reorderings

Proc. P1	Proc. P2
I_1 : r_1 = Ld $[b]$	I_3 : r_2 = Ld $[a]$
I_2 : St $[a]$ 1	I_4 : St $[b]$ 1
TSO forbids, but Alpha, RMO, and ARM allow: $r_1 = r_2 = 1$	

(d) LB: test for load-store reordering

Figure 2-4: Litmus tests for instruction reordering

2.4 Atomic versus Non-Atomic Memory

The coherent memory systems in multiprocessors can be classified into two types: *atomic* and *non-atomic* memory systems, which we explain next.

2.4.1 Atomic Memory

For an atomic memory system, a store issued to it will be advertised to all processors simultaneously. Such a memory system can be abstracted to a *monolithic memory* that processes loads and stores instantaneously. Implementations of atomic memory systems are well understood and used pervasively in practice. For example, a coherent write-back cache hierarchy with a MSI/MESI protocol can be an atomic memory system [134, 139]. In such a cache hierarchy, the moment a store request is written to the L1 data array corresponds to processing the store instantaneously in the monolithic memory abstraction; and the moment a load request gets its value corresponds to the instantaneous processing of the load in the monolithic memory.

The abstraction of atomic memory can be relaxed slightly to allow the a processor issuing a store to see the store before any other processors. It should be noted that the store still becomes visible to processors other than the issuing one at the same time. This corresponds to adding a private store buffer for each processor on top of the coherent cache hierarchy in the implementation.

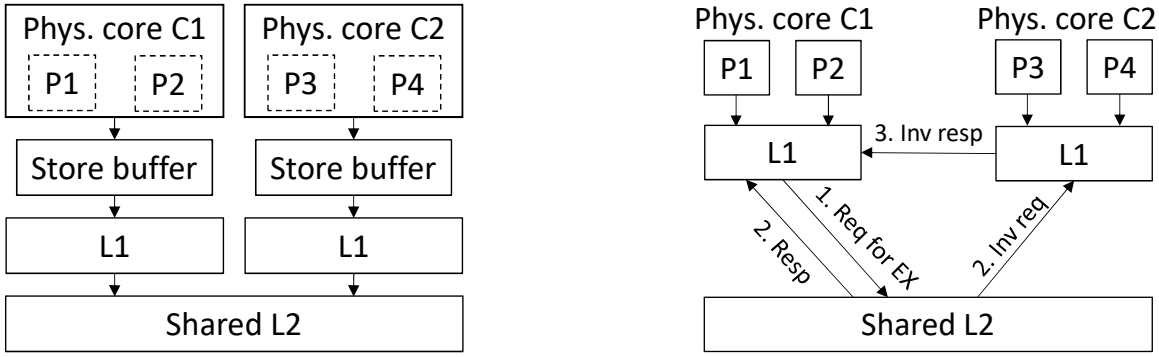
2.4.2 Non-Atomic Memory

In a non-atomic memory system, a store becomes visible to different processors at different times. According to our knowledge, nowadays only the memory systems of POWER processors are non-atomic. (GPUs may have non-atomic memories, but they are beyond the scope of this thesis which is about CPU memory models only.)

A memory system can become non-atomic because of shared store buffers or shared write-through caches. Consider the multiprocessor in Figure 2-5a, which contains two physical cores C1 and C2 connected via a two-level cache hierarchy. L1 caches are private to each physical core while L2 is the shared last level cache. Each physical

core has enabled simultaneous multithreading (SMT), and appears as two logical processors to the programmer. That is, logical processors P1 and P2 share C1 and its store buffer, while logical processors P3 and P4 share C2. We consider the case where each store in the store buffer is not tagged with the processor ID and it can be read by both logical processors sharing the store buffer. In this case, if P1 issues a store, the store will be buffered in the store buffer of C1. Then P2 can read the value of the store while P3 and P4 cannot. Besides, if P3 or P4 issues a store for the same address at this time, this new store may hit in the L1 of C2 while the store by P1 is still in the store buffer. Thus, the new store by P3 or P4 is ordered before the store by P1 in the coherence order for the store address. As a result, the shared store buffers (without processor-ID tags) together with cache hierarchy form a non-atomic memory system.

We can force each logical processor to tag its stores in the shared store buffer so that other processors do not read these stores in the store buffer. However, if L1s are write-through caches, the memory system can become non-atomic for a similar reason, and it is much more difficult to tag values in the L1s.



(a) Shared store buffers

(b) DASH protocol

Figure 2-5: Examples of non-atomic memory systems

Even if we make L1s write-back, the memory system can still fail to be an atomic memory system, for example, if it uses the DASH coherence protocol [85] as shown in Figure 2-5b. Consider the case when both L1s hold address a in the shared state, and P1 is issuing a store to a. In this case, the L1 of core C1 will send a request for exclusive permission to the shared L2. When L2 sees the request, it sends the

response to C1 and the invalidation request to C2 *simultaneously*. When the L1 of C1 receives the response, it can directly write the store data into the cache without waiting for the invalidation response from C2. At this moment, P2 can read the more up-to-date store data from the L1 of C1, while P3 can only read the original memory value for a. Note that in case P3 or P4 issues another store for a at this moment, this new store must be ordered after the store by P1 in the coherence order of address a, because L2 has already acknowledged the store by P1. This is different from non-atomic memory systems with shared store buffers or shared write-through caches.

2.4.3 Litmus Tests for Memory Atomicity

Figure 2-6 shows three litmus tests to distinguish non-atomic memory from non-atomic memory. In all the litmus tests, the instruction execution on each processor is serialized either by data dependencies or fence instructions, so any non-SC behaviors can be caused only by the non-atomicity of the memory system. For example, in P2 of Figure 2-6a, store I_3 cannot be issued to memory until load I_2 gets its result from memory, because the store data of I_3 depends on the result of load I_2. For another example, in P3 of Figure 2-6a, I_6 cannot be issued to memory until I_4 gets its result from memory because of the load-load fence I_5. In the following, we explain each litmus test briefly.

WRC (write-read-causality, Figure 2-6a): Assuming the store buffer is private to each processor (i.e., atomic memory), if one observes $r_1 = 2$ and $r_2 = 1$ then r_3 must be 2. However, if an architecture allows a store buffer to be shared by P1 and P2 but not P3 (as shown in Figure 2-5a), then P2 can see the value of I_1 from the shared store buffer before I_1 has updated the memory, allowing P3 to still see the old value of a. As explained in Section 2.4.2, a write-through cache shared by P1 and P2 but not P3, and the DASH coherence protocol in Figure 2-5b can also cause this non-atomic behavior.

WWC (write-write-causality, Figure 2-6b): This litmus test is similar to WRC but replaces the load in I_6 with a store. The behavior is possible if P1 and P2 share a

write-through cache or store buffer (as shown in Figure 2-5a). However, as explained in Section 2.4.2, the DASH coherence protocol cannot generate this behavior.

IRIW (independent-reads-independent-writes, Figure 2-6c): This behavior is possible if P1 and P2 share a write-through cache or a store buffer and so do P3 and P4 (as shown in Figure 2-5a). It is also possible using the DASH protocol in Figure 2-5b.

Proc. P1	Proc. P2	Proc. P3
I_1 : St $[a]$ 2	$I_2 : r_1 =$ Ld $[a]$ I_3 : St $[b]$ $(r_1 - 1)$	$I_4 : r_2 =$ Ld $[b]$ I_5 : FenceLL $I_6 : r_3 =$ Ld $[a]$
Atomic memory forbids, but non-atomic memory allows: $r_1 = 2,\ r_2 = 1,\ r_3 = 0$		

(a) Test WRC (write-read-causality) [125]. Store I_1 is observed by load I_2 but not by load I_6 which is causally after I_2.

Proc. P1	Proc. P2	Proc. P3
I_1 : St $[a]$ 2	$I_2 : r_1 =$ Ld $[a]$ I_3 : St $[b]$ $(r_1 - 1)$	$I_4 : r_2 =$ Ld $[b]$ I_5 : St $[a]$ r_2
Atomic memory and *DASH protocol* forbid, but shared store buffers and shared write-through L1 allow: $r_1 = 2,\ r_2 = 1,\ m[a] = 2$		

(b) Test WWC (write-write-causality) [98, 15]. Store I_1 is observed by load I_2 but writes memory after store I_5 which is causally after I_2.

Proc. P1	Proc. P2	Proc. P3	Proc. P4
I_1 : St $[a]$ 1	$I_2 : r_1 =$ Ld $[a]$ I_3 : FenceLL $I_4 : r_2 =$ Ld $[b]$	I_5 : St $[b]$ 1	$I_6 : r_3 =$ Ld $[b]$ I_7 : FenceLL $I_8 : r_4 =$ Ld $[a]$
Atomic memory forbids, but non-atomic memory allows: $r_1 = 1,\ r_2 = 0,\ r_3 = 1,\ r_4 = 0$			

(c) Test IRIW (independent-reads-indenpdent-writes) [125]. P1 and P3 perform two independent stores, which are observed by P2 and P4 in different order.

Figure 2-6: Litmus tests for non-atomic memory

2.4.4 Atomic and Non-Atomic Memory Models

Because of the drastic difference in the nature of atomic and non-atomic memory systems, memory models are also classified into atomic memory models and non-

atomic memory models according to the type of memory systems that the model supports in implementations. Most memory models are atomic memory models, e.g., SC, TSO, RMO, Alpha, and ARMv8. The only non-atomic memory model today is the POWER memory model. In general, non-atomic memory models are much more complicated. In fact, ARM has recently changed its memory model from non-atomic to atomic in its version 8. Due to the prevalence of atomic memory models, *this thesis focuses mainly on atomic memory models.*

2.5 Problems with Existing Memory Models

Here we review existing weak memory models and explain their problems.

2.5.1 SC for Data-Race-Free (DRF)

Data-Race-Free-0 (DRF0) is an important class of software programs where races for shared variables are restricted to locks [19]. Adve et al. [19] have shown that the behavior of DRF0 programs is sequentially consistent. DRF0 has also been extended to DRF-1 [20], DRF-x [99], and DRF-rlx [131] to cover more programming patterns. There are also hardware schemes [46, 137, 130] that accelerate DRF programs. While DRF is a very useful programming paradigm, we believe that a memory model for an ISA needs to specify the behaviors of all programs, including non-DRF programs.

2.5.2 Release Consistency (RC)

RC [60] is another important software programming model. The programmer needs to distinguish synchronizing memory accesses from ordinary ones, and label synchronizing accesses as acquire or release. Intuitively, if a load-acquire in processor P1 reads the value of a store-release in processor P2, then memory accesses younger than the load-acquire in P1 will happen after memory accesses older than the store-release in P2. Gharachorloo et al. [60] define what is a *properly-labeled* program, and shown that the behaviors of such programs are SC.

The RC definition attempts to define the behaviors for all programs in terms of the reorderings of events, and an event refers to performing a memory access with respect to a processor. However, it is not easy to derive the value that each load should get based on the ordering of events, especially when the program is not properly labeled.

Furthermore, the RC definition (both RC_{SC} and RC_{PC} in [60]) admits some behaviors unique to non-atomic memory models, but still does not support all non-atomic memory systems in the implementation. In particular, the RC definition allows the behaviors of WRC and IRIW (Figures 2-6a and 2-6c), but it disallows the behavior of WWC (Figure 2-6b). In WWC, when I_2 reads the value of store I_1, the RC definition says that I_1 is performed with respect to (w.r.t.) P2. Since store I_5 has not been issued due to the data dependencies in P2 and P3, I_1 must be performed w.r.t. P2 before I_5. The RC definition says that "all writes to the same location are serialized in some order and are performed in that order with respect to any processor" [60, Section 2]. Thus, I_1 is before I_5 in the serialization order of stores for address a, and the final memory value of a cannot be 2 (the value of I_1), i.e., RC forbids the behavior of WWC and thus forbids non-atomic memory systems that have shared store buffers or shared write-through caches.

2.5.3 RMO and Alpha

RMO [142] and Alpha [16] can be viewed as variants of RC in the class of atomic memory models. They both allow all four load/store reorderings. However, they have different problems regarding the ordering of dependent instructions. The RMO definition is too restrictive in the ordering of dependent instructions, while the Alpha definition is too liberal. Next we explain the problems in more details.

RMO: RMO intends to order dependent instructions in certain cases, but its definition is too restrictive in the sense that it forbids implementations from performing speculative load execution and store forwarding simultaneously without performing additional checks. Consider the litmus test in Figure 2-7 (MEMBAR is the fence in RMO). In P2, the execution of I_6 is conditional on the result of I_4, I_7 loads from the address that I_6 stores to, and I_9 uses the results of I_7. According the definition

of dependency ordering in RMO [142, Section D.3.3], I_9 depends on I_4 transitively. Then the RMO axioms [142, Section D.4] dictate that I_9 must be after I_4 in the memory order, and thus forbid the behavior in Figure 2-7. However, this behavior is possible in hardware with speculative load execution and store forwarding, i.e., I_7 first speculatively bypasses from I_6, and then I_9 executes speculatively to get 0.

A more sophisticated implementation can still perform speculative load execution and store forwarding to let I_9 get value 0 speculatively, but it will detect the violation of RMO and squash I_9 when the cache line loaded by I_9 is evicted from the L1 of P2 because of store I_1. However, it should be noted that monitoring L1 evictions for every load is an overkill, and it may not be easy to determine exactly which loads should be affected by L1 evictions in a RMO implementation.

Proc. P1	Proc P2
I_1 : St a 1	$I_4 : r_1 =$ Ld b
I_2 : MEMBAR	I_5 : if$(r_1 \neq 1)$ exit
I_3 : St b 1	I_6 : St c 1
	$I_7 : r_2 =$ Ld c
	$I_8 : r_3 = a + r_2 - 1$
	$I_9 : r_4 =$ Ld r_3
RMO forbids: $r_1 = 1,\ r_2 = 1, r_3 = a,\ r_4 = 0$	

Figure 2-7: RMO dependency order

Alpha: Alpha is much more liberal in that it allows the reordering of dependent instructions. However, this gives rise to an out-of-thin-air (OOTA) problem [40].

Proc. P1	Proc. P2
$I_1 : r_1 =$ Ld $[a]$	$I_3 : r_2 =$ Ld $[b]$
I_2 : St $[b]$ r_1	I_4 : St $[a]$ r_2
All models should forbid: $r_1 = r_2 = 42$	

Figure 2-8: OOTA

Figure 2-8 shows an example OOTA behavior, in which value 42 is generated out of thin air. If allowing all load/store reorderings is simply removing the $\text{InstOrder}_{\text{SC}}$ axiom from the the SC axiomatic definition, then the behavior would be legal. OOTA behaviors must be forbidden by the memory-model definition, because they can never be generated in any existing or reasonable hardware implementations and they make

formal analysis of program semantics almost impossible. To forbid OOTA behaviors, Alpha introduces an axiom which requires looking into *all possible* execution paths to determine if a younger store should not be reordered with an older load to avoid cyclic dependencies [16, Chapter 5.6.1.7]. This axiom complicates the model definition significantly, because most axiomatic models only examine a single execution path at a time.

2.5.4 ARM

As noted in Chapter 1, ARM has recently changed its memory model. The latest ARM memory model is also a variant of RC in the class of atomic memory models. It allows all four load/store reorderings. It enforces orderings between certain dependent instructions, and is free from the problems of RMO or Alpha regarding dependencies. However, it introduces complications in the ordering of loads for the same address, which we will discuss in Section 3.1.5.

2.6 Other Related Memory Models

The tutorial by Adve et al. [18] has described the relations between some of the models discussed above as well as some other models [62, 52]. Recently, there has been a lot of work on the programming models for emerging computing resources such as GPU [23, 69, 57, 107, 29, 28], and storage devices such as non-volatile memories [79, 105, 71, 129]. There are also efforts in specifying the semantics of high-level languages, e.g., C/C++ [133, 39, 34, 33, 73, 72, 106] and Java [97, 44, 95]. As mentioned in Chapter 1, weak memory models are driven by optimizations in the implementations. The optimizations used in the implementations of CPUs, GPUs, and high-level-language compilers are drastically different. For example, compilers can perform constant propagation which is extremely difficult (if not possible) to do in hardware like CPUs. Therefore, the ideas behind the memory models of CPUs, GPUs and high-level languages also differ from each other, and *this thesis is about CPU memory models only.*

Model-checking tools are useful in finding memory-model related bugs; prior works [27, 91, 96, 138, 92, 93] have presented tools for various aspects of memory-model testing.

2.7 Difficulties of Using Simulators to Evaluate Memory Models

To use a simulator to evaluate memory models, the simulator should not only run at high speed to be able to complete realistic benchmarks, but also model accurately the microarchitectural features affected by memory models and the timing of synchronizations between processors. However, we find most simulators cannot meet these two requirements at the same time.

Fast simulators often use Pin [90] or QEMU [9] as the functional front-end to gain simulation speed. However, these simulators may not be able to model accurately the interaction or synchronization between processors. For example, consider a producer-consumer case. In this case, the producer thread first writes data to memory and then sets the flag in memory, while the consumer thread first spins on the flag until the flag is set and then reads the data. The number of spins on the flag in the simulation would be determined by the functional front-end instead of the timing of the target system.

A cycle-accurate simulator like GEM5 [37] may be able to simulate accurately the inter-processor interaction. However, the simulation speed is too slow to finish running realistic benchmarks, and sometimes even a "cycle-accurate" simulator may fail to model all the microarchitectural details. For example, in GEM5, a load instruction occupies an instruction-issue-queue (or reservation-station) entry until it gets its value[1], and the instruction-issue-queue entry is used as the retry starting point in case the load misses in TLB or is stalled from being issued to memory (e.g., because the address range of an older store overlaps partially with that of the load). This

[1]See `https://github.com/gem5/gem5/blob/91195ae7f637d1d4879cc3bf0860147333846e75/src/cpu/o3/inst_queue_impl.hh`. Accessed on 03/13/2019.

unnecessarily increases the pressure on the instruction issue queue, because the load can be removed from the instruction issue queue as soon as its address operand is ready and the processor can use the load-queue entry as the retry starting point. As a result, the instruction issue queue may become a major bottleneck that overshadows the performance differences between different memory models.

2.8 Open-Source Processor Designs

There is a long, if not very rich, history of processors that were designed in an academic setting. Examples of early efforts include the RISC processors like MIPS [66], RISC I and RISC II [111], and SPUR [67], and dataflow machines like Monsoon [110], Sigma1 [68], EM4 [122], and TRIPS [123]. All these attempts were focused on demonstrating new architectures; there was no expectation that other people would improve or refine an existing implementation. Publication of the RISC-V ISA in 2010 has already unleashed a number of open-source processor designs [56, 13, 7, 12, 8, 11, 2, 102] and probably many more are in the works which are not necessarily open source. There are also examples of SoCs that use RISC-V processors. e.g.: [31, 53, 64, 49, 83].

Most open source RISC-V designs are meant to be used by others in their own SoC designs and, to that extent, they provide a framework for generating the RTL for a variety of specific configurations of the design. We discuss several examples of such frameworks:

- *Rocket chip generator* [11, 84]: generates SoCs with RISC-V cores and accelerators. The RISC-V cores are parameterized by caches, branch predictors, degree of superscalarity, and ISA extensions such as hardware multipliers (M), atomic memory instructions (A), FPUs (F/D), and compressed instructions (C). At the SoC level, one can specify the number of cores, accelerators chosen from a fixed library, and the interconnect. Given all the parameters, the Rocket chip generator produces synthesizable Verilog RTL.

  Rocket chip generator has been used for many SoC designs [21, 152, 83, 75, 153, 141]. Some modules from the Rocket chip generator have also been used

to implement Berkeley's Out-of-Order BOOM processor [2]. The Rocket chip generator is written in Chisel [4].

- *FabScalar* [47]: allows one to assemble a variety of superscalar designs from a set of predefined pipeline-stage blocks, called CPSL. A template is used to instantiate the desired number of CPSL for every stage and then glue them together. For example, one can generate the RTL for a superscalar core with 4 fetch stages, 6 issue/register read/write back, and a chosen set of predictors. FabScalar have been successful in generating heterogeneous cores using the same ISA for a multicore chip [136, 48]. They report results which are comparable to commercial hand-crafted RTL. These CPSLs are not intended to be modified themselves.

- *PULP* [8]: attempts to make it easy to design ultra-low power IoT SoC. It focuses on processing data coming from a variety of sensors, each one of which may require a different interface. The processor cores themselves are not intended to be refined within this framework. A number of SoCs have been fabricated using PULP [49].

All these frameworks are structural, that is, they guarantee correctness only if each component meets its timing assumptions and functionality. For some blocks, the timing assumptions are rigid, that is, the block takes a fixed known number of cycles to produce its output. For some others, like cache accesses, the timing assumption is latency insensitive. In putting together the whole system, or in replacing a block, if the user observes all these timing constraints, the result should be correct. However no mechanical verification is performed to ensure that the timing assumptions were not violated, and often these timing violations are not obvious due to interactions across blocks with different timing assumptions.

The goal of our CMD framework is more ambitious, in the sense that, in addition to parameterized designs, we want the users to be able to incorporate new microarchitectural ideas. For example, replace a central instruction issue queue in an OOO design, with several instruction issue queues, one for each functional unit. Tradition-

ally, making such changes requires a deep knowledge of the internal functioning of the other blocks, otherwise, the processor is unlikely to function. We want to encapsulate enough properties in the interface of each block so that it can be composed without understanding the internal details.

A recent paper [82] argued for agile development of processors along the lines of agile development of software. The methodological concerns expressed in that paper are orthogonal to the concerns expressed in this thesis, and the two methodologies can be used together. However, we do advocate going beyond simple structural modules advocated in that paper to achieve true modularity which is amenable to modular refinement.

Chapter 3

GAM: a Common Base Model for Weak Memory Models

The problems of existing memory models discussed in Section 2.5 further illustrates the importance of having a common base model for weak memory models; otherwise we can be easily drowned by the subtleties of different memory models. In this chapter, we construct a common base model, i.e., the *General Atomic Memory Model (GAM)*. We restrict the discussion to atomic memory models because most CPU memory models (except POWER) are atomic memory models. In Section 3.1, we derive GAM intuitively by constructing a multiprocessor from uniprocessors and an atomic memory system. Along the construction of the multiprocessor, we discover more and more constraints on the possible behaviors of the processor. Section 3.2 translates these informal constraints into operational and axiomatic definitions of GAM, and proves the equivalence of the two definitions. During the construction procedure of GAM, we also show that there are places where different memory models may make different choices. The impact of these choices on model definitions is studied in Section 3.1, and the impact on performance is evaluated in Section 3.3. In general, these choices has little impact on performance, so GAM makes the choices that match the common assumptions in multithreaded programs. (We have not tried to parameterized the definition of GAM by the different choices.)

3.1 Intuitive Construction of GAM

We begin by studying a highly optimized out-of-order uniprocessor, OOO^U, and show that even such an aggressive implementation still observes some ordering constraints to preserve single-thread semantics. When multiple OOO^{U} processors are connected via an atomic memory system to form a multiprocessor OOO^{MP}, these constraints can be extended to form a base memory model that can characterize the behaviors of OOO^{MP} and meet the goal of preserving uniprocessor optimizations.

However, the base model is not programmable, because there is no way to restore SC for every multithreaded program. Therefore, we introduce fence instructions to control the exeuction order in OOO^{MP}. We also want to make the constructed memory model amenable for programming, i.e., the model should not break the orderings that programmers commonly assume even when programming machines with weak memory models. To match programmers' intuitions, we introduce more constraints to the constructed model, which means extra restrictions on implementations. We will study the impact of these restrictions on performance in Section 3.3.

3.1.1 Out-of-Order Uniprocessor (OOO^{U})

Figure 3-1 shows the structure of OOO^{U} which is connected to a write-back cache hierarchy. In case a memory access gets a cache miss, the processor fetches the line to L1 and then accesses it. The memory system can process multiple requests in parallel and out of order, but will process requests for the same address in the order that they are issued to the memory system. To simplify the description, we skip details that are unrelated to memory models. OOO^{U} fetches the next instruction speculatively, and every fetched instruction is inserted into the ROB in order. Loads and stores will also be inserted in the same order into the load buffer (LB) and the store buffer (SB), respectively.

OOO^{U} executes instructions out of order and speculatively, but we assume the following two restrictions on speculations:

1. A store request sent to the memory system cannot be withdrawn and its effect

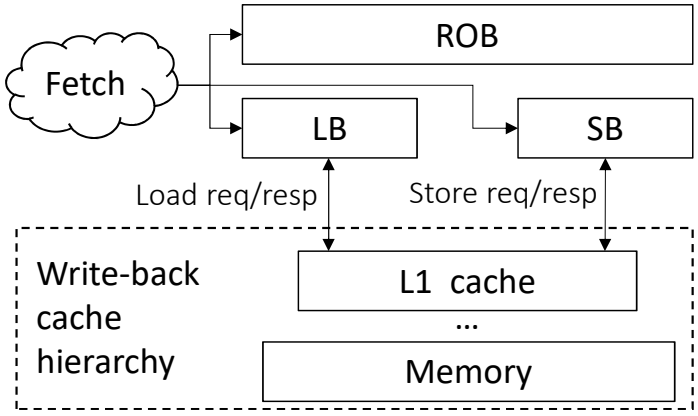


Figure 3-1: Structure of OOO^U

cannot be undone, i.e., a store cannot be sent to memory speculatively.

2. The value of any source register other than the PC of an instruction is never predicted (i.e., OOO^U does not perform any value prediction [87, 88, 58, 101, 112, 113, 128]).

While the first restriction is easy to accept, the second one will be justified in Section 3.1.4. The restrictions on speculation imply necessary conditions when an instruction can be issued to start execution. For example, an instruction cannot be issued until all its source operands are ready (i.e., have been computed by older instructions). We will discuss later about other constraints on issuing an instruction (especially a store). After being issued, a reg-to-reg or branch instruction is executed by just local computation. The execution of a store sends a store request to the memory system. The execution of a load first searches the SB for data forwarding from a store that has not completed its store request to the memory system.[1] In case forwarding is not possible, the load will send a request to the memory system.

In spite of out-of-order execution, OOO^U still commits instructions from the ROB in order. A store does not need to complete its store request in the memory system when being committed, while load, reg-to-reg and branch instructions should have got their values at commit time. In the following, when we say an instruction I_1

[1]Forwarding cannot be done after the store has been written into the L1 data array, because in a multiprocessor setting, other processors may have overwritten the value of that store.

is older than another instruction I_2, by default we mean that I_1 is before I_2 in the commit order (or equivalently, I_1 is inserted into ROB before I_2).

Instruction reordering in the uniprocessor: By instruction reordering, we mean that the *execution order* of two instructions is different from the *commit order*. The execution order is the order of the times when instructions *finish execution*. A reg-to-reg or branch instruction finishes execution when it computes its destination register value or resolves the next PC, respectively. A load finishes execution when it gets forwarding from SB or reads the data from L1. A store finishes execution when it writes the store data into the data array of the L1 cache. An instruction that is squashed (e.g., due to mis-speculation) before being committed is not a member of the execution order.

3.1.2 Constraints in OOO^{U}

All the constraints on the execution order in OOO^{U} are listed in Figure 3-2, and we will derive them one by one in this section. These constraints can be classified into two categories. The first set of constraints (SAMemSt and SAStLd) are between memory instructions for the same address, and are essential in maintaining single-thread correctness. The second set of constraints (RegRAW, BrSt and AddrSt) reflects the necessary conditions that need to be met before issuing an instruction to start execution. Although speculative execution can remove many of such conditions, some are still preserved since we have assumed some restrictions on speculation.

Constraints for memory instructions of the same address: Assume I_1 and I_2 are two memory instructions for the same address a, and I_1 is older than I_2. If both I_1 and I_2 are loads, then their executions do not need to be ordered. If I_2 is a store, it cannot write L1 before I_1 finishes execution no matter whether I_1 is a load or a store. Therefore we have the *SAMemSt* constraint in Figure 3-2.

Now consider the case that I_1 is a store and I_2 is a load. If I_2 is executed by reading L1, then it cannot do so before I_1 has written L1. Thus, the only way for these two instructions to get reordered is when I_2 gets forwarding from a store S as shown in Figure 3-3. S should be the youngest store that is older than I_2. While

- **Constraint SAMemSt** (same-address-memory-access-to-store): A store must be ordered after older memory instructions for the same address.
- **Constraint SAStLd** (same-address-store-to-load): A load must be ordered after every instruction that produce the address or data of the immediately preceding store for the same address.
- **Constraint RegRAW** (register-read-after-write): An instruction must be ordered after an older instruction that produce one of its source operands other than PC.
- **Constraint BrSt** (branch-to-store): A store must be ordered after an older branch.
- **Constraint AddrSt** (address-to-store): A store must be ordered after an instruction which produces the address of a memory instruction that is older than the store.

Figure 3-2: Constraints on execution orders in OOO^{U}

there cannot be direct ordering constraints between I_1 and I_2 due to the forwarding, if I_2 eventually gets committed without being squashed, then I_2 cannot start execution before the address and data of S have been computed by older instructions. This gives the *SAStLd* constraint in Figure 3-2.

Proc. P1
$I_1 : \mathsf{St}\ [a]\ 1$
$S : \mathsf{St}\ [a]\ r_1$
$I_2 : r_2 = \mathsf{Ld}\ [a]$

Figure 3-3: Store forwarding

Proc. P1
$I_1 : r_1 = \mathsf{Ld}\ [a]$
$I_2 : \mathsf{St}\ [r_1]\ 1$
$I_3 : r_2 = \mathsf{Ld}\ [b]$

Figure 3-4: Load speculation

Constraints for issuing to start execution: Since an instruction cannot be issued to execution without all its source operands being ready, we have the *RegRAW* constraint in Figure 3-2. Note that we have excluded PC from this constraint. This is because OOO^{U} does branch prediction, and every fetched instruction already knows its PC and can use it for execution.

Constraint RegRAW has already covered the issuing requirement for reg-to-reg, branch and load instructions. In particular, there are no more constraints regarding the issue of loads because of speculations. For example, consider the program in

Figure 3-4. OOO^{U} can issue the load in I_3 before the store address of I_2 is computed (i.e., before I_1 finishes execution), even though the address of I_2 may turn out to be the same as I_3. In case I_1 indeed writes value b into r_1, OOO^{U} will squash I_3 and re-execute it, and the execution ordering between I_1 and I_3 has been captured by constraint SAStLd.

Now we consider the constraints induced by the restriction of no speculative store issue. A simple case is that a store cannot be issued to memory (and thus cannot finish execution) when an older branch is not executed, i.e., constraint *BrSt* in Figure 3-2. This is because the branch may be mis-predicted at fetch time and will cause an ROB squash in the future. Another case is that a store cannot be issued to memory (and thus cannot finish execution) when the address of an older memory instruction is not ready, i.e., constraint *AddrSt* in Figure 3-2. This is because if we issue the store to memory and later the address of the older memory instruction turns out to be same as the store address, then we may violate single-thread correctness (i.e., constraint SAMemSt).

3.1.3 Extending Constraints to Multiprocessors

Consider the multiprocessor OOO^{MP} which connects multiple OOO^{U}s to an atomic memory system which may be implemented as a coherent write-back cache hierarchy. The constraints on local execution order in Figure 3-2 still apply to each OOO^{U} in OOO^{MP}, but they are not enough to describe the behaviors of the overall multiprocessor. The only difference between a uniprocessor and a multiprocessor is about the load values. In the uniprocessor setting, a load always gets the value of the youngest store that is older than the load. However, in OOO^{MP}, if a load gets its value from the atomic memory system, the value may come from a store of a different processor.

In order to understand such interaction via the atomic memory system, recall that the atomic memory system can be abstracted by a monolithic memory, and the time that a load/store request reads/writes the L1 data array in the atomic memory system corresponds to the instantaneous processing of the request in the monolithic memory (Section 2.4). Therefore, we can put all memory instructions into an *atomic memory*

order based on their L1 access times, which are also their execution finish times. Hence, the atomic memory order should respect local execution order (constraint $LMOrd_{Atomic}$ in Figure 3-5), and the load that accesses the memory should read from the immediate preceding store for the same address in atomic memory order (constraint $LdVal_{Atomic}$ in Figure 3-5).

In case the load does not access the memory, it gets data forwarded from the immediate preceding store from the same processor for the same address in the commit order (constraint *LdForward* in Figure 3-5), same as OOO^{U}.

- **Constraint $\mathbf{LMOrd_{Atomic}}$** (local-to-atomic-memory-order): The atomic memory order of two memory instructions from the same processor is the same as the execution order of these two instructions in that processor.
- **Constraint $\mathbf{LdVal_{Atomic}}$** (atomic-memory-load-value): A load that executes by requesting the memory system should get the value of the youngest store for the same address that is ordered before the load in the atomic memory order.
- **Constraint LdForward** (load-forward): A load that executes using locally forwarded values should get the value of the immediate preceding store from the same processor for the same address in the commit order.

Figure 3-5: Constraints for load values in OOO^{MP}

These three constraints can be restated as the two constraints LMOrd and LdVal in Figure 3-6. To do so, we put all memory instructions, including loads that forward from local stores, from all processors for all addresses in OOO^{MP} into a *global memory order* according to their execution finish times (not commit times). Thus, the global memory order should respect the atomic memory order and the execution order (constraint *LMOrd*). Note that the way a load L is executed can be distinguished by the global memory order of L and its immediate preceding store S from the same processor for the same address in the commit order. If L is ordered before S in the global memory order (i.e., L finishes execution before S is written to L1), then L must get its value forwarded from S. Otherwise, L is ordered after S in the global memory order, and L should be executed by sending a load request to the atomic memory system. Therefore, the constraints for load values in the two cases ($\text{LdVal}_{\text{Atomic}}$ and

LdForward) can be combined into constraint *LdVal* using the following observations:

1. In case of forwarding, S is before L in the commit order, and it is younger than (after) any store which is older than (before) L in the global memory order.

2. In case of reading the memory system, all stores that are before L in the commit order are also before L in the global memory order.

Constraint LdVal also appears in RMO [142] and Alpha [16].

- **Constraint LMOrd** (local-to-global-memory-order): The global memory order of two memory instructions from the same processor is the same as the execution order of these two instructions in that processor.
- **Constraint LdVal** (load-value): A load should get the value of the youngest store for the same address in the global memory order that is ordered before the load in either the global memory order or the local commit order of the processor of the load.

Figure 3-6: Additional constraints in OOO^{MP}

Atomic read-modify-write (RMW): Multiprocessors often provide atomic read-modify-write (RMW) instructions to implement synchronization primitives like locks in multithreaded programs. Here we discuss briefly the semantics of RMW instructions (which are not included in our formal definitions in Section 3.2). There are actually multiple choices for the constraints that an RMW instruction should observe. A simple way is to say that an RMW instruction for address a should obey all the constraints that apply to a load a and a store a, and that RMW must be executed by accessing the memory system.

A more complicated choice treats an RMW instruction as two separate instructions, i.e., a load L and a store S. L and S do not need to stick together in the global memory order. Assume L reads from another store S', which should be older than both L and S in the global memory order. The only requirement is that there is no other store for the same address sitting between S' and S in the global memory order. In this choice, the RMW instructions is like a pair of load-reserve and store-conditional. The meaning of atomicity is weakened in this choice, so we opt for the prior simpler semantics.

3.1.4 Constraints Required for Programming

Up to now, the constraints in Figures 3-2 and 3-6 are enough to describe the behaviors of loads and stores in OOO^{MP}: constraints in Figure 3-2 specify which local commit order should be preserved in the local execution order, constraint LMOrd translates the local execution order of memory instructions to the global memory order, and finally constraint LdVal specifies the value of each load given the global memory order and the commit order of each processor. However, these constraints are not enough for parallel programming especially when programmers want to restore SC. *Memory fence instructions* and *enforceable dependencies* are two mechanisms to control load/store reorderings. We will first introduce fence instructions and associated new constraints, and then discuss enforceable dependencies that have already been provided by the current constraints. The inclusion of these new constraints results in memory model *GAM0*, an initial version of GAM.

Fences to Control Orderings

Here we provide four basic fences: FenceLL, FenceLS, FenceSL, and FenceSS. These fences order all memory instructions of a given type before the fence with all memory instructions of another given type after the fence in the execution order. For example, FenceLS orders all loads before the fence with all stores after the fence in the execution order. To align with our previous descriptions that each instruction has an execution finish time, we can consider that a fence also needs to be executed but acts as a NOP. A fence restricts execution order according to the *FenceOrd* (fence-ordering) constraint in Figure 3-7. It should be noted that a fence can only be ordered with a memory instruction, and two fences are not ordered (directly) with respect to each other. Because of constraint LMOrd, the execution ordering enforced by fences will also be reflected in the global memory order.

These fences can be combined to produce stronger fences, such as the following three which are commonly used. We expect most users to not go beyond the following three fences because of their direct relation to programming language memory models.

- Acquire fence (FenceAcq): FenceLL; FenceLS.
- Release fence (FenceRel): FenceLS; FenceSS.
- Full fence (FenceFull): FenceLL; FenceLS; FenceSL; FenceSS.

- **Constraint FenceOrd** (fence-ordering): A FenceXY must be execution-ordered after all older memory instructions of type X (from the same processor), and execution-ordered before all younger memory instructions of type Y (from the same processor).

Figure 3-7: Additional constraints for fences

Data Dependencies to Enforce Ordering

The most commonly used enforceable dependency in programming is the data dependency. Consider litmus test MP+addr (message passing with dependency on address) in Figure 3-8a. Since the address of the load in I_5 depends on the result of I_4 (i.e., I_4 and I_5 are data-dependent loads), most programmers will assume that the two loads in P2 should not be reordered, and thus the non-SC behavior $\langle r_1 = a, r_2 = 0 \rangle$ should never happen even if there is no FenceLL between the two loads in P2. GAM0 matches this assumption of programmers because constraints RegRAW and LMOrd indeed keep I_4 before I_5 in the execution order and global memory order.

Programmers can in fact exploit the feature of data-dependent load-load ordering to replace FenceLL with artificial data dependencies. Consider the program in Figure 3-8b. The intent is that P2 should execute load b (I_4) before load a (I_6). To avoid inserting a fence between the two loads, one can create an artificial dependency from the result of the first load to the address of the second load. In this way, GAM0 will still forbid the non-SC behavior. This optimization can be useful when only I_6, but not any instruction following I_6, needs to be ordered after I_4, i.e., the execution of instructions following I_6 will not be stalled by any fence. It should be noted that P2 should not optimize I_5 into $r_2 = a$; otherwise there will not be any dependency from I_4 to I_6. That is, implementations of GAM must respect *syntatic* data dependency.

Proc. P1	Proc. P2
I_1 : St $[a]$ 1	$I_4 : r_1 =$ Ld $[b]$
I_2 : FenceSS	$I_5 : r_2 =$ Ld $[r_1]$
I_3 : St $[b]$ a	
GAM0 forbids $r_1 = a, r_2 = 0$	

(a) MP+addr

Proc. P1	Proc. P2
I_1 : St $[a]$ 1	$I_4 : r_1 =$ Ld $[b]$
I_2 : FenceSS	$I_5 : r_2 = a + r_1 - r_1$
I_3 : St $[b]$ 1	$I_6 : r_3 =$ Ld $[r_2]$
GAM0 forbids $r_1 = 1, r_2 = a, r_3 = 0$	

(b) MP+artificial-addr

Proc. P1	Proc. P2
I_1 : St $[a]$ 1	$I_4 : r_1 =$ Ld $[b]$
I_2 : FenceSS	I_5 : St $[c]$ r_1
I_3 : St $[b]$ 1	$I_6 : r_2 =$ Ld $[c]$
	$I_7 : r_3 = a + r_2 - r_2$
	$I_8 : r_4 =$ Ld $[r_3]$
GAM0 forbids $r_1 = r_2 = 1$, $r_3 = a, r_4 = 0$	

(c) Dependency via memory

Proc. P1	Proc. P2
I_1 : St $[a]$ 1	$I_4 : r_1 =$ Ld $[a]$
I_2 : FenceSS	$I_5 : r_2 =$ Ld $[b]$
I_3 : St $[b]$ a	$I_6 : r_3 =$ Ld $[r_2]$
GAM0 forbids $r_1 = 0$, $r_2 = a$, $r_3 = 0$	

(d) MP+prefetch

Figure 3-8: Litmus tests of data-dependency ordering

Data dependencies can not only be created by read-after-write (RAW) on registers, but also by RAW on memory locations. GAM0 will still order two loads which are related by a chain of data dependencies via registers and memory locations. Consider the program in Figure 3-8c. P2 first loads from address b, then stores the result to address c, next loads from address c again, and finally loads from an address a which is computed using the load result on c. There is a chain of data dependencies from the first load to the last load in P2, and programmers would assume that these two loads are ordered. GAM0 indeed enforces this ordering by constraint SAStLd, which says I_6 should be ordered after I_4, i.e., the instruction that produce the data of I_5.

Restrictions on implementations: Enforcing data-dependency ordering does not come at no cost. As mentioned in Section 3.1.1, the processor should not perform value prediction. To understand why, consider again the program in Figure 3-8a. If P2 is allowed to perform value prediction, then it can predict the result of I_4 to be a, and issues I_5 to the memory system even before P1 issues any store. This will make the non-SC behavior possible. Martin et al. [101] have also noted that it is difficult to implement value prediction for weak memory models that enforce data-dependency ordering.

While value prediction is a still-evolving technique, a processor can break data-dependency ordering by just allowing a load to get data forwarding from an older executed load (i.e., load-load forwarding). Consider the MP+prefetch litmus test in Figure 3-8d. In case load-load forwarding is allowed, P2 can first execute I_4 by reading 0 from memory. Then, P1 executes all its instructions in order, and finishes writing both stores to memory. Next P2 executes I_5 by reading the up-to-date value a for address b from memory, and finally executes I_6 by forwarding the stale value 0 from I_4. This generates the non-SC behavior. To keep the data-dependency ordering, OOO^{U} is only allowed to forward data from older stores as described in Section 3.1.1.

Another technique that can break data-dependency ordering is the *delayed invalidation* in the L1 cache. That is, L1 can respond to an invalidation from the parent cache immediately without truly evicting the stale cache line. Consider the MP+addr litmus test in Figure 3-8a. We consider the case that address a is initially in the L1 cache of P2, while address b is not. In this case, let P1 first executes the two stores sequentially and P2 will delay the invalidation of a. When P2 executes the two loads afterwards, P2 can get value 1 for b from the parent cache (or main memory) while still getting stale value 0 for a from its local L1. This is as if the two data-dependent loads in P2 were reordered. To keep data-dependency ordering, the stale lines must be evicted if L1 is waiting for any response from the parent. Even if the memory model does not enforce data-dependency ordering, fences have to do extra work to clear these stale lines in L1.

An extreme of delayed invalidation is not to invalidate shared copies at all. This idea has been exploited in several recently proposed coherence protocols [46, 119]. We will describe and evaluate such a self-invalidation (SI) coherence protocol in Chapter 6. The main idea of the SI protocol is that the directory tracks only the child cache that owns the data in the exclusive state, and does not track any shared copies. The child cache will self-invalidate all the shared copies in it if the core executes a fence instruction.

Enforcing data-dependency ordering is a balance between programming and processor implementation. Nevertheless, not enforcing this ordering will result in extra

fences in program patterns like pointer-chasing. In Section 3.3, we will show that forbidding load-load forwarding has negligible performance impact. We do not evaluate the performance impact of value prediction, because it strongly depends on the effectiveness of the predictors and is beyond the scope of this paper. In Sections 6.2 to 6.4, we will show that the SI coherence protocol, which is admitted only if the memory model does not enforce data-dependency ordering, does not improve performance or energy efficiency. It may even cause degradation in case of multithreaded benchmarks with frequent synchronizations.

The constraints in Figures 3-2, 3-6 and 3-7 have now formed a complete memory model, which preserves uniprocessor optimizations in implementations and has sufficient ordering mechanisms for programming. Since this memory model targets multiprocessors with atomic memory systems, we refer to this model as *General Atomic Memory Model 0* (GAM0).

3.1.5 To Order or Not to Order: Same-Address Loads

GAM0 does not have the *per-location SC* [42] property which many programmers expect a memory model to have. Per-location SC requires that all accesses to a single address appear to execute in a sequential order which is consistent with the commit order of each processor. In terms of the orderings of memory instructions for the same address, GAM0 already enforces the ordering between an older memory instruction to a younger store. Although GAM0 allows a younger load to be reordered with an older store, the load will get the value of the store, so these two instructions can still be put into the sequential order. The only place where GAM0 violates per-location SC is when there are two consecutive loads for the same address. Consider the CoRR (coherent read-read) litmus test in Figure 3-9a. Models with per-location SC would disallow the non-SC behavior $\langle r_1 = 1, r_2 = 0 \rangle$. However, OOO^{U} can execute I_2 and I_3 out of order and there is no constraint in GAM0 to order these two loads. Thus, the global memory order in GAM0 can be $I_3 \rightarrow I_1 \rightarrow I_2$, causing the non-SC behavior. It should be noted that GAM0 is not the only memory model that violates per-location SC; RMO can also reorder two consecutive loads for the same address.

Strengthen GAM0 for Per-Location SC

To meet the programmers' requirement of per-location SC, we introduce the following *SALdLd* constraint.

- **Constraint SALdLd** (same-address-load-load): For any pair of loads for the same address and in the same processor, if there is no intervening store for the same address between them in the same processor, then the execution order of these two loads should match their commit order.

After introducing the above constraint to GAM0, the new memory model will forbid the non-SC behavior in Figure 3-9a, and we refer to the new memory model as *GAM*. Note that in constraint SALdLd, we do not order two loads with the same address in case there is a store also for the same address between them. This is because the younger load can get forwarding from the intervening store before the older load even starts execution, and this will not violate per-location SC. To better illustrate this point, consider the program in Figure 3-9b. I_4 and I_6 are both loads for address b, but there is a also a store I_5 for b between them. If we force I_6 to be after I_4 in the execution order and global memory order, then I_7 will also be ordered after I_4, forbidding I_7 from getting value 0. However, OOO^{U} can have I_6 bypass from I_5 and then execute I_7 by reading 0 from memory before any store in P1 has been issued. Note that all memory accesses to b can still be put into a sequential order ($I_3 \rightarrow I_4 \rightarrow I_5 \rightarrow I_6$) which is consistent with the commit orders of P1 and P2.

To implement constraint SALdLd correctly, when a load resolves its address, the processor should kill younger loads for the same address which have been issued to memory or have got data forwarded from a store older than the load. And when a load attempts to start execution, it needs to search not only older stores for the same address for forwarding but also older loads for the same address which have not started execution. In case it finds an older load before any store, it needs to be stalled until the older load has started execution. It should be noted that constraint SALdLd is a restriction on implementations purely for the purpose of matching programmers' needs. In theory, the squashes caused by this load-load ordering constraint should

affect single-thread performance. However, in Section 3.3, we will show via simulation that such squashes are very rare and the influence on performance is actually negligible.

Proc. P1	Proc. P2
I_1 : St $[a]$ 1	$I_2 : r_1 =$ Ld $[a]$ $I_3 : r_2 =$ Ld $[a]$
Per-location SC forbids, but GAM0 and RMO allow $r_1 = 1$, $r_2 = 0$	

(a) CoRR

Proc. P1	Proc. P2
I_1 : St $[a]$ 1 I_2 : FenceSS I_3 : St $[b]$ 1	$I_4 : r_1 =$ Ld $[b]$ I_5 : St $[b]$ 2 $I_6 : r_2 =$ Ld $[b]$ $I_7 : r_3 =$ Ld $[a + r_2 - r_2]$
Both per-location SC and GAM allow $r_1 = 1$, $r_2 = 2$, $r_3 = 0$	

(b) Loads with an intervening store

Proc. P1	Proc. P2
I_1 : St $[a]$ 1 I_2 : FenceSS I_3 : St $[b]$ 1	$I_4 : r_1 =$ Ld $[b]$ $I_5 : r_2 = c + r_1 - r_1$ $I_6 : r_3 =$ Ld $[r_2]$ $I_7 : r_4 =$ Ld $[c]$ $I_8 : r_5 = a + r_4 - r_4$ $I_9 : r_6 =$ Ld $[r_5]$
ARM allows but GAM forbids $r_1 = 1$, $r_2 = c$, $r_3 = 0$, $r_4 = 0$, $r_5 = a$, $r_6 = 0$	

(c) RSW

Proc. P1	Proc. P2
I_1 : St $[a]$ 1 I_2 : FenceSS I_{10} : St $[c]$ 0 I_{11} : FenceSS I_3 : St $[b]$ 1	$I_4 : r_1 =$ Ld $[b]$ $I_5 : r_2 = c + r_1 - r_1$ $I_6 : r_3 =$ Ld $[r_2]$ $I_7 : r_4 =$ Ld $[c]$ $I_8 : r_5 = a + r_4 - r_4$ $I_9 : r_6 =$ Ld $[r_5]$
Both ARM and GAM forbid $r_1 = 1$, $r_2 = c$, $r_3 = 0$, $r_4 = 0$, $r_5 = a$, $r_6 = 0$	

(d) RNSW

Figure 3-9: Litmus tests for same-address loads

Alternative Solution by ARM

The ARM memory model uses a different constraint (shown below), which we refer to as $SALdLd_{ARM}$, to enforce the ordering of same-address loads and achieve per-location SC.

- **Constraint SALdLd$_{\mathbf{ARM}}$**: The execution order of two loads for the same address (in the same processor) that do not read from the same *store* (not just same value) must match the commit order.

Constraint SALdLd$_{\text{ARM}}$ is strictly weaker than constraint SALdLd. To exploit the relaxation, the processor should not kill younger loads when a load resolves its address. Instead, when a load gets its value from the memory system, the processor kills

all younger loads whose values have been overwritten by other processors. Such younger loads can be identified by keeping track of evictions from L1. The above implementation should have less ROB squashes than the implementation of GAM with constraint SALdLd. However, we already mentioned that the squashes in GAM are very rare, so the relaxation in constraint $\text{SALdLd}_{\text{ARM}}$ will not lead to extra performance. We will confirm this point in Section 3.3.

Besides little gain in performance, constraint $\text{SALdLd}_{\text{ARM}}$ actually gives rise to confusing program behaviors. Consider the RSW (read-same-write) litmus test in Figure 3-9c and the RNSW (read-not-same-write) litmus test in Figure 3-9d. These two tests are very similar. In both tests, P1 first stores to a (I_1) and then stores to b (I_3); P2 first loads from b (I_4) and finally loads from a (I_9); memory location c always has value 0. The only difference between them is that in RNSW (Figure 3-9d), P1 performs an extra store I_{10} which writes the initial memory value 0 again into address c. We focus on the following non-SC behavior: P2 first gets the up-to-date value 1 from b (I_4) but finally gets the stale value 0 from a (I_9). Given the similarity between these two tests, one may expect that a memory model should either allow the non-SC behavior in both tests or forbid the behavior in both tests.

GAM indeed forbids this non-SC behavior in both tests, because I_4 and I_6 are data-dependent loads, I_6 and I_7 are consecutive loads for the same address c, and I_7 and I_9 are again data-dependent loads. As a result, in P2, the last load must be after the first load in the global memory order in GAM, forbidding I_9 from getting value 0.

In contrast, ARM allows the non-SC behavior in RSW but forbids it in RNSW. In RSW (Figure 3-9c), I_6 and I_7 both reads the initial memory value and are not ordered by constraint $\text{SALdLd}_{\text{ARM}}$, so the behavior is allowed by ARM. However, in RNSW (Figure 3-9d), if I_6 and I_7 are still executed out of order to produce the non-SC behavior, then I_7 first reads the initial memory value and I_6 later reads the value of I_{10}. Although the values read by I_6 and I_7 are equal, the values are supplied by different stores (initialization store and I_{10}), violating constraint $\text{SALdLd}_{\text{ARM}}$. Therefore, ARM forbids the non-SC behavior in RNSW. We can also verify that

per-location SC forbids that I_7 reads the initial memory value and I_6 reads from I_{10} simultaneously, because I_{10} must be ordered after the initialization of c if all memory accesses for c are put into a sequential order.

We believe it is confusing for constraint $\text{SALdLd}_{\text{ARM}}$ to allow RSW while forbidding RNSW, especially when the difference between the tests is so small. Therefore, we resort to the much simpler SALdLd constraint in GAM which forbids both behaviors without losing any performance in practice.

3.2 Formal Definitions of GAM

In this section, we give the axiomatic and operational definitions of GAM in a formal manner. Since the axioms of GAM are similar to the constraints derived in the previous section, we give the axiomatic definition first.

3.2.1 Axiomatic Definition of GAM

As introduced in Section 2.1, the axiomatic definition is a set of axioms that check if a combination of program order ($<_{po}$), global memory order ($<_{mo}$) and read-from relation ($\xrightarrow{rf}$) is legal or not. Program order and global memory order correspond to the commit order and the global memory order in Section 3.1, respectively. The core of the axiomatic definition of GAM is to define a *preserved program order* ($<_{ppo}^{gam}$). $<_{ppo}^{gam}$ relates two instructions in the same processor when their execution order must match the commit order. That is, $<_{ppo}^{gam}$ is a summary of constraints SAMemSt, SAStLd, SALdLd, RegRAW, BrSt, AddrSt and FenceOrd. After defining $<_{ppo}^{gam}$, we will give the two axioms of GAM, which reflect constraints LMOrd and LdVal, respectively.

Before defining $<_{ppo}^{gam}$, we define the RAW dependencies via registers as follows (all definitions ignore the PC register):

Definition 1 (RS: Read Set)**.** *$RS(I)$ is the set of registers an instruction I reads.*

Definition 2 (WS: Write Set)**.** *$WS(I)$ is the set of registers an instruction I can write.*

Definition 3 (ARS: Address Read Set)**.** *$ARS(I)$ is the set of registers a memory instruction I reads to compute the address of the memory operation.*

Definition 4 (data dependency $<_{ddep}$)**.** *$I_1 <_{ddep} I_2$ if $I_1 <_{po} I_2$ and $WS(I_1) \cap RS(I_2) \neq \emptyset$ and there exists a register r in $WS(I_1) \cap RS(I_2)$ such that there is no instruction I such that $I_1 <_{po} I <_{po} I_2$ and $r \in WS(I)$.*

Definition 5 (address dependency $<_{adep}$)**.** *$I_1 <_{adep} I_2$ if $I_1 <_{po} I_2$ and $WS(I_1) \cap ARS(I_2) \neq \emptyset$ and there exists a register r in $WS(I_1) \cap ARS(I_2)$ such that there is no instruction I such that $I_1 <_{po} I <_{po} I_2$ and $r \in WS(I)$.*

Data dependency, i.e., $I_1 <_{ddep} I_2$ in Definition 4, means that I_2 will use a result of I_1 as its source operand. Address-dependency, i.e., $I_1 <_{adep} I_2$ in Definition 5, means that I_2 will use a result of I_2 as the source operands to compute its load or store address. Thus, data dependency includes address dependency, i.e., $I_1 <_{adep} I_2 \implies I_1 <_{ddep} I_2$.

Now we define $<_{ppo}^{gam}$ as a summary of all the constraints for execution order:

Definition 6 (Preserved program order $<_{ppo}^{gam}$)**.** *Instructions $I_1 <_{ppo}^{gam} I_2$ if $I_1 <_{po} I_2$ and at least one of the following is true:*

1. *(Constraint SAMemSt) I_1 is a load or store, and I_2 is a store for the same address.*

2. *(Constraint SAStLd) I_2 is a load, and there exists a store S to the same address such that $I_1 <_{ddep} S <_{po} I_2$, and there is no other* store *for the same address between S and I_2 in $<_{po}$.*

3. *(Constraint SALdLd) both I_1 and I_2 are loads for the same address, and there is no store for the same address between them in $<_{po}$.*

4. *(Constraint RegRAW) $I_1 <_{ddep} I_2$.*

5. *(Constraint BrSt) I_1 is a branch and I_2 is a store.*

6. *(Constraint AddrSt)* I_2 *is a store, and there exists a memory instruction* I *such that* $I_1 <_{adep} I <_{po} I_2$.

7. *(Constraint FenceOrd part 1)* I_1 *is a fence* FenceXY *and* I_2 *is a memory instruction of type* Y.

8. *(Constraint FenceOrd part 2)* I_2 *is a fence* FenceXY *and* I_1 *is a memory instruction of type* X.

9. *(Transitivity) there exists an instruction* I *such that* $I_1 <_{ppo}^{gam} I$ *and* $I <_{ppo}^{gam} I_2$.

The last case in Definition 6 says that $<_{ppo}^{gam}$ is transitive.

With $<_{ppo}^{gam}$, we now give the two axioms of GAM in Figure 3-10. The $\text{LoadValue}_{\text{GAM}}$ axiom is just a formal way of stating constraint LdVal. The $\text{InstOrder}_{\text{GAM}}$ axiom interprets constraint LMOrd. That is, if two memory instructions $I_1 <_{ppo}^{gam} I_2$, then I_1 should be ordered before I_2 in the execution order, and thus they are also ordered in the global memory order, i.e., $I_1 <_{mo} I_2$.

Axiom $\text{InstOrder}_{\text{GAM}}$ (preserved instruction ordering):
$$I_1 <_{ppo}^{gam} I_2 \;\Rightarrow\; I_1 <_{mo} I_2$$
Axiom $\text{LoadValue}_{\text{GAM}}$ (the value of a load):
$$\mathsf{St}\ [a]\ v \xrightarrow{rf} \mathsf{Ld}\ [a] \;\Rightarrow\; \mathsf{St}\ [a]\ v = \max_{<mo}\{\mathsf{St}\ [a]\ v' \mid \mathsf{St}\ [a]\ v' <_{mo} \mathsf{Ld}\ [a] \;\vee\; \mathsf{St}\ [a]\ v' <_{po} \mathsf{Ld}\ [a]\}$$

Figure 3-10: Axioms of GAM

3.2.2 An Operational Definition of GAM

The operational definition of GAM describes an abstract machine, and how to operate the machine to run a program. Figure 3-11 shows the structure of the abstract machine.

The abstract machine contains a monolithic memory (same as the one in SC) connected to each processor. Each processor Pi contains a ROB and a PC register. The PC register contains the address of the next instruction to be fetched (speculatively) into the ROB. The ROB has one entry per instruction; each ROB entry contains the following information for the instruction I in it:

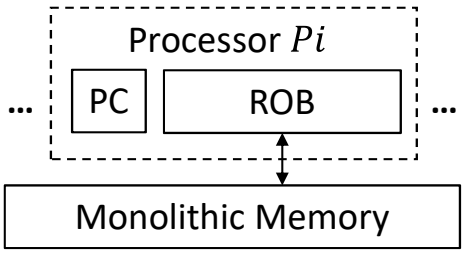


Figure 3-11: Abstract machine of GAM

- A *done* bit to denote if I is done or not-done (i.e., has finished execution or not).
- The execution result of I, e.g., load value or ALU result (valid only when the done bit is set).
- The *address-available* bit, which denotes whether the memory address has been computed in case I is a load or a store.
- The computed load or store address.
- The *data-available* bit, which denotes if the store data has been computed in case I is a store.
- The computed store data.
- The predicted branch target in case I is a branch.

An instruction in ROB can search through older entries to determine if its source operands are ready and to get the source operand values.

The abstract machine runs a program in a step-by-step manner. In each step, we can pick a processor and fire one of the rules listed in Figures 3-12 and 3-13. That is, no two processors can be active in the same step, and the active processor in this step can fire only one rule. Each rule consists of a *guard* condition and an *action*. The rule cannot be fired unless the guard condition is satisfied. When a processor fires a rule, it takes the action described in the rule. The choices of the processor and the rule are arbitrary, as long as the processor state can meet the guard condition of the rule.

- **Rule GAM-Fetch:** Fetch a new instruction.

  Guard: True.

  Action: Fetch a new instruction from the address stored in the PC register. Add the new instruction into the tail of ROB. If the new instruction is a branch, predict the branch target address of the branch, update PC to be the predicted address, and record the predicted address in the ROB entry of the branch; otherwise we just increment PC.

- **Rule GAM-Execute-Reg-to-Reg:** Execute a reg-to-reg instruction I.

  Guard: I is marked not-done and all source operands of I are ready.

  Action: Do the computation, record the result in the ROB entry, and mark I as done.

- **Rule GAM-Execute-Branch:** Execute a branch instruction I.

  Guard: I is marked not-done and all source operands of I are ready.

  Action: Compute the branch target address and mark I as done. If the computed target address is different from the previously predicted address (which is recorded in the ROB entry), then we kill all instructions which are younger than I in the ROB (excluding I). That is, we remove those instructions from the ROB, and update the PC register to the computed branch target address.

- **Rule GAM-Execute-Load:** Execute a load instruction I for address a.

  Guard: I is marked not-done, its address-available bit is set and all older FenceXL instructions are done.

  Action: Search the ROB from I towards the oldest instruction for the first not-done memory instruction with address a:

  1. If a not-done load to a is found then instruction I cannot be executed, i.e., we do nothing.
  2. If a not-done store to a is found then if the data for the store is ready, then execute I by bypassing the data from the store, and mark I as done; otherwise, I cannot be executed (i.e., we do nothing).
  3. If nothing is found then execute I by reading $m[a]$, and mark I as done.

  If we mark I as done, we record the load value as the execution result in the ROB entry of I.

Figure 3-12: Rules to operate the GAM abstract machine (part 1 of 2)

At a high level, these rules abstract the operation of processor implementations OOO^{U} and OOO^{MP}, and preserve the constraints in Section 3.1. The order of accessing monolithic memory is consistent with the global memory order in OOO^{MP}.

- **Rule GAM-Compute-Store-Data:** compute the data of a store instruction I.

  Guard: The data-available bit is not set and the source registers for the data computation are ready.

  Action: Compute the data of I and record it in the ROB entry; set the data-available bit of the entry.

- **Rule GAM-Execute-Store:** Execute a store I for address a.

  Guard: I is marked not-done and in addition all the following conditions must be true:

  1. The address-available bit of I is set,
  2. The data-available bit of I is set,
  3. All older branch instructions are done,
  4. All older loads and stores have their address-available bits set,
  5. All older loads and stores for address a are done,
  6. All older FenceXS instructions are done.

  Action: Update $m[a]$ and mark I as done.

- **Rule GAM-Compute-Mem-Addr:** Compute the address of a load or store instruction I.

  Guard: The address-available bit is not set and the address operand is ready with value a.

  Action: We first set the address-available bit and record the address a into the ROB entry of I. Then we search the ROB from I towards the youngest instruction (excluding I) for the first memory instruction with address a. If the instruction found is a done load, then we kill that load and all instructions that are younger than the load in the ROB, i.e., we remove the load and all younger instructions from ROB and set the PC register to the PC of the load. Otherwise no instruction needs to be killed.

- **Rule GAM-Execute-Fence:** Execute a FenceXY instruction I.

  Guard: I is marked not-done, and all older memory instructions of type X are done.

  Action: Mark I as done.

Figure 3-13: Rules to operate the GAM abstract machine (part 2 of 2)

Marking an instruction as done corresponds to finishing the execution of the instruction in OOO^{U}. Thus, the order of marking instructions as done in this abstract machine corresponds to the execution order in OOO^{U}. Instructions, especially loads, can be executed (i.e., marked as done) speculatively; in case this eager execution

turns out to violate the constraints later on, the rules will detect the violation and squash the ROB. Next we explain each rule.

Rule GAM-Fetch corresponds to the speculative instruction fetch in OOO^{U}. Rule GAM-Execute-Reg-to-Reg and GAM-Execute-Branch corresponds to finishing the execution of a reg-to-reg or branch instruction in OOO^{U}; the guard conditions that source operands should be ready preserves constraint RegRAW. The guard of Rule Execute-Fence preserves constraint FenceOrd. In rule GAM-Execute-Load, the guard that checks older fences preserves constraint FenceOrd; doing nothing in case of finding a not-done load the ROB search preserves constraint SALdLd; doing nothing in case of finding a not-done store without store data preserves constraint SAStLd. Notice that a load can be issued without waiting for all older memory instructions to resolve their addresses; this corresponds to the speculative execution in OOO^{U}. In the guard of rule GAM-Execute-Store, case 3 preserves constraint BrSt, case 4 preserves constraint AddrSt, case 5 preserves constraint SAMemSt, and case 6 preserves cosntraint FenceOrd. In rule GAM-Compute-Mem-Addr, in case a store address is computed and a younger load is killed in the ROB search, constraints LdVal and SAStLd are preserved; in case a load address is computed and a younger load is killed, constraint SALdLd is preserved.

3.2.3 Proof of the Equivalence of the Axiomatic and Operational Definitions of GAM

We have proved the equivalence of the axiomatic and operational definitions of GAM, i.e., Theorems 1 and 2. Below we give the sketch of the proofs and the details can be found in [149]. (The proofs can be skipped without affecting the understanding of the rest of the thesis.)

Theorem 1 (Soundness)**.** *GAM operational model* $\subseteq$ *GAM axiomatic model.*

Proof. The goal is to show that for any execution of the operational model, we can construct $\langle <_{po}, <_{mo}, \xrightarrow{rf} \rangle$ which satisfies the GAM axioms and has the same program behavior as the operational execution. To do this, we need to introduce some ghost

states to the operational model, and show invariants that hold after every step in the operational model.

In the operational model, we assume there is a (ghost) global time which is incremented whenever a rule fires. We also assume each instruction I in an ROB has the following ghost states which are accessed only in the proofs (all states start as $\top$):

- I.doneTS: Records the current global time when a rule R fires and marks I as done.

- I.addrTS: Records the current global time for memory instruction I when a GAM-Compute-Mem-Addr rule R fires to compute the address of I.

- I.sdataTS: Records the current global time for a store instruction I, when a GAM-Compute-Store-Data rule R fires to compute the store data of I.

- I.from: Records the store read by I if I is a load. That is, the store is either the not-done store I bypasses from or the done store with the maximum doneTS among all done stores for a when I is marked as done.

For convenience, we use I.ldval to denote the load value if I is a load, use I.addr to denote the memory access address if I is a memory instruction, and use I.sdata to denote the store data if I is a store. These fields are $\top$ if the corresponding values are not available.

Eventually we will use the states at the end of the operational execution to construct the axiomatic edges. $<_{po}$ will be constructed by the order of instructions in ROB, $\xrightarrow{rf}$ will be constructed by the from states of loads, and $<_{mo}$ will be constructed by the order of doneTS timestamps of all memory instructions.

Given the model state at any time in the execution of the operational model, we can define the program order $<_{po\text{-}rob}$, data-dependency order $<_{ddep\text{-}rob}$, address-dependency order $<_{adep\text{-}rob}$, and a new relation $<_{ntppo\text{-}rob}$ (non-transitive preserved program order) which is similar to the preserved program order (we add suffix *rob* to distinguish from the definitions in the axiomatic model):

- $<_{po\text{-}rob}$: Instructions $I_1 <_{po\text{-}rob} I_2$ iff both I_1 and I_2 are in the same ROB and I_1 is older than I_2 in the ROB.

- $<_{ddep\text{-}rob}$: $I_1 <_{ddep\text{-}rob} I_2$ iff $I_1 <_{po\text{-}rob} I_2$ and I_2 needs the result of I_1 as a source operand.

- $<_{adep\text{-}rob}$: $I_1 <_{adep\text{-}rob} I_2$ iff $I_1 <_{po\text{-}rob} I_2$, and I_2 is a memory instruction, and I_2 needs the result of I_1 as a source operand to compute the memory address to access.

- $<_{ntppo\text{-}rob}$: $I_1 <_{ntppo\text{-}rob} I_2$ iff $I_1 <_{po\text{-}rob} I_2$ and at least one of the following conditions hold:

  1. $I_1 <_{ddep\text{-}rob} I_2$.
  2. I_1 is a branch, and I_2 is a store.
  3. I_2 is a store, and there exists a memory instruction I such that $I_1 <_{adep\text{-}rob} I <_{po\text{-}rob} I_2$.
  4. I_2 is a load with $I_2.\mathsf{addr} = a \neq \top$, and there exists a store S with $S.\mathsf{addr} = a$, and $I_1 <_{ddep\text{-}rob} S <_{po\text{-}rob} I_2$, and there is no store S' such that $S'.\mathsf{addr} = a$ and $S <_{po\text{-}rob} S' <_{po\text{-}rob} I_2$.
  5. I_1 is a load with $I_1.\mathsf{addr} = a \neq \top$, and I_2 is a store with $I_2.\mathsf{addr} = a$.
  6. Both I_1 and I_2 are stores with $I_1.\mathsf{addr} = I_2.\mathsf{addr} = a \neq \top$.
  7. Both I_1 and I_2 are loads with $I_1.\mathsf{addr} = I_2.\mathsf{addr} = a \neq \top$, and there is no store S such that $S.\mathsf{addr} = a$ and $I_1 <_{po\text{-}rob} S <_{po\text{-}rob} I_2$.
  8. I_1 is a fence $\mathsf{FenceXY}$ and I_2 is a memory instruction of type Y, or I_2 is a fence $\mathsf{FenceXY}$ and I_1 is a memory instruction of type X.

It should be noted that the way to compute $<_{ntppo\text{-}rob}$ from $<_{po\text{-}rob}$ is almost the same as the way to compute $<_{ppo}^{gam}$ from $<_{po}$ except for two differences. The first difference is that $<_{ntppo\text{-}rob}$ is not made transitively closed; this is for simplifying the proof to some degree. The second difference is that in case the definition needs the address of

memory instructions, $<_{ntppo\text{-}rob}$ ignores memory instructions which have not computed their addresses. Since the address of every memory instruction will be computed at the end of the operational execution, the second difference will diminish by that time. Since $<_{po}$ is defined by the $<_{po\text{-}rob}$ at the end of the operational execution, $<_{ppo}^{gam}$ will be the transitive closure of $<_{ntppo\text{-}rob}$ at the end of the operational execution.

With the above definitions, the following invariants during the execution of the operational model:

1. If $I_1 <_{ntppo\text{-}rob} I_2$ and $I_2.\mathsf{doneTS} \neq \top$, then $I_1.\mathsf{doneTS} \neq \top$ and $I_1.\mathsf{doneTS} < I_2.\mathsf{doneTS}$.

2. If $I_1 <_{adep\text{-}rob} I_2$ and $I_2.\mathsf{addrTS} \neq \top$, then $I_1.\mathsf{doneTS} \neq \top$ and $I_1.\mathsf{doneTS} < I_2.\mathsf{addrTS}$.

3. If $I_1 <_{ddep\text{-}rob} I_2$, and not $I_1 <_{adep\text{-}rob} I_2$, and I_2 is a store, and $I_2.\mathsf{sdataTS} \neq \top$, then $I_1.\mathsf{doneTS} \neq \top$ and $I_1.\mathsf{doneTS} < I_2.\mathsf{sdataTS}$.

4. If $I_1 <_{po\text{-}rob} I_2$, and I_1 is a memory instruction, and I_2 is a store, and $I_2.\mathsf{doneTS} \neq \top$, then $I_1.\mathsf{addrTS} \neq \top$ and $I_1.\mathsf{addrTS} < I_2.\mathsf{doneTS}$.

5. We never kill a done store.

6. For any address a, let S be the store with the maximum doneTS among all the done stores for address a. The monolithic memory value for a is equal to $S.\mathsf{sdata}$.

7. For any done load L, let $S = L.\mathsf{from}$ (i.e., S is the store read by L). All of the following properties are satisfied:

   (a) S still exists in an ROB (i.e., S is not killed).

   (b) $S.\mathsf{addr} = L.\mathsf{addr}$ and $S.\mathsf{sdata} = L.\mathsf{ldval}$.

   (c) If S is done, then there is no not-done store S' such that $S'.addr = a$ and $S' <_{po\text{-}rob} L$.

(d) If S is done, then for any other done store S' with $S'.\mathsf{addr} = L.\mathsf{addr}$, if $S' <_{po\text{-}rob} L$ or $S'.\mathsf{doneTS} < L.\mathsf{doneTS}$, then $S'.\mathsf{doneTS} < S.\mathsf{doneTS}$.

(e) If S is not done, then $S <_{po\text{-}rob} L$, and there is no store S' such that $S'.\mathsf{addr} = L.\mathsf{addr}$ and $S <_{po\text{-}rob} S' <_{po\text{-}rob} L$.

Invariant 1 is a similar statement to the InstOrder axiom, and will become exactly the same as that axiom at the end of the operational execution. Invariants 2 and 3 captures the ordering effects of dependencies carried to the computation of memory address and store data. Invariant 4 captures part of the guard of the GAM-Execute-Store rule, and is also related to constraint AddrSt in the definition of $<_{ppo}^{gam}$. Invariant 5 is an important property saying that stores are never written to the shared memory speculatively, so the model does not need any system-wide rollback. Invariant 6 constrains the current monolithic memory value. Invariant 7 constrains the store read by a load, and in particular, invariant 7d will become the $\text{LoadValue}_{\text{GAM}}$ axiom at the end of the operation execution. The detailed proof for these invariants can be found in [149, Appendix A].

Now we can complete the proof by constructing $\langle <_{po}, <_{mo}, \xrightarrow{rf} \rangle$ using the ending state of the operational execution as follows:

- $<_{po}$ is constructed as the order of instructions in each ROB.
- $<_{mo}$ is constructed by the ordering of doneTS, i.e., for two memory instructions I_1 and I_2, $I_1 <_{mo} I_2$ iff $I_1.\mathsf{doneTS} < I_2.\mathsf{doneTS}$.
- $\xrightarrow{rf}$ is constructed by the from fields, i.e., for a load L and a store S, $S \xrightarrow{rf} L$ iff $S = L.\mathsf{from}$.

Invariant 7b ensures that the constructed $\xrightarrow{rf}$ and $<_{po}$ are consistent with each other (e.g., it rules out the case that $\xrightarrow{rf}$ says a load should read a store with value 1, but $<_{po}$ says the load has value 2).

Since all instructions are done at the end of execution, then invariant 7d becomes the $\text{LoadValue}_{\text{GAM}}$ axiom. Therefore, the constructed $\langle <_{po}, <_{mo}, \xrightarrow{rf} \rangle$ satisfy the $\text{LoadValue}_{\text{GAM}}$ axiom.

At the end of execution, invariant 1 becomes: if $I_1 <_{ntppo\text{-}rob} I_2$, then $I_1.\mathsf{doneTS} < I_2.\mathsf{doneTS}$. Note that the $<_{ppo}^{gam}$ computed from $<_{po}$ is actually the transitive closure of $<_{ntppo\text{-}rob}$. Since instructions are totally ordered by doneTS fields, we have if $I_1 <_{ppo}^{gam} I_2$, then $I_1.\mathsf{doneTS} < I_2.\mathsf{doneTS}$. Since $<_{mo}$ is defined by the order of doneTS fields, the $\text{InstOrder}_{\text{GAM}}$ axiom is also satisfied.

□

Theorem 2 (Completeness)**.** *GAM axiomatic model* $\subseteq$ *GAM operational model.*

Proof. The goal is that for any legal axiomatic relations $\langle <_{po}, <_{mo}, \xrightarrow{rf} \rangle$ (which satisfy the GAM axioms), we can run the operational model to give the same program behavior. The strategy to run the operational model consists of two major phases. In the first phase, we only fire GAM-Fetch rules to fetch all instructions into all ROBs according to $<_{po}$. During the second phase, in each step we fire a rule that either marks an instruction as done or computes the address or data of a memory instruction. Which rule to fire in a step depends on the current state of the operational model and $<_{mo}$. Here we give the detailed algorithm that determines which rule to fire in each step:

1. If in the operational model there is a not-done reg-to-reg or branch instruction whose source registers are all ready, then we fire an GAM-Execute-Reg-to-Reg or GAM-Execute-Branch rule to execute that instruction.

2. If the above case does not apply, and in the operational model there is a memory instruction, whose address is not computed but the source registers for the address computation are all ready, then we fire a GAM-Compute-Mem-Addr rule to compute the address of that instruction.

3. If neither of the above cases applies, and in the operational model there is a store instruction, whose store data is not computed but the source registers for the data computation are all ready, then we fire a GAM-Compute-Store-Data rule to compute the store data of that instruction.

4. If none of the above cases applies, and in the operational model there is a fence instruction and the guard of the GAM-Execute-Fence rule for this fence is ready, then we fire the GAM-Execute-Fence rule to execute that fence.

5. If none of the above cases applies, then we find the oldest instruction in $<_{mo}$, which is not-done in the operational model, and we fire an GAM-Execute-Load or GAM-Execute-Store rule to execute that instruction.

The above algorithm essentially prioritize the firing of local-computation rules in each processor. If there is no local computation to be done in any processors, then the algorithm choose the oldest not-done memory instruction in $<_{mo}$ to execute. That is, $<_{mo}$ gives the order of executing load and store instructions.

Before giving the invariants, we give a definition related to the ordering of stores for the same address. For each address a, all stores for a are totally ordered by $<_{mo}$, and we refer to this total order of stores for a as $<^{a}_{co}$.

Now we show the invariants. After each step, we maintain the following invariants:

1. The order of instructions in each ROB in the operational model is the same as the $<_{po}$ of that processor in the axiomatic relations.

2. The results of all the instructions that have been marked as done so far in the operational model are the same as those in the axiomatic relations.

3. All the load/store addresses that have been computed so far in the operational model are the same as those in the axiomatic relations.

4. All the store data that have been computed so far in the operational model are the same as those in the axiomatic relations.

5. No kill has ever happened in the operational model.

6. For the rule fired in each step that we have performed so far, the guard of the rule is satisfied the at that step (i.e., the rule can fire).

7. In each step that we have performed so far, if we fire a rule to execute an instruction (especially a load) in that step, the instruction must be marked as done by the rule.

8. For each address a, the order of all the store updates on monolithic memory address a that have happened so far in the operational model is a prefix of $<_{co}^{a}$.

The detailed proof of the invariants can be found in [149, Appendix B].

□

3.3 Performance Evaluation

We evaluate the performance impact caused by enforcing same-address load-load ordering and disallowing load-load forwarding in GAM, and show that the influence on performance is in fact negligible.

3.3.1 Methodology

As mentioned in Sections 3.1.5, the same-address load-load ordering constraint (SALdLd) places extra restrictions on uniprocessor implementations to cater to the needs of programmers. Disallowing load-load forwarding is also mainly affecting single-thread performance. Therefore, we study the performance of a *single* processor of the following four memory models using the SPEC CPU2006 benchmarks:

- GAM: OOO^{U} with constraint SALdLd.
- ARM: OOO^{U} with constraint $\text{SALdLd}_{\text{ARM}}$.
- GAM0: OOO^{U} (i.e., no constraint on same-address loads).
- Alpha*: OOO^{U} with load-load data forwarding.

The comparison of GAM against ARM and GAM0 will show the performance impact of same-address load-load ordering constraint SALdLd, and the comparison of

GAM against Alpha* will illustrate the performance implications of disallowing load-load forwarding to enforce data-dependency ordering. Here we do not evaluate value prediction. The self-invalidation coherence protocol is evaluated in Chapter 6.

Besides, GAM0 can be viewed as a corrected version of RMO [142] (they both allow the reordering of same-address loads). Alpha* is similar to Alpha [16] in allowing load-load forwardings; it is more liberal than Alpha in that it does not enforce any same-address load-load ordering; but it does not account for delayed invalidations. Thus, the comparison of GAM versus ARM, GAM0 and Alpha* will be an estimate of the performance comparison of GAM versus existing memory models including ARM, RMO and Alpha.

We modeled these four processors in GEM5 [37]. The implementation details have been described in Section 3.1. For ARM, we ignore the kills when loads read values from the memory system, so the performance of ARM is an optimistic estimation. Note that when a load is ready to issue in the ARM processor, it still searches older loads for stalls. Table 3.1 shows the detailed parameters[2]; the sizes of the important buffers (ROB, load buffer and store buffer) are chosen to match a Haswell processor.

Single core @2.5GHz with x86 ISA (modified O3 CPU model)	
Width	4-way fetch/decode/rename/commit, 6-way issue to execution, 6-way write-back to register file
Function units	4 Int ALUs, 1 Int multiply, 1 Int Divide, 2 FP ALUs, 1 FP multiply, 1 FP divide and sqrt, 2 load/store units
Buffers	192-entry ROB, 72-entry load buffer, 42-entry store buffer (holding both speculative and committed stores)
Classic memory system with 64B cache lines	
L1 inst	32KB, 8-way, 4-cycle hit latency, 4 MSHRs
L1 data	32KB, 8-way, 4-cycle hit latency, 8 MSHRs
Unified L2	256KB, 8-way, 12-cycle hit latency, 20 MSHRs
L3	1MB, 16-way, 35-cycle hit latency, 30 MSHRs
Memory	80ns (200-cycle) latency and 12.8GB/s bandwidth

Table 3.1: Processor parameters

[2]As explained in Section 1.3, in GEM5, a load instruction occupies an instruction-issue-queue entry until it gets its value. This occupation time is much longer than that in normal implementations which release the instruction-issue-queue entry when the source registers are ready. Since our focus is not on the instruction issue queue, we simulate an unlimited instruction issue queue.

We run all reference inputs of all SPEC CPU benchmarks (55 inputs in total) in full-system mode. For each input, we simulate from 10 uniformly distributed checkpoints. For each checkpoint, we first warm up the memory system for 25M instructions, then warm up the processor pipeline for 200K instructions, and finally simulate 100M instructions in detail. For each benchmark, we summarize the statistics of all the input checkpoints to produce the final performance numbers.

Since GEM5 cracks an instruction into micro-ops (uOPs), we will use uOP counts instead of instruction counts in the rest of this section, and performance is characterized by uOPs per cycle, i.e., uPC.

3.3.2 Results and Analysis

Figure 3-14 shows the percentage of performance improvement (in terms of uPC) of ARM, GAM0 and Alpha* over GAM for each benchmark. The last column in the figure is the average across all benchmarks. The performance improvements of ARM, GAM0 and Alpha* over GAM are all negligible (0.17%, 0.17%, and 0.3% on average, respectively) and never exceed 3%. This shows that the performance penalty for GAM to enforce the same-address load-load ordering and data-dependency ordering is very small. Next we analyze the influence of these two orderings in more details.

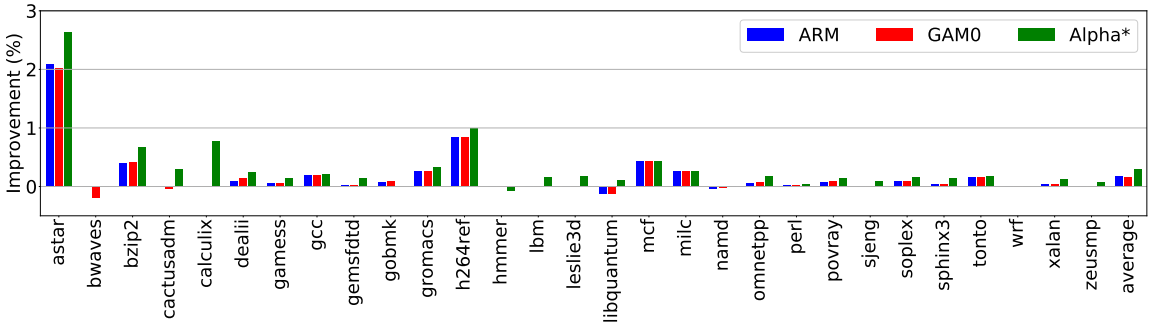


Figure 3-14: Relative Performance (uPC) improvement (in percentage) of ARM, GAM0, and Alpha* over GAM

Same-address load-load ordering: Constraint SALdLd in GAM puts the following two restrictions on implementations:

1. **Kills**: when a load L computes its address, the processor kills any younger load

which has finished execution but has not got its value from a store younger than L.

2. **Stalls**: when a load L is ready to issue to start execution, if there is an older unissued load for the same address and L cannot get forwarding from any store younger than the unissued load, then L will be stalled.

In contrast, ARM will not have any kills, but it is still subject to the stalls; GAM0 is not affected by the kills or the stalls. Figure 3-15 shows the number of kills (caused by same-address load-load ordering) per thousand uOPs in GAM. The average number of kills per thousand uOPs in GAM is 0.2, and the maximum is 2.5. That is, kills caused by same-address load-load ordering are extremely rare.

Figure 3-16 shows the number of stalls (caused by same-address load-load ordering) per thousand uOPs in GAM and ARM. The number of stalls in GAM and ARM are similar. The average number of stalls per thousand uOPs is 0.9, and the maximum is 8. Since the penalty of stalls is much less than that of kills, these small numbers of stalls will not make GAM (and ARM) slower than GAM0.

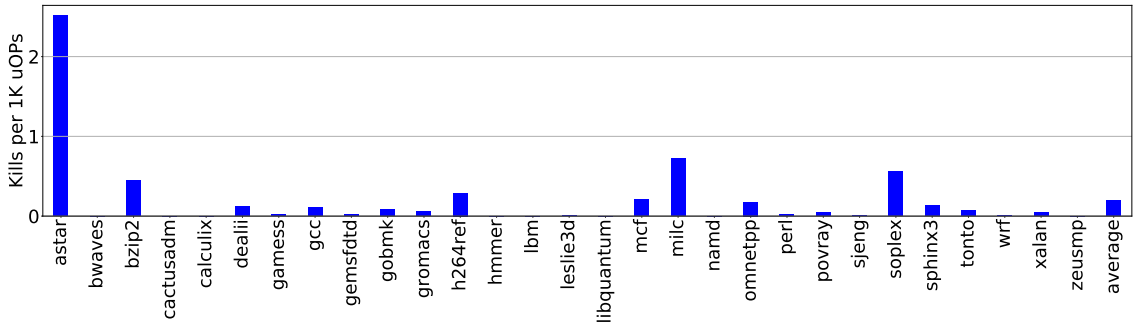


Figure 3-15: Number of kills caused by same-address load-load orderings per thousand uOPs in GAM

Load-Load forwarding: In case data-dependency ordering is not enforced, the processor (i.e., Alpha*) can forward data from an older executed load to a younger unexecuted load. However, this forwarding is beneficial only in case that the younger load would get a cache miss if it were issued to the memory system. Figure 3-17 shows the number of load-to-load forwardings per thousand uOPs in Alpha*, and Figure 3-18 shows the reduction of Alpha* over GAM in the number of L1 load misses per

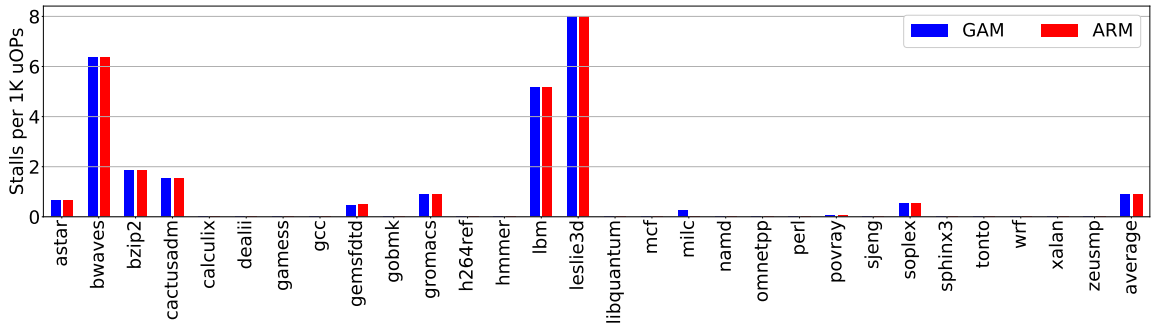


Figure 3-16: Number of stalls caused by same-address load-load orderings per thousand uOPs in GAM and ARM

thousand uOPs. As we can see, load-load forwardings can happen quite frequently: the average number of forwardings is 26 and the maximum is 103. However, the number of L1 load misses is not reduced significantly: the average reduction is 0.06 misses per thousand uOPs and the maximum reduction is 2.8. That is, the load that gets the forwarding from the older load can also read the data from the L1 cache. This explains why load-load forwardings do not translate to performance improvement over GAM.

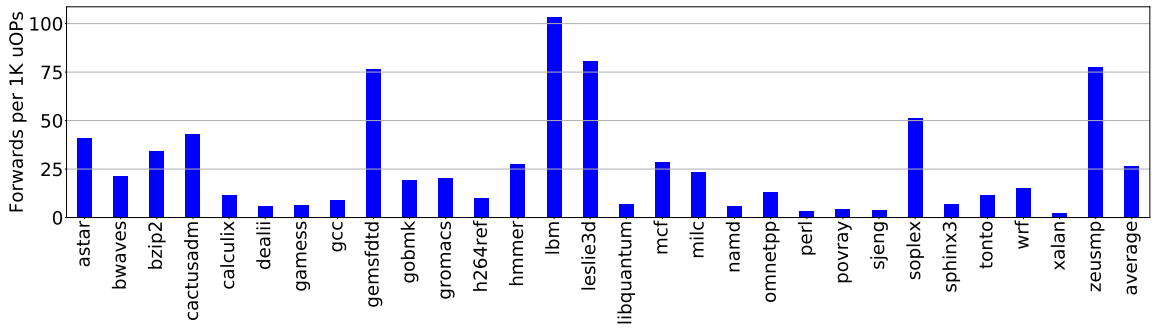


Figure 3-17: Number of load-to-load forwardings per thousand uOPs in Alpha*

3.4 Summary

We have constructed a common base model, GAM, for atomic memory models. GAM preserves all uniprocessor optimizations except those breaking the common assumptions in multithreaded programs. The construction of GAM starts from the constraints on execution orders in uniprocessors, then extends the constraints to a multi-

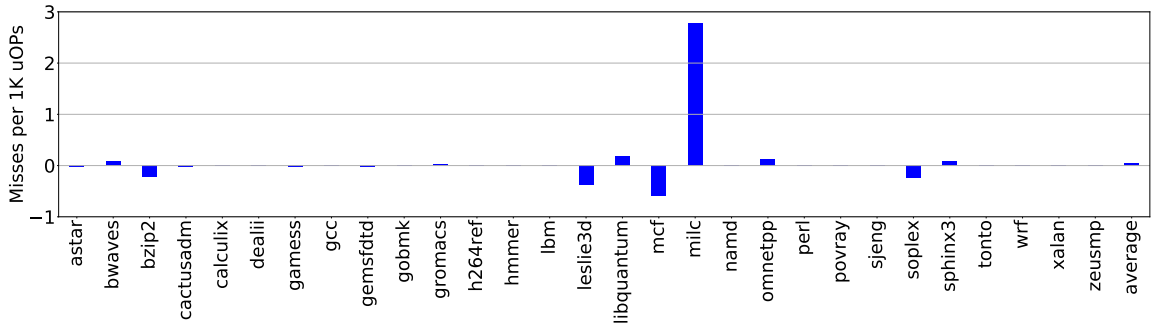


Figure 3-18: Reduced number of L1 load misses per thousand uOPs for Alpha* over GAM

processor setting, and finally introduces additional constraints necessary for parallel programming. This construction procedure makes GAM a memory model that preserves most uniprocessor optimizations. It also explains why each ordering constraint is introduced, and which uniprocessor optimizations are sacrificed for programming purposes. Other weak memory models differ from GAM in terms of same-address load-load ordering and data-dependency ordering. Our evaluation shows that these differences, especially same-address load-load ordering, have little impact on performance. Therefore, GAM has chosen to enforce these orderings to match the common assumptions in multithreaded programs. It should be noted that the definition of GAM is just for one specific choice and is not parameterized by different choices.

Chapter 4

WMM: a New Weak Memory Model with a Simpler Definition

The definition of GAM introduced in Chapter 3 is still quite complicated, and we identify the source of the complexity to be allowing load-store reordering (Section 4.1). Based on this insight, we define a new memory model, *WMM*, which has a much simpler definition, by forbidding load-store reordering completely (Section 4.2). We compare WMM against GAM in Section 4.3, and describe how WMM can be implemented using a conventional out-of-order processor in Section 4.4. In Section 4.5, we evaluate the performance of WMM and show that forbidding load-store reordering has little cost in performance.

4.1 Defintional Complexity of GAM

4.1.1 Complexity in the Operational Definition of GAM

The abstract machine of GAM (Figures 3-12 and 3-13) is still quite complicated. It contains an ROB for each processor to buffer multiple in-flight instructions, and executes instructions partially, e.g., a load needs to compute its address and read memory in two different rules. In contrast, the abstract machine of SC is much simpler, because it considers only the next instruction in each processor and executes

an instruction atomically in each rule.

To understand the reason for the definitional complexity of the abstract machine of GAM, consider the program in Figure 4-1. GAM allows load I_2 to end up with value 1, which comes a future store I_3 in the same processor but to a different address. This is as if load I_2 and store I_3 were reordered in P1. GAM allows load-store reordering, and the abstract machine of GAM can achieve this behavior in the following way by (1) making I_1 read 0 from monolithic memory, (2) computing the address of I_2, (3) writing store I_3 to monolithic memory, (4) making I_4 and I_5 access monolithic memory sequentially, and (5) making I_2 read 1 from monolithic memory.

Proc. P1	Proc. P2
$I_1 : r_1 = \mathsf{Ld}\ [c]$ $I_2 : r_2 = \mathsf{Ld}\ [a + r_1]$ $I_3 : \mathsf{St}\ [b]\ 1$	$I_4 : r_3 = \mathsf{Ld}\ [b]$ $I_5 : \mathsf{St}\ [a]\ r_3$
GAM allows $r_1 = 0, r_2 = r_3 = 1$	

Figure 4-1: Behavior caused by load-store reordering

This behavior is not possible in any abstract machine without buffering multiple instructions. Consider an abstract machine that looks at only the next instruction to execute in each processor. When the machine executes load I_2, store I_3 and its store value 1 are not yet in the system. Therefore, load I_2 cannot get value 1 when it is executed.

This behavior is also impossible in any abstract machine which cannot execute load I_2 partially. Consider the case that the abstract machine has to execute I_2 atomically, i.e., compute the load address and access memory in one single rule. In that case, store I_3 can never be sent to memory before I_2 is executed atomically; otherwise there will be risk that I_3 and I_2 access the same address and single-threaded correctness is violated.

As we can see, in order to allow a load to see the effect of a future store (i.e., allow load-store reordering), the abstract machine of GAM has to bear the complexity of buffering multiple instructions and partially executing instructions.

4.1.2 Complexity in the Axiomatic Definition of GAM

The core of the axiomatic definition of GAM is the definition of preserved program order ($<_{ppo}^{gam}$). the definition of $<_{ppo}^{gam}$ is quite complicated because it tracks various dependencies between instructions, e.g., data dependencies, address dependencies and control dependencies (Section 3.2.1). To understand why such complexity is needed in the definition of GAM, we can try to simplify the definition by getting rid of all the dependency related constraints. This results in the following definition of a different preserved program order, $<_{ppo}^{new}$:

Definition 7 ($<_{ppo}^{new}$)**.** *Instructions $I_1 <_{ppo}^{new} I_2$ if $I_1 <_{po} I_2$ and at least one of the following is true:*

1. *(Constraint SAMemSt) I_1 is a load or store, and I_2 is a store for the same address.*
2. *(Constraint SALdLd) both I_1 and I_2 are loads for the same address, and there is no store for the same address between them in $<_{po}$.*
3. *(Constraint FenceOrd part 1) I_1 is a fence* FenceXY *and I_2 is a memory instruction of type* Y*.*
4. *(Constraint FenceOrd part 2) I_2 is a fence* FenceXY *and I_1 is a memory instruction of type* X*.*
5. *(Transitivity) there exists an instruction I such that $I_1 <_{ppo}^{new} I$ and $I <_{ppo}^{new} I_2$.*

The above definition of $<_{ppo}^{new}$ keeps constraint SAMemSt to ensure single-threaded correctness, keeps constraint SALdLd for per-location SC, and keeps constraint FenceOrd for fence instructions. This new definition is self-contained and much simpler than the original definition of $<_{ppo}^{gam}$ which involves six definitions (Definitions 1 to 6). However, when we combine $<_{ppo}^{new}$ with the two axioms of GAM (Figure 3-10), the resulting memory model will allow the out-of-thin-air (OOTA) behavior shown in Figure 2-8. The OOTA behavior is as if the load and the store in the same processor were reordered even though the store data depends on the load result. OOTA behaviors must

be forbidden by the memory-model definition, because they can never be generated in any existing or reasonable hardware implementations and they make formal analysis of program semantics almost impossible. GAM allows load-store reorderings in general, but forbids the reordering of dependent load-store pairs by including dependencies in the definition of $<_{ppo}^{gam}$. That is, the complexity of tracking dependencies in GAM is needed to avoid OOTA problems while still allowing general load-store reorderings.

4.2 WMM Model

The analysis in Section 4.1 has revealed that the source of the complexity in the GAM defintions is allowing load-store reordering. Allowing load-store reordering forces the operational definition of GAM to model an ROB and partial instruction-execution, and forces the axiomatic definition of GAM to track various dependencies between instructions. Therefore, in order to construct a weak memory model with a simpler definition, we consider forbidding load-store reordering completely. This results in a new memory model, *WMM*, which allows store-load, store-store and load-load reorderings.

By giving up load-store reordering, the abstract machine of WMM no longer needs to model ROB-like structures or partial instruction execution. Instead, it can be defined based on *Instantaneous Instruction Execution (I^{2}E)*. An I^{2}E abstract machine can execute instructions in-order and instantaneously (Sections 4.2.1 and 4.2.2), and consequently every processor has up-to-date state after each instruction is executed. The I^{2}E property makes it much easier to understand operational behaviors allowed by a memory model. The SC abstract machine in Section 2.1.1 is an example of I^{2}E. It should be noted that *the I^{2}E abstract machine is purely for definitional purposes, and it does not preclude out-of-order implementations*. In particular, we will show in Section 4.3 how the I^{2}E abstract machine of WMM simulates the behavior of a variant of the GAM abstract machine which executes instructions out of order.

The axiomatic definition of WMM also becomes much simpler. It does not track

any dependencies between instructions while still avoiding OOTA problems (Section 4.2.3). We have also proved the equivalence between the operational definition and the axiomatic definition.

It should be noted that WMM has its own fence instructions (Sections 4.2.1 and 4.2.2), which are different from the FenceXY instructions of GAM. We will explain the differences in Section 4.3.

4.2.1 Operational Definitions with I^{2}E

Before getting into the details of WMM, we first explain the relation between load-store reordering and I^{2}E abstract machine. We can prove the following theorem:

Theorem 3 (Forbidding load-store reordering implies the I^{2}E model)**.** *Any processor implementation that prohibits load-store reordering can be modeled by an I^{2}E abstract machine.*

Proof. We consider an arbitrary processor implementation that prohibits load-store reordering. This implementation should consist of n processors connected to a shared memory system. Since the implementation prohibits Ld-St reordering, then the processor will not issue a store to the memory before all preceding loads (in that processor) have got their results. Therefore, any execution on this implementation will satisfy the following property: $\mathsf{acyclic}(<_{po} \cup \xrightarrow{rf})$, i.e., the union of program order $<_{po}$ and read-from relation $\xrightarrow{rf}$ cannot form any cycle.

For any execution on this implementation, we can simulate its behavior using an I^{2}E abstract machine which consists of n I^{2}E processors connected to a *magic memory*. The magic memory responds to each load L instantly. If the store read by L in the implementation execution has been issued to the magic memory, then the magic memory returns that store; otherwise the magic memory returns a random store value it has seen. We use the following algorithm to operate the I^{2}E abstract machine to simulate the implementation execution:

1. If there is a non-memory instruction (i.e., neither load nor store) on an I^{2}E processor, then execute that instruction.

2. Otherwise, if there is any store to execute on an I^{2}E processor, then execute that store by issuing it to the magic memory.

3. Otherwise, the next instruction to execute on every I^{2}E processor is a load. We pick a processor i, whose next instruction to execute is a load L and the store read by L in the implementation execution has been issued to the magic memory. We execute L, and the magic memory will return the store that L reads in the implementation execution.

In each step of the simulation algorithm, we maintain the following invariants:

1. The order of instructions executed in each I^{2}E processor matches the program order in the implementation execution.

2. The values of every instruction (e.g. load results, store data, load/store addresses, etc.) executed in each I^{2}E processor match those in the implementation execution.

3. The magic memory never returns a random store for any load it receives; the magic memory always returns the store read by the load in the implementation execution.

We can prove inductively that the simulation algorithm maintains the invariants in each step and never gets stuck. Most of the proof is straightforward. The ony part worth noticing is that in case 3 of the simulation algorithm, we can always find such a load L because of the $\mathsf{acyclic}(<_{po} \cup \xrightarrow{rf})$ property of the implementation execution. □

The dual of Theorem 3 is also true:

Theorem 4 (I^{2}E cannot model load-store reordering)**.** *If a memory model admits load-store reordering, it cannot be expressed as an I^{2}E operational definition, i.e., it cannot have a simple operational description.*

Proof. Consider the LB litmus test in Figure 2-4d. Any memory model that admits Ld-St reordering will allow this behavior by simply reordering the load and the store

on either processor. However, this behavior can never be allowed in any I^{2}E abstract machine, because neither store I_1 nor store I_3 can be executed before any of the loads is executed in an I^{2}E abstract machine. □

From Theorems 3 and 4, we can see that load-store reordering and I^{2}E operational definitions are incompatible, and that forbidding load-store reordering is the key to construct a weak memory model with a simple operational definition.

Examples of I^{2}E Operational Definitions

The abstract machine of SC in Section 2.1.1 is an I^{2}E model. As another example, we give the I^{2}E abstract machine of TSO.

Figure 4-2 shows the I^{2}E abstract machine of TSO proposed in [109, 127]. The abstract machine consist of n atomic processors and an n-ported monolithic memory m. Each processor contains a register state s, which represents all architectural registers, including both the general purpose registers and special purpose registers, such as PC. Each processor also contains a *store buffer sb*. In the abstract machines all buffers are unbounded. Since each processor in the I^{2}E model executes instructions instantaneously, the register state of the processor is always up-to-date. In particular, the next instruction to execute in a processor always refers to the instruction at the address stored in the PC register of that processor.

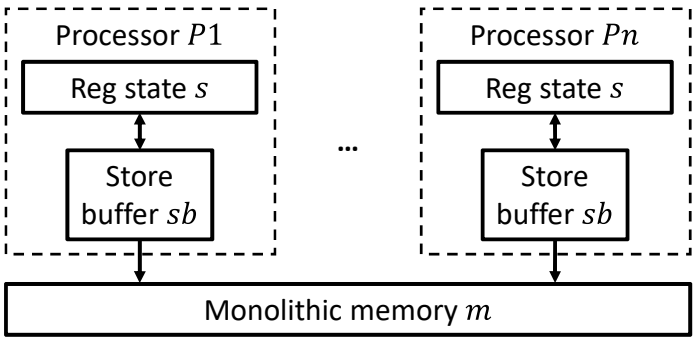


Figure 4-2: I^{2}E abstract machine of TSO

Just like SC, any processor can execute an instruction atomically, and if the instruction is a non-memory instruction, it just modifies the local register state. A store

is executed by inserting its ⟨address, value⟩ pair into the local *sb* instead of writing the data in memory. A load first looks for the load address in the local *sb* and returns the value of the *youngest* store for that address. If the address is not in the local *sb*, then the load returns the value from the monolithic memory. TSO can also perform a *background* operation, which removes the *oldest* store from a *sb* and writes it into the monolithic memory. Having a *sb* allows TSO to do store-load reordering, e.g., the model allows the non-SC behavior in the SB litmus test (Figure 2-4a).

In order to enforce ordering in accessing the memory and to rule out non-SC behaviors, TSO has a fence instruction, which we refer to as Commit. When a processor executes a Commit fence, it gets blocked unless its *sb* is empty. Eventually, any *sb* will become empty as a consequence of the background operations that move data from the *sb* to the memory. For example, we need to insert a Commit fence after each store in Figure 2-4a to forbid the non-SC behavior in TSO.

We summarize the rules to operate the TSO abstract machine in Figure 4-3. Similar to the rules of the GAM abstract machine, each rule consists of a *guard* and an *action*. The rule can be fired by taking the action only when the guard is true. Each time we fire only one rule (either instruction execution or *sb* dequeue) atomically in the whole system (e.g., no two processors can execute instructions simultaneously). The choice of which rule to fire is nondeterministic.

Enabling store-store reordering: We can extend TSO to PSO by changing the background rule to dequeue the oldest store for *any* address in *sb* (see the PSO-DeqSb operation in Figure 4-4). This extends TSO by permitting store-store reordering.

4.2.2 Operational Definition of WMM

WMM allows load-load reordering in addition to the reorderings allowed by PSO. Since a reordered load may read a stale value, we introduce a conceptual device called *invalidation buffer*, *ib*, for each processor in the I^2E abstract machine shown in Figure 4-5. *ib* is an unbounded buffer of ⟨address, value⟩ pairs, each representing a stale memory value for an address that can be observed by the processor. Multiple stale values for an address in *ib* are kept ordered by their staleness.

- **Rule TSO-Nm:** non-memory execution.

  Guard: The next instruction of a processor is a non-memory instruction.

  Action: Instruction is executed by local computation.

- **Rule TSO-Ld:** load execution.

  Guard: The next instruction of a processor is a load.

  Action: Assume the load address is a. The load returns the value of the *youngest* store for a in sb if a is present in the sb of the processor, otherwise, the load returns $m[a]$, i.e., the value of address a in the monolithic memory.

- **Rule TSO-St:** store execution.

  Guard: The next instruction of a processor is a store.

  Action: Assume the store address is a and the store value is v. The processor inserts the store $\langle a, v\rangle$ into its sb.

- **Rule TSO-Com:** Commit execution.

  Guard: The next instruction of a processor is a Commit and the sb of the processor is empty.

  Action: The Commit fence is executed simply as a NOP.

- **Rule TSO-DeqSb:** background store buffer dequeue.

  Guard: The sb of a processor is not empty.

  Action: Assume the $\langle$address, value$\rangle$ pair of the oldest store in the sb is $\langle a, v\rangle$. Then this store is removed from sb, and the monolithic memory $m[a]$ is updated to v.

Figure 4-3: Operations of the TSO abstract machine

- **Rule PSO-DeqSb:** background store buffer dequeue.

  Guard: The sb of a processor is not empty.

  Action: Assume the value of the oldest store for *some* address a in the sb is v. Then this store is removed from sb, and the monolithic memory $m[a]$ is updated to v.

Figure 4-4: PSO background rule

The rules of the WMM abstract machine are similar to those of PSO except for the background operation and the load execution. When the background rule moves a store from sb to the monolithic memory, the original value in the monolithic memory, i.e., the stale value, enters the ib of every other processor. A load first searches the

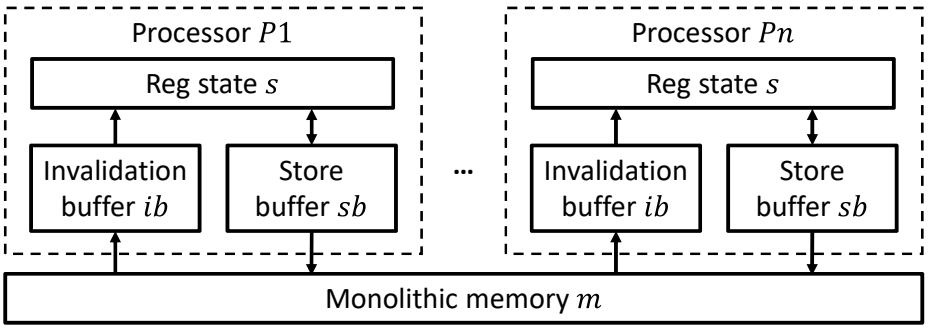


Figure 4-5: I^{2}E abstract machine of WMM

local sb. If the address is not found in sb, it either reads the value in the monolithic memory or any stale value for the address in the local ib, the choice between the two being nondeterministic.

The rules of the abstract machine maintain the following invariants: once a processor *observes* a store, it cannot observe any staler store for that address. Therefore, (1) when a store is executed, values for the store address in the local ib are purged; (2) when a load is executed, values staler than the load result are flushed from the local ib; and (3) the background operation does not insert the stale value into the ib of a processor if the sb of the processor contains the address.

Just like introducing the Commit fence in TSO, to prevent loads from reading the stale values in ib, we introduce the Reconcile fence to clear the local ib. Figure 4-6 summarizes the rules of the WMM abstract machine.

Properties of WMM

The I^{2}E abstract machine of WMM executes instructions instantaneously and in order, but because of store buffers (sb) and invalidation buffers (ib) in the abstract machine, it can model the effects of instruction reorderings. Similar to TSO/PSO, WMM allows store-load and store-store reorderings because of sb, e.g., WMM allows the behaviors in Figures 2-4a and 2-4b (FenceLL should be replaced by a Reconcile). To forbid the behavior in Figure 2-4a, we need to insert a Commit followed by a Reconcile after the store in each processor. Reconcile is needed to prevent loads from getting stale values from ib. The sequence of Commit; Reconcile acts as a full fence. To forbid

- **Rule WMM-Nm:** non-memory execution. Same as TSO-Nm.
- **Rule WMM-Ld:** load execution.

  Guard: The next instruction of a processor is a load.

  Action: Assume the load address is a. If a is present in the sb of the processor, then the load returns the value of the *youngest* store for a in the local sb. Otherwise, the load is executed in either of the following two ways (the choice is arbitrary):

  1. The load returns the monolithic memory value $m[a]$, and all values for a in the local ib are removed.
  2. The load returns some value for a in the local ib, and all values for a older than the load result are removed from the local ib. (If there are multiple values for a in ib, the choice of which one to read is arbitrary).
- **Rule WMM-St:** store execution.

  Guard: The next instruction of a processor is a store.

  Action: Assume the store address is a and the store value is v. The processor inserts the store $\langle a, v\rangle$ into its sb, and removes all values for a from its ib.
- **Rule WMM-Com:** Commit execution. Same as TSO-Com.
- **Rule WMM-Rec:** execution of a Reconcile fence.

  Guard: The next instruction of a processor is a Reconcile.

  Action: All values in the ib of the processor are removed.
- **Rule WMM-DeqSb:** background store buffer dequeue.

  Guard: The sb of a processor is not empty.

  Action: Assume the value of the oldest store for *some* address a in the sb is v. First, the stale $\langle$address, value$\rangle$ pair $\langle a, m[a]\rangle$ is inserted to the ib of every other processor whose sb does not contain a. Then this store is removed from sb, and $m[a]$ is set to v.

Figure 4-6: Rules to operate the WMM abstract machine

the behavior in Figure 2-4b, we need to insert a Commit between the two stores in P1, and the Commit gives release semantics. The I^{2}E definition of WMM automatically forbids load-store reordering (Figure 2-4d) and out-of-thin-air behaviors (Figure 2-8).

load-load reordering: WMM allows the behavior in Figure 2-4c (FenceSS should be replaced by a Commit), because I_4 can read the stale value 0 from ib. This is as if the two loads in P2 were reordered. We need a Reconcile between the two loads in

P2 to forbid this behavior in WMM, and the Reconcile fence gives acquire semantics.

No dependency ordering: WMM does not enforce any dependency ordering, and it requires Reconcile fences to enforce dependency ordering in WMM. For example, WMM allows the all behaviors in Figure 3-8 (FenceSS should be replaced by Commit), because the last load in P2 can always get the stale value 0 from ib in each litmus test. All those behaviors are as if data-dependent loads were reordered. This is different from GAM which forbids all the behaviors in Figure 3-8.

Besides data-dependency ordering, WMM does not obey control-dependency ordering (Figure 4-7) or potential-memory-dependency ordering (Figure 4-8). In Figure 4-7, the execution of second load in P2 is conditional on the result of the first load. In Figure 4-8, there is a potential memory dependency in P2 betweent the store and the second load before the first load gets its result. WMM allows both behaviors which reorder loads in P2 in Figures 4-7 and 4-8.

Proc. P1	Proc. P2
I_1 : St $[a]$ 1	$I_4 : r_1 =$ Ld $[b]$
I_2 : Commit	I_5 : if$(r_1 \neq 0)$ exit
I_3 : St $[b]$ 1	$I_6 : r_2 =$ Ld $[a]$
Both WMM and GAM allow $r_1 = 1, r_2 = 0$	

Figure 4-7: MP+Ctrl: litmus test for control-dependnecy ordering

Proc. P1	Proc. P2
I_1 : St $[a]$ 1	$I_4 : r_1 =$ Ld $[b]$
I_2 : Commit	I_5 : St $[r_1 + a]$ 42
I_3 : St $[b]$ 100	$I_6 : r_2 =$ Ld a
Both WMM and GAM allow $r_1 = 100, r_2 = 0$	

Figure 4-8: MP+Mem: litmus test for potential-memory-dependency ordering

Atomic memory: WMM is an atomic memory model. A store can be read by a load only from the same processor while the store is in sb. However, if the store is ever pushed from sb to the monolithic memory, it becomes visible to all other processors simultaneously. Thus, WMM forbids the behaviors in non-atomic-memory litmus tests in Figures 2-6a, 2-6b and 2-6c (FenceLL should be Reconcile in these tests).

Per-location SC: WMM enforces per-location SC (Figure 3-9a), because both sb and ib enforce FIFO on same address entries.

4.2.3 Axiomatic Definition of WMM

The axiomatic definition of WMM still uses the two axioms of GAM in Figure 3-10. The only difference is in the definition of preserved program order. The preserved program order for WMM, i.e., $<_{ppo}^{wmm}$, only applies to memory or fence instructions in the same processor. Table 4.1 shows the truth table for a boolean function $\mathsf{order}_{\mathsf{wmm}}(X, Y)$, which indicates whether an older instruction X should be ordered before a younger instruction Y in $<_{ppo}^{wmm}$. For example, entry $\langle \mathsf{Ld}\ [a], \mathsf{Ld}[b] \rangle$ says that X ($\mathsf{Ld}\ [a]$) should be ordered before Y ($\mathsf{Ld}\ [b]$) only if the load addresses are the same (i.e., $a = b$). This corresponds to the same-address load-load ordering. Entry $\langle \mathsf{Ld}\ [a], \mathsf{St}\ [b]\ v' \rangle$ says that an older load is always ordered before a younger store, i.e., no load-store reordering. A Reconcile fence is always ordered before younger instructions, so it has acquire semantics; and a Commit fence is always ordered after older instructions, so it has release semantics. A Commit followed by a Reconcile would act as a full fence which orders instructions older than the commit before instructions younger than the Reconcile.

$\mathsf{order}_{\mathsf{wmm}}(X, Y)$		Y			
		$\mathsf{Ld}\ [b]$	$\mathsf{St}\ [b]\ v'$	Reconcile	Commit
X	$\mathsf{Ld}\ [a]$	$a = b$	True	True	True
	$\mathsf{St}\ [b]\ v$	False	$a = b$	False	True
	Reconcile	True	True	True	True
	Commit	False	True	True	True

Table 4.1: Truth table for $\mathsf{order}_{\mathsf{wmm}}(X, Y)$

With function $\mathsf{order}_{\mathsf{wmm}}$, we can easily define $<_{ppo}^{wmm}$ as follows:

Definition 8 (WMM preserved program order $<_{ppo}^{wmm}$)**.** *Memory or fence instructions $I_1 <_{ppo}^{wmm} I_2$ if $I_1 <_{po} I_2$ and at least one of the following is true:*

1. $\mathsf{order}_{\mathsf{wmm}}(I_1, I_2)$ *returns true.*

2. *(Transitivity) there exists an memory or fence instruction I such that $I_1 <_{ppo}^{wmm} I$ and $I <_{ppo}^{wmm} I_2$.*

4.2.4 Proof of the Equivalence of the Axiomatic and Operational Definitions of WMM

We have proved the equivalence of the axiomatic and operational definitions of WMM, i.e., Theorems 5 and 6. Below we give the sketch of the proofs and the details can be found in [148]. (The proofs can be skipped without affecting the understanding of the rest of the thesis.)

Theorem 5 (Soundness)**.** *WMM I^2E model $\subseteq$ WMM axiomatic model.*

Proof. The goal is that for any execution in the WMM I^2E model, we can construct relations $\langle <_{po}, <_{mo}, \xrightarrow{rf} \rangle$ that have the same program behavior and satisfy the WMM axioms. To do this, we first introduce the following ghost states to the I^2E model:

- Field source in the monolithic memory: For each address a, we add state $m[a]$.source to record the store that writes the current memory value.
- Fields source and overwrite in the invalidation buffer: For each stale value $\langle a, v \rangle$ in an invalidation buffer, we add state v.source to denote the store of this stale value, and add state v.overwrite to denote the store that overwrites v.source in the memory.
- Per-processor list $<_{po\text{-}i2e}$: For each processor, $<_{po\text{-}i2e}$ is the list of all the instructions that has been executed by the processor. The order in $<_{po\text{-}i2e}$ is the same as the execution order in the processor. We also use $<_{po\text{-}i2e}$ to represent the ordering relation in the list (the head of the list is the oldest/minimum in $<_{po\text{-}i2e}$).
- Global list $<_{mo\text{-}i2e}$: $<_{mo\text{-}i2e}$ is a list of all the executed loads, executed fences, and stores that have been dequeued from the store buffers. $<_{mo\text{-}i2e}$ contains instructions from all processors. We also use $<_{mo\text{-}i2e}$ to represent the ordering relation in the list (the head of the list is the oldest/minimum in $<_{mo\text{-}i2e}$).
- Read-from relations $\xrightarrow{rf\text{-}i2e}$: $\xrightarrow{rf\text{-}i2e}$ is a set of edges. Each edge points from a

store to a load, indicating that the load had read from the store in the I^{2}E model. Every executed load in I^{2}E is pointed to by a $\xrightarrow{rf\text{-}i2e}$ edge.

$m[a]$.source initially points to the initialization store, and $<_{po\text{-}i2e}, <_{mo\text{-}i2e}, \xrightarrow{rf\text{-}i2e}$ are all initially empty. We now show how these states are updated in the operations of the WMM I^{2}E model.

1. WMM-Nm, WMM-Com, WMM-Rec, WMM-St: Assume the operation executes an instruction I in processor i. We append I to the tail of list $<_{po\text{-}i2e}$ of processor i. If I is a fence (i.e., the operation is WMM-Com or WMM-Rec), then we also append I to the tail of list $<_{mo\text{-}i2e}$.

2. WMM-DeqSb: Assume the operation dequeues a store S for address a. In this case, we update $m[a]$.source to be S. Let S_0 be the original $m[a]$.source before this operation is performed. Then for each new stale value $\langle a, v\rangle$ inserted into any invalidation buffer, we set v.source $= S_0$ and v.overwrite $= S$. We also append S to the tail of list $<_{mo\text{-}i2e}$.

3. WMM-Ld: Assume the operation executes a load L for address a in processor i. We append L to the tail of list $<_{po\text{-}i2e}$ of processor i. The remaining actions depends on how L gets its value in this operation:

   - If L reads from a store S in the local store buffer, then we add edge $S \xrightarrow{rf\text{-}i2e} L$, and append L to the tail of list $<_{mo\text{-}i2e}$.
   - If L reads the monolithic memory $m[a]$, then we add edge $m[a]$.source $\xrightarrow{rf\text{-}i2e} L$, and append L to the tail of list $<_{mo\text{-}i2e}$.
   - If L reads a stale value $\langle a, v\rangle$ in the local invalidation buffer, then we add edge v.source $\xrightarrow{rf\text{-}i2e} L$, and we insert L to be right before v.overwrite in list $<_{mo\text{-}i2e}$ (i.e., L is older than v.overwrite, but is younger than any other instruction which is older than v.overwrite).

As we will see later, at the end of the I^{2}E execution, $<_{po\text{-}i2e}$, $<_{mo\text{-}i2e}$ and $\xrightarrow{rf\text{-}i2e}$ will become the $\langle <_{po}, <_{mo}, \xrightarrow{rf}\rangle$ relations that satisfy the WMM axioms. The only slight

difference is that $<_{mo\text{-}i2e}$ contains fence instructions while $<_{mo}$ does not. We can simply remove fences from $<_{mo\text{-}i2e}$ to construct $<_{mo}$ without affecting any invariants. Before getting there, we show that the I^{2}E model has the following invariants after each operation is performed:

1. For each address a, $m[a]$.source in the I^{2}E model is the youngest store for a in $<_{mo\text{-}i2e}$.

2. All loads and fences that have been executed in the I^{2}E model are in $<_{mo\text{-}i2e}$.

3. An executed store is either in $<_{mo\text{-}i2e}$ or in store buffer, i.e., for each processor i, the store buffer of processor i contains exactly every store that has been executed in the I^{2}E model but is not in $<_{mo\text{-}i2e}$.

4. For any two stores S_1 and S_2 for the same address in the store buffer of any processor i in the I^{2}E model, if S_1 is older than S_2 in the store buffer, then $S_1 <_{po\text{-}i2e} S_2$.

5. For any processor i and any address a, address a cannot be present in the store buffer and invalidation buffer of processor i at the same time.

6. For any stale value v for any address a in the invalidation buffer of any processor i in the I^{2}E model, the following invariants hold:

   (a) v.source and v.overwrite are in $<_{mo\text{-}i2e}$, and v.source $<_{mo\text{-}i2e}$ v.overwrite, and there is no other store for a between them in $<_{mo\text{-}i2e}$.

   (b) For any Reconcile fence F that has been executed by processor i in the I^{2}E model, $F <_{mo\text{-}i2e} v$.overwrite.

   (c) For any store S for a that has been executed by processor i in the I^{2}E model, $S <_{mo\text{-}i2e} v$.overwrite.

   (d) For any load L for a that has been executed by processor i in the I^{2}E model, if store $S \xrightarrow{rf\text{-}i2e} L$, then $S <_{mo\text{-}i2e} v$.overwrite.

7. For any two stale values v_1 and v_2 for the same address in the invalidation buffer of any processor i in the I^{2}E model, if v_1 is older than v_2 in the invalidation buffer, then $v_1.\mathsf{source} <_{mo\text{-}i2e} v_2.\mathsf{source}$.

8. For any instructions I_1 and I_2, if $I_1 <_{po\text{-}i2e} I_2$ and $\mathsf{order}_{\mathsf{wmm}}(I_1, I_2)$ and I_2 is in $<_{mo\text{-}i2e}$, then $I_1 <_{mo\text{-}i2e} I_2$.

9. For any load L and store S, if $S \xrightarrow{rf\text{-}i2e} L$, then the following invariants hold:

   (a) If S not in $<_{mo\text{-}i2e}$, then S is in the store buffer of the processor of L, and $S <_{po\text{-}i2e} L$, and there is no store S' for the same address in the same store buffer such that $S <_{po\text{-}i2e} S' <_{po\text{-}i2e} L$.

   (b) If S is in $<_{mo\text{-}i2e}$, then $S = \max_{mo\text{-}i2e}\{S' \mid S'.\mathsf{addr} = L.\mathsf{addr} \ \wedge \ (S' <_{po\text{-}i2e} L \ \vee \ S' <_{mo\text{-}i2e} L)\}$, and there is no other store S'' for the same address in the store buffer of the processor of L such that $S'' <_{po\text{-}i2e} L$.

The detailed proof of the above invariants can be found in [148, Appendix A].

It is easy to see that at the end of the I^{2}E execution (of a program), there is no instruction to execute in each processor and all store buffers are empty (i.e., all exected loads stores and fences are in $<_{mo\text{-}i2e}$). At that time, we can define axiomatic relations $<_{po}$, $<_{mo}$, and $\xrightarrow{rf}$ as $<_{po\text{-}i2e}$, $<_{mo\text{-}i2e}$ without fences, and $\xrightarrow{rf\text{-}i2e}$, respectively. Then invariants 8 and 9b ensures that the InstOrder and LoadValue axioms of WMM are satisfied.

□

Theorem 6 (Completeness)**.** *WMM axiomatic model* $\subseteq$ *WMM I^{2}E model.*

Proof. The goal is that for any axiomatic relations $\langle <_{po}, <_{mo}, \xrightarrow{rf} \rangle$ that satisfy the WMM axioms, we can run the same program in the I^{2}E model and get the same program behavior. We first make a small modification to $<_{mo}$. We insert all the fence instructions into $<_{mo}$ such that $<_{mo}$ still respects $<_{ppo}^{wmm}$. This is always doable because $<_{ppo}^{wmm}$ only relates instructions in the same processor. There can be multiple ways to insert fences into $<_{mo}$, and we can just pick any one of them. From now on, we assume $<_{mo}$ contains also fence instructions, and the two WMM axioms still hold.

We use an algorithm to operate the I^{2}E model to get the same program behavior as in axiomatic relations $\langle <_{po}, <_{mo}, \xrightarrow{rf} \rangle$. During the operation of the I^{2}E model, the instructions executed in each processor should match the $<_{po}$ of that processor, and thus, we can associate instructions in the I^{2}E model to instructions in the axiomatic relations.

The algorithm begins with the I^{2}E model (in initial state), an empty set Z, and a queue Q which contains all the memory and fence instructions in $<_{mo}$. The order of instructions in Q is the same as $<_{mo}$, i.e., the head of Q is the oldest instruction in $<_{mo}$. In each step of the algorithm, we perform one of the followings actions:

1. If the next instruction of some processor in the I^{2}E model is a non-memory instruction, then we perform the WMM-Nm operation to execute it in the I^{2}E model.

2. Otherwise, if the next instruction of some processor in the I^{2}E model is a store, then we perform the WMM-St operation to execute that store in the I^{2}E model.

3. Otherwise, if the next instruction of some processor in the I^{2}E model is mapped to a load L in set Z, then we perform the WMM-Ld operation to execute L in the I^{2}E model, and we remove L from Z.

4. Otherwise, we pop out instruction I from the head of Q and process it in the following way:

   (a) If I is a store, then I must have been mapped to a store in some store buffer (we will prove this), and we perform the WMM-DeqSb operation to dequeue I from the store buffer in the I^{2}E model.

   (b) If I is a Reconcile fence, then I must have been mapped to the next instruction to execute in some processor (we will prove this), and we perform the WMM-Rec operation to execute I in the I^{2}E model.

   (c) If I is a Commit fence, then I must have been mapped to the next instruction to execute in some processor (we will prove this), and we perform the WMM-Com operation to execute I in the I^{2}E model.

(d) I must be a load in this case. If I has been mapped, then it must be mapped to the next instruction to execute in some processor in the I^{2}E model (we will prove this), and we perform the WMM-Ld operation to execute I in the I^{2}E model. Otherwise, we just add I into set Z.

For proof purposes, we introduce source field to each value in the monolithic memory and invalidation buffers in the I^{2}E model. The source field records the store that supplies the value.

We also define a function overwrite. For each store S in $<_{mo}$, overwrite(S) returns the store for the same address such that $S <_{mo}$ overwrite(S), and there is no store S' for the same address such that $S <_{mo} S' <_{mo}$ overwrite(S). That is, overwrite(S) returns the store that overwrites S in $<_{mo}$. (overwrite(S) does not exist if S is the last store for its address in $<_{mo}$.)

With the above definitions and new states, we introduce the invariants of the algorithm. After each step of the algorithm, we have the following invariants for the states of the I^{2}E model, Z and Q:

1. For each processor i, all executed instructions and the next-to-execute instruction in processor i in the I^{2}E model is a prefix of the $<_{po}$ of processor i in the axiomatic relations.

2. The predicate of any operation performed in this step is satisfied.

3. If we execute an instruction in the I^{2}E model in this step, the operation is able to get the same instruction result as that of the corresponding instruction in the axiomatic relations.

4. The instruction type, load/stores address, and store data of every executed instruction in the I^{2}E model are the same as those of the corresponding instruction in the axiomatic relations.

5. All loads that have been executed in the I^{2}E model are exactly all the loads that are in $<_{mo}$ but not in Q or Z.

6. All fences that have been executed in processor i are exactly all that fences that are in $<_{mo}$ but not in Q.

7. All stores that have been executed and dequeued from the store buffers in the I^{2}E model are exactly all the stores that are in $<_{mo}$ but not in Q.

8. For each address a, $m[a]$.source in the I^{2}E model is the youngest store for a, which has been popped from Q, in $<_{mo}$.

9. For each processor i, the store buffer of processor i contains exactly every store that has been executed in the I^{2}E model but still in Q.

10. For any two stores S_1 and S_2 for the same address in the store buffer of any processor i in the I^{2}E model, if S_1 is older than S_2 in the store buffer, then $S_1 <_{po} S_2$.

11. For any processor i and any address a, address a cannot be present in the store buffer and invalidation buffer of processor i at the same time.

12. For any processor i, if a store S meets all the following conditions, then the invalidation buffer of processor i contains an entry whose source field is S:

    (a) The store buffer of processor i does not contain the address of S.

    (b) overwrite(S) exists and overwrite(S) has been popped from Q.

    (c) For each Reconcile fence F that has been executed by processor i in the I^{2}E model, $F <_{mo}$ overwrite(S).

    (d) For each store S' for the same address that has been executed by processor i in the I^{2}E model, $S' <_{mo}$ overwrite(S).

    (e) For each load L for the same address that has been executed by processor i in the I^{2}E model, if store $S' \xrightarrow{rf} L$ in the axiomatic relations, then $S' <_{mo}$ overwrite(S).

13. For any stale value $\langle a, v \rangle$ in any invalidation buffer, overwrite(v.source) exists and overwrite(v.source) is not in Q.

14. For any two stale values v_1 and v_2 for the same address in the invalidation buffer of any processor i in the I^{2}E model, if v_1 is older than v_2 in the invalidation buffer, then $v_1.\mathsf{source} <_{mo} v_2.\mathsf{source}$.

These invariants guarantee that the algorithm will operate the I^{2}E model to produce the same program behavior as the axiomatic model.

□

4.3 Comparing WMM and GAM

4.3.1 Bridging the Operational Definitions of WMM and GAM

The abstract machines of WMM and GAM are defined in drastically different styles. In particular, WMM does not model out-of-order execution explicitly. To understand the relations between the two, we first define GAM^{VP}, an abstract machine in the style of the GAM abstract machine, i.e., using an ROB. Next we explain how the I^{2}E abstract machine of WMM simulates the out-of-order execution in GAM^{VP}, and prove that GAM^{VP} is contained within WMM. It should be noted that GAM^{VP} is not equivalent to GAM.

At a high level, GAM^{VP} is derived by making three major changes on the GAM abstract machine:

1. replace the fence instructions in GAM with the fence instructions in WMM;
2. restrict the execution of a store to forbid load-store reordering; and
3. introduce load-value prediction because WMM does not need to obey data-dependency orderings (and this is why we name the model as GAM^{VP}).

Next we give the details of the GAM^{VP} abstract machine.

The structure of the GAM^{VP} abstract machine is exactly the same as GAM, which has been shown in Figure 3-11. The abstract machine contains a monolithic memory m connected to each processor, and each processor has an ROB and a PC

register. The PC register contains the address of the next instruction to be fetched (speculatively) into the ROB. The ROB has one entry per instruction. Each ROB entry for an instruction I in GAM^{VP} contains all the fields of an ROB entry in GAM, but the ROB entry in GAM^{VP} differs from that in GAM in the following two aspects:

1. the ROB entry in GAM^{VP} contains an extra *load-value-predicted* bit, which indicates if the load value has been predicted in case I is a load; and

2. the execution result in the ROB entry in GAM^{VP} is considered valid (i.e., readable by younger instructions) if the done bit *or the load-value-predicted bit* is set.

Figure 4-9 and 4-10 show the rules to operate the GAM^{VP} abstract machine. Figure 4-9 contains all the rules in GAM^{VP} that are the same as those in GAM, including instruction fetch, execution of reg-to-reg and branch instructions, and computing store data and memory addresses. Figure 4-10 contains the rules that are different from GAM. The rule to execute fence instructions in GAM is replaced by rules GAM^{VP}-Execute-Commit and GAM^{VP}-Execute-Reconcile in GAM^{VP}, because WMM uses Commit and Reconcile fences. The guards of these two rules match the ordering constraints of WMM (Table 4.1). GAM^{VP}-Predict-Load-Value is the newly introduced rule for load-value prediction. It should be noted that we cannot predict the value for a load if the load is already executed or predicted before. Rule GAM^{VP}-Execute-Load executes a load in almost the same way as GAM. The two differences are: (1) the guard is changed to match the ordering constraint of WMM, and (2) younger instructions need to be squashed in case the load value has been mispredicted earlier. Rule GAM^{VP}-Execute-Store executes a store in the same way as GAM but with a different guard to match the ordering constraint of WMM. In particular, case 4 in the guard forbids load-store reordering.

We can show that the I^{2}E abstract machine of WMM can simulate all the behaviors of GAM^{VP}, i.e., the following Theorem 7.

Theorem 7. *GAM$^{VP}\subseteq$ WMM.*

- **Rule GAM$^{\mathbf{VP}}$-Fetch:** Fetch a new instruction. Same as GAM-Fetch.
- **Rule GAM$^{\mathbf{VP}}$-Execute-Reg-to-Reg:** Execute a reg-to-reg instruction I. Same as GAM-Execute-Reg-to-Reg.
- **Rule GAM$^{\mathbf{VP}}$-Execute-Branch:** Execute a branch instruction I. Same as GAM-Execute-Branch.
- **Rule GAM$^{\mathbf{VP}}$-Compute-Store-Data:** compute the data of a store instruction I. Same as GAM-Compute-Store-Data.
- **Rule GAM$^{\mathbf{VP}}$-Compute-Mem-Addr:** Compute the address of a load or store instruction I. Same as GAM-Compute-Mem-Addr.

Figure 4-9: Operations on the GAM^{VP} abstract machine (part 1 of 2: rules same as GAM)

The key to relate WMM and GAM^{VP} is to find an in-order serialization point, which is commonly known as the *instruction-commit* in most processors, in GAM^{VP}. We can hypothetically mark instructions in GAM^{VP} as committed while the abstract machine is running. It should be noted that only the oldest uncommitted instruction in an ROB can be marked as committed, i.e., instructions are committed in order. The necessary (but insufficient) conditions for an instruction to be committed are that:

1. all older instructions in the same ROB are committed,
2. all older fence and load instructions in the same ROB are done, and
3. all older not-done branches are not mispredicted (we can determine mispredictions because all load values are available).

Committing a reg-to-reg or branch or store instruction does not need to meet any extra conditions. In particular, the instruction does not need to be done, and in case of a store, the store address or data does not need to be computed. This is because the result of a reg-to-reg or branch instruction or the address and data of a store are determined by the results of older loads. For a load instruction, it should be done and obey the store-to-load memory dependency for all older stores when it is committed. Otherwise the load cannot be committed. If the load does not obey all the memory

- **Rule GAM$^{\mathbf{VP}}$-Execute-Commit:** Execute a Commit fence I.

  Guard: I is marked not-done, and all older memory and fence instructions are done.

  Action: Mark I as done.

- **Rule GAM$^{\mathbf{VP}}$-Execute-Reconcile:** Execute a Reconcile fence I.

  Guard: I is marked not-done, and all older load and fence instructions are done.

  Action: Mark I as done.

- **Rule GAM$^{\mathbf{VP}}$-Predict-Load-Value:** Predict the result of a load instruction I.

  Guard: I is marked not-done, and its load-value-predicted bit has not been set.

  Action: Set the load-value-predicted bit of I, and set the execution-result in the ROB entry of I to a random value v.

- **Rule GAM$^{\mathbf{VP}}$-Execute-Load:** Execute a load instruction I for address a.

  Guard: I is marked not-done, its address-available bit is set, and all older Reconcile fences are done.

  Action: Perform the same action as GAM-Execute-Load. In addition, if we mark I as done and the load-value-predicted bit of I has been set before, we compare the load value with the original execution result (i.e., the predicted value) stored in the ROB entry of I. In case they do not match, we kill all instructions younger than I (excluding I), i.e., we remove all younger instructions from ROB and set the PC register to the next PC of I.

- **Rule GAM$^{\mathbf{VP}}$-Execute-Store** Execute a store I for address a.

  Guard: I is marked not-done, and all the following conditions must be true:

  1. The address-available bit of I is set,
  2. The data-available bit of I is set,
  3. All older branch instructions are done,
  4. All older loads are done,
  5. All older stores have their address-available bits set,
  6. All older stores for address a are done,
  7. All older fence instructions are done.

  Action: Update $m[a]$ and mark I as done.

Figure 4-10: Rules to operate the GAM^{VP} abstract machine (part 2 of 2: rules different from GAM)

dependencies, it will be squashed when an older store computes its address later, and thus, it should not be committed. For a fence instruction, it needs to be done when

it is committed. Since the guards of rules that execute fence instructions require older load and fence instructions to be done, if a fence instruction is committed, its commit should happen immediately after it is marked as done. The guard of GAM^{VP}-Execute-Store ensures that a store cannot become done (i.e., modify the monolithic memory and become readable by other processors) before it is committed.

Given the hypothetical commit point of instructions, WMM can simulate the behavior of GAM^{VP} as follows. Whenever we mark an instruction in GAM^{VP} as committed, we also execute that instruction in WMM. Each time GAM^{VP} writes a store to monolithic memory, we have WMM dequeue that store to monolithic memory. The store is in the store buffer in WMM is because a store cannot become done before it is committed. A committed load in GAM^{VP} is able to read from the same store in the following way. If the store read by the load in GAM^{VP} is still not-done in GAM^{VP}, then WMM lets the load read the store from the local store buffer. If the store is currently in the monolithic memory of GAM^{VP}, then WMM lets the load read the store from the monolithic memory. If the store has been overwritten in the monolithic memory of GAM^{VP}, then WMM lets the load read the store from the local invalidation buffer. It should be noted that in the last case (i.e., the store is overwritten in GAM^{VP}), the store cannot be removed from the invalidation buffer by the execution of older loads or stores. This is because the rules in GAM^{VP} (especially the kills and stalls of load instructions) ensure that a committed load never reads from a store which is older than any local older store or any stores already observed by a local older load. Any loads violating this invariant will be squashed before becoming committed. The store cannot be removed from the invalidation buffer because of older Reconcile fences either. This is because an older Reconcile fence should be committed right after it is done, and by that time, the store cannot be overwritten (otherwise it cannot be read by the load).

It should be noted that the above reasoning process does not mention load-value prediction at all. This is because we only consider committed loads which are done and have validated their value predictions. Load-value prediction allows loads in GAM^{VP} to observe stale values that may not be visible in GAM, but such stale values can

still be captured by the invalidation buffer in WMM. The proof below formalizes the above reasoning. (The proof can be skipped without affecting the understanding of the rest of the thesis.)

Proof for Theorem 7. The proof outline is as follows. We first give a procedure to operate the WMM I^2E abstract machine to simulate the behavior of the WAM abstract machine. Then we give the invariants of the simulation process. Finally we prove that all the invariants hold throughout the simulation.

Simulation procedure: The simulation procedure requires adding the following ghost fields to each ROB entry in GAM^{VP}:

- A *committed* bit (initially unset) to indicate that the instruction cannot be squashed and has been simulated in WMM.
- A *mispredicted* bit (initially unset) for each branch instruction to indicate that the branch should be mispredicted according to the WMM execution. This bit can be set only when the committed bit is also set.
- A *wmm-store-address* field for each store instruction to record the store address computed in the WMM execution.

These fields will be manipulated by the simulation procedure, and cannot be unset after being set once. In particular, the simulation procedure will set instructions in each processor as committed monotonically from the oldest to the youngest. Whenever an instruction is committed, we let WMM fire a rule to execute the instruction. The detailed simulation procedure is as follows:

1. Let GAM^{VP} fire a rule. If the rule is GAM^{VP}-Execute-Store, then go to step 2. Otherwise, go to step 3.
2. *GAM^{VP} just fired a GAM^{VP}-Execute-Store rule to write a store into monolithic memory*: In this case, we have WMM fire a WMM-DeqSb rule to dequeue the same store from store buffer to monolithic memory, and then go back to step 1.

3. *GAM^{VP} just fired a rule (which is not GAM^{VP}-Execute-Store) in processor P_i:* In this case, we try to mark more instructions in P_i as committed. If there is no uncommitted instruction in the ROB of P_i, then we do nothing and go back to step 1. If the youngest committed instruction in the ROB of P_i is a not-done branch which is marked as mispredicted (i.e., GAM^{VP} has not yet corrected the branch misprediction), then we do nothing and go back to step 1. Otherwise, we examine the oldest uncommitted instruction I in the ROB of P_i, and take actions according the type of I:

   - I is a reg-to-reg instruction: In this case, we set I as committed, and let WMM fire a WMM-Nm rule to execute I.
   - I is a branch instruction: In this case, we set I as committed, and let WMM fire a WMM-Nm rule to execute I. If I is not-done in GAM^{VP} and the predicted branch target does not match the next address computed in WMM, then we set the mispredicted bit of I.
   - I is a store instruction: In this case, we set I as committed, let WMM fire a WMM-St rule to execute I, and record the address computed in WMM in the wmm-store-address field of I.
   - I is a Commit or Reconcile instruction: In this case, if I is not-done, then we do nothing. Otherwise, we set I as committed, and let WMM fire a WMM-Com or WMM-Rec rule, respectively, to execute I.
   - I is a load instruction: In this case, we check if both of the following conditions are met:
     - I is done in GAM^{VP}.
     - If there are any committed stores in P_i whose wmm-store-addresses are equal to the load address, then the youngest among those stores must have already computed its address in GAM^{VP}.

     If any of the above conditions is not met, then we do nothing. Otherwise, both conditions are met, and in this case, we set I as committed, and let

WMM fire a WMM-Ld to execute I to read from the same store. That is, if the store read by I in GAM^{VP} is still not-done in GAM^{VP}, then WMM reads the store from the local store buffer. If the store is currently in the monolithic memory of GAM^{VP}, then WMM reads the store from the monolithic memory. If the store has been overwritten in the monolithic memory of GAM^{VP}, then WMM reads the store from the local invalidation buffer.

If I is set as committed, then we restart this step to commit more instructions. Otherwise, we go back to step 1.

It should be noted that the process of marking instructions as committed in step 3 can stop only for three reasons: (1) there is no more uncommitted instruction, (2) the youngest committed branch is mispredicted, and (3) the oldest uncommitted instruction is a load which does not meet the specific conditions.

Invariants: To help state the invariants, we first introduce a global clock, which is incremented each time after GAM^{VP} fires a rule. With the global clock, we can track the following timestamps for each instruction in GAM^{VP}:

- The *done-time* which records the time when the instruction is marked as done.
- The *overwritten-time*, which, in case the instruction is a store, records the time when the store is being overwritten by another store for the same address in the monolithic memory.
- The *committed-time* which records the time when the instruction is marked as committed.

The simulation procedure maintains the following invariants after each rule fires in WMM or GAM^{VP}:

1. Committed instructions are never squashed in GAM^{VP}.
2. If an instruction I is committed in the ROB of P_i, then all instructions older than I in the ROB of P_i are committed.

3. In GAM^{VP}, if a store S for address a in P_i is done, then all of the followings are true:

   (a) All memory instructions older than S in the ROB of P_i has computed their addresses.

   (b) For any store S' older than S in the ROB of P_i, if S' is also for address a, then S' is done, and the done-time of $S' <$ that of S.

   (c) For any load or fence instruction I older than S in the ROB of P_i, I is done, and the done-time of $I <$ that of S.

   (d) S is committed.

4. In GAM^{VP}, if a load L for address a in P_i is done and it reads from store S, then all of the followings are true:

   (a) If S is not-done, then S must be in the ROB of P_i and is older than L.

   (b) For any load L' older than L in the ROB of P_i, if L' has computed its address to be a and is not-done, then S must be younger than L' in the ROB of P_i.

   (c) For any load L' older than L in the ROB of P_i, if L' has computed its address to be a, and L' is done by reading store S', and S' is not-done, then either S is just S' or S is younger than S' in the ROB of P_i.

   (d) For any load L' older than L in the ROB of P_i, if L' has computed its address to be a, and L' is done by reading store S', and S' is done, then either S is not-done or the done-time of $S \geq$ that of S'.

   (e) For any store S' older than L in the ROB of P_i, if S' has computed its address to be a and S' is not-done, then either S is just S' or S is younger than S' in the ROB of P_i.

   (f) For any store S' older than L in the ROB of P_i, if S' has computed its address to be a and S' is done, then either S is not-done or the done-time of $S \geq$ that of S'.

(g) For any Reconcile fence R older than L in the ROB of P_i, R must be done, and the done-time of $R <$ that of L.

5. In GAM^{VP}, if a Reconcile fence R is done, then for any older load or fence instruction I, I is done, and the done-time of $I <$ that of R.

6. In GAM^{VP}, if a Commit fence C is done, then for any older load or store or fence instruction I, I is done, and the done-time of $I <$ that of R.

7. In GAM^{VP}, if a load L for address a in P_i is committed, then all of the followings are true:

   (a) L is done and should read from some store S.

   (b) For any store S' older than L in the ROB of P_i, if the wmm-store-address of S' is a and S' is not-done, then either S is just S' or S is younger than S' in the ROB of P_i.

   (c) The committed-time of L is equal to one plus the maximum done-time of L and all older load and fence instructions in the ROB of P_i, i.e., $1 + \max\{\text{done-time of } I \mid I \text{ is } L \ \vee \ (I \text{ is older than } L \text{ in ROB of } P_i \ \wedge \ I \text{ is a load or fence})\}$

8. In GAM^{VP}, if a Reconcile fence R is committed, then R is done, and the committed-time of R is equal to one plus the done-time of R.

9. In GAM^{VP}, if a Commit fence C is committed, then C is done, and the committed-time of C is equal to one plus the done-time of C.

10. Every WMM rule can truly fire, i.e., the guard is satisfied.

11. Instructions executed in WMM in each processor are the committed instructions in the ROB of each processor in GAM^{VP}.

12. For each committed load or store instruction in GAM^{VP}, if the address or store-data field is computed, then it matches that of the corresponding instruction executed in WMM.

13. For each done and committed instruction in GAM^{VP}, its execution result (including destination register value and next PC) matches that of the corresponding instruction executed in WMM, and in particular, if the instruction is a load, then the load reads from the same store in GAM^{VP} and WMM.

14. The content of the monolithic memory of WMM matches that of GAM^{VP}, and the sequence of stores that modifies the monolithic memory in WMM also matches that of GAM^{VP}.

15. For any address a, the stores for a in the store buffer of P_i in WMM are exactly all the not-done committed stores with wmm-store-address equal to a in the ROB of P_i in GAM^{VP}, and the order of these stores in the store buffer matches the order of them in the ROB.

16. For any address a, if there is any not-done committed store with wmm-store-address equal to a in the ROB of P_i in GAM^{VP}, then the invalidation buffer of P_i in WMM does not contain address a.

17. For any address a, if there is no not-done committed store with wmm-store-address equal to a in the ROB of P_i in GAM^{VP}, then the stores for a in the invalidation buffer of P_i are exactly every store S which meets all the following conditions:

    (a) In GAM^{VP}, S is done, has address a, and has been overwritten in the monolithic memory (i.e., the overwritten-time of S is valid).

    (b) For each committed store S' with wmm-store-address equal to a in the ROB of P_i, the done-time of $S \geq$ that of S'.

    (c) For each store S' read by any committed loads for address a in the ROB of P_i, S' is done, and the done-time of $S \geq$ that of S'.

    (d) For each committed Reconcile fence R in the ROB of P_i, the overwritten-time of $S >$ the done-time of R.

The order of these stores in the invalidation buffer matches the order of their done-times.

Invariants 1 to 9 are properties of GAM^{VP}. Invariants 1 and 2 state the monotonicity of the process of marking instructions as committed. Invariants 3 to 6 state the execution ordering in GAM^{VP}, and in particular, the ordering of same-address memory instructions and the load-to-store ordering. Invariants 7 to 9 state when an instruction should become committed.

Invariants 10 to 17 are about the properties of WMM and the relation between WMM and GAM^{VP}. Invariants 10 to 13 state the correctness of our simulation procedure. Invariants 14 to 17 show the relation between the states in WMM and the states in GAM^{VP}.

It should be noted that for any instruction I in P_i, if I and all instructions older than I are done, then our simulation procedure (step 3) will mark I and all these older instructions as committed. This guarantees forward progress of the simulation procedure.

Proving the correctness of the invariants: Most of the invariants are not difficult to prove, so we skip the detailed proof. Here we just consider the most complicated case as an example. We consider the case that step 3 of the simulation procedure marks a load L for address a in the ROB of P_i as committed and L reads from a store S which has been overwritten in the monolithic memory in GAM^{VP}. In this case, we will prove that WMM can fire a WMM-Ld rule to execute L to read from S in the invalidation buffer of P_i (i.e., invariants 10 and 13 still hold), and the contents of the invalidation buffer of P_i still obey invariant 17.

We first show by contradiction that when we mark L as committed, there is no not-done committed store with wmm-store-address equal to a in the ROB of P_i. We assume such stores exist and let S' be the youngest among them. Since S' is already committed, S' is older than L in the ROB of P_i. The condition for marking L as committed in step 3 requires S' to have already computed its address. According to invariant 12, the address of S' must be a. Then, according to invariant 4e, S should be not-done, contradicting with our initial assumption that S has been overwritten.

Since there is no not-done committed store with wmm-store-address equal to a in the ROB of P_i, all committed store with wmm-store-address equal to a in the ROB of P_i are done and has computed their addresses to be a (invariant 12), and invariant 17 can be applied. We now show that S is in the invalidation buffer of P_i in WMM, i.e., S meets all the requirements in invariant 17. Requirement 17a is met because of the initial assumption. Requirement 17b is met because of invariant 4f. Requirement 17c is met because of invariants 4c and 4d. Requirement 17d is met because of invariant 4g and the fact that S cannot be overwritten by the time L is marked as done. Since all requirements are met, WMM can indeed have L read from S in the invalidation buffer of P_i, i.e., invariants 10 and 13 still hold.

When the WMM-Ld rule fires, WMM removes all stores for a older than S from the invalidation buffer of P_i. According to invariant 17 before the rule fires, the order of stores match the order of done-times. That is, WMM removes all stores for a which have smaller done-times than S from the invalidation buffer of P_i. This ensures that invariant 17 still holds (especially regarding requirement 17c) after the rule fires.

□

4.3.2 Same-Address Load-Load Ordering

In WMM, loads for the same address are always ordered by $<_{ppo}^{wmm}$ (Table 4.1). However, in GAM, loads for the same address are ordered by $<_{ppo}^{gam}$ only when there is no intervening stores for the same address between them (constraint SALdLd). It may seem that WMM is unnecessarily more restrictive in same-address load-load ordering, but this is actually not the case. A short explanation is that the GAM^{VP} abstract machine, which is contained by WMM, allows two loads for the same address with an intervening store to be executed out of order.

We can understand this more intuitively by considering the instruction sequence in Figure 4-11. GAM allows I_3 to overtake I_1 in the global memory order $<_{mo}$, while WMM forbids this reordering in $<_{mo}$. It should be noted that the reordering of I_1 and I_3 will not create any more behavior in WMM. To understand this point, consider a future instruction I_4 in the same processor. To produce more behaviors,

the reordering of I_1 and I_3 should allow I_4 to overtake more instructions in $<_{mo}$. If I_4 is a fence (either Commit or Reconcile) or a store, it has to be ordered after both I_1 and I_3 in $<_{mo}$. Thus, the reordering of I_1 and I_3 has no influence on the position of I_4 in $<_{mo}$. If I_4 is a load for a different address, then I_4 needs not to be ordered with either I_1 or I_3. Thus, the reordering of I_1 and I_3 still does not impact I_4. In I_4 is a load for address a, even if I_4 also overtakes I_1 in $<_{mo}$, I_4 will get the value from a local store like I_2. However, a local store like I_2 can be read by I_4 even if I_4 is after I_1 in $<_{mo}$. Therefore, the reordering of I_1 and I_3 cannot produce more behaviors in WMM.

Proc. P1
$I_1 : r_1 = \mathsf{Ld}\ [a]$
$I_2 : \mathsf{St}\ [a]\ 1$
$I_3 : r_2 = \mathsf{Ld}\ [a]$
$\cdots$
I_4 : Any future instruction

Figure 4-11: Loads for the same address with an intervening store for the same address in between

It should be noted that the above reasoning and claim will not hold if WMM enforces data-dependent instructions. In fact, Alpha, another memory model that does not enforce data-dependency ordering, uses the same representation for same-address load-load ordering in its axiomatic model.

The lack of dependency ordering in WMM can actually make WMM implementations a little bit more flexible than GAM implementations. Consider litmus test RSW in Figure 3-9c. WMM allows this behavior because all the data dependencies in P2 does not imply any ordering. This implies that a WMM processor can handle loads for the same address in the same way as ARM. That is, a WMM processor can issue loads for the same address out of order, but kill the younger one if its value has been overwritten when the older load gets its value (Section 3.1.5). The eager execution of the younger load can be viewed as performing a value prediction (rule GAM^{VP}-Predict-Load-Value), and the later check on whether the value has been overwritten can be viewed as truly executing the younger load (rule GAM^{VP}-Execute-Load).

4.3.3 Fence Ordering

The Commit and Reconcile fences in WMM have release and acquire semantics, respectively. However, they are slightly different from the FenceRel (i.e., FenceLS; FenceSS) and FenceAcq (i.e., FenceLL; FenceLS) in GAM. This is because Commit and Reconcile are ordered with each other in $<_{ppo}^{wmm}$ while fences in GAM are never ordered with each other. The ordering in WMM makes Commit followed by a Reconcile a full fence, while the combination of FenceRel and FenceAcq is not a full fence (because it lacks FenceSL).

It is possible to extend WMM to include a *light-weight commit fence*, i.e., LWCommit, which can be overtaken by a younger Reconcile. In the I^{2}E abstract machine, the execution of a LWCommit has no guard, and it inserts the LWCommit into the store buffer. LWCommit controls the ordering of removing stores from the store buffer. Stores younger than the LWCommit in the store buffer cannot be removed, and the LWCommit can be removed from the store buffer only when there is no older stores in the buffer. The LWCommit fence defined in this way is in fact a pure store-store fence. Since WMM enforces load-store ordering by default, LWCommit is closer to FenceRel than Commit is.

4.4 WMM Implementation

WMM can be implemented using conventional OOO multiprocessors, and even the most aggressive speculative techniques on load execution are admitted by the semantics of WMM. To demonstrate this, we describe an OOO implementation of WMM (Figure 4-12), and show simultaneously how the WMM operational definition (i.e., the I^{2}E abstract machine) captures the behaviors of the implementation. The implementation is described abstractly to skip unrelated details (e.g., ROB entry reuse). The implementation consists of n OOO processors and a coherent write-back cache hierarchy which we discuss next.

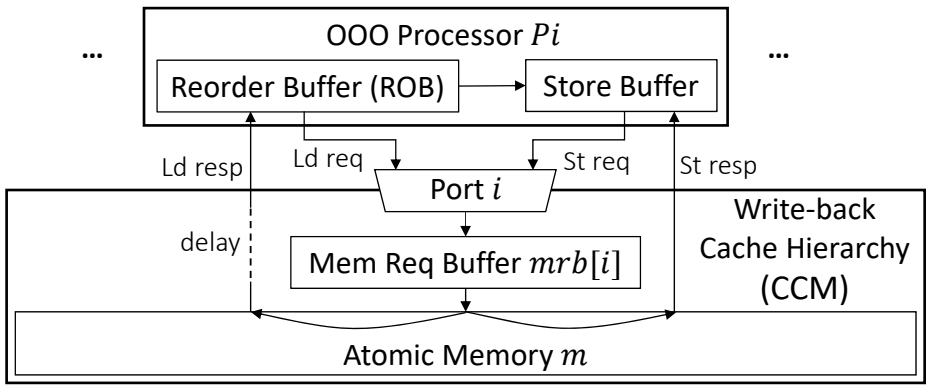


Figure 4-12: CCM+OOO: implementation of WMM

4.4.1 Write-Back Cache Hierarchy (CCM)

We describe CCM as an abstraction of a conventional write-back cache hierarchy to avoid too many details. In the following, we explain the function of such a cache hierarchy, abstract it to CCM, and relate CCM to the WMM model.

Consider a real n-ported write-back cache hierarchy with each port i connected to processor Pi. A request issued to port i may be from a load instruction in the ROB of Pi or a store in the store buffer of Pi. In conventional coherence protocols, all memory requests can be *serialized*, i.e., each request can be considered as taking effect at some time point within its processing period [139]. For example, consider the non-stalling MSI directory protocol in the Primer by Sorin et. al. [134, Chapter 8.7.2]. In this protocol, a load request takes effect immediately if it hits in the cache; otherwise, it takes effect when it gets the data at the directory or a remote cache with M state. A store request always takes effect at the time of writing the cache, i.e., either when it hits in the cache, or when it has received the directory response and all invalidation responses in case of miss. We also remove the requesting store from the store buffer when a store request takes effect. Since a cache cannot process multiple requests to the same address simultaneously, we assume requests to the same address from the same processor are processed in the order that the requests are issued to the cache.

CCM (Figure 4-12) abstracts the above cache hierarchy by operating as follows: every new request from port i is inserted into a memory request buffer $mrb[i]$, which

keeps requests to the same address in order; at any time we can remove the oldest request for an address from a mrb, let the request access the monolithic memory m, and either send the load result to ROB (which may experience a delay) or immediately dequeue the store buffer. m represents the coherent memory states. Removing a request from mrb and accessing m captures the moment when the request takes effect.

It is easy to see that the monolithic memory in CCM corresponds to the monolithic memory in the WMM model, because they both hold the coherent memory values. We will show shortly that how WMM captures the combination of CCM and OOO processors. Thus any coherent protocol that can be abstracted as CCM can be used to implement WMM.

4.4.2 Out-of-Order Processor (OOO)

The major components of an OOO processor are the ROB and the store buffer (see Figure 4-12). Instructions are fetched into and committed from ROB in order; loads can be issued (i.e., search for data forwarding and possibly request CCM) as soon as its address is known; a store is enqueued into the store buffer only when the store commits (i.e., entries in a store buffer cannot be killed). To maintain the per-location SC property of WMM, when a load L is issued, it kills younger loads which have been issued to memory or have got forwarded values from stores older than L. Next we give the correspondence between OOO and WMM.

Store buffer: The state of the store buffer in OOO is represented by the sb in WMM. Entry into the store buffer when a store commits in OOO corresponds to the WMM-St rule. In OOO, the store buffer only issues the oldest store for some address to CCM. The store is removed from the store buffer when the store updates the monolithic memory in CCM. This corresponds to the WMM-DeqSb rule.

ROB and eager loads: Committing an instruction from ROB corresponds to executing it in WMM, and thus the architectural register state in both WMM and OOO must match at the time of commit. Early execution of a load L to address a with a return value v in OOO can be understood by considering where $\langle a, v\rangle$ resides in OOO

when L *commits*. Reading from sb or monolithic memory m in the WMM-Ld rule covers the cases that $\langle a, v\rangle$ is, respectively, in the store buffer or the monolithic memory of CCM when L commits. Otherwise $\langle a, v\rangle$ is no longer present in CCM+OOO at the time of load commit and must have been overwritten in the monolithic memory of CCM. This case corresponds to having fired the WMM-DeqSb rule to insert $\langle a, v\rangle$ into ib previously, and now using the WMM-Ld rule to read v from ib.

Speculations: OOO can issue a load speculatively by aggressive predictions, such as branch prediction (Figure 4-7), memory dependency prediction (Figure 4-8) and even load-value prediction (Figure 3-8). As long as all predictions related to the load eventually turn out to be correct, the load result got from the speculative execution can be preserved. *No further check is needed.* Speculations effectively reorder dependent instructions, e.g., load-value speculation reorders data-dependent loads. Since WMM does not require preserving any dependency ordering, speculations will neither break WMM nor affect the above correspondence between OOO and WMM.

Although WMM allows maximum flexibility in speculative load execution, it constrains the execution of stores. As described earlier, in OOO, a store cannot be issued to memory until it is committed from ROB and enters the store buffer.

Fences: Fences never go into store buffers or CCM in the implementation. In OOO, a Commit can commit from ROB only when the local store buffer is empty. Reconcile plays a different role; at the time of commit it is a NOP, but while it is in the ROB, it stalls all younger loads (unless the load can bypass directly from a store which is younger than the Reconcile). The stall prevents younger loads from reading values that would become stale when the Reconcile commits. This corresponds to clearing ib in WMM.

Summary: For any execution in the CCM+OOO implementation, we can operate the WMM model following the above correspondence. Each time CCM+OOO commits an instruction I from ROB or dequeues a store S from a store buffer to memory, the monolithic memory of CCM, store buffers, and the results of committed instructions in CCM+OOO are exactly the same as those in the WMM model when the WMM model executes I or dequeues S from sb, respectively.

4.5 Performance Evaluation

In this section, we compare the performance of WMM against GAM to show that the cost of having a simpler memory-model definition is minimal.

4.5.1 Methodology

We simulate a WMM out-of-order processor and a GAM out-of-order processor using the GEM5 simulator [37]. Since the major difference between WMM and GAM is whether load-store reordering is allowed or not, i.e., how early a store can be issued to memory, we first explain briefly the behavior of store.

In a typical processor implementation, a store instruction is kept in either the store queue or the store buffer after it is renamed and entered into the ROB. The store buffer holds stores that are safe to be issued to memory, while the store queue keeps the rest. When we described the implementation of WMM in Section 4.4, the store queue is subsumed by the ROB, and a store is moved into the store buffer (from the store queue or ROB) after it is committed from ROB. It should be noted that the partition of store queue and store buffer is logical. A different implementation could use a unified buffer to hold both structures. The GEM5 simulator, which we are using for this evaluation, takes the latter approach. Therefore, in the rest of this section, we do not distinguish between store queue and store buffer, i.e., we assume there is a *unified store buffer* that holds both committed and uncommitted stores.

Since WMM disallows load-store reordering, the simulated WMM processor issues only committed stores to memory. In the simulated GAM processor, the store issue does not need to wait for instruction commit. A store can be issued to memory as long as it satisfies all the requirement listed in the GAM operational definition (Section 3.2.2), i.e., there is no pending interrupts, no older instruction can trigger an exception, all older branches have been resolved, all older memory instructions have resolved their addresses, all older loads with overlapping addresses have got their values and all older stores with overlapping addresses have been issued. Since a store can be issued earlier in GAM, the store-buffer entry occupied by the store can be

recycled earlier, thus reducing the chance that the store buffer becomes full. Since a full store buffer will stall the renaming of a newly fetched store instruction, GAM has the potential performance benefit of reducing stalls at the renaming stage.

Benchmark selection: It should be noted that the reduction on renaming stalls mostly affect single-threaded performance. Therefore, to evaluate the performance difference between WMM and GAM, we can simply run single-threaded benchmarks. We run all reference inputs of all SPEC CPU benchmarks (55 inputs in total) using the GEM5 simulator in full-system mode. For each input, we simulate from 10 uniformly distributed checkpoints. For each checkpoint, we first warm up the memory system for 25M instructions, then warm up the processor pipeline for 200K instructions, and finally simulate 100M instructions in detail. For each benchmark, we summarize the statistics of all the input checkpoints to produce the final performance numbers.

Processor configuration: We reuse the parameters in Table 3.1 as the baseline configuration for both WMM and GAM processors. The buffer sizes in Table 3.1 match those in a Haswell processor. Since the difference between WMM and GAM is related to how often the store buffer becomes full, we also study alternative configurations which have smaller store buffers. The three different store-buffer sizes, i.e., *SB42*, *SB20* and *SB10*, are summarized in Table 4.2.

SB42	The default unified store buffer, i.e., 42 entries.
SB20	A smaller store buffer, i.e., 20 entries.
SB10	A tiny store buffer, i.e., 10 entries.

Table 4.2: Different store-buffer sizes used in the evaluation

Another knob in the processor configuration is when a store-buffer entry can be recycled. The default behavior in GEM5 is to recycle the entry when the store has modified the L1 cache line. An alternative way is to recycle the store-queue entry as soon as the store is issued to the memory system. We refer to the default recycle policy in GEM5 as *LATE*, and the alternative policy as *EARLY*. These two policies are summarized in Table 4.3.

Since we can change the store-buffer size and recycle policy in the comparison between GAM and WMM, we present each specific performance metric by plotting

LATE	Recycle a store-queue entry after the store modifies L1.
EARLY	Recycle a store-queue entry after the store is issued to memory.

Table 4.3: Different recycle policies of store-queue entries

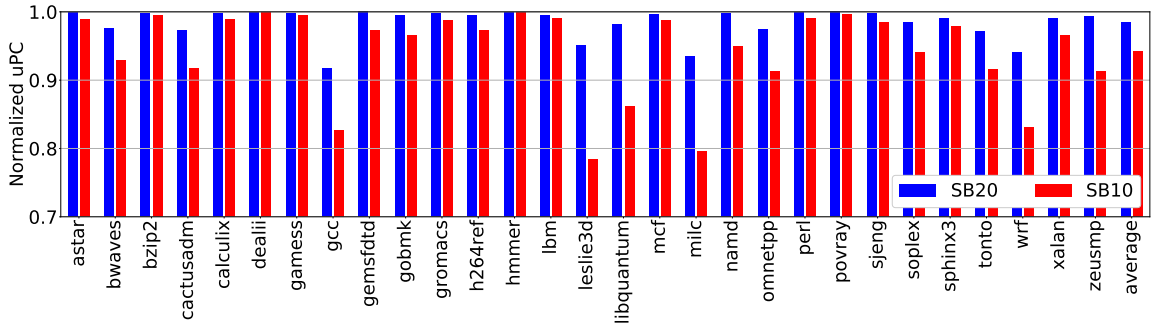
the results with the same recycle policy but different store-buffer sizes into one figure.

4.5.2 Results and Analysis

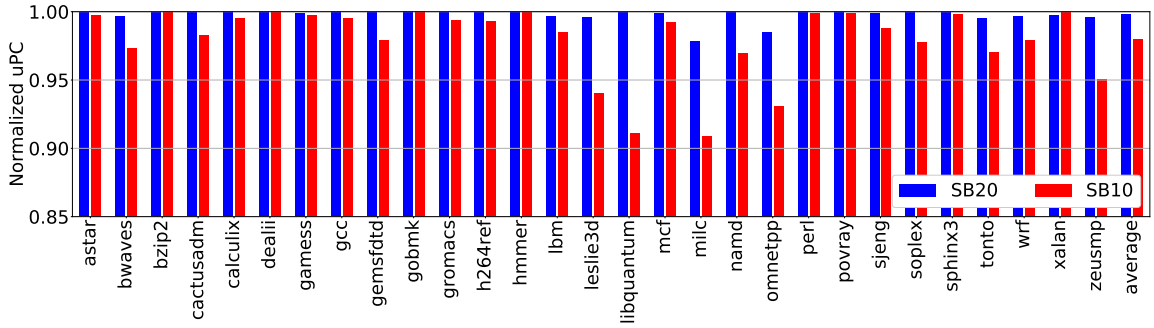
We first study how the store-buffer size affects performance, and then analyze the effects of the load-store reordering which is allowed in GAM but disallowed in WMM.

Performance impact of store-buffer size: Figure 4-13 shows the performance of WMM processors with 20-entry and 10-entry store buffers (i.e., SB20 and SB10) for each recycle policy. The performance numbers are normalized to that of the WMM processor with the default 42-entry store buffer (i.e., SB42) with the same recycle policy. Higher values mean better performance. The rightmost columns are the average numbers across all benchmarks. When the store-buffer size decreases from 42 to 20, there is already observable performance degradation in some benchmarks (e.g., the performance of benchmark gcc drops by 8.3% with the LATE recycle policy), though the average performance drop is still very small (1.6% in case of the LATE policy and almost zero in case of the EARLY policy). The performance degradation becomes more obvious when the store-buffer size decreases from 42 to 10. In this case, though the average performance drop is still insignificant (5.7% with the LATE policy and 2.1% with the EARLY policy), the maximum drop can reach 21.6% with the LATE policy (benchmark leslie3d) and 9.1% with the EARLY policy (benchmark milc).

As explained earlier, the performance drop is caused by the increased renaming stalls when the store buffer becomes full. Figure 4-14 shows the renaming stall cycles in the WMM processor caused by a full store buffer for each store-buffer size and each recycle policy. The stall cycles have been normalized to the execution time of WMM processor with the default store buffer (i.e., WMM-SB42). The renaming stalls are negligible with the default 42-entry store buffer. However, the stalls become



(a) LATE recycle policy

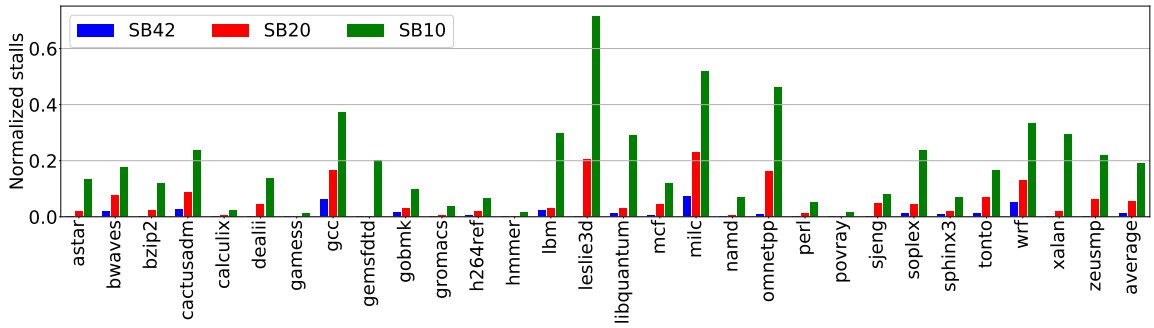


(b) EARLY recycle policy

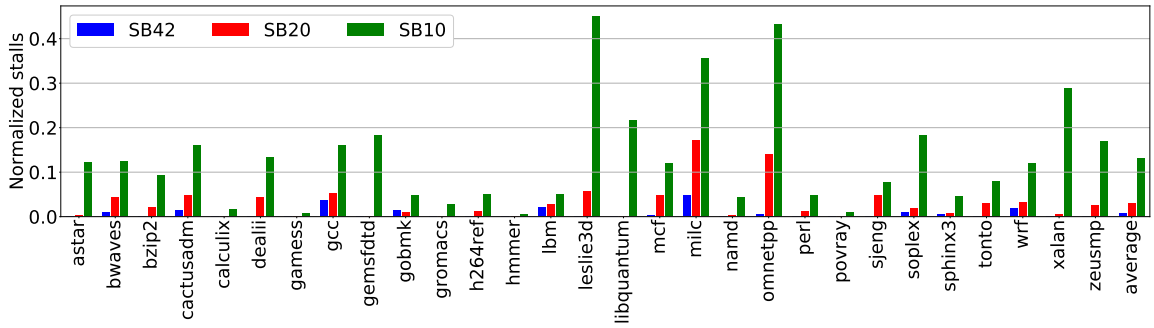
Figure 4-13: Performance (uPC) of WMM-SB20 and WMM-SB10 normalized to that of WMM-SB42

significant when the store buffer has only 10 entries (i.e., SB10). In case of the LATE recycle policy, the renaming stalls can be 19% of the execution time on average, and as high as 72% in benchmark leslie3d, which has the most performance drop in Figure 4-13a. In case the EARLY recycle policy, the renaming stalls can also be 13% of the execution time on average, and as high as 45% in benchmark leslie3d. It should be noted that increased renaming stalls caused by full store buffers may not be proportional to the performance loss. This is because renaming stalls caused by other factors (e.g., a full ROB or slow instruction fetch) may decrease at the meantime.

Another observation is that the EARLY recycle policy makes performance less susceptible to the decrease of the store-buffer size as compared to the LATE policy. This is because the EARLY policy can reduce the time that a store stays in the store buffer, thus reducing the chance that the store buffer becomes full. Figure 4-15 shows the percentage that the EARLY policy has reduced over the LATE policy on the time that a store occupies a store-buffer entry. The reduction is quite significant, e.g., in



(a) LATE recycle policy



(b) EARLY recycle policy

Figure 4-14: Renaming stall cycles due to a full store buffer in WMM in configurations SB42, SQ20 and SQ10. Stall cycles are normalized to the execution time of WMM-SB42.

case of SB10, the EARLY policy can reduce the store-buffer-entry occupation time by 22% on average.

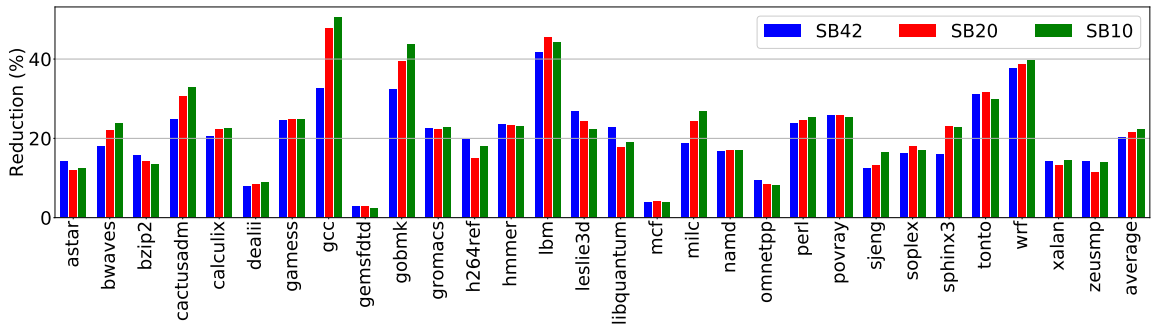
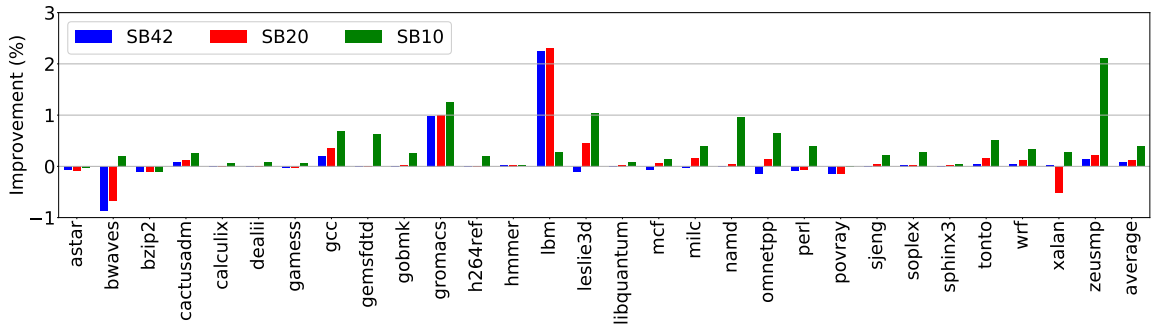


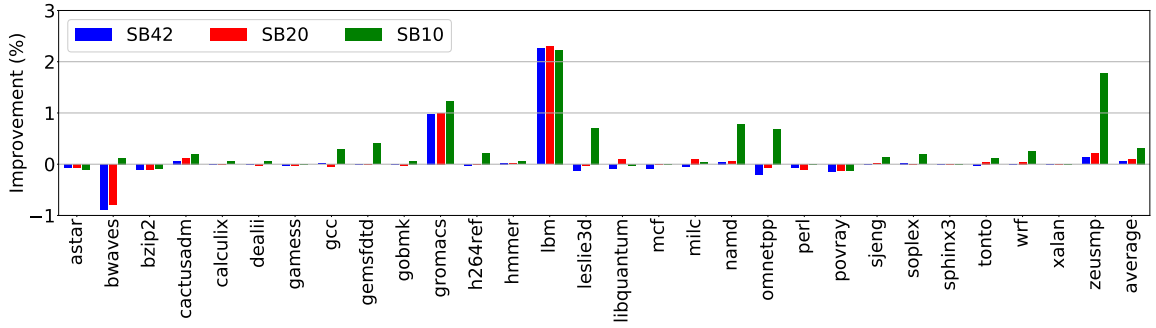
Figure 4-15: Reduction (in percentage) for the EARLY policy over the LATE policy on the time that a store lives in the store buffer in the WMM processor with different store-buffer sizes

Effects of load-store reordering: Since allowing load-store reordering in GAM can make a store issue earlier, we expect a GAM processor to have less renaming stalls

caused by a full store buffer, and thus better performance, than a WMM processor. However, the evaluation results contradict our expectation. Figure 4-16 shows the percentage of performance improvement of GAM over WMM for each store-buffer size and each recycle policy. GAM has little performance improvement over WMM in the default SB42 case (average improvement is 0.1%). This is not surprising because the 42-entry store buffer is sufficiently large and the renaming stalls cased by a full store buffer are infrequent according to Figure 4-14. However, GAM still cannot improve performance when the store buffer becomes smaller. For example, in case of SB10 and the LATE policy, the average improvement is 0.4% and maximum is 2.1%; and in case of SB10 and the EARLY policy, the average improvement is 0.3% and the maximum is 2.2%.



(a) LATE recycle policy

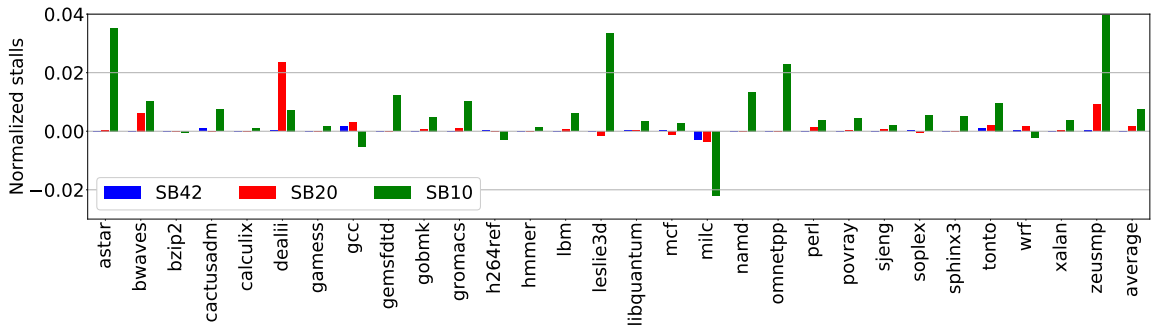


(b) EARLY recycle policy

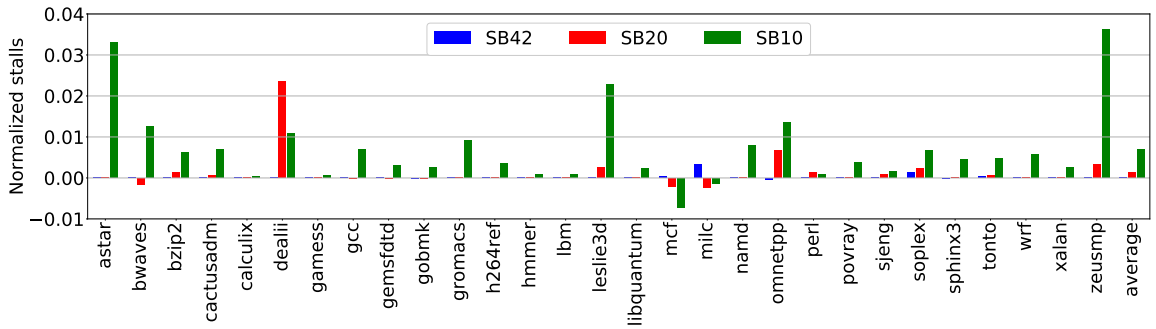
Figure 4-16: Relative performance improvement (in percentage) of GAM over WMM in configurations SB42, SB20 and SB10

In addition, Figure 4-17 shows how much GAM has reduced the renaming stalls due to full store buffers as compared to WMM (for each store-buffer size and each recycle policy). The reduced stall cycles are normalized to the execution time of

WMM-SB42 (with the same recycle policy). As we can see, the reduction on renaming stalls is also negligible. For example, in case of SB10 and the LATE policy, the average reduction is merely 0.7% of the execution time as compared to the average 19% stall rate in Figure 4-14a. And for benchmark lesli3d which stalls 72% of the execution time due to a full store buffer in case of SB10 and the LATE policy (Figure 4-14a), GAM can only bring down the stall time by 3.4% of the execution time.



(a) LATE recycle policy

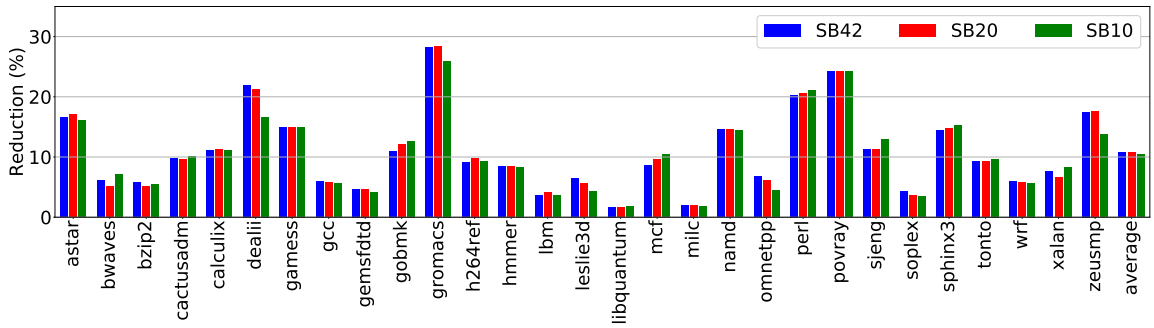


(b) EARLY recycle policy

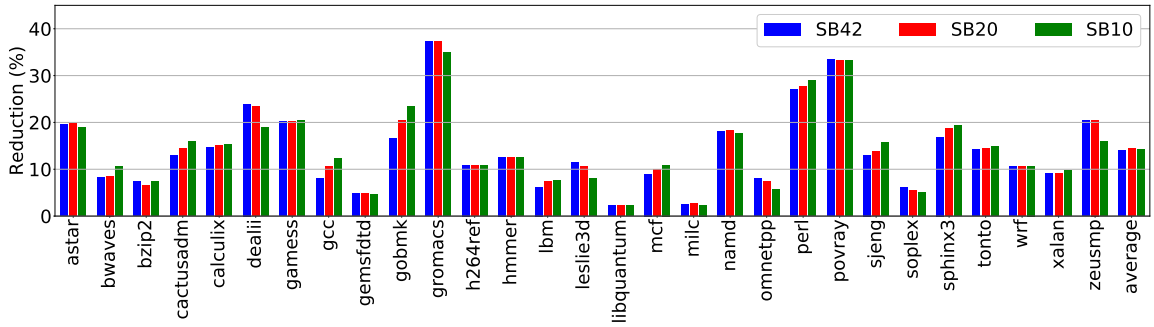
Figure 4-17: Reduced renaming stall cycles caused by full stores buffers for GAM over WMM. Reduced cycles are normalized to the execution time of WMM-SB42.

The ineffectiveness of allowing load-store reordering can be understood by looking at the time that a store lives in the store buffer. Figure 4-18 shows the percentage that GAM has reduced over WMM on the time that a store occupies a store-buffer entry (for each store-buffer size and each recycle policy). We can see an obvious reduction. In case of SB10, the average reduction of GAM over WMM can reach 10% and 14% for the LATE and EARLY policies, respectively. However, the amount of reduction in fact varies a lot across different benchmarks. In case of SB10 with both LATE and EARLY policies, benchmarks gromacs, perl and povray all have

significant reduction on the store-buffer-entry occupation time, but these benchmarks have neither performance loss nor increased renaming stalls when the store-buffer size reduces (Figures 4-13 and 4-14). In contrast, for benchmarks leslie3d and milc, which have significant performance loss and increased renaming stalls when the store-buffer size decreases to SB10 in both recycle policies (Figures 4-13 and 4-14), GAM fails to reduce the store-buffer-entry occupation time. The maximum reduction for these benchmarks in case of SB10 is merely 5.6% with the LATE policy and 8% with the EARLY policy, much below the corresponding average reductions.



(a) LATE recycle policy



(b) EARLY recycle policy

Figure 4-18: Reduction (in percentage) for GAM over WMM on the time that a store lives in the store buffer in configurations SB42, SB20 and SB10, respectively

4.6 Summary

We have identified that the source of complexity in the definitions of GAM is load-store reordering. By forbidding load-store-reordering, we constructed WMM, which has much simpler axiomatic and operational definitions than GAM. In theory, by

allowing load-store reordering, a GAM processor can reduce the time that a store stays in the store buffer, avoid renaming stalls due to full store buffers, and attain better performance than WMM. However, our evaluation shows that the reduction in the store-buffer occupation time is not large enough to get any observable performance benefits. Therefore, by forbidding load-store reordering, WMM strikes a balance between performance and definitional simplicity.

Chapter 5

RiscyOO: a Modular Design of Out of Order Processors[1]

In order to reduce the engineering efforts of implementing different memory models, we need a flexible processor-design framework. The framework should allow us to do *modular refinement*. That is, a module can be refined without knowing the implementation details of the other modules, and the refined modules should still compose with other modules.

Most processors [11, 2, 47, 8] have been designed in a *structural* way, i.e., modules are physical blocks connected by wires. The implementation of one module may make implicit assumptions about the timing of input signals coming from other modules, and thus, the composed processor functions correctly if each module meets its timing assumptions and functionality. Such timing assumptions are difficult to specify, which makes the mechanical verification of timing-assumption violations impossible. In order to avoid rigid timing assumptions, some designers use the latency-insensitive framework, where modules communicate with each other using FIFOs. In such frameworks, a module cannot depend on the timing of other modules, and that significantly improves the flexibility in modular refinement. Although the latency-insensitive framework has proven useful in building hardware accelerators, it is not expressive enough for processors. In processors, a microarchitectural event may need

[1]The work presented in this chapter is done jointly with Andrew Wright and Thomas Bourgeat.

to access and modify the states in multiple modules *atomically*. This is because different microarchitectural events may race with each other in accessing the states inside modules, and such "data races" can make the processor implementation incorrect if an event fails to perform all its accesses atomically with respect to other events. Here we list several race cases in the out-of-order (OOO) processor we have built, and we will discuss a detailed example later in Section 5.1.1:

1. *Speculation bits*: Often an instruction flowing through the execution pipeline carries bits to indicate the speculation events which can cause its squashing. There is a race in the management of these bits, because they are cleared by asynchronous events that show that the speculations were correct.

2. *Memory address disambiguation*: A load in the load queue searches the store queue for forwarding. At the same time, a store in the store queue may get its address filled, and it searches the load queue for detecting memory-dependency violations. There is a race if the addresses are the same.

3. *Partially overlapped addresses*: We stall the execution of a younger load-word if there is an older store-byte on the same word. For example, a load-word searches older stores for detecting such stalls. A store-byte should wake up younger loads stalled by it when the store is written to the L1 cache. There is a race if the load and store are accessing the same word.

4. *Distributed cache coherence protocols*: A race condition arises when a parent is trying to downgrade a child's entry while it is receiving the eviction of the same entry from the child.

A non-modular solution to these race problems is to put all the interacting components in one module. However, doing so in complex designs leads to big monolithic modules.

Since the existing design methodologies cannot satisfy our need, we developed the *Composable Modular Design (CMD)* framework to support modular refinement

and composability of models. The framework permits state changes in multiple modules *atomically*, so it is amenable to processor designs. CMD uses the following two techniques to achieve composability and atomicity:

1. *modules with guarded interface methods*, and
2. *atomic rules* that glue modules together by calling the methods of modules.

In CMD, a module is like an object in an object-oriented programming language, and can be manipulated only by its interface methods. A method provides combinational access and performs atomic updates to the state elements inside the module. In addition, every interface method is *guarded*, that is, there is an implicit guard (a ready signal) on every method which must be true before the method can be invoked (enabled). For example, the guard signal for the enqueue method of a FIFO is simply the not-full condition. CMD subsumes the traditional latency-insensitive framework by admitting systems where each interface method of every module simply enqueues or dequeues FIFOs.

Unlike the structural designs, modules in CMD are manipulated by the special glue logic, i.e., atomic rules, which is a collection of calls to the methods of one or more modules. An atomic rule is like an atomic transaction, which either updates the states of all the called modules or does nothing. Of course, a method can execute only if its guard is true; therefore, the guards of all the methods called by a rule must be true simultaneously. In CMD, each microarchitectural event, which is supposed to happen in a single cycle, is expressed as an atomic rule. Atomicity ensures that the refinement of a module does not affect how the module is used by other rules.

Using the CMD framework, we developed a parameterized and modular OOO processor, *RiscyOO*, which is released at `https://github.com/csail-csg/riscy-OOO` under the MIT License. We will base off from RiscyOO to evaluate different memory models in the next chapter.

In the following, we first describe the CMD framework in Section 5.1, and then introduces the core microarchitecture and the memory system of RiscyOO in Sections 5.2 and 5.3, respectively. We evaluate the performance of RiscyOO in Sec-

tion 5.4.

5.1 Composable Modular Design (CMD) Framework

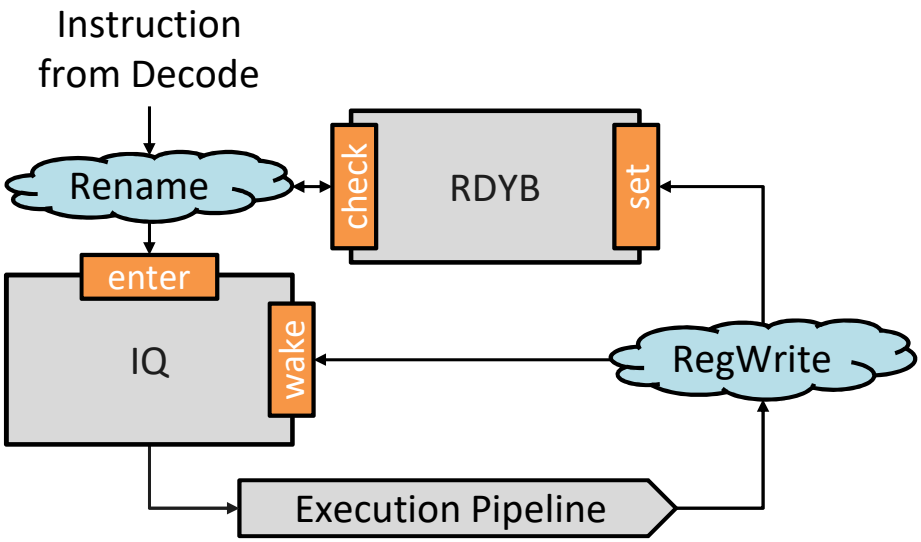
As mentioned earlier, latency-insensitive frameworks are insufficient for processor designs because of races between microarchitectural events. We first study an example race case which involves the instruction-issue queue (IQ) in the OOO processor, and then show how CMD maintains atomicity and solves the race problem by extending module interfaces with concurrency properties.

5.1.1 Race between Microarchitectural Events

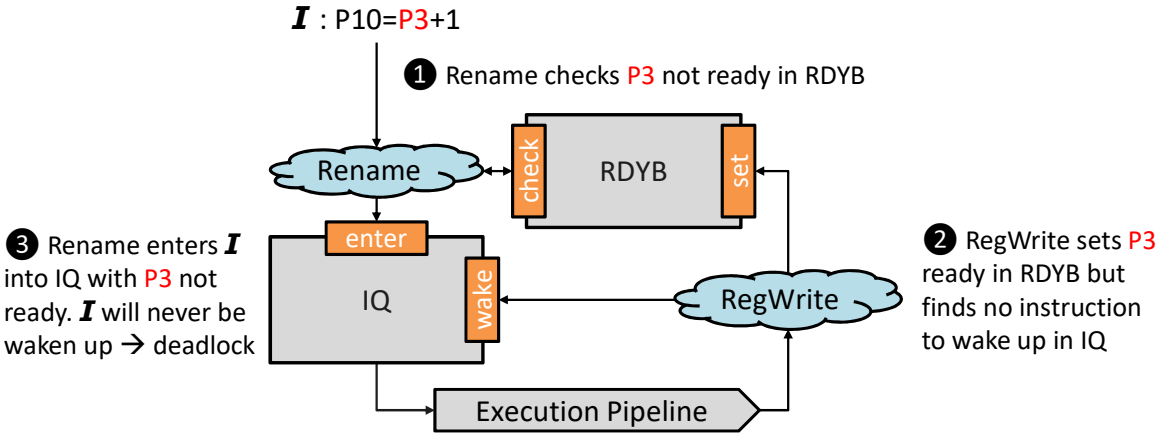
Figure 5-1a shows the modules and microarchitectural events that participate in a race in the OOO processor. Module IQ keeps unissued instructions and tracks whether their (physical) source registers are ready or not. Module RDYB keeps one bit for each physical register, indicating whether the register has valid data or not. Microarchitectural event Rename gets a new instruction from the decode stage, does renaming, *checks* RDYB to see if the source registers of the instructions are ready, and *enters* the instruction with the register-ready bits into IQ. Microarchitectural event RegWrite happens at the end of execution pipeline when an instruction gets the value for its destination register. The event *sets* the corresponding bit in RDYB and *wakes* up dependent instructions in IQ. (There are other actions performed by the two events, but they are unrelated to the race case.)

Both events are accessing the states in IQ and RDYB, and thus, form a race. If Rename does not happen atomically with respect to RegWrite, then the race can lead to deadlock in the processor. Consider the case in Figure 5-1b. Rename is processing instruction I with physical source register $P3$, while RegWrite is writing the same register $P3$. It is possible that Rename first checks RDYB and finds $P3$ not ready, then RegWrite happens and cannot wake up instruction I which is not yet in IQ, and finally Rename enters I into IQ. In this case, I will be stuck in IQ forever, i.e., the processor deadlocks.

It is difficult for latency-insensitive frameworks to keep the atomicity of events and resolve this race problem. A structural solution is to introduce bypass wires either in RDYB (to have Rename see the updated register-ready bits) or in IQ (to have RegWrite wake up instruction I). However, the bypass wires break latency insensitivity and reduce composability.



(a) Modules and microarchitectural events involved in the race. Modules IQ and RDYB are represented by blocks, while microarchitectural events Rename and RegWrite are represented by clouds. Event Rename calls method *check* of RDYB and method *enter* of IQ. Event RegWrite calls method *set* of RDYB and method *wake* of IQ.



(b) Operation sequence that leads to deadlock. Event Rename is processing instruction I with source register P3 while event RegWrite is writing P3.

Figure 5-1: Race between microarchitectural events Rename and RegWrite in an OOO processor

5.1.2 Maintaining Atomicity in CMD

To resolve the race problem in Figure 5-1, CMD expresses events Rename and RegWrite as two separate rules and guarantees the atomicity of each rule. That is, event Rename will be expressed as rule Rename which calls method *check* of RDYB and method *enter* of IQ, and event RegWrite will be expressed as rule RegWrite which calls method *set* of RDYB and method *wake* of IQ. Figure 5-2 shows the pseudo codes of the module interface methods and the atomic rules. Here we use the syntax of Bluespec SystemVerilog (BSV) [3].

However, enforcing atomicity is challenging because the two rules manipulate the same states. In this case, it has to be ensured that the rules appear to execute one after another. Whether two rules can execute concurrently and in which order depend on the properties of the called methods. Thus, for each module, we use a *Conflict Matrix (CM)*, which specifies which methods of the module can be called concurrently. The relation between each pair of methods may be described as follows:

- *conflict-free*: the methods do not manipulate the same states, and thus, can be called concurrently;
- *conflicting*: the methods cannot be called in the same cycle, and thus, it is illegal for a rule to call conflicting methods;
- *happen-before*: the methods can be called concurrently but functionally they behave as if one executed before the other. This involves bypass logic inside the module in case a write method happens before a read method.

Given the CMs of all the modules, it is straightforward to derive the concurrency relation between every pair of rules. Consider two rules $R1$ and $R2$. If any method called by $R1$ is conflict-free with any method called by $R2$, then $R1$ is conflict-free with $R2$. Otherwise, if any method called by $R1$ either happens before or conflicts free with any method called by $R2$, then $R1$ happens before $R2$, and vice versa. $R1$ is conflicting with $R2$ in all other cases. Support for CMD requires a stall signal in the glue logic to suppress the execution of one rule in a pair of conflicting rules.

```
interface IQ;
  method Action enter(decodedRenamedInst, rdy1, rdy2);
  method Action wake(dstReg);
  // other methods ...
endinterface
IQ iq <- mkIQ; // IQ module

interface RDYB;
  method Bool check1(srcReg1); // check reg ready bit
  method Bool check2(srcReg2);
  method Action set(dstReg); // set reg ready bit
  // other methods ...
endinterface
RDYB rdyb <- mkRDYB; // RDYB module

rule doRename; // Rename event
  // dInst has been decoded and renamed
  Bool rdy1 = rdyb.check1(dInst.src1);
  Bool rdy2 = rdyb.check2(dInst.src2);
  iq.enter(dInst, rdy1, rdy2);
  // other actions ...
endrule

rule doRegWrite; // RegWrite event
  let wbInst <- exec_pipeline.resp(); // inst leaves exe pipeline
  iq.wake(wbInst.dst);
  rdyb.set(wbInst.dst);
  // other actions ...
endrule
```

Figure 5-2: Pseudo codes for the interfaces of IQ and RDYB and the atomic rules of Rename and RegWrite

Back to the race problem in Figure 5-1, if there is no bypass wire in the implementation of module IQ or module RDYB, then the CM of IQ will have method *wake* happen before *enter*, and the CM of RDYB will have method *check* happen before *set*. In this case, rules Rename and RegWrite conflict with each other and must be executed one by one. CMD will generate a stall signal to enforce such a schedule. To make rules RegWrite and Rename execute concurrently, one solution is to change the CM of RDYB to have method *set* happen before *check*. That is, the implementation

of RDYB contains a bypass wire from method *set* to *check*. We will show shortly that this bypass wire can be generated implicitly in CMD. In this case, RegWrite will appear to execute before Rename.

5.1.3 Expressing CMD in Hardware Description Languages (HDLs)

We expressed the CMD framework in Bluespec SystemVerilog (BSV). The BSV compiler automatically (1) derives the CM for each module implementation, (2) derives the concurrency relations between each pair of rules according to the CMs, and (3) generates stall signals in the glue logic. *The compiler statically resolves all the dynamic concurrency issues, making it possible to apply mechanical verification techniques to designs.*

It is possible to use other HDLs to express CMD, but then we may not get all the benefits of the automatic concurrency analysis done by the BSV compiler.

5.1.4 CMD Design Flow

We develop designs in two phases. We first focus on functionality and do not try to get maximum hardware concurrency. After this phase, we often discover that two rules are conflicting, and this affects performance adversely. To execute such rules concurrently invariably requires introduction of bypass logic. In CMD, one does not need to write bypass logic explicitly. Instead, one can specify the order in which rules should execute, from which one can derive the desired CM of each module. Rosenband and Arvind [121] have given a systematic way to transform the module implementation to satisfy a given CM using Ephemeral History Registers (EHRs) which implicitly introduce bypass logic. This transformation does not affect the functional correctness of the overall design.

5.1.5 Modular Refinement in CMD

When a refinement on a module does not affect the interface methods or the CM, the local correctness of the module guarantees the preservation of the overall correctness. If the CM of the refined module is changed, the BSV compiler can re-analyze the relations between rules and generate new stall signals to keep the design functionally correct.

In some cases, a refinement may entail several modules simultaneously or may add new methods to a module for increased functionality. Any changes in interface methods imply that the rules that call these methods have to be changed. However, making this change does not require knowing the internal details of other modules, because other modules have been encapsulated by their interface specifications.

In summary, by employing modules with guarded interfaces and atomic rules, CMD is able to provide strong composability and atomicity.

5.2 Out-of-Order Core of RiscyOO

RiscyOO is a parameterized out-of-order superscalar cache-coherent multiprocessor built using CMD. Figure 5-3 shows the microarchitecture of the OOO core of RiscyOO. The salient features of our OOO microarchitecture are the physical register file (PRF), reorder buffer (ROB), a set of instruction issue queues (IQ) – one for each execution pipeline (only two are shown to avoid clutter), and a load-store unit, which includes LSQ, non-blocking D cache, etc.

The Fetch module contains three different branch predictors (branch target buffer, tournament direction predictor, and return address stack) and it enters instructions into ROB and IQs after renaming. We use *epochs* for identifying wrong path instructions. Instructions can be flushed because of branch mispredictions, load miss-speculations on memory dependencies, and page faults on address translation. Each instruction that may cause a flush is assigned a *speculation tag* [144], and the subsequent instructions that can be affected by it carry this tag. These speculation tags are managed as a finite set of bit masks which are set and cleared as instruction

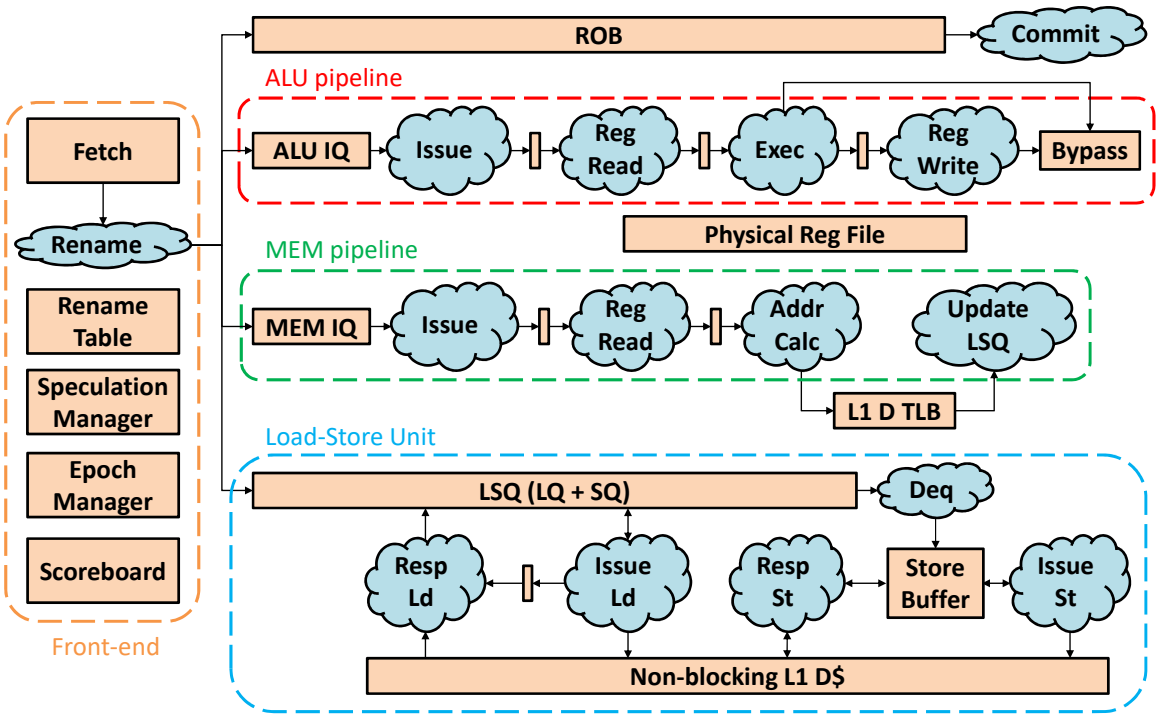


Figure 5-3: Top-level moduels and rules of the OOO core. Modules are represented by rectangles, while rules are represented by clouds. The core contains four execution pipelines: two for ALU and branch instructions, one for memory instructions, and one for floating point and complex integer instructions (e.g., multiplication). Only two pipelines are shown here for simplicity.

execution proceeds. When an instruction can no longer cause any flush, it releases its speculation tag, and the corresponding bit is reset in the bit masks of subsequent instructions so that the tag can be recycled. To reduce the number of mask bits, we only assign speculation tags to branch instructions, while deferring the handling of interrupts, exceptions and load speculation failures until the commit stage. Every module that keeps speculation-related instructions must keep speculation masks and provide a *correctSpec* method to clear bits from speculation masks, and a *wrongSpec* method to kill instructions. We do not repeatedly describe these two methods in the rest of this section.

We also maintain two sets of PRF register-ready bits to reduce latency between dependent instructions. The true ready bits are used in the Reg-Read stage to stall instructions. Another set of ready bits (Scoreboard in Figure 5-3) are set optimistically when it is known that the register would be set by an older instruction with small

predictable latency. These optimistic bits are maintained as a scoreboard, and are used when instructions are entered in IQ and can improve throughput for instructions with back-to-back dependencies.

In Figure 5-3, boxes represent the major modules in the core, while clouds represent the top-level rules. Next we describe the interfaces of all the salient modules, and some important rules. We will also use the LSQ as an example to show how we implement modules according to conflict matrices.

5.2.1 Interfaces of Salient Modules

The Front-End

The main modules in the front-end are Fetch, Epoch Manager, Speculation Manager and the Renaming Table. Fetch is an in-order pipeline that does PC translation, L1 I access, instruction decode and all types of branch predictions. The Epoch Manager keeps the epoch, detects wrong-path instructions at Rename stage, and recycle unused epoch values. The Speculation Manager assigns tags to branch instructions at Rename stage. The Renaming Table keeps the mapping from architectural registers to physical registers. The Scoreboard keeps the optimistic register-ready bits as mentioned earlier. The front-end is superscalar, and can supply multiple instructions to the execution engine every cycle.

Fetch: Figure 5-4 shows the in-order pipeline of the Fetch module. It contains PC, I TLB, I cache, and all the branch predictors. The pipeline first sends PC to I TLB for translation and predicts the next PC, then accesses I cache to fetch instructions, and finally decodes the instructions, and predicts the directions for conditional branches and the targets for indirect jumps. The module also attaches an *epoch* to every fetched instructions at PC translation time to identify wrong-path instructions in later stages. This module has the following methods:

- *setWaitRedirect*: stops fetching instructions.
- *redirect*: changes the PC register, increments the epoch, and resumes instruction fetch in case it has been stopped before.

- *trainPredictor:* updates the branch predictors for training.

The Fetch module also exports interface methods to connect I TLB to an L2 TLB, and connect I cache to the L2 cache. The L2 TLB can perform hardware page walk, which is not shown in Figure 5-3.

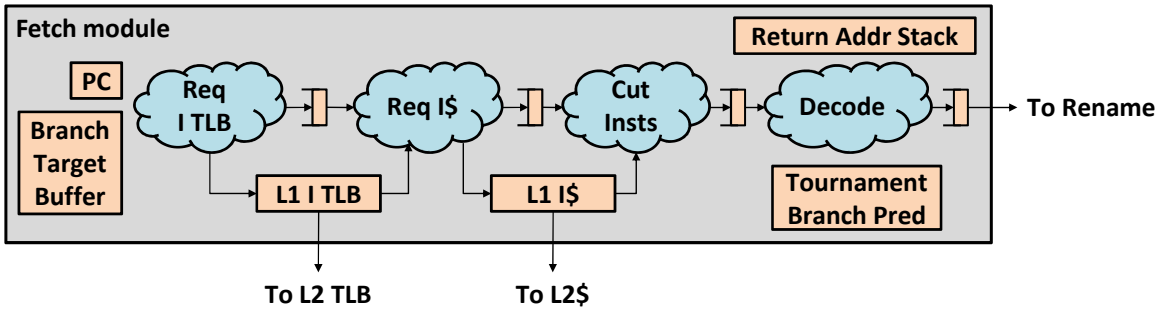


Figure 5-4: In-order pipeline of the Fetch module

Epoch Manager: The epoch is incremented each time a later stage redirects the control flow. Since the epoch can only be of finite bit width, this module also recycles unused epoch values. This module has the following methods:

- *check:* checks whether an incoming instruction (from Fetch) is a wrong-path instruction or not.
- *updatePrev:* recycles unused epoch based on the epoch of the instruction from Fetch.
- *increment:* increments the epoch.

Speculation Manager: The Speculation Manager module assigns speculation tags to branch instructions and masks to every instruction before the instructions are entered into ROB and IQ. It has the following methods:

- *specMask:* returns the appropriate speculation mask for the instruction at the Rename rule.
- *claimSpecTag:* checks out a speculation tag for an instruction at the Rename rule which may cause ROB flush in the future.

Renaming Table: The renaming table holds a non-speculative architectural rename mapping, together with the deltas on this mapping for each instruction that is entered in the ROB. Since the deltas also need to be flushed in case of speculation failure, each delta also carries the speculation mask. This module has the following methods:

- *getRename:* returns the renamed physical registers for source and destination registers of instructions at the Rename rule.
- *claimRename:* record the delta on rename mapping made by the instruction at the Rename rule.
- *commit:* is called when an instruction is committed from ROB. It commits the delta of this instruction in the renaming map, and frees the original physical register in the modified architectural rename mapping entry.

As we can see, every module that keeps speculation-related instructions must keep speculation masks and provide the *correctSpec* and *wrongSpec* methods to handle the two cases. In the following, we will not repeat the functionality of these two methods. It should also be noted that an alternative implementation using checkpoints of the rename mapping can use a similar interface.

Scoreboard: The Scoreboard provides the following methods to access the optimistic register-ready bits:

- *setReady:* sets a physical register to have valid data.
- *setBusy:* sets a physical register to have invalid data.
- *lookup:* returns if a physical register has valid data or not.

The Execution Engine

The execution engine consists of multiple parallel execution pipelines, and instructions can be issued from the IQs in different pipelines simultaneously. The number of execution pipelines is parameterized. Though instructions execute and write the register file out of order, the program order is always kept by ROB. The Physical

Register File (PRF) is shared by all execution pipelines, and it stores a register-ready bit for each physical register. Unlike the Scoreboard, the ready bit can be set only when the data is written to the physical register. Each IQ is responsible for tracking read-after-write (RAW) dependencies, and issuing instructions with all source operands ready, as discussed in Section 5.1.1.

ROB: ROB keeps in program order all in-flight instructions which have been renamed but not yet committed. Each entry has a PC, instruction type, a speculation mask, a completion bit, detected exceptions, index to LSQ and page-fault virtual address for memory instructions, and a few more miscellaneous status bits. Instructions that manipulate system special registers (CSRs in RISC-V) overload the fault-address field as a data field. In a different design, it may be possible to reduce the width of ROB entries by keeping these data or address fields in a separate structure, without affecting the ROB interface. ROB can use a single register to hold CSR data, because we allow only one CSR instruction in flight, and another register to store the oldest faulting address. However, LSQ may need to keep virtual addresses in each of its slots, because in RISC-V, a memory access can cause exceptions even after address translation, and the virtual address needs to be written into a CSR in case of an exception. ROB has the following methods:

- *enq:* enqueue a new instruction to ROB.
- *getEnqIndex:* returns the index for the slot where the next entry will be allocated.
- *deq:* dequeue the oldest instruction from ROB.
- *first:* returns the information associated with the instruction in the commit slot of ROB.
- *setNonMemCompleted:* marks the instruction at the specified ROB index to have completed (so that it can be committed later).
- *setAfterTranslation*: is called when a memory instruction has finished address translation. It tells the ROB whether the memory instruction can only access

memory non-speculatively (so ROB will notify LSQ when the instruction reaches the commit slot), and also marks it complete in case of a normal store.

- *setAtLSQDeq:* is called when load or memory-mapped store is dequeued from LSQ. It marks exception or load speculation failure or complete.

Instruction Issue Queue (IQ): Each IQ has the following methods:

- *enter:* enters a new instruction to reservation station in the Rename rule.
- *issue:* removes and returns an instruction whose source registers are all ready.
- *wakeup:* is called whenever a physical register is updated.

Bypass: Instead of bypassing values in an ad-hoc manner, we have created a structure to bypass ALU execution results from the Exec and Reg-Write rules in the ALU pipeline to the Reg-Read rule of every pipeline. It provides a *set* method for each of the Exec and Reg-Write rules to pass in the ALU results, and a *get* method for each of the Reg-Read rules to check for the results passed to the *set* methods in the same cycle. These methods are implemented such that $set < get$.

Load-Store Unit

The load-store unit consists of an LSQ, a store buffer (SB) and a non-blocking L1 D cache. LSQ contains a load queue (LQ) and a store queue (SQ) to keep in-flight loads and stores in program order, respectively. The SB holds committed stores that have not been written into L1 D.

When a memory instruction leaves the front-end, it enters the IQ of the memory pipeline and allocates an entry in LQ or SQ. A fence instruction will not enter IQ, but will allocate an entry in SQ. The memory pipeline computes the virtual address in the Addr-Calc stage and then sends it to L1 D TLB for address translation (see Figure 5-3). When the translation result is available, the Update-LSQ stage checks if there is a page fault, and if the memory instruction is accessing the normal cached memory region or the memory mapped IO (MMIO) region. It also updates the ROB,

and LQ or SQ entry for this instruction accordingly. In case of a normal load, it is executed speculatively either by sending it to L1 or by getting its value from SB or SQ; otherwise, LSQ will provide a read-to-(re)issue load which will be executed speculatively. However, it may have to be stalled because of fences, partially overlapped older stores, and other reasons. Thus, LQ needs internal logic that searches for ready-to-issue loads every cycle. Speculative loads that violate memory dependency are detected and marked as to-be-killed when the Update-LSQ stage updates the LSQ with a store address.

Unlike normal loads, MMIO accesses and atomic accesses (i.e., load reserve, store conditional and read-modify-write) can only access memory when the instruction has reached the commit stage.

Normal stores can be dequeued from SQ sequentially after they have been committed from ROB. In case of TSO, there is no SB, and only the oldest store in SQ can be issued to L1 D provided that the store has been committed from ROB. It can be dequeued from SQ only when the store hits in the cache. Though there can be only one store request in L1 D, SQ can issue as many store-prefetch requests as it wants. Currently we have not implemented this feature. In case of a weak memory model like WMM or GAM, the dequeued store will be inserted into a store buffer (SB) without being issued to L1 D. SB can coalesce stores for the same cache line and issue stores to L1 D out of order.

Normal loads can be dequeued from LQ sequentially after they get the load values and all older stores have known addresses, or after they become faulted. A dequeued load marks the corresponding ROB entry as complete, exception, or to-be-killed. In order to check easily if all older stores have known addresses or not, LSQ maintains a pointer which points to the oldest SQ entry without an address, and there is a rule constantly trying to advance this pointer.

LSQ: As mentioned earlier, loads and stores are kept in separate queues. In order to observe the memory dependency between loads and stores, each load in LQ keeps track of the index of the immediately preceding SQ entry. In case a load has been issued from LQ, the load needs to track whether its value will come from the cache

or by forwarding from an SQ entry or an SB entry. When a load tries to issue, it may not be able to proceed because of fences or partially overlapped older stores. In such cases, the load records the source that stalls it, and retries after the source of the stall has been resolved. In case of ROB flush, if a load, which is waiting for the memory response, is killed, then this load entry is marked as waiting for a wrong path response. Because of this bit, we can reallocate this entry to a new load, but not issue it until the bit is cleared. The LSQ module has the following methods:

- *enq:* allocates a new entry in LQ or SQ for the load or store instruction, respectively, at the Rename stage.

- *update:* is called after a memory instruction has translated its address and, in case of a store, the store has computed its data. This fills the physical address (and store data) into the corresponding entry of the memory instruction. In case the memory instruction is a store, this method also searches for younger loads that violate memory dependency ordering and marks them as to-be-killed. Depending upon the memory model, more killings may have to be performed. We have implemented the killing mechanisms for TSO and WMM; it is quite straightforward to implement other weak memory models.

- *getIssueLd:* returns a load in LQ that is ready to issue, i.e., the load does not have any source of stall and is not waiting for wrong path response.

- *issueLd:* tries to issue the load at the given LQ index. This method will search older stores in SQ to check for forwarding or stall. The method also takes as input the search result on store buffer, which is combined with the search result on store queue to determine if the load is stalled or can be forwarded, or should be issued to cache. In case of stall, the source of stall will be recorded in the LQ entry.

- *respLd:* is called when the memory response or forwarding data is ready for a load. This returns if the response is at wrong path, and in case of a wrong path response, the waiting bit will be cleared.

- *wakeupBySBDeq:* is called in the WMM implementation when a store buffer entry is dequeued. This removes the corresponding sources of stall from load queue entries.

- *cacheEvict:* is called in the TSO implementation when a cache line is evicted from L1 D. This searches for loads that read stale values which violate TSO, and marks them as to-be-killed.

- *setAtCommit:* is called when the instruction has reached the commit slot of ROB (i.e., cannot be squashed). This enables MMIO or atomic accesses to start accessing memory, or enables stores to be dequeued.

- *firstLd/firstSt:* returns the oldest load/store in LQ/SQ.

- *deqLd/deqSt:* removes the oldest load/store from LQ/SQ.

Store Buffer: The store buffer has the following methods:

- *enq:* inserts a new store into the store buffer. If the new store address matches an existing entry, then the store is coalesced with the entry; otherwise a new buffer entry is allocated.

- *issue:* returns the address of an unissued buffer entry, and marks the entry as issued.

- *deq:* removes the entry specified by the given index, and returns the contents of the entry.

- *search:* returns the content of the store buffer entry that matches the given address.

L1 D Cache: The L1 D Cache module has the following methods:

- *req:* request the cache with a load address and the corresponding load queue index, or a store address and the corresponding store buffer index.

- *respLd:* returns a load response with the load queue index.

- *respSt:* returns a store buffer index. This means that the cache has exclusive permission for the address of the indexed store buffer entry. The cache will remained locked until the *writeData* method is called to write the store data of the indexed store buffer entry into cache.

- *writeData:* writes data to cache; the data should correspond to the previously responded store buffer index.

L1 D cache also has the interface to connect to the L2 cache.

L1 D TLB: The L1 D TLB is non-blocking and has the following methods:

- *req:* request the TLB with a virtual address and the corresponding LSQ index.

- *resp:* returns the translation result with the LSQ index.

- *flush:* starts flushing the TLB contents.

- *flushDone:* returns true if there is no pending flushing.

It also has methods to be connected to the L2 TLB.

5.2.2 Connecting Modules Together

Our CMD framework uses rules to connect modules together for the OOO core. The rules call methods of the modules, and this implicitly describes the datapaths between modules. More importantly, the rules are guaranteed to fire atomically, leaving no room for concurrency bugs. The challenging part is to have rules fire in the same cycle. To achieve this, the conflict matrix of the methods of each module has to be designed so that the rules are not conflicting with each other. Once the conflict matrix a module is determined, there is a mechanical way to translate an initial implementation whose methods conflict with each other to an implementation with the desired conflict matrix. It should be noted that though the rules fire in the same cycle, they still behave as if they are firing one after another.

There are about a dozen rules at the top level. Instead of introducing all the rules, we explain two rules in detail to further illustrate the atomicity issue. Figure 5-5 shows

the *doIssueLd* and *doRespSt* rules. The *doIssueLd* rules first gets a ready-to-issue load from the LSQ module. Then it searches the store buffer for possible forwarding or stall (due to partially overlapped entry). Next it calls the *issueLd* method of LSQ to combine the search on the store buffer and the search on the store queue in LSQ to derive whether the load can be issued or forwarded. The *respSt* rule first gets the store response from the L1 D cache. Then it dequeues the store from the store buffer and writes the store to L1 D. Finally, it wakes up loads in LSQ that has been stalled by this store earlier.

```
rule doIssueLd;
  let load <- lsq.getIssueLd;
  let sbSearchResult = storeBuffer.search(load.addr);
  let issueResult <- lsq.issueLd(load, sbSearchResult);
  if(issueResult matches tagged Forward .data) begin
    // get forwarding, and save forwarding result in a FIFO which will
    be processed later by the doRespLd rule
    forwardQ.enq(tuple2(load.index, data));
  end else if(issueResult == ToCache) begin
    // issue to cache
    dcache.req(Ld, load.index, load.addr);
  end // otherwise load is stalled
endrule
rule doRespSt;
  let sbIndex <- dcache.respSt;
  let data, byteEn <- storeBuffer.deq(sbIndex);
  dcache.writeData(data, byteEn);
  lsq.wakeupBySBDeq(sbIndex);
endrule
```

Figure 5-5: Rules for LSQ and Store Buffer

Without our CMD framework, when these two rules fire in the same cycle, concurrency bug may arise because both rules race with each other on accessing states in LSQ and the store buffer. Consider the case that the load in the *doIssueLd* rule has no older store in LSQ, but the store buffer contains a partially overlapped entry which is being dequeued in the *doRespSt* rule. In this case, the two rules race on the valid bit of the store buffer entry, and the source of stall for the load. Without CMD, if we pay no attention to the races here and just let all methods read the register states at

the beginning of the cycle, then the *issueLd* method in the *doIssueLd* rule will records the store buffer entry as stalling the load, while the *wakeupBySBDeq* method in the *doRespLd* rule will fail to clear the stall source for the load. In this case, the load may be stalled forever without being waken up for retry. With our CMD framework, the methods implemented in the above way will lead the two rules to conflict with each other, i.e., they cannot fire in the same cycle. To make them fire in the same cycle, we can choose the conflict matrix of LSQ to be $issueLd < wakeupBySBDeq$, and the conflict matrix of the store buffer to be $search < deq$. In this way, the two rules can fire in the same cycle, but rule $doIssueLd$ will appear to take effect before rule $doRespSt$.

5.2.3 Module Implementations

As mentioned in Section 5.1.4, we do not care about CMs in the first phase of implementation, i.e., we only implement the functionality. In the second phase, we need to choose a proper CM for each module to optimize for performance. the main consideration is to minimize the combinational delay without hurting microarchitectural performance (e.g., IPC). After choosing a CM, we need to implement the module to adhere to the CM. The first obvious way is to build explicitly the bypass logic as in a traditional design. In RiscyOO, we often use an alternative approach proposed by Rosendband and Arvind [121, 120] to enforce the CM of a module. We use the LSQ as an examples to illustrate this approach.

LSQ: Module Implementation using Ephemeral History Registers (EHRs)

Figure 5-6 shows the internal state elements and rules of LSQ. The LQ and SQ are two circular buffers. The internal rule *findIssue*[2] finds a ready-to-issue rule searches through the LQ to find a ready-to-issue load and enqueues the LQ index into FIFO *readyQ*. The rule also sets a bit in the LQ entry so that the rule will not select this entry again in the next cycle. The *getIssueLd* method will dequeue an LQ index

[2]The RiscyOO implementation further splits this rule into two rules. We skip this detail here for simplicity.

from *readyQ*, and reset the bit in the LQ entry (so the load can be reissued in case it gets stalled this time). *validatePtr* is the pointer that points to the oldest SQ entry which does not have a valid address, and the internal rule *validateStAddr* advances this pointer.

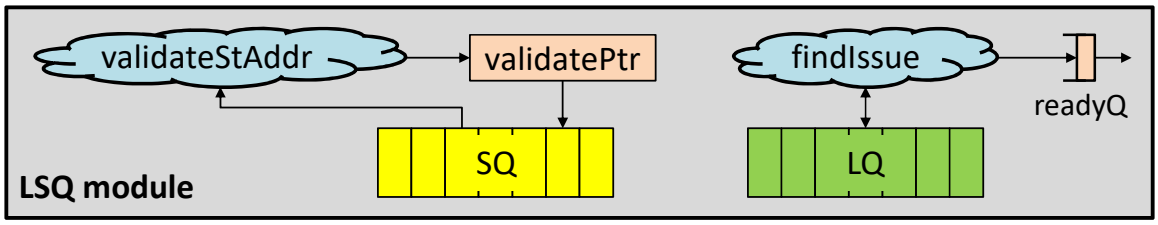


Figure 5-6: Internal states and rules of LSQ

The two internal rules and the interface methods (Section 5.2.1) of LSQ race with each other on the state elements like LQ and SQ entries. To make the rules and methods fire concurrently in the same cycle, a simple way of choosing CM is to put the concurrent rules and methods in a total order. We pick the following total order for LSQ ($A < B$ means A happens before B in CM):

- *findIssue* < *deqLd* < *validateStAddr* < *cacheEvict* < *update* < *getIssueLd* < *issue* < *wakeupStallBySBDeq* < *setAtCommit* < *respLd* < {*enqLd, enqSt*} < *correctSpec*

Methods *enqLd* and *enqSt* are put together because we make them conflicting with each other[3]. In fact only one of them can be called in a cycle because there is only a single memory pipeline. Method *wrongSpec* which is not shown above will be conflicting with every method or rule. It should be noted that the happen-before relations in CM does not need to be transitive or form a total order. We pick a total order here just for simplicity, and we find it sufficiently good in practice.

To enforce this CM in the module implementation, we can simply turn each state element into an Ephemeral History Register (EHR) [120]. For example, each field of an LQ or SQ entry will be an EHR, and *validatePtr* will also be an EHR. An EHR is simply a register with multiple ports, and each port can be read and written. Any

[3] *enqLd* is conflicting with *enqSt* because each load needs to track the immediately preceding store.

read or write accesses on port i will happen before any accesses on port j if $i < j$, i.e., a write on port i will be visible to a read on port j. Read will happen before write on the same port. The final state is determined by the write with the maximum port ID.

Given the properties of EHRs, we can enforce the happen-before relations in CM simply by having each method use a unique port ID to access the EHRs. That is, method *findIssue* acceses only port 0 of every EHR, method *deqLd* accesses only port 1, method *validateStAddr* accesses only port 2, and so on.

5.3 Cache-Coherent Memory System of RiscyOO

We have connected the OOO cores together to form a multiprocessor as shown in Figure 5-7. The L1 caches communicate with the shared L2 via a cross bar, and the L2 TLBs sends uncached load requests to L2 via another cross bar to perform hardware page walk. The page-walk load requests follow a miss-no-allocate policy in L2, because each core already has a large L2 TLB and a translation cache to cache the page-walk results. All memory accesses, including memory requests to L1 D made by memory instructions, instruction fetches, and loads to L2 for page walk, are coherent.

We implemented an MESI coherence protocol which has been formally verified by Vijayaraghavan et al. [140]. According to the protocol, each link between L1 and L2 (i.e., the links between L1s and the cross bar, and the link between L2 and the cross bar) contains three independent FIFOs to transfer (1) upgrade requests from the L1, (2) downgrade responses from the L1, and (3) upgrade responses and downgrade requests from the LLC, respectively. Next we show the details of the L2 cache as another example of using CMD. (L1 caches have a similar microarchitecture, so we do not repeat them.)

5.3.1 L2 Cache

Figure 5-8 shows the microarchitecture of the L2 cache. L2 contains two submodules, MSHR (Miss Status Handling Registers) and the cache-access pipeline. MSHR keeps

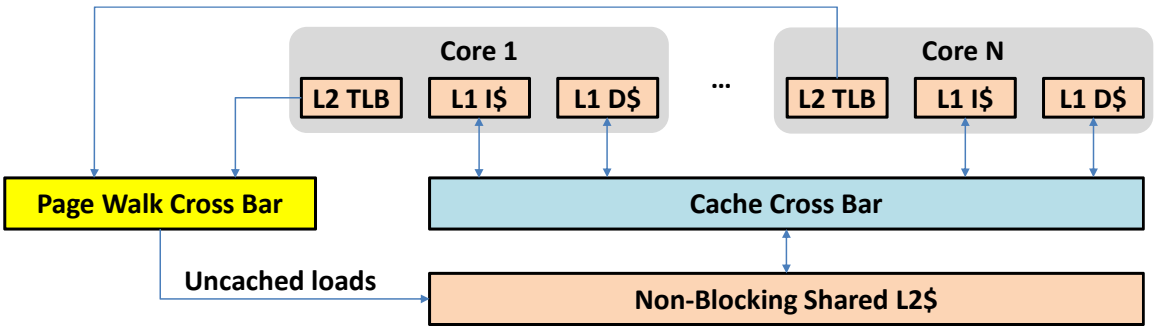


Figure 5-7: RiscyOO multiprocessor

all the in-flight requests from L1s and corresponding bookkeeping information, and provides interface methods to manipulate them. The cache-access pipeline has an interface like an FIFO. Any message entered into the pipeline will first read the tag SRAM to perform tag matching, and then read the data SRAM. The dequeue method of the pipeline can also update the tag SRAM and the data SRAM. The update is bypassed to earlier stages in the pipeline so that accesses to the tag and data SRAMs can get up-to-date contents.

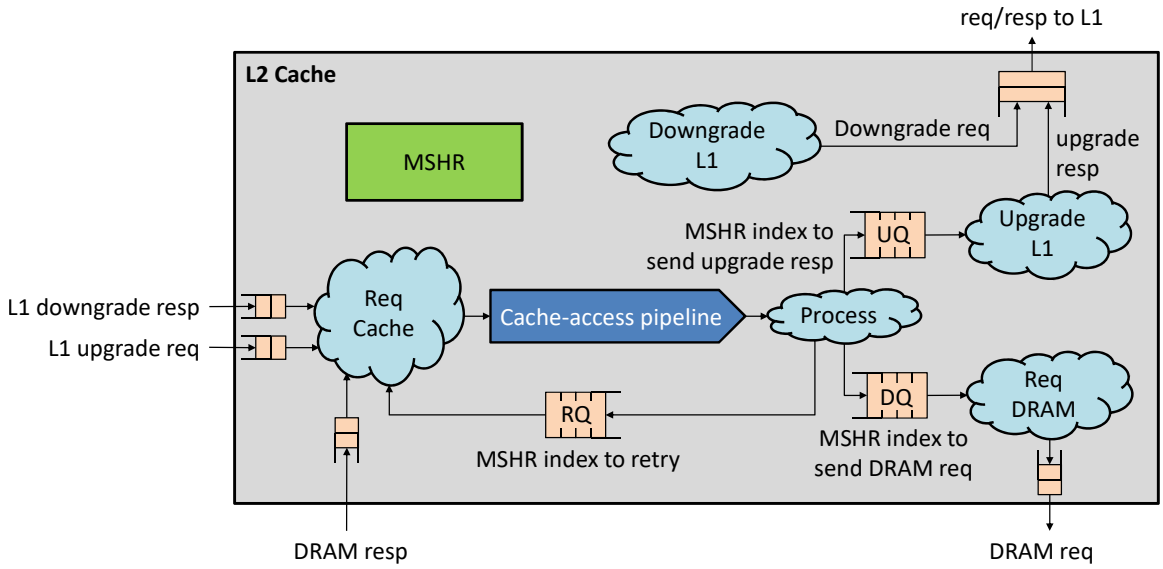


Figure 5-8: Microarchitecture of the L2 cache. Modules are represented by blocks, while rules are represented by clouds. All the rules access the MSHR module; arrows pointing to MSHR are not shown to avoid cluttering. Uncached loads from TLBs are also not shown for simplicity; they are handled in a similar way as L1 requests.

Rule *Req-Cache* enters a cache message, including a response from DRAM, a

downgrade response from L1, a new upgrade request from L1, and an old upgrade request from L1 that is retrying, into the cache-access pipeline. The arbitration of these messages follows a fixed priority (DRAM responses are more urgent than L1 responses which are more urgent than L1 requests). A new upgrade request from L1 also needs to allocate an MSHR entry.

After a message finishes accessing the SRAMs in the pipeline, it is processed by the *Process* rule which implements the actions to handle the message as specified in the coherence protocol. After the processing, an L1 upgrade request could be ready to respond, and in this case, the *Process* rule enters the MSHR index of the request into a FIFO, i.e., UQ in Figure 5-8, and the response data is buffered in the corresponding MSHR entry. The depth of UQ is equal to the number of MSHRs so it will never back-pressure the pipeline. In other cases that a cache replacement or a cache miss occurs, the L1 upgrade request that causes the replacement or cache miss needs to request DRAM. In this case, the *Process* rule enters the MSHR index into a FIFO, i.e., DQ in Figure 5-8, and buffers the data in the MSHR entry if writeback is needed. The depth of DQ is also equal to the number of MSHRs, so it will also never backpressure the pipeline.

The coherence protocol requires that each cache line can be manipulated by one request at a time, so the *Process* rule will put an L1 request to sleep in MSHR if the requested cache line is already occupied by another request (either for upgrading L1 or replacement). The tag SRAM contains an owner field for each cache line to identify which L1 request is currently occupying the line. The sleeping request will also be recorded in the MSHR entry of the owning request so that the sleep request can be waken up later. The *Process* rule wakes up a sleeping request for retry by enqueuing its MSHR index into a FIFO, i.e., RQ in Figure 5-8.[4] The depth of RQ is also equal to the number of MSHRs, so it will also never backpressure the pipeline.

Rules *Updagrade-L1* and *Req-DRAM* simply dequeue MSHR indexes from UQ and DQ, and send responses or requests to L1s or DRAM, respectively.

[4]The actual implementation in RiscyOO has made another optimization by directly swapping the sleeping request to the end of the cache-access pipeline if the owning request is responded. This simply requires modifying the dequeue method of the cache-access pipeline.

Rule *Downgrade-L1* searches through MSHR to find an entry that needs to downgrade any L1s, and sends the downgrade request, and sends the downgrade request if such an entry is found. Although *Downgrade-L1* and *Upgrade-L1* are contending for the FIFO connecting to L1s, the protocol requires that *Downgrade-L1* should not fire if UQ is not empty, i.e., when *Upgrade-L1* can fire.

As we can see, all the rules need to access the MSHR entries. To resolve these racing accesses, we can again turn each field of an MSHR entry into an EHR, and let each rule access EHRs using a unique port ID.

5.4 Evaluation of RiscyOO

We synthesized the RiscyOO processor on AWS F1 FPGA [1]. The processor boots Linux on FPGA, and benchmarking is done under this Linux environment (i.e., there is no syscall emulation). We evaluate single-core performance of RiscyOO by running SPEC CINT2006 benchmarks with ref input to completion. For benchmarks with multiple inputs, we just ran one input. The instruction count of each benchmark ranges from 64 billion to 2.7 trillion. With the processor running at 40 MHz on FPGA, we are able to complete the longest benchmark in about two days. We leave the multicore evaluation to Chapter 6 in which we will compare performance of multicores with different memory models.

The goal of this evaluation is to demonstrate the effectiveness of the CMD framework in improving performance, and to show that the microarchitecture of RiscyOO is realistic enough to deliver competitive performance. We first present the single-core performance results (Sections 5.4.2 to 5.4.5), and then give the ASIC synthesis results (Section 5.5).

5.4.1 Methodology

Table 5.1 shows the basic configuration, referred to as *RiscyOO-B*, of our RiscyOO processor. Since the number of cycles needed for a memory access on FPGA is much lower than that in a real processor, we model the memory latency and bandwidth for

a 2GHz clock in our FPGA implementation.

We compare our design with the four processors shown in Table 5.2: Rocket[5] (RISC-V ISA), A57 (ARM ISA), Denver (ARM ISA), and BOOM (RISC-V ISA). In Table 5.2, we have also grouped these processors into three categories: Rocket is an in-order processor, A57 and Denver are both commercial ARM processors, and BOOM is the state-of-the-art academic OOO processor.

The memory latency of Rocket is configurable, and is 10 cycles by default. We use two configurations of Rocket in our evaluation, i.e., *Rocket-10* with the default 10-cycle memory latency, and *Rocket-120* with 120-cycle memory latency which matches our design. Since Rocket has small L1 caches, we instantiated a *RiscyOO-C$^-$* configuration of our processor, which shrinks the caches in the RiscyOO-B configuration to 16KB L1 I/D and 256KB L2.

To illustrate the flexibility of CMD, we created another configuration *RiscyOO-T$^+$*, which improves the TLB microarchitecture of RiscyOO-B. In RiscyOO-B, both L1 and L2 TLBs block on misses, and a L1 D TLB miss blocks the memory execution pipeline. RiscyOO-T$^+$ supports parallel miss handling and hit-under-miss in TLBs (maximum 4 misses in L1 D TLB and 2 misses in L2 TLB). RiscyOO-T$^+$ also includes a split translation cache that caches intermediate page walk results [32]. The cache contains 24 fully associative entries for each level of page walk. We implemented all these microarchitectural optimizations using CMD in merely *two* weeks.

We also instantiated a *RiscyOO-T$^+$R$^+$* configuration which extends the ROB size of RiscyOO-T$^+$ to 80, in order to match BOOM's ROB size and compare with BOOM.

Table 5.3 has summarized all the variants of the RiscyOO-B configuration, i.e., RiscyOO-C$^-$, RiscyOO-T$^+$ and RiscyOO-T$^+$R$^+$.

The evaluation uses all SPEC CINT2006 benchmarks except perlbench which we were not able to cross-compile to RISC-V. We ran all benchmarks with the **ref** input to completion on all processors except BOOM, whose performance results are taken directly from [77]. We did not run BOOM ourselves because there is no publicly

[5]The prototype on AWS is said to have an L2 [74], but we have confirmed with the authors that there is actually no L2 in this particular released version.

released FPGA image of BOOM. Since the processors have different ISAs and use different fabrication technology, we measure performance in terms of one over the number of cycles needed to complete each benchmark (i.e., 1 / cycle count). Given so many different factors across the processors, this performance evaluation is informative but not rigorous. The goal here is to show that RiscyOO can achieve reasonable performance.

5.4.2 Effects of TLB microarchitectural optimizations

Before comparing with other processors, we first evaluate the effects of the TLB microarchitectural optimizations employed in RiscyOO-T$^+$. Figure 5-9 shows the performance of RiscyOO-T$^+$, which has been normalized to that of RiscyOO-B, for each benchmark. Higher values imply better performance. The last column is the geometric mean across all benchmarks. The TLB optimizations in RiscyOO-T$^+$ turn out to be very effective: on average, RiscyOO-T$^+$ outperforms RiscyOO-B by 29% and it doubles the performance of benchmark astar.

To better understand the performance differences, we show the number of L1 D TLB misses, L2 TLB misses, branch mispredictions, L1 D cache misses and L2 cache misses per thousand instructions of RiscyOO-T$^+$ in Figure 5-10. Benchmarks mcf, astar and omnetpp all have very high TLB miss rates. Although RiscyOO-B has a very large L2 TLB, the blocking nature of L1 and L2 TLBs still makes TLB misses incur a huge penalty. The non-blocking TLB designs and translation caches in RiscyOO-T$^+$ mitigate the TLB miss penalty and result in a substantial performance gain.

This evaluation shows that microarchitectural optimizations can bring significant performance benefits. It is because of CMD that we can implement and evaluate these optimizations in a short time. Since RiscyOO-T$^+$ always outperforms RiscyOO-B, we will use RiscyOO-T$^+$ instead of RiscyOO-B to compare with other processors.

Front-end	2-wide superscalar fetch/decode/renam 256-entry direct-mapped BTB tournament branch predictor as in Alpha 21264 [76] 8-entry return address stack
Execution Engine	64-entry ROB with 2-way insert/commit Total 4 pipelines: 2 ALU, 1 MEM, 1 FP/MUL/DIV 16-entry IQ per pipeline
Ld-St Unit	24-entry LQ, 14-entry SQ, 4-entry SB (each 64B wide)
TLBs	Both L1 I and D are 32-entry, fully associative L2 is 2048-entry, 4-way associative
L1 Caches	Both I and D are 32KB, 8-way associative, max 8 requests
L2 Cache	1MB, 16-way, max 16 requests, coherent with I and D 10-cycle hit latency
Memory	120-cycle latency, max 24 req (25.6GB/s for 2GHz clock)

Table 5.1: RiscyOO-B configuration of our RISC-V OOO uniprocessor

Name	Description	Catergory
Rocket	Prototype on AWS F1 FPGA for FireSim Demo v1.0 [5]. RISC-V ISA, in-order core, 16KB L1 I/D, no L2, 10-cycle or 120-cycle memory latency.	In-order
A57	Cortex-A57 core on Nvidia Jetson Tx2. ARM ISA, 3-wide superscalar. OOO core, 48KB L1 I, 32KB L1 D, 2MB L2.	Commercial ARM
Denver	Denver core [41] on Nvidia Jetson Tx2. ARM ISA, 7-wide superscalar. 128KB L1 I, 64KB L1 D, 2MB L2.	
BOOM	Performance results taken from [77]. RISC-V ISA, 2-wide superscalar. OOO core, 80-entry ROB, 32KB L1 I/D, 1MB L2, 23-cycle L2 latency, 80-cycle memory latency.	Academic OOO

Table 5.2: Processors to compare against

Variant	Difference	Specifications
RiscyOO-C^-	Smaller Caches	16 KB L1 I/D, 256 KB L2
RiscyOO-T^+	Improved TLB	Non-blocking TLBs, page table walk cache
RiscyOO-T^+R^+	Larger ROB	RiscyOO-T^+ with 80-entry ROB

Table 5.3: Variants of the RiscyOO-B configuration

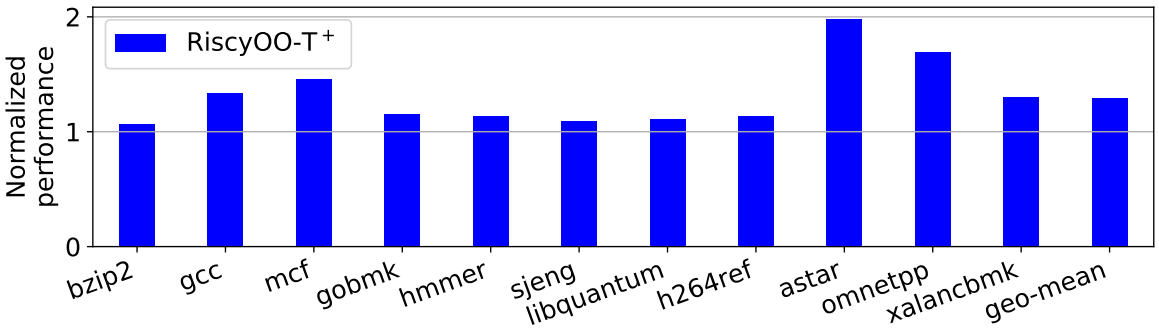


Figure 5-9: Performance of RiscyOO-T$^+$ normalized to RiscyOO-B. Higher is better.

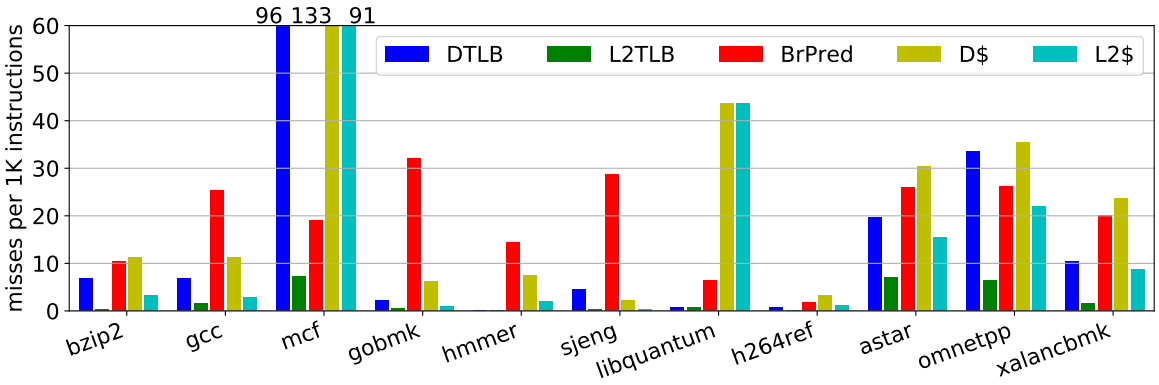


Figure 5-10: Number of L1 D TLB misses, L2 TLB misses, branch mispredictions, L1 D misses and L2 misses per thousand instructions of RiscyOO-T$^+$

5.4.3 Comparison with the in-order Rocket processor

Figure 5-11 shows the performance of RiscyOO-C$^-$, Rocket-10, and Rocket-120 for each benchmark. The performance has been normalized to that of RiscyOO-T$^+$. We do not have libquantum data for Rocket-120 because each of our three attempts to run this benchmark ended with an AWS server crash after around two days of execution.

As we can see, Rocket-120 is much slower than RiscyOO-T$^+$ and RiscyOO-C$^-$ on every benchmark, probably because its in-order pipeline cannot hide memory latency. On average, RiscyOO-T$^+$ and RiscyOO-C$^-$ outperform Rocket-120 by 319% and 196%, respectively. Although Rocket-10 has only 10-cycle memory latency, RiscyOO-T$^+$ still outperforms Rocket-10 in every benchmark, and even RiscyOO-C$^-$ can outperform or tie with Rocket-10 in many benchmarks. On average, RiscyOO-T$^+$ and RiscyOO-C$^-$ outperforms Rocket-10 by 53% and 8%, respectively. This comparison shows that our OOO processor can easily outperform in-order processors.

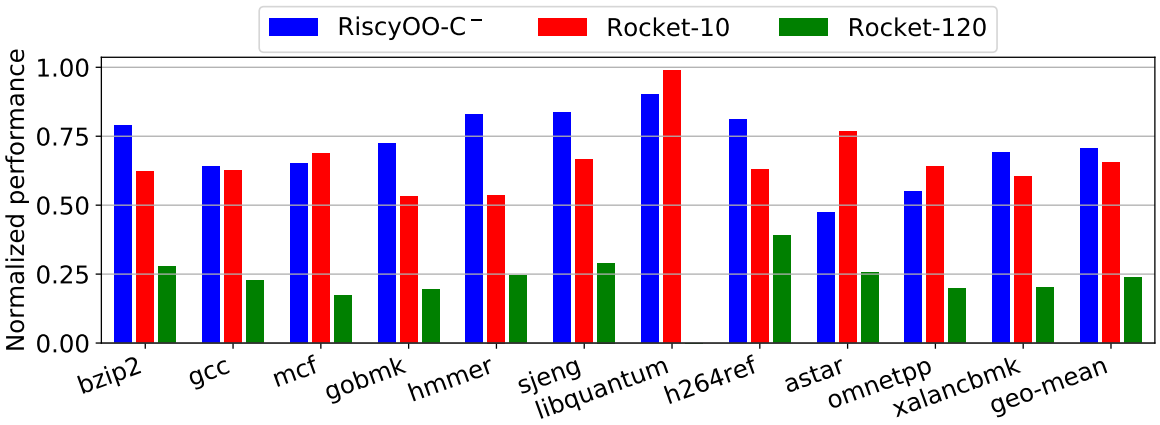


Figure 5-11: Performance of RiscyOO-C$^-$, Rocket-10, and Rocket-120 normalized to RiscyOO-T$^+$. Higher is better.

5.4.4 Comparison with commercial ARM processors

Figure 5-12 shows the performance of ARM-based processors, A57 and Denver, for each benchmark. The performance has been normalized to that of RiscyOO-T$^+$. A57 and Denver are generally faster than RiscyOO-T$^+$, except for benchmarks mcf, astar and omnetpp. On average, A57 outperforms RiscyOO-T$^+$ by 34%, and Denver outperforms RiscyOO-T$^+$ by 45%.

To better understand the performance differences, we revisit the miss rates of RiscyOO-T$^+$ in Figure 5-10. Because of the high TLB miss rates in benchmarks mcf, astar and omnetpp, the TLB optimizations enable RiscyOO-T$^+$ to catch up with or outperform A57 and Denver in these benchmarks. Commercial processors have significantly better performance in benchmarks hmmer, h264ref, and libquantum. Benchmarks hmmer and h264ref both have very low miss rates in TLBs, caches and branch prediction, so the higher performance in A57 and Denver may be caused by their wider pipelines (our design is 2-wide superscalar while A57 is 3-wide and Denver is 7-wide). Benchmark libquantum has very high cache miss rates, and perhaps commercial processors have employed memory prefetchers to reduce cache misses.

Since we do not know the details of commercial processors, we cannot be certain about our reasons for the performance differences. In spite of this, the comparison still shows that the performance of our OOO design is not out of norm. However, we do believe that to go beyond 2-wide superscalar our processor will require more

architectural changes especially in the front-end.

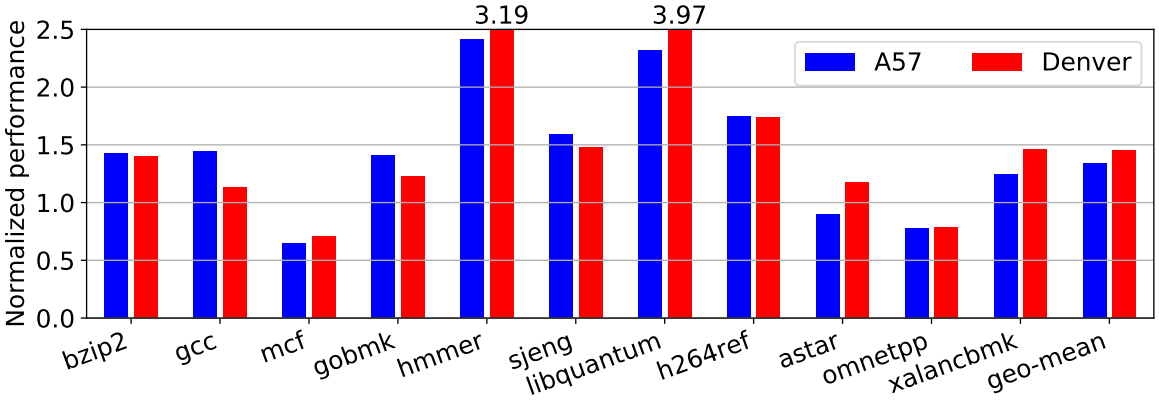


Figure 5-12: Performance of A57 and Denver normalized to RiscyOO-T$^+$. Higher is better.

5.4.5 Comparison with the academic OOO processor BOOM

Figure 5-13 shows the IPCs of BOOM and our design RiscyOO-T$^+$R$^+$[6]. We have tried our best to make the comparison fair between RiscyOO-T$^+$R$^+$ and BOOM. RiscyOO-T$^+$R$^+$ matches BOOM in the sizes of ROB and caches, and the influence of the longer L2 latency in BOOM can be partially offset by the longer memory latency in RiscyOO-T$^+$R$^+$. BOOM did not report IPCs on benchmarks gobmk, hmmer and libquantum [77], so we only show the IPCs of remaining benchmarks. The last column shows the harmonic mean of IPCs over all benchmarks.

On average, RiscyOO-T$^+$R$^+$ and BOOM have similar performance, but they outperform each other in different benchmarks. For example, in benchmark mcf, RiscyOO-T$^+$R$^+$ (IPC=0.16) outperforms BOOM (IPC=0.1), perhaps because of the TLB optimizations. In benchmark sjeng, BOOM (IPC=1.05) outperforms RiscyOO-T$^+$R$^+$ (IPC=0.73). This is partially because RiscyOO-T$^+$R$^+$ suffers from 29 branch mispredictions per thousand instructions while BOOM has about 20 [77].

This comparison shows that our OOO processor designed using CMD has matching performance with the state-of-the-art academic processors.

[6]For each benchmark, we ran all the ref inputs (sometimes there are more than one), and computed the IPC using the aggregate instruction counts and cycles. This makes our instruction counts close to those reported by BOOM.

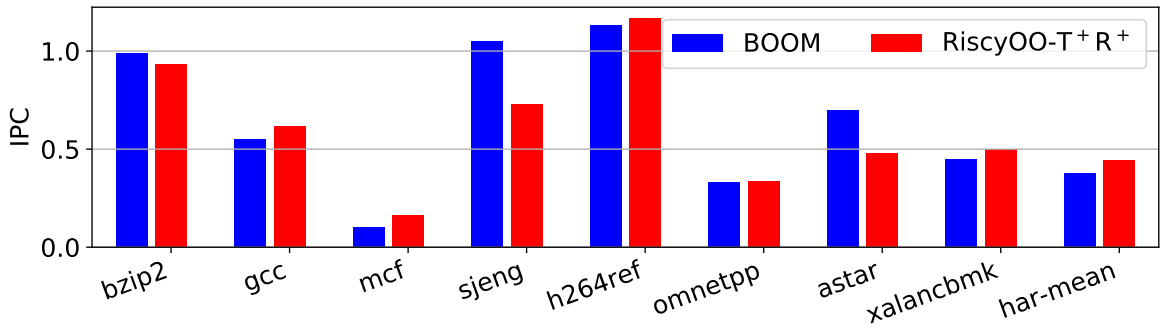


Figure 5-13: IPCs of BOOM and RiscyOO-T$^+$R$^+$ (*BOOM results are taken from [77]*)

5.5 ASIC Synthesis[7]

To evaluate the quality of the produced designs, we synthesized a single core (processor pipeline and L1 caches) of the RiscyOO-T$^+$ and RiscyOO-T$^+$R$^+$ processor configurations for ASIC. Our synthesis flow used a 32 nm SOI technology and SRAM blackboxes using timing information from CACTI 6.5 [104]. We performed topographical synthesis using Synopsys's Design Compiler, i.e., we performed a timing-driven synthesis which performs placement heuristics and includes resistive and capacitive wire delays in the timing model. This approach significantly reduces the gap between post-synthesis results and post-placement and routing results. The synthesis does not take into account the floating point unit and the integer multiplier and divider. We produced a maximum frequency for each configuration by reporting the fastest clock frequency which was successfully synthesized. We produced a NAND2-equivalent gate count by taking the total cell area and dividing by the area of a default-width NAND2 standard cell in our library. As a result, our NAND2-equivalent gate count is logic-only and does not include SRAMs.

Core Configuration	RiscyOO-T$^+$	RiscyOO-T$^+$R$^+$
Max Frequency	1.1 GHz	1.0 GHz
NAND2-Equivalent Gates	1.78 M	1.89 M

Table 5.4: ASIC synthesis results

Results: The synthesis results are shown in Table 5.4. Both processors can operate at 1.0 GHz or above. The area of the RiscyOO-T$^+$R$^+$ configuration is only 6.2% more

[7]ASIC synthesis is done by Andrew Wright.

than that of the RiscyOO-T$^+$ configuration, because RiscyOO-T$^+$R$^+$ increases only the ROB size and the number of speculation tags over RiscyOO-T$^+$. The NAND2-equivalent gate counts of the processors are significantly affected by the size of the branch predictors. This can be reduced by either reducing the size of the tournament branch predictor and/or utilizing SRAM for part of the predictor.

5.6 Summary

We have developed the CMD framework, in which modules have guarded interface methods, and are composed together using atomic actions. With the atomicity guarantee in CMD, modules can be refined selectively relying only on the interface details, including Conflict Matrix, of other modules. Using CMD, we designed and implemented an out-of-order superscalar cache-coherent multiprocessor, RiscyOO, which can boot Linux and complete benchmarks of trillions of instructions without errors. Our evaluation shows that RiscyOO can easily outperform in-order processors (e.g., Rocket) and matches state-of-the-art academic OOO processors (e.g., BOOM), though not as good as highly optimized commercial processors. We will leverage the modularity and flexibility of the design of RiscyOO to implement and evaluate different memory models in the next chapter.

Chapter 6

Evaluation of WMM versus TSO

Although we have greatly simplified the definitions of weak memory models by proposing WMM, the definition of WMM is still more complex than the definitions of strong memory models like TSO. In this chapter, we study whether the extra definitional complexity can make processors with weak memory models have better PPA (performance/power/area) than processors with strong memory models. In this study, we use WMM as the representative weak memory model because it admits most microarchitectural optimizations and still has a reasonable definition, and use TSO as the representative strong memory model, because TSO is the memory model of the widely used Intel processors. We use RiscyOO (Chapter 5) as the baseline processor because of its realistic microarchitecture and its fast speed which makes it possible to finish real-world workloads in a reasonable amount of time.

The evaluation is still very difficult because it is affected by many factors in both the benchmark programs and the processor microarchitectures. In terms of benchmarks, the frequency of synchronization and communication between threads may affect how well a memory model behaves. In our study, we evaluate single-threaded benchmarks as well as multithreaded benchmarks with different degrees of synchronizations. Multithreaded benchmarks are parallelized using portal multithreaded libraries and compiler built-ins, which we believe is the common case in multithreaded programming.

As for processor microarchitectures, TSO implementations have two major per-

formance bottlenecks compared to WMM. The first one is regarding load execution. It is clear that a naive implementation of TSO which executes loads sequentially will have poor performance. Therefore, most TSO implementations execute loads out of order and speculatively while snooping cache evictions to squash speculative loads that violate the memory ordering required by TSO. Frequent squashes can hurt performance. Squashes can be reduced by training predictors on whether a load should be issued speculatively. The other performance bottleneck is the speed of recycling entries of the store queue (SQ). TSO requires stores in SQ to be written to the memory system sequentially, and thus, frequent store misses in the cache will slow down the recycle of SQ entries. In RiscyOO, if the SQ is full and the register-renaming stage gets a store instruction, then the renaming stage will be stalled and the front-end fetch-decode pipeline will also be back-pressured. This is because renaming of a store requires allocating an entry in SQ in RiscyOO. This can be mitigated by prefetching for stores and having a deep SQ. Therefore, a sophisticated TSO microarchitecture with enough resources (e.g., predictors, prefetchers and deep buffers) will be as fast as a weak-memory machine. This point is supported by the fact that Intel processors, which are TSO machines, dominate the high-end server market.

In this study, we focus on a different scenario where processors have limited resources (e.g., buffer sizes are small). Focusing on lower-end machines also saves our efforts in engineering the baseline RiscyOO processors to catch up with commercial products. For example, the branch predictors of RiscyOO may need signficant improvement in order to support the large ROB size and wide superscalarity in Intel processors.

This evaluation is by no means a comprehensive comparison of weak and strong memory models. Nevertheless, our results show the following characteristics of the PPA of TSO and WMM:

- TSO can have single-threaded performance overhead because the dequeue of SQ is slow, but introducing store-prefetch in TSO can recover most of the performance loss.

- WMM can be slower than TSO in multithreaded benchmarks that synchronize frequently, because fences can unnecessarily serialize load execution at runtime while loads in TSO can speculate over fences to avoid the unnecessary stalls.

- There is little difference between TSO and WMM in energy efficiency or area cost.

- WMM admits more flexible implementations, e.g., a self-invalidation (SI) coherence protocol, than TSO does. However, the new SI coherence protocol does not improve, but rather degrades, performance and energy efficiency, especially in case of multithreaded benchmarks with frequent synchronizations.

In the following, we first explain our methodology (Section 6.1), then present the evaluation results at the microarchitecture level (Sections 6.2 to 6.4), and finally give the ASIC synthesis results (Section 6.5).

6.1 Methodology

6.1.1 Benchmarks

We evaluate single-threaded performance of memory models by running SPEC CINT2006 benchmarks with *ref* input to completion. For benchmarks with multiple inputs, we just ran one input. The instruction count of each benchmark ranges from 64 billion to 2.7 trillion.

For multithreaded performance evaluation, we ran the PARSEC benchmark suite [36] and the GAP benchmark suite [35, 6]. Among the 13 PARSEC benchmarks, we could not cross-compile raytrace, vips and dedup to RISC-V. Though we have managed to compile bodytrack and canneal, they cannot run even on the RISC-V ISA simulator [14], which is the golden model for RISC-V implementations. We ran the remaining 8 PARSEC benchmarks with *native* input to completion. The user-level instruction count of each benchmark ranges from 560 billion to 7.4 trillion.

The GAP benchmark suite contains 6 graph analytics algorithms, i.e., *bc* (between centrality), *bfs* (breadth-first search), *cc* (connected components), *pr* (page rank), *sssp* (single-source shortest paths), and *tc* (triangle counting). The software implementation [6] of the algorithms are parallelized using openMP. We ran all 6 GAP benchmarks on the USA-road graph [17] to completion.[1] We also follow the measurement methodology given by the GAP suite, including the number of trials to repeat the kernels, the source vertex, etc. The only exception is that we increased the number of trials of tc from 3 to 10 in order to get more instruction counts. Table 6.1 has summarized the measurement parameters of GAP benchmarks. The user-level instruction count of each benchmark ranges from 7.7 billion to 70 billion.

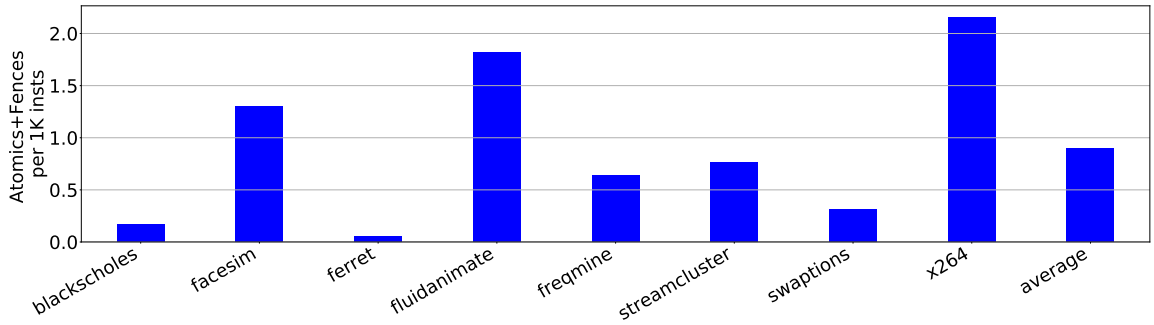
Benchmark	Parameters
bc	16 trials, each from 4 sources
bfs	64 trials from 64 sources
cc	16 trials
pr	16 trials
sssp	64 trials from 64 sources
tc	10 trials

Table 6.1: Measurement parameters of GAP benchamrks (adapted from [35, Table 1])

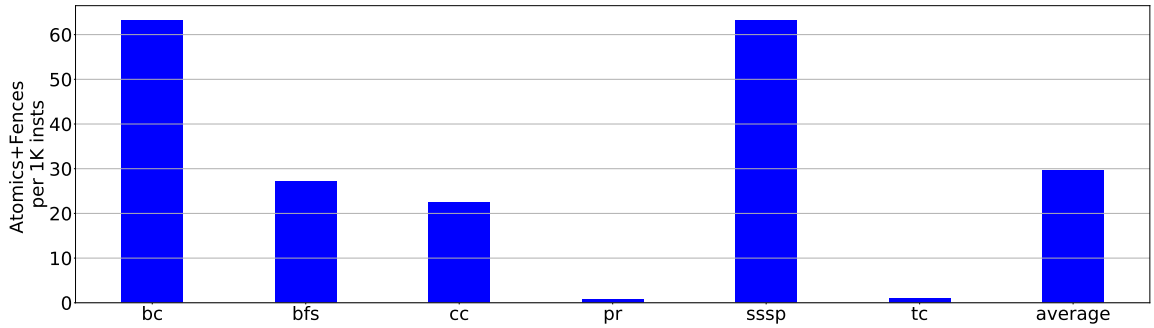
We ran both PARSEC and GAP benchmarks because they have different frequencies of synchronizations between threads. The number of atomic instructions (load reserve, store conditional, and atomic read-modify-write) and fences can be used as an indicator for the degree of synchronizations. Figure 6-1 shows the number of atomic instructions and fences per thousand user-level instructions in PARSEC and GAP benchmarks on a 4-core WMM multiprocessor. PARSEC benchmarks (Figure 6-1a) have generally infrequent synchronizations. On average, there is less than one atomic or fence instruction per thousand user-level instructions. In contrast, among the six GAP benchmarks (Figure 6-1b), bc, bfs, cc and sssp all have frequent synchronizations, while pr and tc are infrequent in synchronizations. The average number

[1]We did not run larger graphs like twitter because RiscyOO does not have enough memory. Currently RiscyOO supports only 16GB memory and the file system is a ramfs, but the file holding a twitter graph is already more than 10GB large.

of atomic and fence instructions can reach 30 per thousand user-level instructions, which is orders of magnitude larger than that of PARSEC.



(a) PARSEC benchmarks



(b) GAP benchmarks

Figure 6-1: Number of atomic instructions and fences per thousand user-level instructions in PARSEC and GAP benchmarks on a 4-core WMM multiprocessor

Porting Multithreaded Benchmarks to Different Memory Models

The correctness of multithreaded programs depends on the memory model. It may require inserting more fences to run a multithreaded program, which is written under the assumption of TSO, on a WMM processor. Both PARSEC and GAP benchmarks are targeting Intel processors with the TSO memory model. Fortunately, they are parallelized in a data-race-free fashion using the pthread and openMP libraries. When we cross-compile the benchmarks to RISC-V, which has a weak memory model like GAM, the gcc compiler will automatically insert fences in the library routines of pthread and openMP. GAP benchmarks also use a couple of gcc built-in atomics (e.g., `__sync_bool_compare_and_swap`), and the gcc compiler will also surround these atomics with fences during cross-compilation. Therefore, we do not need to

manually insert fences in the benchmarks for running them on WMM processors.

We need to manually insert fences in the Linux kernel if the WMM processor can reoreder data-dependent loads (RISC-V memory model does not reorder data-dependent loads). We follow the fence-insertion scheme in the Alpha port of Linux (Alpha memory model allows the reordering of data-dependent loads). That is, we define the dependent-load-load-fence macro `read_barrier_depends()` as an RISC-V acquire fence (i.e., FENCE ir,iorw in RISC-V assembly), and insert this macro in several places in the software page-walk code as Alpha does.

Since RISC-V assumes a weak memory model like GAM, which allows more reorderings than WMM and TSO, the cross-compilation may end up having unnecessary fences. Fortunately, the RISC-V fence instruction (i.e., FENCE in RISC-V assembly) specifies which load/store orderings it is enforcing, so the WMM or TSO processor can dynamically translate unnecessary fences to NOPs at decode stage (see Section 6.1.3).

6.1.2 Processor Configurations

Table 6.2 shows the configuration of uniprocessors which will run SPEC benchmarks to evaluate single-threaded performance of memory models. The sizes of ROB, LQ and SQ are 1/3 of those in an Intel Haswell processor which has 192 ROB entries, 72 LQ entries and 42 SQ entries. Table 6.3 shows the configuration of 4-core multiprocessors which will run PARSEC and GAP benchmarks to evaluate multithreaded performance of memory models. The L2 cache size is increased to 2MB. The size of each core is further reduced to 1/4 of an Intel Haswell core, and the associativity of L1 TLBs and L1 caches is reduced by half. This is due to the limited logic resources on the AWS FPGA. Synthesizing a 4-core multiprocessor using this configuration takes up to 95% of the logic resources on the FPGA.

These configurations are shared by all memory models. In Section 6.1.3, we explain the microarchitectural differences between the implementations of WMM and TSO.

Front-end	2-wide superscalar fetch/decode/renam 256-entry direct-mapped BTB tournament branch predictor as in Alpha 21264 [76] 8-entry return address stack
Execution Engine	64-entry ROB with 2-way insert/commit Total 4 pipelines: 2 ALU, 1 MEM, 1 FP/MUL/DIV 16-entry IQ per pipeline
Ld-St Unit	24-entry LQ, 14-entry SQ, 4-entry SB (each 64B wide)
TLBs	Both L1 I and D are 32-entry, fully associative L2 is 1024-entry, 4-way associative Split translation cache [32] with 24 entries for each level
L1 Caches	Both I and D are 32KB, 8-way associative, max 8 requests LRU replacement, 2-cycle hit latency
L2 Cache	1MB, 16-way, max 16 requests, coherent with I$ and D$ random replacement, 10-cycle hit latency
Memory	120-cycle latency, max 24 req (25.6GB/s for 2GHz clock)

Table 6.2: Baseline configuration of uniprocessors

Front-end	2-wide superscalar fetch/decode/renam 256-entry direct-mapped BTB tournament branch predictor as in Alpha 21264 [76] 8-entry return address stack
Execution Engine	48-entry ROB with 2-way insert/commit Total 4 pipelines: 2 ALU, 1 MEM, 1 FP/MUL/DIV 10-entry IQ per pipeline
Ld-St Unit	18-entry LQ, 10-entry SQ, 2-entry SB (each 64B wide)
TLBs	Both L1 I and D are 16-entry, fully associative L2 is 1024-entry, 4-way associative Split translation cache [32] with 24 entries for each level
L1 Caches	Both I and D are 32KB, 4-way associative, max 4 requests LRU replacement, 2-cycle hit latency
Shared L2 Cache	2MB, 16-way, max 16 requests, random replacement, 10-cycle hit latency, MESI coherence protocol, coherent with all four I$ and D$
Memory	120-cycle latency, max 24 req (12.8GB/s for 2GHz clock)

Table 6.3: Baseline configuration of 4-core multiprocessors

6.1.3 Memory-Model Implementations

We implemented four processors, *WMM-Base*, *WMM-SI*, *TSO-Base*, and *TSO-SP*, for the evaluation. All four processors are derived from RiscyOO. WMM-Base and WMM-SI are implementations of WMM. The major difference between the two is that

WMM-SI uses a self-invalidation coherence protocol and can reorder data-dependent loads, while WMM-Base uses the common MESI coherence protocol and does not reorder data-dependent loads. The comparison between WMM-Base and WMM-SI also shows the performance implications of allowing dependent-load-load reorderings in weak memory models. TSO-Base and TSO-SP are implementations of TSO. They both use the common MESI coherence protocol. TSO-SP prefetches exclusive cache permissions for store instructions (SP stands for store-prefetch), while TSO-Base does not.

All four processors can boot Linux (WMM-SI requires the changes in Linux mentioned in Section 6.1.1 because of the reorderings of data-dependent loads), and all the benchmarkings were done under Linux.

The four implementations differ in coherence protocols, L1 I cache, decode, dequeue of SQ, load execution, dequeue of LQ, and store-prefetch. Next we explain these differences one by one.

Coherence Protocol

WMM-Base, TSO-Base and TSO-SP: all use the default MESI coherence protocol in RiscyOO. This MESI protocol is a variant of the 4-hop MSI protocol formally verified by Vijayaraghavan et al. [140]. The shared L2 will respond a load request (i.e., upgrade-to-S request) from an L1 with E state if the L1 request causes a DRAM access that brings the cache line into L2.

WMM-SI: uses a self-invalidation coherence protocol which is a variant of the DeNovo coherence protocol [46]. We refer to this protocol as *SI*. In the SI protocol, the directory of the shared L2 tracks only the L1 that owns the cache line in the exclusive state (i.e., E or M), and does not track any shared copies of a cache line (i.e., L1 data in the S state). An L1 upgrade request sent to L2 will not induce any invalidations of shared copies in other L1s. L2 needs to downgrade only the L1 that owns the cache line in the exclusive state. As a result, L1 data in the S state can be stale, and can also be evicted silently without notifying L2. The stale data in L1 enables WMM-SI to reorder data-dependent loads (see Section 3.1.4). The Reconcile fence is responsible

for clearing all the L1 data in S state to ensure that there is no stale data in the local L1. Since L1 D is only 32KB large with 512 cache lines, we clear all S-state L1 data in a single cycle by keeping all the state bits in registers.

It should be noted that keeping stale data in L1 may affect the forward progress of programs. A common programming pattern is to spin on a memory location until it is changed. For example, when the program tries to acquire a lock, it may first use normal loads to spin on the lock variable until it sees the lock is released, and then tries to use atomic instructions to truly acquire the lock. If a stale copy of the lock variable is kept in the L1, then the program may never leave the spinning phase. Inserting fences for every occurrence of such programming patterns in all the common software, including Linux and libraries like pthread and openMP, is extremely tedious. To guarantee forward progress, the SI protocol will voluntarily evict an L1 cache line in the S state if the number of consecutive load hits on the line exceeds a threshold. We refer to this threshold as the *self-eviction threshold*. There is clearly a tradeoff in the choice of the threshold. A high threshold reduces self-evictions but makes forward progress more difficult; a low threshold is exactly the opposite. We choose a default threshold of 64 consecutive load hits, and refer to the specific instantiation of WMM-SI with this threshold as *WMM-SI64*.

SI differs from DeNovo in the sense that DeNovo requires the program to be data-race-free so as to avoid handling racing accesses from different cores, while SI reuses the mechanisms in the MESI protocol to handle racing accesses and can support arbitrary programs, including Linux.

The potential benefit of SI is that the size of the L2 directory grows logarithmically with the number of L1s, while in the default MESI protocol, the size of the L2 directory grows linearly with the number of L1s. That is, SI could be more scalable in a many-core system. Besides, SI does not send invalidations to shared copies in L1s, so network traffic may be reduced. Furthermore, L1s are not inclusive with L2 (because there is no invalidations on shared copies), and thus, the effective total cache size may increase.

L1 I Cache

Most ISAs would provide a special instruction to synchronize instruction and data streams (e.g., the ISB instruction in ARM). In RISC-V, such special instruction is called FENCE.I. The differences in coherece protocols can influence the implementation of FENCE.I.

WMM-Bas, TSO-Base and TSO-SP: All the L1 I caches are also involved in the MESI coherence protocol, and thus, are coherent with all the L1 D caches. Therefore, FENCE.I only needs to squash all the in-flight instructions when it is committed from ROB.

WMM-SI: Since the L2 in the SI coherence does not track L1 data in the S state, L1 I caches cannot get involved in the SI protocol. When a FENCE.I instruction is committed from ROB, not only all the in-flight instruction are squashed, but also the L1 I cache needs to be cleared. Similar to clearing L1 D, L1 I can be cleared in a single cycle.

Decode

WMM and TSO processors dynamically translate RISC-V fences into WMM or TSO fences or NOPs at decode stage. There are two types of fences in RISC-V: a normal fence instruction which specifies the memory orderings to enforce, and an atomic instruction which carries acquire or release bits.

WMM-Base and WMM-SI: decode fence instructions that order anything before a younger load as a Reconcile fence, and decode fence instructions that order an older store before anything as a Commit fence. Fence instructions that satisfy both conditions are decoded as a full fence, i.e., a Commit followed by a Reconcile. The acquire and release bits on an atomic instruction behave as Reconcile and Commit fences, respectively. An atomic instruction with both acquire and release bits has the effect of a full fence.

TSO-Base and TSO-SP: keep only fence instructions that order an older store before a younger store. Other fence instructions are decoded into NOPs. The acquire

and release bits on atomic instructions are ignored, but all atomic instructions are treated as fences in TSO. In Intel processors, an atomic read-modify-write also has the effect of a fence.

Dequeue of SQ

Both TSO and WMM processors dequeue all SQ entries, including normal stores, atomic instructions (store-conditionals and read-modify-writes), and fence instructions, sequentially. A normal store can be dequeued from SQ only after the corresponding store instruction has been committed from the ROB. That is, stores are never sent to memory (speculatively) before they are committed.

WMM-Base and WMM-SI: have a store buffer (SB) to receive stores dequeued from SQ. Stores for the same cache line are coalesced into a single entry in SB. Each entry in SB can initiate a store request to the memory system (i.e., L1 D). When the corresponding cache line in L1 D gets the exclusive permission, the entry is removed from SB an written into L1 D. There is no ordering relation between different entries in SB.

If the oldest SQ entry is a fence, then it can be dequeued from SQ only when the fence instruction reaches the commit slot of ROB. Dequeuing a Commit fence additionally requires SB to be empty. Dequeuing a Reconcile fence in WMM-Base does not need to meet other constraints. However, dequeuing a Reconcile in WMM-SI requires flushing the L1 D cache, because the self-invalidation protocols used in WMM-SI allows L1 D to keep stale values. The fence instruction will not be committed from ROB after it is dequeued from SQ. We use this rather conservative fence-dequeue scheme to simplify the dequeue logic and ensure correct implementation of fences.

If the oldest SQ entry is an atomic instruction, then it will be issued to memory after the atomic instruction reaches the commit slot of ROB, i.e., it becomes non-speculative. If the atomic instruction carries a release bit, then it cannot be issued until SB is empty (similar to a Commit fence). The atomic instruction can be committed from ROB after its memory access completes. In WMM-SI, if the atomic instruction carries an acquire bit, then L1 D needs to be flushed after the memory

access completes (similar to a Reconcile fence).

TSO-Base and TSO-SP: do not have an SB. If the oldest SQ entry is a normal store and the store instruction has been committed from ROB, then the store can be issued to the memory system. When the store finishes writing L1 D, the entry is dequeued from SQ. If the oldest SQ entry is a fence and the fence instruction is at the commit slot of ROB, then the entry is dequeued from SQ and the instruction is committed from ROB. If the oldest SQ entry is an atomic instruction and the instruction is at the commit slot of ROB, then the atomic access can be sent to the memory system. After the memory access completes, the entry is is dequeued from SQ and the instruction is committed from ROB.

Load Execution

Both TSO and WMM processors issue normal loads speculatively and out of order, but speculative loads in different memory models are subject to different types of stalls or kills. (A load-reserve is always executed non-speculatively when the instruction reaches the commit slot of ROB.)

WMM-Base and WMM-SI: When a load tries to issue, it searches through older memory and fence instructions in LSQ. If there is an older Reconcile or an older unissued load to the same address, and the issuing load cannot forward from a store younger than the Reconcile or the unissued load, then the load cannot issue. Stalling load issue because of older unissued same-address loads is for enforcing same-address load-load ordering. However, stalling load issue is not enough for enforcing this ordering. In addition, when a load gets issued successfully, it needs to kill any younger loads, which access the same address, have been issued, and do not get forwarding from any store younger than the issuing load. The stalls and kills for enforcing same-address load-load ordering are disabled in the uniprocessor in order to estimate the upper bound of single-threaded performance of weak memory models.

TSO-Base and TSO-SP: are not subject to all the above load-issue stalls in WMM. In TSO-Base, when a load tries to issue, it only needs to search through older stores for forwarding or stalls due to partially overlapped address. (WMM-Base and WMM-

SI are also subject to stalls on partially overlapped stores.) It should be noted that *TSO-Base can issue a load to memory even if there is an older fence in LSQ, i.e., a load can speculate over a fence.* To ensure the implementation conforms to TSO, TSO-Base needs to snoop L1-cache evictions (including both invalidations by other cores and replacement to serve cache misses). In case a cache line is evicted from L1 D, TSO-Base kills any loads in LQ that have been issued to memory or have got forwarding from stores which have left SQ (i.e, have been written to memory). To support this check, every load in LQ tracks where its value comes from, and this field will be updated if a store is dequeued from SQ. It should be noted that this bookkeeping of load-value sources is also needed for enforcing store-to-load memory dependency, and WMM processors also have this bookkeeping. It should also be noted that snooping cache evictions cannot guarantee the correctness of TSO completely, because cache evictions can only kill loads in LQ. We need to ensure that loads dequeued from LQ can no longer violate TSO memory orderings. The conditions for dequeuing LQ are explained next.

Dequeue of LQ

Both WMM and TSO processors dequeue executed loads from LQ sequentially, but the detailed conditions for dequeuing differ slightly.

WMM-Base and WMM-SI: can dequeue the oldest load in LQ as long as all stores older than the load have computed and translated their addresses. That is, the load cannot be killed by any older stores.

TSO-Base and TSO-SP: can dequeue the oldest load in LQ only when (1) all stores older than the load have computed and translated their addresses, and (2) *there is no atomic or fence instruction in SQ that is older than the load.* The second condition is important because it keeps the load in LQ and subject to cache evictions until all older atomic and fence instructions are committed. That is, the load will read from the same store if it is replayed immediately after all older atomic and fence instructions are committed. This enforces the memory-ordering effects carried by the atomic and fence instructions in TSO, i.e., a younger loads cannot overtake any older

atomic or fence instructions.

Store-prefetch

Only TSO-SP prefetches the exclusive permission in L1 for store instructions. It should be noted that prefetches in TSO-SP are non-binding and do not affect functional correctness. TSO-SP has a small FIFO to hold the store addresses to be prefetched. When a store instruction finishes translating its address in the memory execution pipeline, it will enqueue its address into the FIFO. If the FIFO is full because of back pressure from the L1 cache, then the store address is simply dropped. At every cycle, TSO-SP tries to dequeue an address from the FIFO and issue to L1 a prefetch request that upgrades the address to the E state. The issue of prefetch requests will yield to the issue of other memory requests (e.g., loads) when there is contention on the port of L1. After the issue request enters the L1, if there is another request that upgrades the same address to the E or M state, then the prefetch request is simply dropped. In other cases, the prefetch request is handled like a normal memory request except that it does not generate any response back to the core pipeline.

6.1.4 Energy Analysis

Besides performance, we are also concerned with energy efficiency. Although we do not have a detailed power model, we can still perform semi-quantitative analysis on energy based on the number of certain events that are commonly believed to dominate energy consumption. As far as the comparison of WMM and TSO is concerned, we consider the following three events that can make difference in energy consumption: (1) mis-speculative loads, (2) DRAM accesses, and (3) network traffic between cores and L2. We now explain why WMM and TSO processors may differ from each other in these events.

Mis-speculative loads: A speculative load in both TSO and WMM processors can be killed by an older store if the load does not observe memory dependency. TSO

and WMM differ in the other sources that can kill the load. In TSO, the load can also be killed by cache eviction before it is dequeued from LQ. In WMM, the load can also be killed by a younger load for the same address due to same-address load-load ordering. (The extra kills in WMM for same-address loads are considered only in the multicore evaluation.)

DRAM accesses: If TSO and WMM processors use the same memory system, then the number of DRAM accesses is unlikely to be different. However, WMM admits the SI coherence protocol (i.e., the WMM-SI processor), which allows L1s not to be inclusive with L2. In theory, this could increase the effective total cache size, and thus reduce DRAM accesses. However, since the total L1 cache size is still much smaller than the L2 cache size, we do not expect a big reduction on DRAM accesses. Nevertheless, we will present results on DRAM accesses. It should be noted that DRAM accesses can also be caused by uncached load requests for page walks (Section 5.3), and these accesses may not be reduced by using the SI coherence protocol.

Network traffic between cores and L2: The amount of network traffic can be different between TSO and WMM processors also because of the SI coherence protocol which is admitted only by WMM. The SI protocol removes the invalidations of shared copies in L1s, and thus, it may reduce network traffic. However, the self-invalidation mechanisms in L1s may create more L1 misses, which in turn increase the network traffic.

It should be noted that RiscyOO does not have an on-chip network or any routers. RiscyOO just uses an cross bar to connect all L1s to the shared L2. However, this does not prevent us from calculating the amount of data being transferred between cores and L2. As mentioned earlier, there are four types of cache messages being transferred between L1s and L2: (1) upgrade requests from L1s to L2, (2) upgrade responses from L2 to L1s, (3) downgrade requests from L2 to L1s, and (4) downgrade responses (including voluntary cache evictions) from L1s to L2. All messages contain an address field, and response messages may also contain a data field which is a cache line. As a first-order approximation, we assume that an address-only message is 8-

byte large, while a message with both address and data is 72-byte large (each cache line is 64-byte large).

For completeness, we also include the requests and responses for page-walk loads into the total network traffic. The request and response for a page-walk load are both assumed to be 8-byte large. It should be noted that the downgrade messages between L1s and L2 can be caused by page-walk loads. Consider the case that Linux has modified a page-table entry on core 0, and later a user process is accessing the page corresponding to the page-table entry. In this case, the page-walk load will hit in L2, but the L1 D of core 0 has the data in M state, which needs to be downgraded.

6.2 Results of Single-threaded Evaluation

6.2.1 Performance Analysis

Figure 6-2 shows the normalized execution time of each processor for each SPEC benchmark. The execution time of WMM-SI64, TSO-Base and TSO-SP has been normalized to that of WMM-Base, so the execution time of WMM-Base is always one. Lower is better. First notice that the single-threaded performance of WMM-Base and WMM-SI64 is very close; the average difference is almost zero. As for the comparison between TSO and WMM, TSO-Base is substantially slower than WMM in several benchmarks, including bzip2, gcc, hmmer and astar. The average performance overhead (in terms of execution time) of TSO-Base over WMM-Base is 5.6%, and the maximum can reach 18% (benchmark astar). However, after introducing store-prefetch in TSO-SP, most of the performance costs of TSO are recovered. The average performance overhead of TSO-SP over WMM-Base is only 0.8%, and the maximum is reduced to 4.2% (benchmark gcc). To understand the performance differences, recall that the single-threaded performance of TSO can be degraded because (1) loads are killed by cache evictions or (2) SQ becomes full. We now examine these two events.

Figure 6-3 shows the number of loads killed by cache evictions per thousand instructions in TSO-Base and TSO-SP. Lower is better. Kills caused by cache evictions

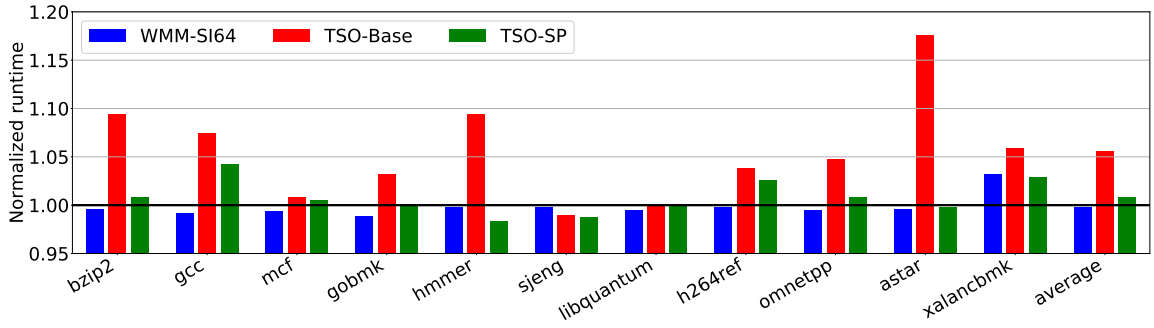


Figure 6-2: Execution time of WMM-SI64, TSO-Base and TSO-SP in SPEC benchmarks. Numbers are normalized to the execution time of WMM-Base.

are very rare. The number of kills per thousand instructions never exceed 0.02 times. Thus, the performance difference between TSO-Base and WMM-Base is not caused by cache-eviction kills. The low frequency of kills is expected, because the benchmarks were run on a single core. Figure 6-4 also shows the load-to-use latency for each processor. There is no observable difference between TSO and WMM processors in terms of load latency, i.e., the performance difference between TSO and WMM is unrelated to load execution.

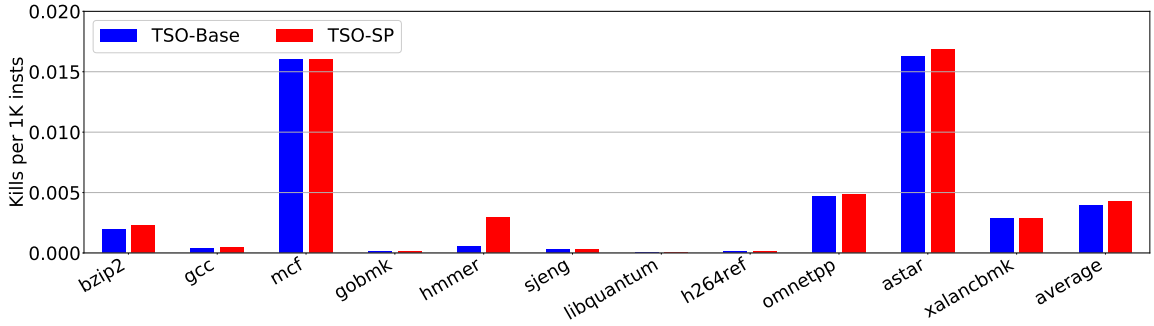


Figure 6-3: Number of loads being killed by cache evictions per thousand instructions in TSO-Base and TSO-SP in SPEC benchmarks

Figure 6-5 shows the number of cycles that SQ is full in each processor. The cycle counts are all normalized to the execution time of WMM-Base. Lower is better. In general, the SQ in TSO-Base becomes full more frequently than the SQ in WMM processors. For benchmarks that TSO-Base is substantially slower (i.e., bzip2, gcc, hmmer and astar), TSO-Base has much more SQ-full cycles than WMM-Base. In particular, the SQ of TSO-Base is full for about 27% of the execution time in

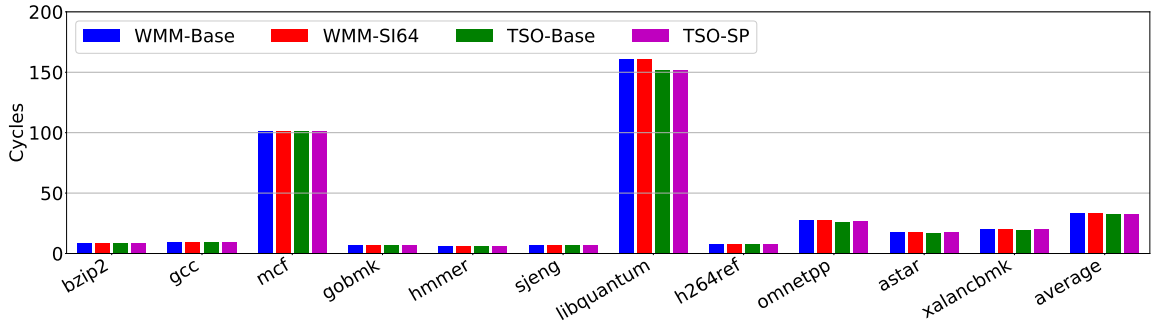


Figure 6-4: Load-to-use latency in SPEC benchmarks

benchmark astar, the benchmark where TSO-Base has the largest slowdown. That is, the slowdown of TSO-Base is mainly caused by the blocking dequeue scheme of SQ which makes SQ become full much more frequently. After introducing store-prefetch in TSO-SP, the SQ-full cycles have dropped significantly in benchmarks bzip2, hmmer and astar. This is why TSO-SP can reduce the performance overheads of TSO-Base.

One may notice that TSO-Base and TSO-SP both have lots of SQ-full cycles in benchmark libquantum. However, these full cycles do not translate to any performance overheads in Figure 6-2. This is because the load latency in benchmark libquantum is extremely high (Figure 6-4), and load execution becomes the most significant bottleneck for both TSO and WMM processors.

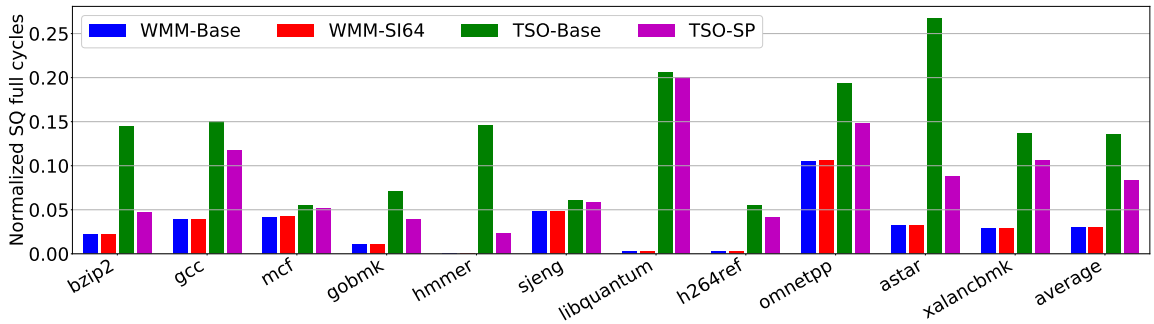


Figure 6-5: Cycles that SQ is full in SPEC benchmarks. The numbers are normalized to the execution time of WMM-Base.

6.2.2 Energy Analysis

Mis-speculative loads: The extra kills of speculative loads introduced in TSO processors are those caused by cache evictions. Figure 6-3 has already shown that such kills are extremely rare, so TSO-Base or TSO-SP should not have energy overheads caused by mis-speculations.

DRAM accesses: Figure 6-6 shows the number of DRAM accesses per thousand instructions in each processor. There is no observable differences in the number of DRAM accesses, i.e., the SI coherence protocol fails to save DRAM accesses in this case. This is not surprising, because even if both L1 I and D are exclusive to L2, the total cache size can increase from 1MB to 1MB+64KB, i.e., only 6.3% increase.

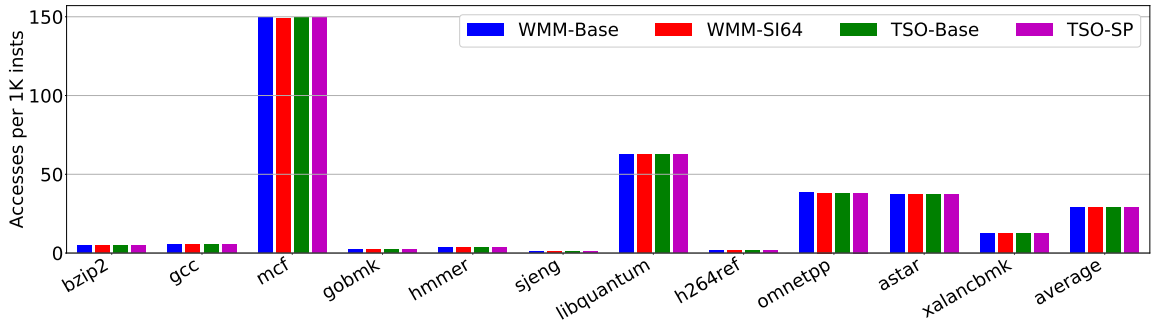


Figure 6-6: Number of DRAM accesses per thousand instructions in SPEC benchmarks

Network traffic between cores and L2: Figure 6-7 shows the number of bytes that have been transferred between cores and L2 in each processor. The number of bytes are normalized against the number of instructions. Again, the SI coherence protocol does not make a difference. This is because loads in the single-core processors are optimized to fetch data to E state in L1 D. This can avoid extra upgrade requests to L2 if the data will be modified later. Therefore, the SI coherence protocol can reduce only the invalidations to L1 I, which are caused by cache replacement in L2. Such invalidations are expected to be rare.

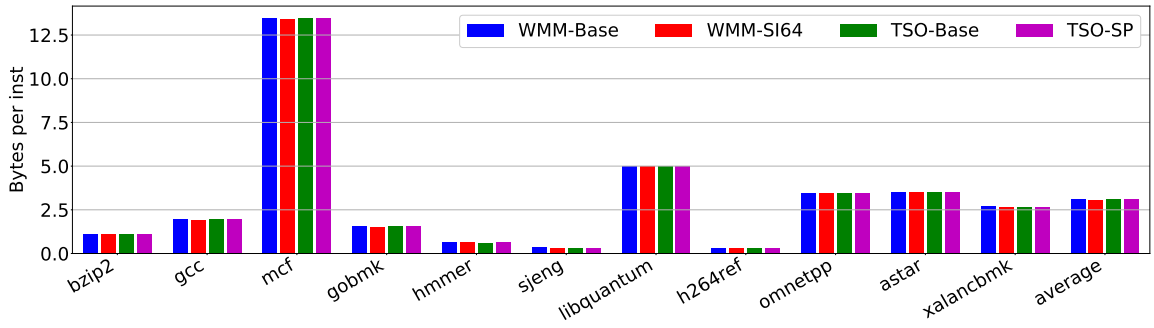


Figure 6-7: Number of bytes per instruction transferred between cores and L2 in SPEC benchmarks

6.3 Results of Multithreaded Evaluation: PARSEC Benchmark Suite

6.3.1 Performance Analysis

Figure 6-8 shows the normalized execution time of each processor for each PARSEC benchmark. The execution time of WMM-SI6, TSO-Base and TSO-SP has been normalized to that of WMM-Base, so the execution time of WMM-Base is always one. Lower is better. The performance of WMM-Base and WMM-SI64 is still similar, and on average there is almost no difference. We have also tried to increase the self-eviction threshold in WMM-SI from 64 to 256 and 1024. Figure 6-9 shows the execution time of WMM-SI64, WMM-SI256 and WMM-SI1024 normalized to that of WMM-Base. The changes in the execution time are insignificant, and the average performance of each WMM-SI processor is still very close to WMM-Base. That is, the SI coherence protocol, which is admitted only by memory models that can reorder data-dependent loads, does not improve performance. It should be noted that the implementation of SI protocol is indeed simpler than that of an MESI protocol.

As for the performance of TSO in Figure 6-8, TSO-Base is still slower than WMM-Base. The maximum performance overhead (in terms of execution time) of TSO-Base over WMM-Base can reach 9.5% (benchmark ferret), while the average is only 2.9%, which is pretty small. After introducing store-prefetch, the maximum performance overhead of TSO-SP over WMM-Base is reduced to 4.8% (benchmark ferret), and

the average is merely 1.9%. Since there are very little synchronizations in PARSEC benchmarks, we expect the reasons for the performance overheads of TSO to be similar to the single-threaded case. Thus, we examine the kills caused by cache evictions and the cycles that SQ is full.

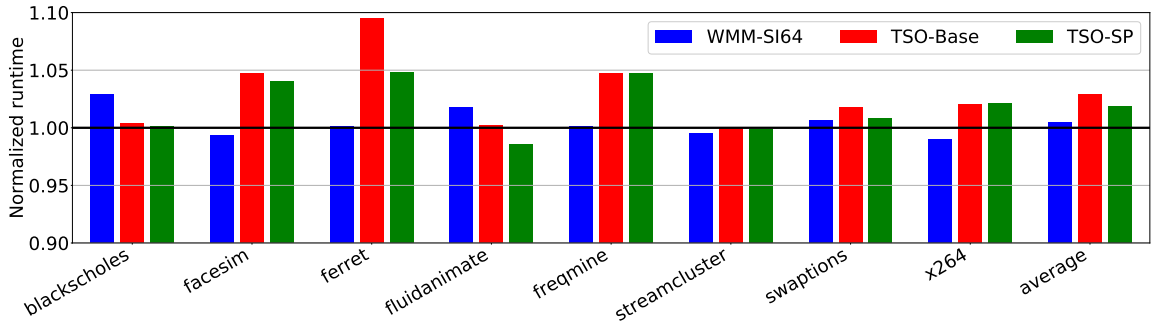


Figure 6-8: Execution time of WMM-SI64, TSO-Base and TSO-SP in PARSEC benchmarks. Numbers are normalized to the execution time of WMM-Base.

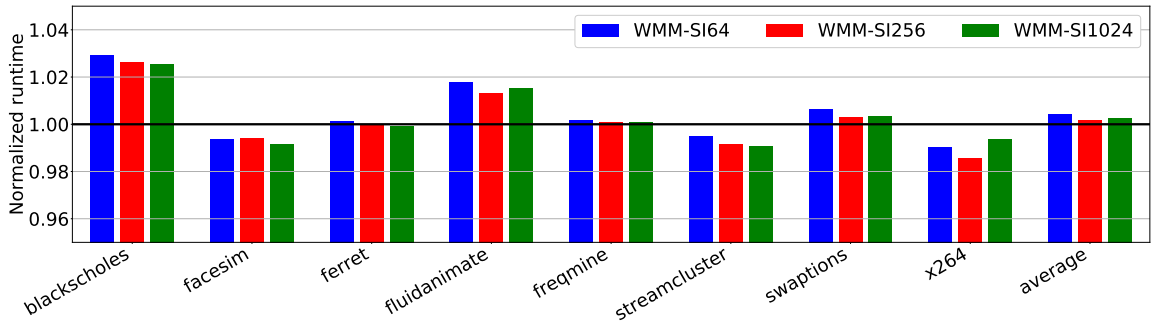


Figure 6-9: Execution time of WMM-SI64, WMM-SI256 and WMM-SI1024 in PARSEC benchmarks. Numbers are normalized to the execution time of WMM-Base.

Figure 6-10 shows the number of loads killed by cache evictions per thousand instructions in TSO-Base and TSO-SP. Lower is better. Since the number of instructions may vary across different processors in multithreaded benchmarks, we always use the number of user-level instructions in WMM-Base in the calculation of the number of events per thousand instructions. The average number of kills per thousand instructions never exceeds 0.015 times. Again, kills caused by cache evictions are very rare and should not affect the performance of TSO.

Figure 6-11 shows the number of cycles that SQ is full in each processor. The cycle counts are all normalized to the execution time of WMM-Base. Lower is better.

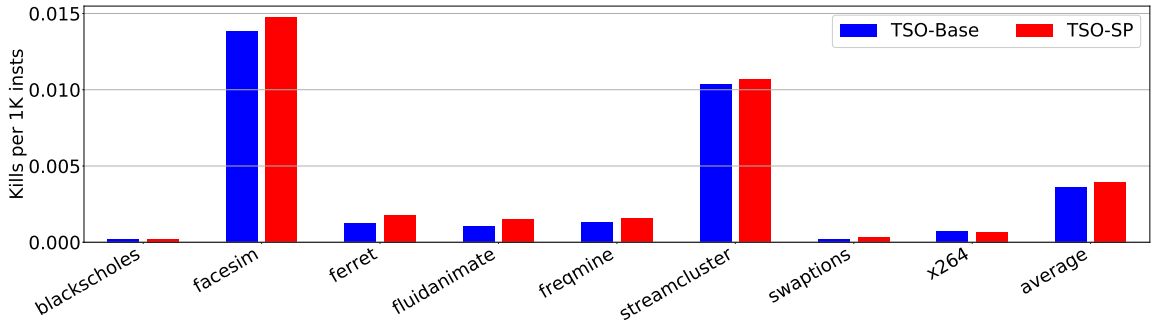


Figure 6-10: Number of loads being killed by cache evictions per thousand instructions in TSO-Base in PARSEC benchmarks

In benchmark ferret, there are a lot more SQ-full cycles in TSO-Base than that in WMM-Base. Although the store-prefetch in TSO-SP has reduced some of the SQ-full cycles, there are still more SQ-full cycles in TSO-SP than WMM-Base. This explains the performance overheads of TSO-Base and TSO-SP in benchmark ferret.

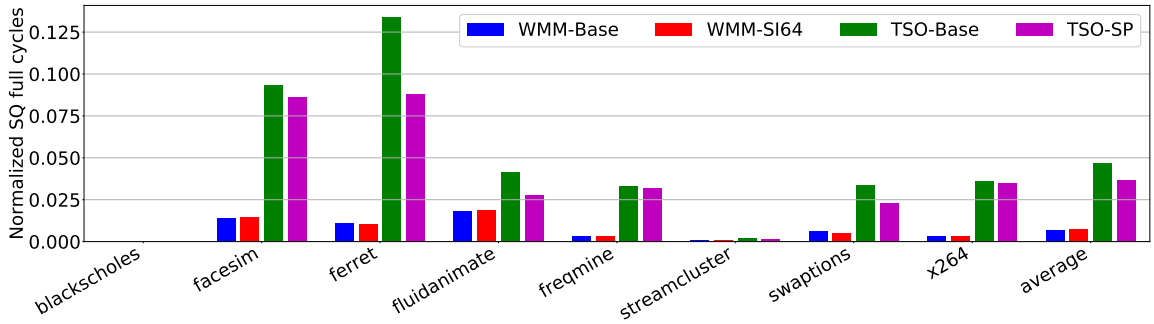


Figure 6-11: Cycles that SQ is full in PARSEC benchmarks. The numbers are normalized to the execution time of WMM-Base.

6.3.2 Energy Analysis

Mis-speculative loads: Figure 6-12 shows the number of mis-speculative loads per thousand instructions in each processor. These mis-speculative loads are killed by older stores, or older loads (WMM only), or cache evictions (TSO only). As we can see, the difference between the number of speculative loads is negligible. In benchmark fluidanimate, WMM-Base and WMM-SI64 both have more mis-speculative loads than TSO-Base. This is because the same-address load-load ordering in WMM can result in kills of speculative loads.

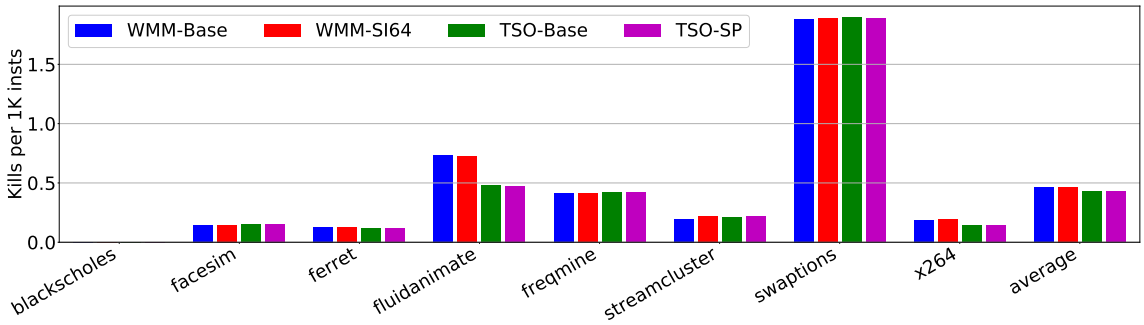


Figure 6-12: Number of mis-speculative loads per thousand instructions in PARSEC benchmarks

DRAM accesses: Figure 6-13 shows the number of DRAM accesses per thousand instructions in each processor. As expected, the SI coherence protocol in WMM-SI64 does not save any DRAM accesses.

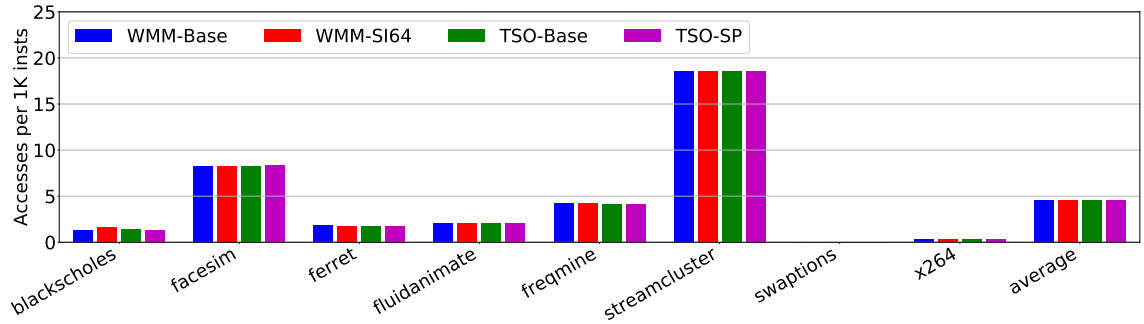


Figure 6-13: Number of DRAM accesses per thousand instructions in PARSEC benchmarks

Network traffic between cores and L2: Figure 6-14 shows the number of bytes that have been transferred between cores and L2 in each processor. The number of bytes are normalized against the number of user-level instructions in WMM-Base. WMM-Base, TSO-Base and TSO-SP have almost the same amount of network traffic, because they share the same cache hierarchy. WMM-SI64, instead of reducing network traffic, generates 50% more traffic than WMM-Base on average.

To understand the results of WMM-SI64, in Figure 6-15, we break down the network traffic into three categories: (1) upgrade requests and responses, (2) downgrade requests and responses, and (3) page-walk load requests and responses. Although the SI protocol in WMM-SI64 has successfully reduced a certain amount of down-

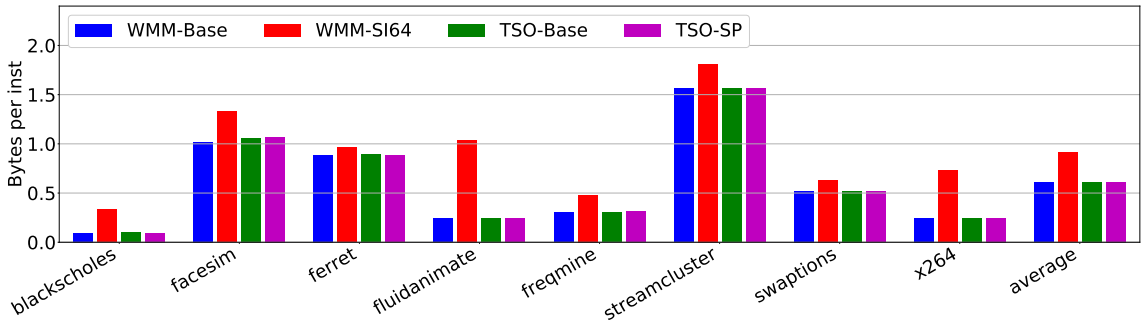


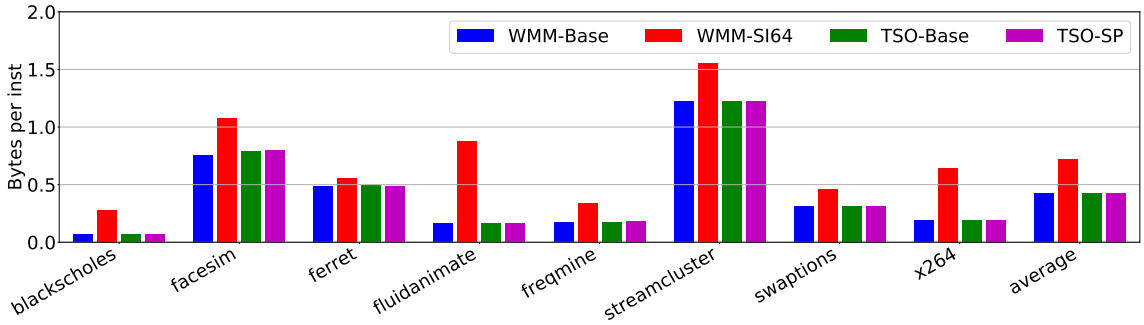
Figure 6-14: Number of bytes per instruction transferred between cores and L2 in PARSEC benchmarks

grade messages (Figure 6-15b), it increases the upgrade messages which constitute the majority of network traffic (Figure 6-15a). The increase of upgrade messages in WMM-SI64 is because the flush of L1 by Reconcile and the voluntary self-evictions both introduce more L1 misses. Raising the self-eviction threshold may mitigate the problem, so we tried to increase the threshold from 64 to 256 and 1024 (i.e., WMM-SI256 ad WMM-SI1024). Figures 6-16 and 6-17 show the upgrade traffic and overall traffic, respectively, of WMM-Base and WMM-SI processors with different self-eviction thresholds. In both figures, we can observe a slight drop in the network traffic of WMM-SI processors, but the traffic of WMM-SI processors is still higher than that of WMM-Base (and thus TSO-Base). Therefore, the SI coherence protocol is unable to reduce network traffic.

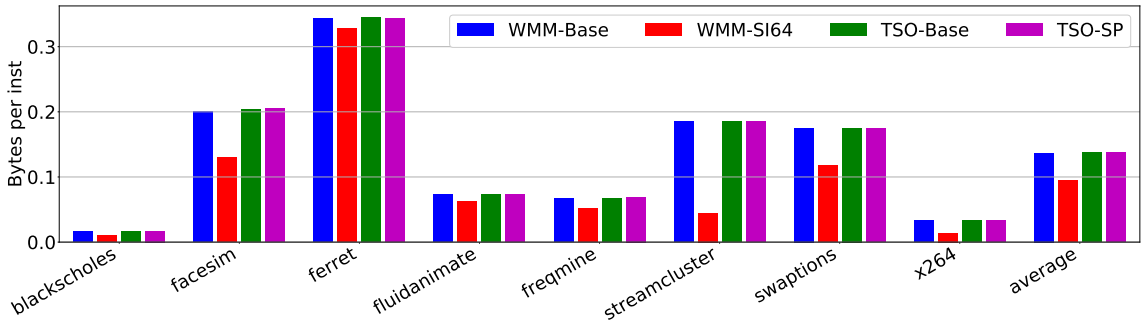
6.4 Results of Multithreaded Evaluation: GAP Benchmark Suite

6.4.1 Performance Analysis

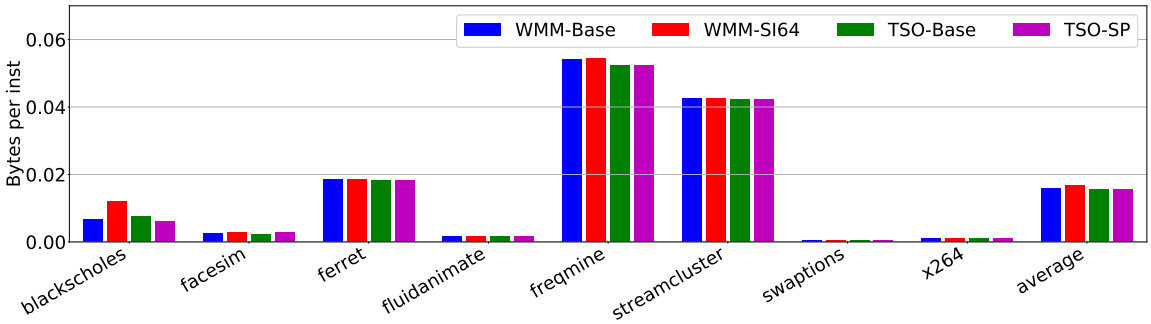
Figure 6-18 shows the normalized execution time of each processor for each GAP benchmark. The execution time of WMM-SI64, TSO-Base and TSO-SP has been normalized to that of WMM-Base, so the execution time of WMM-Base is always one. Lower is better. The performance results of GAP benchmarks are quite different from



(a) Upgrade requests and responses



(b) Downgrade requests and responses



(c) Page-walk load requests and responses

Figure 6-15: Breakdown of Number of bytes per instruction transferred between cores and L2 in PARSEC benchmarks

those of SPEC and PARSEC benchmarks. WMM-SI64 is substantially slower than WMM-Base in many GAP benchmarks. The average performance overhead (in terms of execution time) of WMM-SI64 over WMM-Base is 6.1%, and the maximum can reach 15%. We have also tried to increase the self-eviction threshold in WMM-SI from 64 to 256 and 1024. Figure 6-19 shows the execution time of WMM-SI64, WMM-SI256 and WMM-SI1024 normalized to that of WMM-Base. There is no observable differences on the performance of the three WMM-SI processors, so the SI coherence

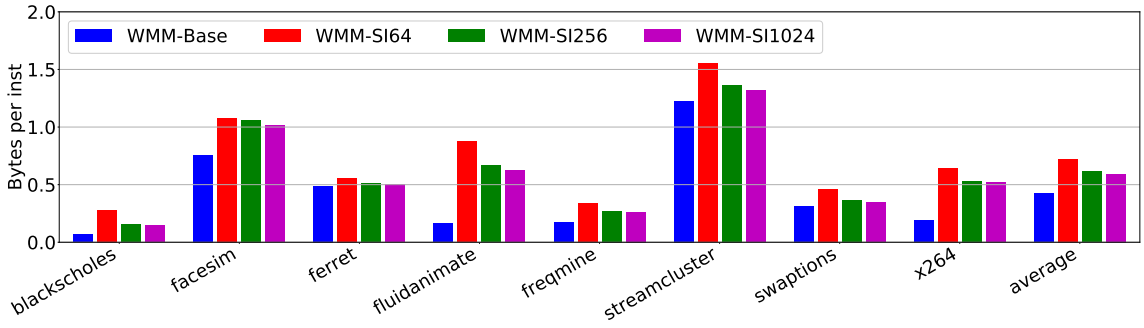


Figure 6-16: Number of bytes per instruction transferred for upgrade requests and responses between cores and L2 in WMM-Base and WMM-SI processors in PARSEC benchmarks

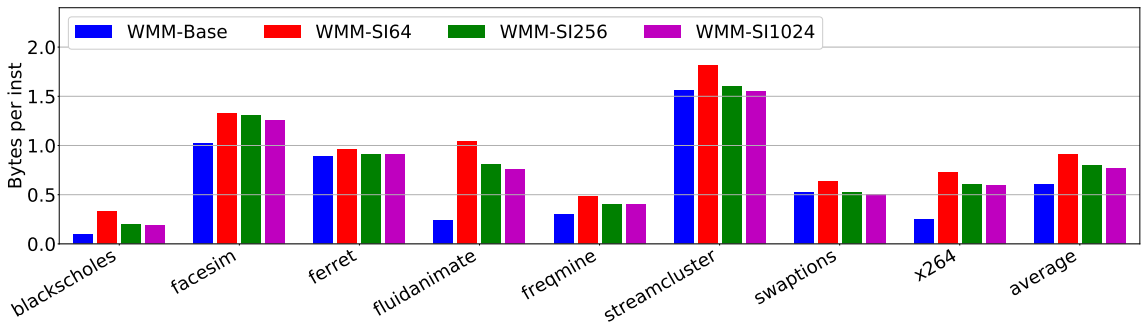


Figure 6-17: Number of bytes per instruction transferred between cores and L2 in WMM-Base and WMM-SI processors in PARSEC benchmarks

protocol indeed hurts performance in case of GAP benchmarks.

As for the performance of TSO in Figure 6-18, both TSO-Base and TSO-SP turn out to be faster than WMM-Base in many benchmarks (e.g., bc, bfs and cc). TSO-Base and TSO-SP can reduce the execution time of WMM-Base by 4.5% and 5.8%, respectively, on average, and maximumly by 10% and 12% (benchmark bc), respectively.

These performance differences can be understood by looking at the number of fences in the programs. It should be noted that Commit fences are unlikely to make WMM processors slower than TSO, because stores in TSO are already serialized. We focus on Reconcile fences (including both fence instructions and acquire bits on atomic instructions), which stall load execution in WMM processors. It should be noted that fences in TSO-Base and TSO-SP (including both fence instructions and atomic instructions) do not stall load execution, even though these instructions have

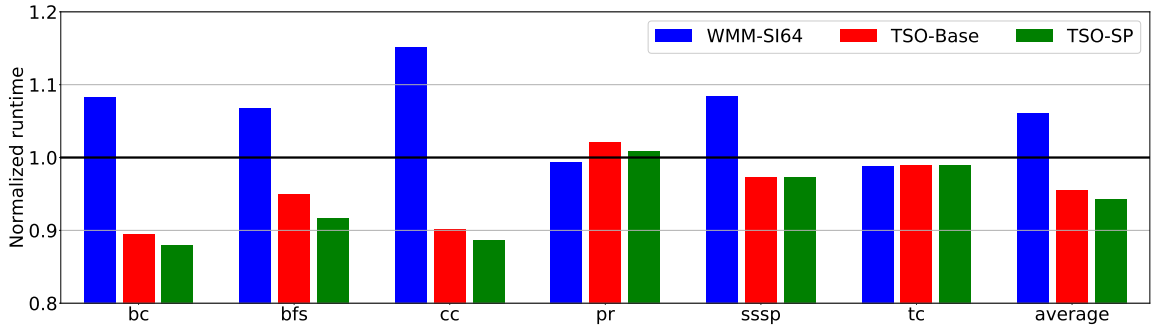


Figure 6-18: Execution time of WMM-SI64, TSO-Base and TSO-SP in GAP benchmarks. Numbers are normalized to the execution time of WMM-Base.

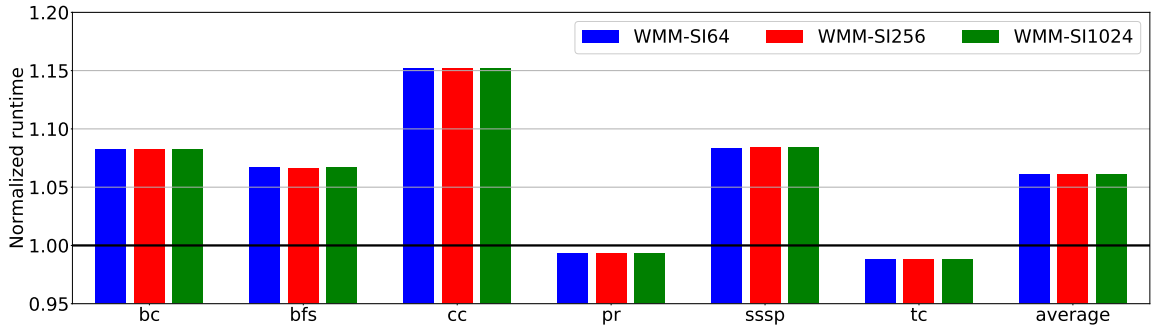


Figure 6-19: Execution time of WMM-SI64, WMM-SI256 and WMM-SI1024 in GAP benchmarks. Numbers are normalized to the execution time of WMM-Base.

ordering semantics according to the TSO memory model. These instructions stall only the dequeue of LQ, i.e., they keep younger loads susceptible to cache evictions until they are committed.

To understand the benefits of speculating loads over fences in TSO, consider the case of a producer thread and a consumer thread. The producer thread first writes chunks of data to memory and then releases the lock. The consumer thread wakes up some amount of time after the lock is released, possibly because Linux have previously scheduled the consumer thread off while it is waiting for the lock. In this case, in the consumer thread, the atomic instruction to grab the lock and the normal loads to consume the data do not need to be ordered. This is because all the data is already in memory by the time the consumer thread wakes up. In this case, WMM still requires a Reconcile fence between the atomic instruction and normal loads, and the fence will impose unnecessary serialization. In TSO, all the normal loads can be issued before

the atomic instruction (which has full-fence semantics) completes, and the loads will not be killed by cache evictions.

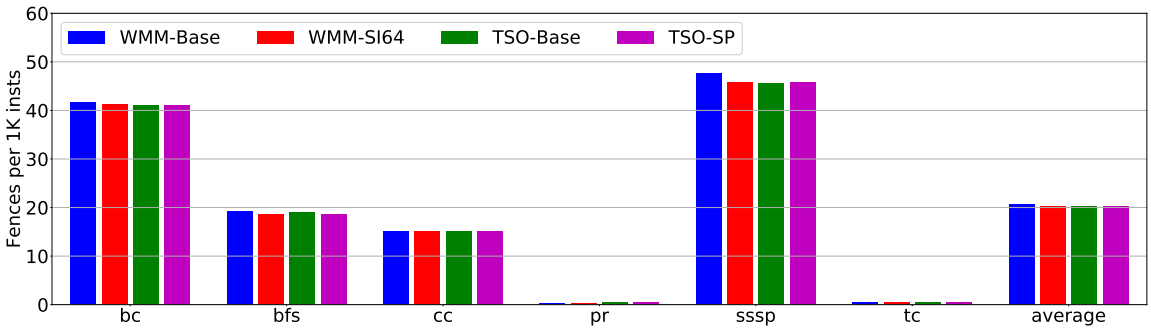


Figure 6-20: Number of Reconcile fences per thousand instructions in WMM-Base and WMM-SI64, and the number of full fences (including atomics) per thousand instructions in TSO-Base and TSO-SP

Figure 6-20 shows the number of Reconcile fences per thousand instructions in WMM-Base and WMM-SI64, and the number of fences per thousand instructions in TSO-Base and TSO-SP. Atomic instructions are also considered as fences in TSO-Base and TSO-SP. Note that we use the number of user-level instructions in WMM-Base to calculate the fence counts per thousand instructions. In benchmarks bc, bfs, cc and sssp, there are significant amount of Reconcile fences which stall the load execution in WMM-Base and flush L1 D in WMM-SI. Increasing the self-eviction threshold in WMM-SI cannot mitigate the performance penalty of Reconcile fences, so we do not see any changes in performance in Figure 6-19. In TSO-Base and TSO-SP, even though the number of fences is similar, load execution is not stalled by fences. Thus, if loads that speculate over fences are not killed, then TSO-Base and TSO-SP will have better performance. Figure 6-21 shows the number of loads killed by cache evictions per thousand instructions in TSO-Base and TSO-SP. As we can see, kills by cache evictions are indeed rare in TSO implementations. This results in the performance improvement of TSO implementations over WMM-Base in benchmarks bc, bfs and cc (Figure 6-18).

One may notice that benchmark sssp has lots of Reconcile fences but TSO-Base and TSO-SP still have similar performance to WMM-Base. This is because all processors execute a lot more *system instructions* in benchmark sssp than in other benchmarks.

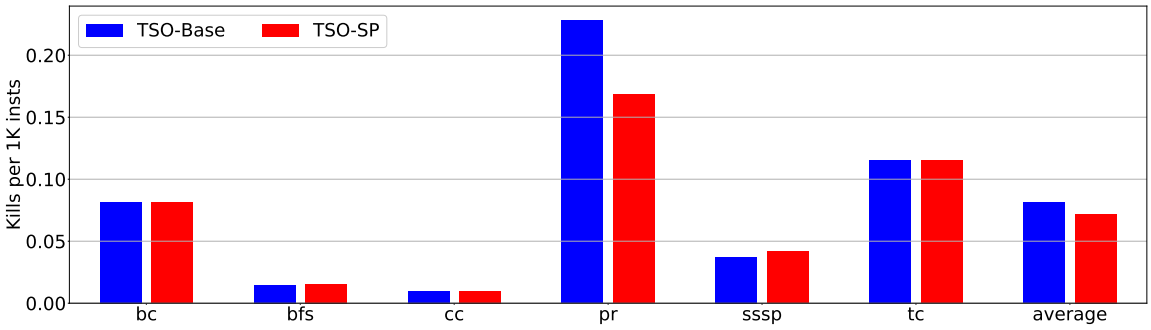


Figure 6-21: Number of loads being killed by cache evictions per thousand instructions in TSO-Base and TSO-SP in GAP benchmarks

System instructions deal with system special registers (CSRs in RISC-V terminology), and are difficult to be implemented in a pipeline fashion. Since system instructions are expected to be rare, RiscyOO simply drains the whole ROB before execution each system instruction (similar to cpuid in Intel processors). An example system instruction is the csrrw instruction which directly reads and writes an CSR. Figure 6-22 shows the number of system instructions per thousand instructions in WMM-Base, TSO-Base and TSO-SP. Again, we use the number of user-level instructions in WMM-Base as the base to calculate the counts per thousand instructions. In benchmark sssp, there are more than 40 system instructions per thousand instructions, far more than other benchmarks. The number of system instructions in benchmark sssp is even similar to the number of Reconcile fences. The slowdown caused by system instructions apply to all processors and dilute the performance difference caused by Reconcile fences.

The abundant system instructions benchmark sssp are likely to be caused by the frequent system calls made in benchmark sssp. Figure 6-23 shows the number of system calls per thousand instructions in WMM-Base and TSO-Base. Benchmark sssp has made far more system calls than other benchmarks. These system calls might be calling synchronization facilities in Linux. If we compare Figures 6-22 and 6-23, we can see that the number of system instructions is roughly proportional to the number of system calls in each benchmark.

Optimizing fence insertions in software: We notice that many fences in GAP

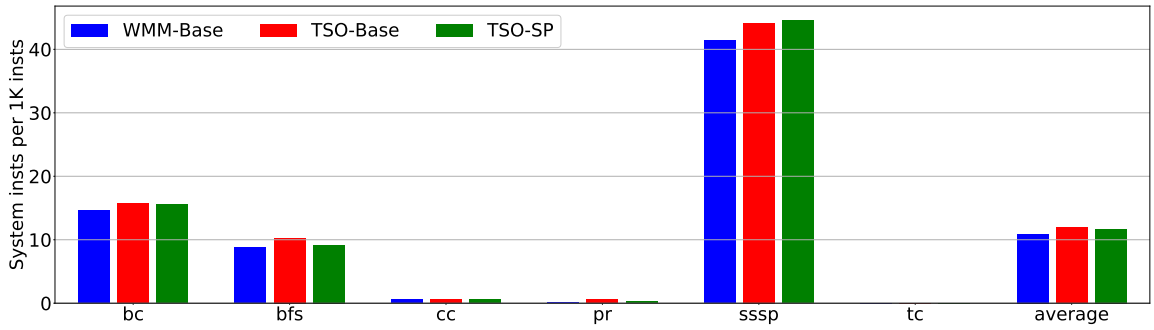


Figure 6-22: Number of system instructions per thousand instructions in GAP benchmarks

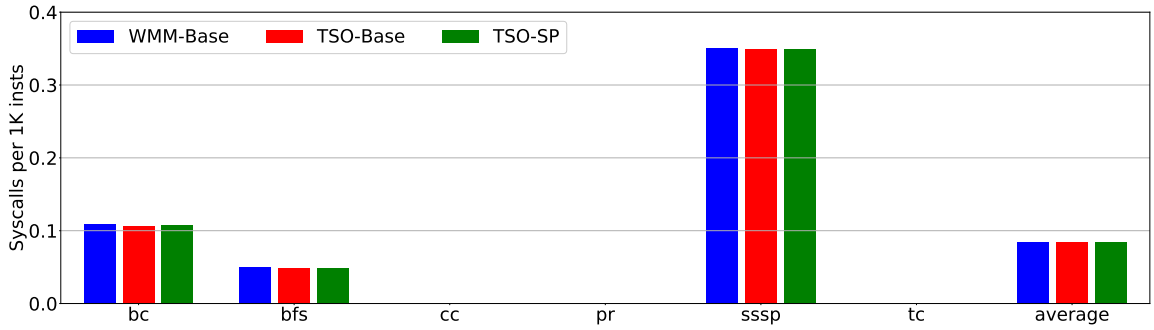


Figure 6-23: Number of system calls per thousand instructions in GAP benchmarks

benchmarks come from the use of built-in functions of the GCC compiler for atomic read-modify-writes (e.g., `__sync_bool_compare_and_swap`). By default, the compiler will translate these built-in functions to RISC-V atomic instructions surrounded by fences. In GAP benchmarks, the purpose of using these atomic read-modify-writes is to let multiple threads update shared memory locations concurrently, while the order of updates does not matter. Therefore, the fences injected by the compiler may not be necessary.

A programmer who is familiar with the algorithms of the benchmarks and the C++ concurrent semantics can pass an additional argument to these built-in functions to specify what fences should be created by the compiler. This manual optimization of fence insertions can benefit the performance of WMM processors by reducing the number of fences. However, it will not affect TSO processors because atomic read-modify-writes are like fences in TSO. To understand the maximum performance impact of this optimization, we remove all the fences associated with all the built-

in function calls for atomic read-modify-writes by passing `__ATOMIC_RELAXED` as the additional argument. Then we rerun the benchmarks on the WMM-Base processor, and we refer to these new results as *WMM-Relax* because we are using C++ relaxed atomics.

Figure 6-24 shows the number of Reconcile fences per thousand instructions in WMM-Base and WMM-Relax, and the number of fences (and atomics) per thousand instructions in TSO-Base and TSO-SP. After removing fences associated with the built-in atomic read-modify-writes, the number of fences in WMM-Relax is much lower than WMM-Base, particularly for the three benchmarks (bc, bfs, and cc) where WMM-Base is slower than TSO processors. Therefore, we expect the performance of WMM-Relax to be better than WMM-Base.

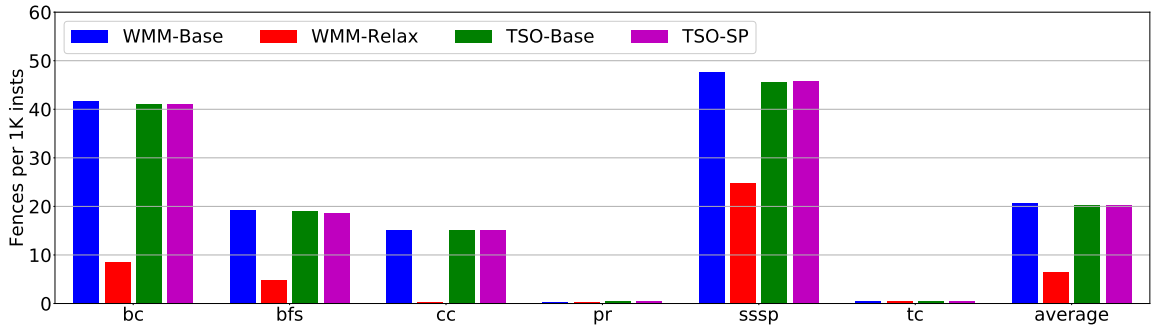


Figure 6-24: Number of Reconcile fences per thousand instructions in WMM-Base and WMM-Relax, and the number of full fences (including atomics) per thousand instructions in TSO-Base and TSO-SP

Figure 6-25 shows the execution time for each GAP benchmark. The execution time of WMM-Relax, TSO-Base and TSO-SP is normalized to that of WMM-Base, so the execution time of WMM-Base is always one. Lower is better. As expected, the performance of WMM-Relax is better than WMM-Base. However, WMM-Relax fails to achieve better performance than TSO-SP. The average performance of WMM-Relax and TSO-SP are the same.

This experiment shows that software programmers may insert unnecessary fences to ensure correctness or portability. However, even if we remove these unnecessary fences, WMM still does not provide better performance than TSO. More importantly, removing these fences requires deep understandings of both the algorithm and the

high-level language model. High-level language models are in fact still an active field of research [39, 34, 33, 72, 106]. It should be noted that we did not prove the correctness of removing all the fences in this experiment, though we have not seen failures of WMM-Relax.

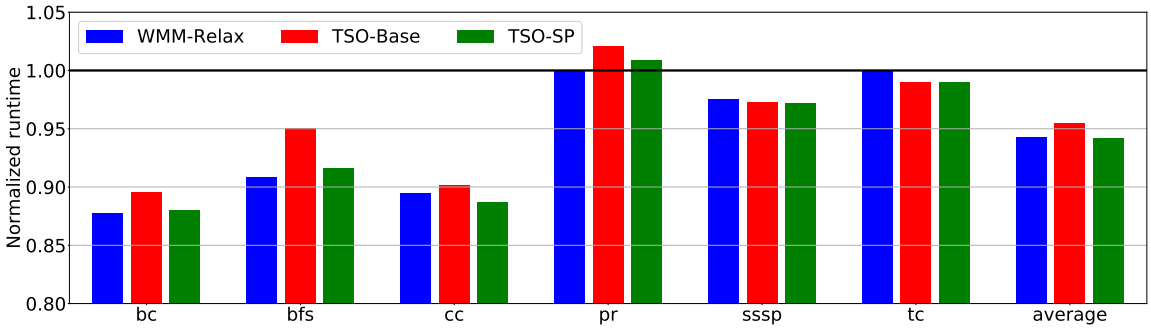


Figure 6-25: Execution time of WMM-Relax, TSO-Base and TSO-SP in GAP benchmarks. Numbers are normalized to the execution time of WMM-Base.

6.4.2 Energy Analysis

Mis-speculative loads: Figure 6-26 shows the number of mis-speculative loads per thousand instructions in each processor. On average, the difference across four processors is less than 0.1 mis-speculative loads per thousand instructions, which is very small. In benchmark sssp, WMM has roughly 0.5 more mis-speculative loads per thousand instructions than TSO because of the same-address load-load ordering in WMM.

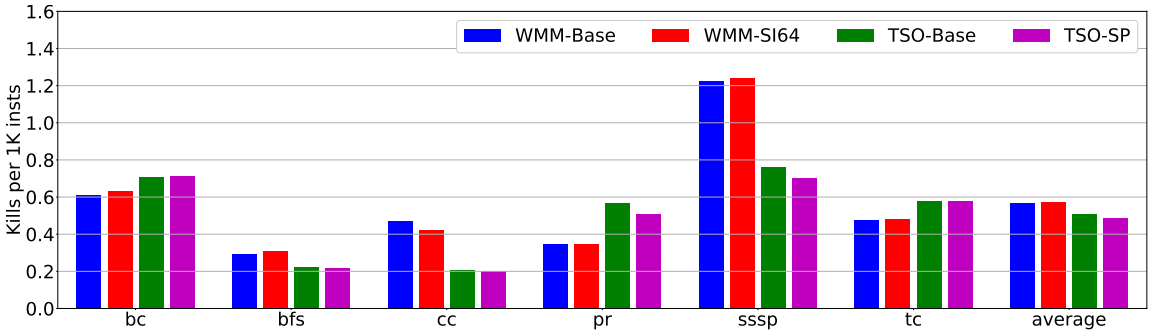


Figure 6-26: Number of mis-speculative loads per thousand instructions in GAP benchmarks

DRAM accesses: Figure 6-27 shows the number of DRAM accesses per thousand instructions in each processor. There is no observable difference across different processors, i.e., the SI coherence protocol does not reduce DRAM accesses.

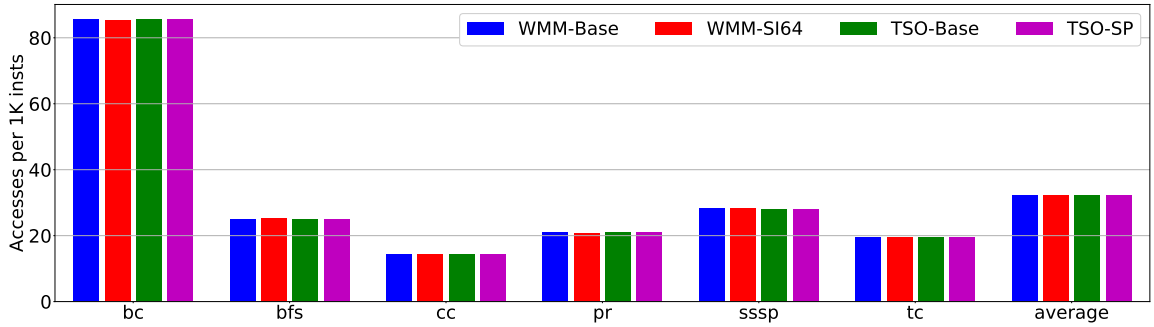


Figure 6-27: Number of DRAM accesses per thousand instructions in GAP benchmarks

Network traffic between cores and L2: Figure 6-28 shows the number of bytes that have been transferred between cores and L2 in each processor. The number of bytes are normalized against the number of user-level instructions in WMM-Base. The results are very similar to those of PARSEC benchmarks. WMM-Base, TSO-Base and TSO-SP have almost the same amount of network traffic, while the average traffic of WMM-SI64 is almost twice of that of WMM-Base, TSO-Base and TSO-SP. The overheads of WMM-SI64 is still because Reconcile fences and self-evictions induce more L1 misses and thus more upgrade messages. We have tried to increase the self-eviction threshold to 256 and 1024, but this does not change the amount of network traffic at all as shown in Figure 6-29. This is because the overheads of WMM-SI is caused mostly by the frequent Reconcile fences that flush L1 data in the S state.

6.5 ASIC Synthesis[2]

We performed topographical synthesis using Synopsys's Design Compiler on the uniprocessor configuration of the four processors, i.e., WMM-Base, WMM-SI64, TSO-Base and TSO-SP. Topographical synthesis is a timing-driven synthesis which performs placement heuristics and includes resistive and capacitive wire delays in the

[2]ASIC synthesis is done by Andrew Wright.

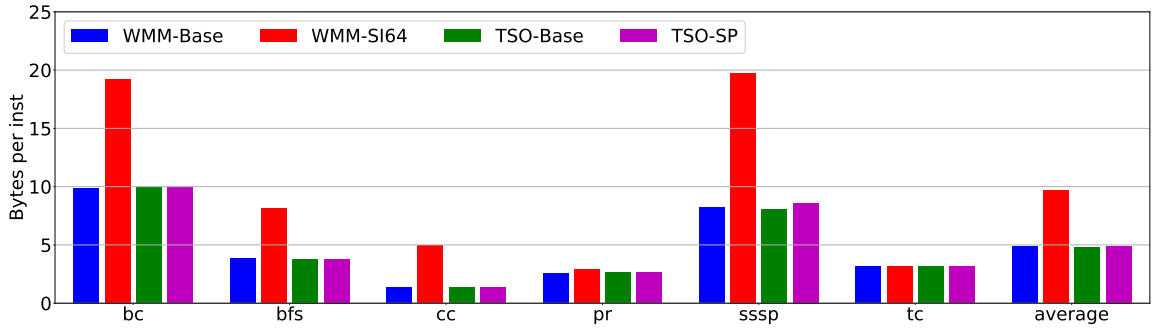


Figure 6-28: Number of bytes per instruction transferred between cores and L2 in GAP benchmarks

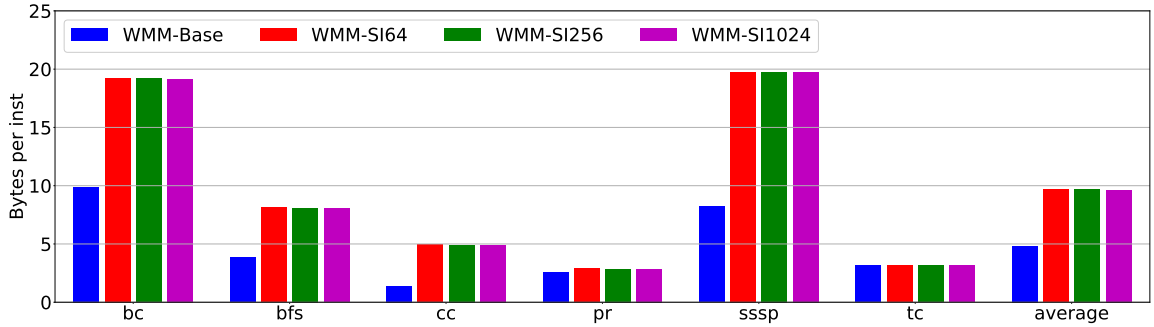


Figure 6-29: Number of bytes per instruction transferred between cores and L2 in WMM-Base and WMM-SI processors in GAP benchmarks

timing model. Thus, it reduces the gap between post-synthesis results and post-placement and routing results. The synthesis flow used a 32 nm SOI technology (same as Section 5.5). We synthesized only the logic in the core (i.e., without L2 cache). We did not consider the latency or area of the floating point unit, the integer multiplier and divider, or any SRAMs.

All processors can be clocked at 1.1GHz (same as the clock speed of RiscyOO-B in Section 5.5). Figure 6-30 shows the area of each processor. The area numbers are normalized to the area of WMM-Base (0.85mm^2). There is no big difference between the areas of different processors. Compared to WMM-Base, TSO-Base and TSO-SP turn out to be more area efficient. TSO-Base can save 3.6% of the area of WMM-Base, and TSO-SP can save 3.2%. This is possibly because, compared to WMM, (1) TSO implementations do not have a store buffer, and (2) loads in TSO do not search for same-address loads or fences at issue time (though cache evictions need to search

all the loads).

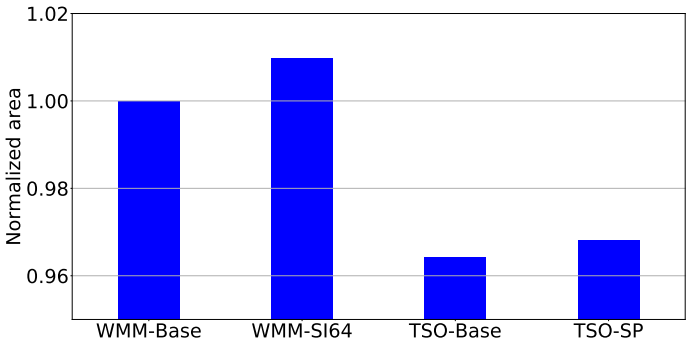


Figure 6-30: Normalized area of each processors. Numbers are normalized to the area of WMM-Base.

6.6 Summary

We compared the PPA of WMM and TSO using small out-of-order multiprocessors and benchmarks written using portable multithreaded libraries and compiler built-ins. Our evaluation shows that TSO without store-prefetch has 5.6% average overheads over WMM in single-threaded performance. The overheads are mainly because the slow dequeue of SQ in TSO makes SQ become full and thus stalls register renaming. After introducing store-prefetch to TSO, most of the performance overhead of TSO is eliminated and the average overhead drops to 0.8%. In multithreaded benchmarks with abundant synchronizations, TSO can actually be faster than WMM. For example, in GAP benchmarks, TSO-Base can reduce the execution time of WMM-Base by 4.5% on average, and maximumly by 10%. This is because the frequent fences in WMM serialize load execution while not all the fences are really needed at runtime. TSO processors can easily have loads speculate over fences and thus get better performance as long as the speculation ends up to be successful, i.e., not killed by cache evictions. Although some of these fences may be unnecessary (e.g., because programmers are being conservative to ensure correctness and portability), our experiment shows that removing these unnecessary fences still cannot make WMM outperform TSO.

It should be noted that the penalty of fences in WMM may be exacerbated if the processor becomes larger, because a fence can affect more in-flight instructions. The speculative loads in TSO may also become more susceptible to cache evictions, so a predictor may be needed to indicate when to speculate over fences. We do not observe frequent failures of speculative loads possibly because our ROB size is small. It is possible to implement a WMM processor that also let loads speculate over Reconcile fences and monitor cache evictions. However, the condition for when a load is no longer affected by cache evictions in WMM is more complicated than that in TSO. Resorting to a simple condition (e.g., when the load is dequeued from LQ) makes WMM implementation equivalent to a TSO/PSO implementation. Even if a WMM processor allows loads to speculate over Reconcile fences and implements a precise checking logic, it is still unclear whether the WMM processor, which is more complicated than TSO processors in both hardware implementation and software programming, can provide strictly better performance than TSO processors. Our experiment on removing fences in GAP benchmarks implies that speculation over fences in WMM can at most make the performance of WMM equal to, but not better than, that of TSO.

As for energy, TSO is almost the same as WMM in terms of the number of mis-speculative loads, DRAM accesses and network traffic between cores and L2. As for area, the area of the core logic of TSO is actually 3% less than that of WMM. That is, we do not observe any benefits of weak memory models in terms of area or energy efficiency.

Another observation is about the SI coherence protocol, which can cause the reordering of data-dependent loads. The SI protocol is admitted only by WMM but not TSO. Although the SI protocol is easier to implement and could be more scalable, it does not improve performance or energy consumption. In fact, in multithreaded benchmarks with frequent synchronizations, the SI protocol can degrade performance and cost significantly more energy. For example, in GAP benchmarks, WMM-SI is 12% slower than TSO-Base and generates 100% more network traffic than TSO-Base on average. *Weak memory models can have more flexible implementations, but which*

are not necessarily the better ones.

The insignificant difference between TSO and WMM also prompts us to rethink if weak memory models are really necessary.

Chapter 7

Conclusion

7.1 Contributions on Weak Memory Models

This thesis has taken a constructive approach to study weak memory models. We have clarified and simplified the definitions of weak memory models, and our evaluation using small out-of-order processors and portable multithreaded benchmarks shows that weak memory models have little benefits over TSO in terms of performance/power/area (PPA). In particular, we have made the following three contributions.

Constructing the common base model for weak memory models: We constructed GAM, the common base model for weak memory models with *atomic* memory. The construction of GAM starts from the constraints on execution orders in uniprocessors, and then extends the constraints to a multiprocessor setting. The construction procedure relates the ordering constraints in the memory-model definition to the microarchitectural optimizations, and reveals the places where memory models can differ from each other. Our evaluation shows that these differences often have little impact on performance, but can affect the complexity of model definitions. In these cases, GAM is defined to match the common assumptions made in multithreaded programs. We have not tried to parameterize the definition of GAM by the different choices in these cases.

Simplifying the definitions of weak memory models: We identified that the source of complexity in the definitions of weak memory models (e.g., GAM) is al-

lowing load-store reordering (i.e., allowing a younger store to be executed before an older load). By forbidding load-store reordering, we constructed a new weak memory model, WMM, which has much simpler axiomatic and operational definitions than GAM. In particular, the operational definition of WMM can be described in the way of instantaneous instruction execution (I^2E), which is also used in the operational definitions of strong memory models like SC and TSO. Our evaluation shows that forbidding load-store reordering has little impact on performance.

Comparing the performance/power/area (PPA) of weak memory models against TSO: We implemented different out-of-order multiprocessors of WMM (the representative for weak memory models) and TSO. Although weak memory models involve much more complexity in their definitions than TSO to admit more flexible implementations, there is no clear advantage of weak memory models over TSO in terms of PPA according to our evaluation. In some cases, TSO can even outperform weak memory models which suffer from the penalty of fence instructions. The simple definition of TSO makes it easy for TSO implementations to speculate beyond the memory ordering required by the model definitions (e.g., the ordering requirement of fences and atomics in TSO). However, it is more difficult to do so in case of weak memory models. This is because the definitions of weak memory models are too complicated, and it becomes difficult to figure out the conditions to check if speculations beyond the required ordering succeed or not. As a result, if we do not want extra complexity in the weak-memory-model implementations, then weak memory models will suffer from the penalty of fence instructions. To make matters worse, software programmers may insert unnecessary fences to increase their confidence in the correctness and portability of their programs. These superfluous fences can degrade further the performance of weak memory models. Even if we spend efforts in improving the performance of weak memory models by optimizing the insertion of fences in software and implementing speculation over fences in hardware, our experiment implies that these optimizations still cannot make weak memory models offer strictly better performance than TSO.

7.2 Future Work on Evaluating Weak Memory Models and TSO

Our evaluation of WMM versus TSO has considered only small out-of-order multiprocessors. As a guidance for future work to prove or disprove the practical usefulness of weak memory models, here we discuss the impact of memory models in case of other types of processors.

7.2.1 High-Performance Out-of-Order processors

In this case, both weak-memory-model machines and TSO machines will execute instructions, in particular loads, out of order and speculatively. The performance bottlenecks of weak memory models are from fence instructions, while the performance of TSO can be hurt because of the in-order dequeue of the store queue and the squashes caused by L1 evictions. Since high-performance out-of-order processors will have larger ROBs and load-store queues (LSQs) than those in our evaluation, fence instructions in weak memory models will stall more instructions and have larger performance penalty than that in our evaluation. In case of TSO, the larger LSQ may keep more speculative loads and L1 evictions may be more likely to create squashes. Now we consider the effects of these potential performance bottlenecks in case of single-threaded and multithreaded programs, respectively.

Single-threaded programs: There are no fence instructions in single-threaded programs, so the performance of single-threaded programs on a weak-memory-model multiprocessor is almost the same as that on a uniprocessor. In case of TSO, squashes by L1 evictions can be avoided if the processor can identify that the load addresses are private (e.g., by page coloring or compiler support [132]). However, the store queue will still be a bottleneck for TSO, and we may see some very small performance benefits of weak memory models similar to those in our evaluation (Section 6.2).

Multithreaded programs: In case of multithreaded performance, fence instructions and L1-eviction squashes will be the major concerns for weak memory models and

TSO, respectively. As mentioned earlier, influence of fences and L1 evictions may be amplified when the processor becomes larger. Therefore, weak-memory-model implementations may also need to implement speculative execution over fences to reduce the penalty of fences. Besides, we may need predictors for both weak memory models and TSO to predict whether we should speculate beyond the memory ordering required by the memory-model definition. With a perfect predictor, there should be little difference between the performance of weak memory models and TSO. Future work can evaluate the practical effectiveness of these predictors. In case the predictors are not effective for TSO, we can also consider adding new instructions to give hints to TSO hardware on whether speculation should be turned on or off, but these new instructions never affect the correctness of the program. (These new instruction are unnecessary for weak memory models because fence instructions already play the role.)

Future work can also investigate the insertion of fences in software, and evaluate the performance impact of unnecessary fences which may be inserted by programmers to increase their confidence. We do not think there are different choices regarding the fence insertions in common synchronization primitives (e.g., locks and conditional variables) in standard multithreaded libraries (e.g., pthread). Therefore, the focus should be on lock-free algorithms, which are typically written under the assumption of SC. It is possible that it may require a fence after every instruction to make the algorithm work correctly, and thus, there may be no difference between weak memory models and SC/TSO.

7.2.2 Energy-Efficient In-Order Processors

For energy-efficient in-order processors, a weak-memory-model implementation can stream memory instructions into the memory system in order and non-speculatively. A load instruction can be committed before it returns from the memory system, and the processor does not need to track the address of this in-flight load. The processor only needs to know which register does not have valid data yet (i.e., a stall-on-use policy). The performance of this implementation is limited by how soon the processor

will be stalled because of a read-after-write hazard on a pending load result. The load slice core microarchitecture [43] aims to reduce this bottleneck, so it may improve the performance of weak memory models.

In case of TSO, if the processor can issue multiple loads to the memory system, then the processor typically needs a load queue (LQ) to keep track of the addresses of these loads to detect violation of the memory ordering required by TSO. A small LQ can stall load instructions at runtime if the LQ becomes full, while a larger LQ may increase the chip area and energy consumption. Recent work [118] on non-speculative and out-of-order execution of loads for TSO may help reduce the pressure on the LQ.

Future work can compare the different implementations of in-order processors mentioned above.

7.2.3 Embedded Microcontrollers

A microcontroller may consist of simple in-order cores with a array of SRAM banks as memory. There may not be caches in each core, so many speculative techniques of TSO, which rely on snooping L1-cache evictions, cannot be applied. If each microcontroller processor does not keep more than one memory request in the memory system, then the microcontroller effectively implements SC. However, if the microcontroller still wants to issue multiple memory requests in parallel, then future work could investigate whether TSO can be implemented efficiently. If TSO is not suitable for microcontrollers, then we also need to answer if there is a weak memory model that can admit all the microcontroller implementations. This question is important because designers of microcontrollers may want maximum freedom in implementations to achieve the desired energy or area efficiency. If we cannot find such a weak memory model, then we can consider what software programming patterns are common in embedded applications, and define the architectural support (e.g., fences) to enforce SC in these programming patterns. This is similar to the definition of SC-for-DRF [19], i.e., the semantics of the hardware is defined only for programs written in certain patterns, but not for all programs.

7.3 Future Work on High-Level Language Models

The memory models of high-level languages, e.g., C++, are influenced by the weak memory models of commercial processors, e.g., ARM [72, 40, 108]. Even though language researchers have developed C++ compilers that perform SC transformations only [100], the compilers still need to pay the price of inserting more fences to prevent hardware from reordering instructions if the goal is to enforce a strong memory model for C++. Given our evaluation results that weak memory models do not have obvious benefits over strong memory models in terms of the PPA of processors, there may be new opportunities for simplifying the memory models of high-level languages.

7.4 Other Contributions and Future Work

A side-product of this memory model study is the CMD framework for processor designs. In CMD, the behaviors of a module are fully captured by the interface information, including the conflict matrix, of the module. Thus, a module can be refined and composed with other modules by relying on the interface information only. We have developed an out-of-order superscalar cache-coherent multiprocessor, RiscyOO, using CMD. Both the synthesis results and the performance results of RiscyOO are very encouraging.

The RiscyOO processor can be used in much broader areas than studying memory models, and one eminent example is security research. Speculative attacks [78, 89] that leverage hardware side channels have become a significant threat to the security of computer systems. To counter these attacks, we need changes in both software and hardware. RiscyOO can serve as a great platform for experimenting these solutions. This is because RiscyOO runs a full software stack, and the modularity of its hardware design makes it easy to modify.

Bibliography

[1] Amazon EC2 F1 instances. https://aws.amazon.com/ec2/instance-types/f1/.

[2] The Berkeley out-of-order RISC-V processor. https://github.com/ucb-bar/riscv-boom. Accessed: 2015-04-07.

[3] Bluespec systemverilog. https://bluespec.com/.

[4] Chisel 3. https://github.com/freechipsproject/chisel3.

[5] FireSim demo v1.0 on Amazon EC2 F1. https://fires.im/2017/08/29/firesim-demo-v1.0.html.

[6] Gap benchmark suite source code. https://github.com/sbeamer/gapbs.git.

[7] Picorv32. https://github.com/cliffordwolf/picorv32.

[8] PULP platform. https://github.com/pulp-platform.

[9] Qemu. https://www.qemu.org/.

[10] The risc-v instruction set. https://riscv.org/.

[11] Rocket chip generator. https://github.com/freechipsproject/rocket-chip. Accessed: 2019-03-12.

[12] Scr1. https://github.com/syntacore/scr1.

[13] SHAKTI. https://bitbucket.org/casl/shakti_public/.

[14] Spike, a RISC-V ISA simulator. https://github.com/riscv/riscv-isa-sim.

[15] Wwc+addrs test result in power processors. http://www.cl.cam.ac.uk/~pes20/ppc-supplemental/ppc051.html#toc11.

[16] *Alpha Architecture Handbook, Version 4*. Compaq Computer Corporation, 1998.

[17] 9th dimacs implementation challenge - shortest paths. http://www.dis.uniroma1.it/challenge9/, 2006.

[18] Sarita V Adve and Kourosh Gharachorloo. Shared memory consistency models: A tutorial. *computer*, 29(12):66–76, 1996.

[19] Sarita V Adve and Mark D Hill. Weak ordering a new definition. In *ACM SIGARCH Computer Architecture News*, volume 18, pages 2–14. ACM, 1990.

[20] Sarita V Adve and Mark D Hill. A unified formalization of four shared-memory models. *IEEE Transactions on Parallel and Distributed Systems*, 4(6):613–624, 1993.

[21] Tutu Ajayi, Khalid Al-Hawaj, Aporva Amarnath, Steve Dai, Scott Davidson, Paul Gao, Gai Liu, Atieh Lotfi, Julian Puscar, Anuj Rao, Austin Rovinski, Loai Salem, Ningxiao Sun, Christopher Torng, Luis Vega, Bandhav Veluri, Xiaoyang Wang, Shaolin Xie, Chun Zhao, Ritchie Zhao, Christopher Batten, Ronald G. Dreslinski, Ian Galton, Rajesh K. Gupta, Patrick P. Mercier, Mani Srivastava, Michael B. Taylor, and Zhiru Zhang. Celerity: An open source RISC-V tiered accelerator fabric. In *Symposium on High Performance Chips (Hot Chips)*, Hot Chips 29. IEEE, August 2017.

[22] Jade Alglave. A formal hierarchy of weak memory models. *Formal Methods in System Design*, 41(2):178–210, 2012.

[23] Jade Alglave, Mark Batty, Alastair F. Donaldson, Ganesh Gopalakrishnan, Jeroen Ketema, Daniel Poetzl, Tyler Sorensen, and John Wickerson. Gpu concurrency: Weak behaviours and programming assumptions. *SIGPLAN Not.*, 50(4):577–591, March 2015.

[24] Jade Alglave, Anthony Fox, Samin Ishtiaq, Magnus O Myreen, Susmit Sarkar, Peter Sewell, and Francesco Zappa Nardelli. The semantics of power and arm multiprocessor machine code. In *Proceedings of the 4th workshop on Declarative aspects of multicore programming*, pages 13–24. ACM, 2009.

[25] Jade Alglave, Daniel Kroening, Vincent Nimal, and Michael Tautschnig. Software verification for weak memory via program transformation. In *Programming Languages and Systems*, pages 512–532. Springer, 2013.

[26] Jade Alglave and Luc Maranget. *Computer Aided Verification: 23rd International Conference, CAV 2011, Snowbird, UT, USA, July 14-20, 2011. Proceedings*, chapter Stability in Weak Memory Models, pages 50–66. Springer Berlin Heidelberg, Berlin, Heidelberg, 2011.

[27] Jade Alglave, Luc Maranget, and Michael Tautschnig. Herding cats: Modelling, simulation, testing, and data mining for weak memory. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 36(2):7, 2014.

[28] Lluc Alvarez, Miquel Moretó, Marc Casas, Emilio Castillo, Xavier Martorell, Jesús Labarta, Eduard Ayguadé, and Mateo Valero. Runtime-guided management of scratchpad memories in multicore architectures. In *2015 International*

Conference on Parallel Architecture and Compilation, PACT 2015, San Francisco, CA, USA, October 18-21, 2015, pages 379–391, 2015.

[29] Lluc Alvarez, Lluís Vilanova, Miquel Moreto, Marc Casas, Marc Gonzàlez, Xavier Martorell, Nacho Navarro, Eduard Ayguadé, and Mateo Valero. Coherence protocol for transparent management of scratchpad memories in shared memory manycore architectures. In *Proceedings of the 42Nd Annual International Symposium on Computer Architecture*, ISCA '15, pages 720–732, New York, NY, USA, 2015. ACM.

[30] ARM. *ARM Architecture Reference Manual: ARMv8, for ARMv8-A architecture profile*. 2017.

[31] U. Banerjee, C. Juvekar, A. Wright, Arvind, and A. P. Chandrakasan. An energy-efficient reconfigurable DTLS cryptographic engine for end-to-end security in IoT applications. In *2018 IEEE International Solid - State Circuits Conference - (ISSCC)*, pages 42–44, Feb 2018.

[32] Thomas W. Barr, Alan L. Cox, and Scott Rixner. Translation caching: Skip, don't walk (the page table). In *Proceedings of the 37th Annual International Symposium on Computer Architecture*, ISCA '10, pages 48–59, New York, NY, USA, 2010. ACM.

[33] Mark Batty, Alastair F. Donaldson, and John Wickerson. Overhauling sc atomics in c11 and opencl. *SIGPLAN Not.*, 51(1):634–648, January 2016.

[34] Mark Batty, Scott Owens, Susmit Sarkar, Peter Sewell, and Tjark Weber. Mathematizing c++ concurrency. In *ACM SIGPLAN Notices*, volume 46, pages 55–66. ACM, 2011.

[35] Scott Beamer, Krste Asanović, and David Patterson. The gap benchmark suite. *arXiv preprint arXiv:1508.03619*, 2015.

[36] Christian Bienia and Kai Li. *Benchmarking modern multiprocessors*. Princeton University Princeton, 2011.

[37] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. The gem5 simulator. *SIGARCH Comput. Archit. News*, 39(2):1–7, August 2011.

[38] Colin Blundell, Milo MK Martin, and Thomas F Wenisch. Invisifence: performance-transparent memory ordering in conventional multiprocessors. In *ACM SIGARCH Computer Architecture News*, volume 37, pages 233–244. ACM, 2009.

[39] Hans-J Boehm and Sarita V Adve. Foundations of the c++ concurrency memory model. In *ACM SIGPLAN Notices*, volume 43, pages 68–78. ACM, 2008.

[40] Hans-J. Boehm and Brian Demsky. Outlawing ghosts: Avoiding out-of-thin-air results. In *Proceedings of the Workshop on Memory Systems Performance and Correctness*, MSPC '14, pages 7:1–7:6, New York, NY, USA, 2014. ACM.

[41] Darrell Boggs, Gary Brown, Nathan Tuck, and KS Venkatraman. Denver: Nvidia's first 64-bit ARM processor. *IEEE Micro*, 35(2):46–55, 2015.

[42] Jason F Cantin, Mikko H Lipasti, and James E Smith. The complexity of verifying memory coherence. In *Proceedings of the fifteenth annual ACM symposium on Parallel algorithms and architectures*, pages 254–255. ACM, 2003.

[43] Trevor E Carlson, Wim Heirman, Osman Allam, Stefanos Kaxiras, and Lieven Eeckhout. The load slice core microarchitecture. In *2015 ACM/IEEE 42nd Annual International Symposium on Computer Architecture (ISCA)*, pages 272–284. IEEE, 2015.

[44] Pietro Cenciarelli, Alexander Knapp, and Eleonora Sibilio. The java memory model: Operationally, denotationally, axiomatically. In *Programming Languages and Systems*, pages 331–346. Springer, 2007.

[45] Luis Ceze, James Tuck, Pablo Montesinos, and Josep Torrellas. Bulksc: bulk enforcement of sequential consistency. In *ACM SIGARCH Computer Architecture News*, volume 35, pages 278–289. ACM, 2007.

[46] Byn Choi, Rakesh Komuravelli, Hyojin Sung, Robert Smolinski, Nima Honarmand, Sarita V. Adve, Vikram S. Adve, Nicholas P. Carter, and Ching-Tsun Chou. Denovo: Rethinking the memory hierarchy for disciplined parallelism. In *2011 International Conference on Parallel Architectures and Compilation Techniques, PACT 2011, Galveston, TX, USA, October 10-14, 2011*, pages 155–166, 2011.

[47] N. Choudhary, S. Wadhavkar, T. Shah, H. Mayukh, J. Gandhi, B. Dwiel, S. Navada, H. Najaf-abadi, and E. Rotenberg. FabScalar: Automating superscalar core design. *IEEE Micro*, 32(3):48–59, May 2012.

[48] R. B. R. Chowdhury, A. K. Kannepalli, S. Ku, and E. Rotenberg. AnyCore: A synthesizable RTL model for exploring and fabricating adaptive superscalar cores. In *2016 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 214–224, April 2016.

[49] Francesco Conti, Davide Rossi, Antonio Pullini, Igor Loi, and Luca Benini. PULP: A ultra-low power parallel accelerator for energy-efficient and flexible embedded vision. *Journal of Signal Processing Systems*, 84(3):339–354, Sep 2016.

[50] Yuelu Duan, Nima Honarmand, and Josep Torrellas. Asymmetric memory fences: Optimizing both performance and implementability. *SIGARCH Comput. Archit. News*, 43(1):531–543, March 2015.

[51] Yuelu Duan, Abdullah Muzahid, and Josep Torrellas. Weefence: toward making fences free in tso. In *ACM SIGARCH Computer Architecture News*, volume 41, pages 213–224. ACM, 2013.

[52] Michel Dubois, Christoph Scheurich, and Fayé Briggs. Memory access buffering in multiprocessors. In *ACM SIGARCH Computer Architecture News*, volume 14, pages 434–442. IEEE Computer Society Press, 1986.

[53] C. Duran, D. L. Rueda, G. Castillo, A. Agudelo, C. Rojas, L. Chaparro, H. Hurtado, J. Romero, W. Ramirez, H. Gomez, J. Ardila, L. Rueda, H. Hernandez, J. Amaya, and E. Roa. A 32-bit RISC-V AXI4-lite bus-based microcontroller with 10-bit SAR ADC. In *2016 IEEE 7th Latin American Symposium on Circuits Systems (LASCAS)*, pages 315–318, Feb 2016.

[54] Marco Elver and Vijay Nagarajan. Tso-cc: Consistency directed cache coherence for tso. In *High Performance Computer Architecture (HPCA), 2014 IEEE 20th International Symposium on*, pages 165–176. IEEE, 2014.

[55] Shaked Flur, Kathryn E. Gray, Christopher Pulte, Susmit Sarkar, Ali Sezgin, Luc Maranget, Will Deacon, and Peter Sewell. Modelling the armv8 architecture, operationally: Concurrency and isa. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL 2016, pages 608–621, New York, NY, USA, 2016. ACM.

[56] N. Gala, A. Menon, R. Bodduna, G. S. Madhusudan, and V. Kamakoti. SHAKTI processors: An open-source hardware initiative. In *2016 29th International Conference on VLSI Design and 2016 15th International Conference on Embedded Systems (VLSID)*, pages 7–8, Jan 2016.

[57] Benedict R Gaster, Derek Hower, and Lee Howes. Hrf-relaxed: Adapting hrf to the complexities of industrial heterogeneous memory models. *ACM Transactions on Architecture and Code Optimization (TACO)*, 12(1):7, 2015.

[58] Walid J Ghandour, Haitham Akkary, and Wes Masri. The potential of using dynamic information flow analysis in data value prediction. In *Proceedings of the 19th international conference on Parallel architectures and compilation techniques*, pages 431–442. ACM, 2010.

[59] Kourosh Gharachorloo, Anoop Gupta, and John L Hennessy. Two techniques to enhance the performance of memory consistency models. In *Proceedings of the 1991 International Conference on Parallel Processing*, pages 355–364, 1991.

[60] Kourosh Gharachorloo, Daniel Lenoski, James Laudon, Phillip Gibbons, Anoop Gupta, and John Hennessy. Memory consistency and event ordering in scalable shared-memory multiprocessors. In *Proceedings of the 17th International Symposium on Computer Architecture*, pages 15–26. ACM, 1990.

[61] Chris Gniady and Babak Falsafi. Speculative sequential consistency with little custom storage. In *Parallel Architectures and Compilation Techniques, 2002. Proceedings. 2002 International Conference on*, pages 179–188. IEEE, 2002.

[62] James R Goodman. *Cache consistency and sequential consistency.* University of Wisconsin-Madison, Computer Sciences Department, 1991.

[63] Dibakar Gope and Mikko H Lipasti. Atomic sc for simple in-order processors. In *High Performance Computer Architecture (HPCA), 2014 IEEE 20th International Symposium on*, pages 404–415. IEEE, 2014.

[64] Jan Gray. Designing a simple FPGA-optimized RISC CPU and system-on-a-chip. In *2000. [Online]. Available: citeseer.ist.psu.edu/article/gray00designing.html*, 2000.

[65] Chris Guiady, Babak Falsafi, and Terani N Vijaykumar. Is sc+ ilp= rc? In *Computer Architecture, 1999. Proceedings of the 26th International Symposium on*, pages 162–171. IEEE, 1999.

[66] John Hennessy, Norman Jouppi, Steven Przybylski, Christopher Rowen, Thomas Gross, Forest Baskett, and John Gill. MIPS: A microprocessor architecture. In *ACM SIGMICRO Newsletter*, volume 13, pages 17–22. IEEE Press, 1982.

[67] Mark D Hill, Susan J Eggers, James R Larus, George S Taylor, Glenn D Adams, Bidyut K Bose, Garth A Gibson, Paul M Hansen, John Keller, Shing I Kong, et al. *SPUR: a VLSI multiprocessor workstation.* University of California, 1985.

[68] Kei Hiraki, Kenji Nishida, Satoshi Sekiguchi, Toshio Shimada, and Toshitsugu Yuba. The SIGMA-1 dataflow supercomputer: A challenge for new generation supercomputing systems. *Journal of Information Processing*, 10(4):219–226, 1987.

[69] Derek R. Hower, Blake A. Hechtman, Bradford M. Beckmann, Benedict R. Gaster, Mark D. Hill, Steven K. Reinhardt, and David A. Wood. Heterogeneous-race-free memory models. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '14, pages 427–440, New York, NY, USA, 2014. ACM.

[70] IBM. *Power ISA, Version 2.07.* 2013.

[71] Arpit Joshi, Vijay Nagarajan, Marcelo Cintra, and Stratis Viglas. Efficient persist barriers for multicores. In *Proceedings of the 48th International Symposium on Microarchitecture*, MICRO-48, pages 660–671, New York, NY, USA, 2015. ACM.

[72] Jeehoon Kang, Chung-Kil Hur, Ori Lahav, Viktor Vafeiadis, and Derek Dreyer. A promising semantics for relaxed-memory concurrency. In *Proceedings of the*

44th ACM SIGPLAN Symposium on Principles of Programming Languages, POPL 2017, pages 175–189, New York, NY, USA, 2017. ACM.

[73] Jeehoon Kang, Chung-Kil Hur, William Mansky, Dmitri Garbuzov, Steve Zdancewic, and Viktor Vafeiadis. A formal c memory model supporting integer-pointer casts. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '15, pages 326–335, New York, NY, USA, 2015. ACM.

[74] Sagar Karandikar, Howard Mao, Donggyu Kim, David Biancolin, Alon Amid, Dayeol Lee, Kyle Kovacs, Borivoje Nikolic, Randy Katz, Jonathan Bachrach, and Krste Asanović. FireSim: Cycle-accurate rack-scale system simulation using FPGAs in the public cloud. *7th RISC-V Workshop*, 2017.

[75] B. Keller, M. Cochet, B. Zimmer, Y. Lee, M. Blagojevic, J. Kwak, A. Puggelli, S. Bailey, P. F. Chiu, P. Dabbelt, C. Schmidt, E. Alon, K. Asanović, and B. Nikolić. Sub-microsecond adaptive voltage scaling in a 28nm FD-SOI processor SoC. In *ESSCIRC Conference 2016: 42nd European Solid-State Circuits Conference*, pages 269–272, Sept 2016.

[76] Richard E Kessler. The alpha 21264 microprocessor. *IEEE micro*, 19(2):24–36, 1999.

[77] Donggyu Kim, Christopher Celio, David Biancolin, Jonathan Bachrach, and Krste Asanovic. Evaluation of RISC-V RTL with FPGA-accelerated simulation. *Workshop on Computer Architecture Research with RISC-V (CARRV)*, 2017.

[78] Paul Kocher, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. Spectre attacks: Exploiting speculative execution. *arXiv preprint arXiv:1801.01203*, 2018.

[79] Aasheesh Kolli, Vaibhav Gogte, Ali G. Saidi, Stephan Diestelhorst, Peter M. Chen, Satish Narayanasamy, and Thomas F. Wenisch. Language-level persistency. In *Proceedings of the 44th Annual International Symposium on Computer Architecture, ISCA 2017, Toronto, ON, Canada, June 24-28, 2017*, pages 481–493, 2017.

[80] George Kurian, Qingchuan Shi, Srinivas Devadas, and Omer Khan. OSPREY: implementation of memory consistency models for cache coherence protocols involving invalidation-free data access. In *2015 International Conference on Parallel Architecture and Compilation, PACT 2015, San Francisco, CA, USA, October 18-21, 2015*, pages 392–405, 2015.

[81] Leslie Lamport. How to make a multiprocessor computer that correctly executes multiprocess programs. *Computers, IEEE Transactions on*, 100(9):690–691, 1979.

[82] Y. Lee, A. Waterman, H. Cook, B. Zimmer, B. Keller, A. Puggelli, J. Kwak, J. Bachrach, D. Patterson, E. Alon, B. Nikolic, and K. Asanović. An agile approach to building RISC-V microprocessors. *IEEE Micro*, PP(99):1–1, 2016.

[83] Y. Lee, B. Zimmer, A. Waterman, A. Puggelli, J. Kwak, R. Jevtic, B. Keller, S. Bailey, M. Blagojevic, P. F. Chiu, H. Cook, R. Avizienis, B. Richards, E. Alon, B. Nikolic, and K. Asanovic. Raven: A 28nm RISC-V vector processor with integrated switched-capacitor DC-DC converters and adaptive clocking. In *2015 IEEE Hot Chips 27 Symposium (HCS)*, pages 1–45, Aug 2015.

[84] Yunsup Lee, Andrew Waterman, Rimas Avizienis, Henry Cook, Chen Sun, Vladimir Stojanovic, and Krste Asanović. A 45nm 1.3 GHz 16.7 double-precision GFLOPS/W RISC-V processor with vector accelerators. In *European Solid State Circuits Conference (ESSCIRC), ESSCIRC 2014-40th*, pages 199–202. IEEE, 2014.

[85] Daniel Lenoski, James Laudon, Kourosh Gharachorloo, Anoop Gupta, and John Hennessy. The directory-based cache coherence protocol for the dash multiprocessor. In *Proceedings of the 17th Annual International Symposium on Computer Architecture*, ISCA '90, pages 148–159, New York, NY, USA, 1990. ACM.

[86] Changhui Lin, Vijay Nagarajan, Rajiv Gupta, and Bharghava Rajaram. Efficient sequential consistency via conflict ordering. In *ACM SIGARCH Computer Architecture News*, volume 40, pages 273–286. ACM, 2012.

[87] Mikko H Lipasti and John Paul Shen. Exceeding the dataflow limit via value prediction. In *Proceedings of the 29th annual ACM/IEEE international symposium on Microarchitecture*, pages 226–237. IEEE Computer Society, 1996.

[88] Mikko H Lipasti, Christopher B Wilkerson, and John Paul Shen. Value locality and load value prediction. *ACM SIGOPS Operating Systems Review*, 30(5):138–147, 1996.

[89] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, and Mike Hamburg. Meltdown. *arXiv preprint arXiv:1801.01207*, 2018.

[90] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. Pin: building customized program analysis tools with dynamic instrumentation. In *Acm sigplan notices*, volume 40, pages 190–200. ACM, 2005.

[91] Daniel Lustig, Michael Pellauer, and Margaret Martonosi. Pipecheck: Specifying and verifying microarchitectural enforcement of memory consistency models. In *Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO-47, pages 635–646, Washington, DC, USA, 2014. IEEE Computer Society.

[92] Daniel Lustig, Geet Sethi, Margaret Martonosi, and Abhishek Bhattacharjee. Coatcheck: Verifying memory ordering at the hardware-os interface. In *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '16, Atlanta, GA, USA, April 2-6, 2016*, pages 233–247, 2016.

[93] Daniel Lustig, Andrew Wright, Alexandros Papakonstantinou, and Olivier Giroux. Automated generation of comprehensive memory model litmus test suites. *22nd ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2017.

[94] Sela Mador-Haim, Luc Maranget, Susmit Sarkar, Kayvan Memarian, Jade Alglave, Scott Owens, Rajeev Alur, Milo MK Martin, Peter Sewell, and Derek Williams. An axiomatic memory model for power multiprocessors. In *Computer Aided Verification*, pages 495–512. Springer, 2012.

[95] Jan-Willem Maessen, Arvind, and Xiaowei Shen. Improving the java memory model using crf. *ACM SIGPLAN Notices*, 35(10):1–12, 2000.

[96] Yatin A. Manerkar, Daniel Lustig, Michael Pellauer, and Margaret Martonosi. Ccicheck: using μhb graphs to verify the coherence-consistency interface. In *Proceedings of the 48th International Symposium on Microarchitecture, MICRO 2015, Waikiki, HI, USA, December 5-9, 2015*, pages 26–37, 2015.

[97] Jeremy Manson, William Pugh, and Sarita V. Adve. The java memory model. In *Proceedings of the 32Nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '05, pages 378–391, New York, NY, USA, 2005. ACM.

[98] Luc Maranget, Susmit Sarkar, and Peter Sewell. A tutorial introduction to the arm and power relaxed memory models. `http://www.cl.cam.ac.uk/~pes20/ppc-supplemental/test7.pdf`, 2012.

[99] Daniel Marino, Abhayendra Singh, Todd Millstein, Madanlal Musuvathi, and Satish Narayanasamy. Drfx: A simple and efficient memory model for concurrent programming languages. In *Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '10, pages 351–362, New York, NY, USA, 2010. ACM.

[100] Daniel Marino, Abhayendra Singh, Todd Millstein, Madanlal Musuvathi, and Satish Narayanasamy. A case for an sc-preserving compiler. In *ACM SIGPLAN Notices*, volume 46, pages 199–210. ACM, 2011.

[101] Milo MK Martin, Daniel J Sorin, Harold W Cain, Mark D Hill, and Mikko H Lipasti. Correctly implementing value prediction in microprocessors that support multithreading or multiprocessing. In *Proceedings of the 34th annual ACM/IEEE international symposium on Microarchitecture*, pages 328–337. IEEE Computer Society, 2001.

[102] E. Matthews and L. Shannon. TAIGA: A new RISC-V soft-processor framework enabling high performance cpu architectural features. In *2017 27th International Conference on Field Programmable Logic and Applications (FPL)*, pages 1–4, Sept 2017.

[103] Adam Morrison and Yehuda Afek. Temporally bounding tso for fence-free asymmetric synchronization. In *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '15, pages 45–58, New York, NY, USA, 2015. ACM.

[104] Naveen Muralimanohar, Rajeev Balasubramonian, and Norman P. Jouppi. CACTI 6.0: A tool to model large caches. Technical report hpl-2009-85, HP Laboratories, 2009.

[105] Sanketh Nalli, Swapnil Haria, Mark D. Hill, Michael M. Swift, Haris Volos, and Kimberly Keeton. An analysis of persistent memory use with WHISPER. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2017, Xi'an, China, April 8-12, 2017*, pages 135–148, 2017.

[106] Kyndylan Nienhuis, Kayvan Memarian, and Peter Sewell. An operational semantics for c/c++11 concurrency. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA 2016, pages 111–128, New York, NY, USA, 2016. ACM.

[107] Marc S. Orr, Shuai Che, Ayse Yilmazer, Bradford M. Beckmann, Mark D. Hill, and David A. Wood. Synchronization using remote-scope promotion. *SIGARCH Comput. Archit. News*, 43(1):73–86, March 2015.

[108] Peizhao Ou and Brian Demsky. Towards understanding the costs of avoiding out-of-thin-air results. *Proceedings of the ACM on Programming Languages*, 2(OOPSLA):136, 2018.

[109] Scott Owens, Susmit Sarkar, and Peter Sewell. A better x86 memory model: x86-tso. In *Theorem Proving in Higher Order Logics*, pages 391–407. Springer, 2009.

[110] Gregory M Papadopoulos and David E Culler. Monsoon: an explicit token-store architecture. In *ACM SIGARCH Computer Architecture News*, volume 18, pages 82–91. ACM, 1990.

[111] David A. Patterson and David R. Ditzel. The case for the reduced instruction set computer. *SIGARCH Comput. Archit. News*, 8(6):25–33, October 1980.

[112] Arthur Perais and André Seznec. Practical data value speculation for future high-end processors. In *International Symposium on High Performance Computer Architecture*, pages 428–439, 2014.

[113] Arthur Perais and André Seznec. Bebop: A cost effective predictor infrastructure for superscalar value prediction. In *21st IEEE International Symposium on High Performance Computer Architecture, HPCA 2015, Burlingame, CA, USA, February 7-11, 2015*, pages 13–25, 2015.

[114] Christopher Pulte, Shaked Flur, Will Deacon, Jon French, Susmit Sarkar, and Peter Sewell. Simplifying arm concurrency: Multicopy-atomic axiomatic and operational models for armv8. *Proceedings of the ACM on Programming Languages*, 2(POPL):19, 2017.

[115] Parthasarathy Ranganathan, Vijay S Pai, and Sarita V Adve. Using speculative retirement and larger instruction windows to narrow the performance gap between memory consistency models. In *Proceedings of the ninth annual ACM symposium on Parallel algorithms and architectures*, pages 199–210. ACM, 1997.

[116] Xiaowei Ren and Mieszko Lis. Efficient sequential consistency in gpus via relativistic cache coherence. In *2017 IEEE International Symposium on High Performance Computer Architecture, HPCA 2017, Austin, TX, USA, February 4-8, 2017*, pages 625–636, 2017.

[117] Alberto Ros, Trevor E. Carlson, Mehdi Alipour, and Stefanos Kaxiras. Non-speculative load-load reordering in TSO. In *Proceedings of the 44th Annual International Symposium on Computer Architecture, ISCA 2017, Toronto, ON, Canada, June 24-28, 2017*, pages 187–200, 2017.

[118] Alberto Ros, Trevor E Carlson, Mehdi Alipour, and Stefanos Kaxiras. Non-speculative load-load reordering in tso. In *ACM SIGARCH Computer Architecture News*, volume 45, pages 187–200. ACM, 2017.

[119] Alberto Ros and Stefanos Kaxiras. Complexity-effective multicore coherence. In *Proceedings of the 21st international conference on Parallel architectures and compilation techniques*, pages 241–252. ACM, 2012.

[120] Daniel L Rosenband. The ephemeral history register: flexible scheduling for rule-based designs. In *Formal Methods and Models for Co-Design, 2004. MEMOCODE'04. Proceedings. Second ACM and IEEE International Conference on*, pages 189–198. IEEE, 2004.

[121] Daniel L Rosenband and Arvind. Hardware synthesis from guarded atomic actions with performance specifications. In *ICCAD-2005. IEEE/ACM International Conference on Computer-Aided Design, 2005.*, pages 784–791. IEEE, 2005.

[122] Shuichi Sakai, Kci Hiraki, Y Kodama, T Yuba, et al. An architecture of a dataflow single chip processor. In *ACM SIGARCH Computer Architecture News*, volume 17, pages 46–53. ACM, 1989.

[123] Karthikeyan Sankaralingam, Ramadass Nagarajan, Haiming Liu, Changkyu Kim, Jaehyuk Huh, Doug Burger, Stephen W Keckler, and Charles R Moore. Exploiting ILP, TLP, and DLP with the polymorphous TRIPS architecture. In *Computer Architecture, 2003. Proceedings. 30th Annual International Symposium on*, pages 422–433. IEEE, 2003.

[124] Susmit Sarkar, Kayvan Memarian, Scott Owens, Mark Batty, Peter Sewell, Luc Maranget, Jade Alglave, and Derek Williams. Synchronising c/c++ and power. In *ACM SIGPLAN Notices*, volume 47, pages 311–322. ACM, 2012.

[125] Susmit Sarkar, Peter Sewell, Jade Alglave, Luc Maranget, and Derek Williams. Understanding power multiprocessors. In *ACM SIGPLAN Notices*, volume 46, pages 175–186. ACM, 2011.

[126] Susmit Sarkar, Peter Sewell, Francesco Zappa Nardelli, Scott Owens, Tom Ridge, Thomas Braibant, Magnus O. Myreen, and Jade Alglave. The semantics of x86-cc multiprocessor machine code. *SIGPLAN Not.*, 44(1):379–391, January 2009.

[127] Peter Sewell, Susmit Sarkar, Scott Owens, Francesco Zappa Nardelli, and Magnus O Myreen. x86-tso: a rigorous and usable programmer's model for x86 multiprocessors. *Communications of the ACM*, 53(7):89–97, 2010.

[128] Rami Sheikh, Harold W Cain, and Raguram Damodaran. Load value prediction via path-based address prediction: avoiding mispredictions due to conflicting stores. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 423–435. ACM, 2017.

[129] Seunghee Shin, James Tuck, and Yan Solihin. Hiding the long latency of persist barriers using speculative execution. *SIGARCH Comput. Archit. News*, 45(2):175–186, June 2017.

[130] Matthew D. Sinclair, Johnathan Alsop, and Sarita V. Adve. Efficient gpu synchronization without scopes: Saying no to complex consistency models. In *Proceedings of the 48th International Symposium on Microarchitecture*, MICRO-48, pages 647–659, New York, NY, USA, 2015. ACM.

[131] Matthew D. Sinclair, Johnathan Alsop, and Sarita V. Adve. Chasing away rats: Semantics and evaluation for relaxed atomics on heterogeneous systems. In *Proceedings of the 44th Annual International Symposium on Computer Architecture, ISCA 2017, Toronto, ON, Canada, June 24-28, 2017*, pages 161–174, 2017.

[132] Abhayendra Singh, Satish Narayanasamy, Daniel Marino, Todd Millstein, and Madanlal Musuvathi. End-to-end sequential consistency. In *ACM SIGARCH Computer Architecture News*, volume 40, pages 524–535. IEEE Computer Society, 2012.

[133] Richard Smith, editor. *Working Draft, Standard for Programming Language C++*. `http://open-std.org/JTC1/SC22/WG21/docs/papers/2015/n4527.pdf`, May 2015.

[134] Daniel J Sorin, Mark D Hill, and David A Wood. A primer on memory consistency and cache coherence. *Synthesis Lectures on Computer Architecture*, 6(3):1–212, 2011.

[135] SPARC International, Inc. *The SPARC Architecture Manual: Version 8*. Prentice-Hall, Inc., 1992.

[136] V. Srinivasan, R. B. R. Chowdhury, E. Forbes, R. Widialaksono, Z. Zhang, J. Schabel, S. Ku, S. Lipa, E. Rotenberg, W. R. Davis, and P. D. Franzon. H3 (Heterogeneity in 3D): A logic-on-logic 3D-stacked heterogeneous multicore processor. In *2017 IEEE International Conference on Computer Design (ICCD)*, pages 145–152, Nov 2017.

[137] Hyojin Sung and Sarita V. Adve. Denovosync: Efficient support for arbitrary synchronization without writer-initiated invalidations. *SIGARCH Comput. Archit. News*, 43(1):545–559, March 2015.

[138] Caroline Trippel, Yatin A. Manerkar, Daniel Lustig, Michael Pellauer, and Margaret Martonosi. Tricheck: Memory model verification at the trisection of software, hardware, and ISA. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2017, Xi'an, China, April 8-12, 2017*, pages 119–133, 2017.

[139] Muralidaran Vijayaraghavan, Adam Chlipala, Arvind, and Nirav Dave. *Computer Aided Verification: 27th International Conference, CAV 2015, San Francisco, CA, USA, July 18-24, 2015, Proceedings, Part II*, chapter Modular Deductive Verification of Multiprocessor Hardware Designs, pages 109–127. Springer International Publishing, Cham, 2015.

[140] Muralidaran Vijayaraghavan, Adam Chlipala, Nirav Dave, et al. Modular deductive verification of multiprocessor hardware designs. In *International Conference on Computer Aided Verification (CAV)*, pages 109–127. Springer, 2015.

[141] Y. Wang, M. Wen, C. Zhang, and J. Lin. RVNet: A fast and high energy efficiency network packet processing system on RISC-V. In *2017 IEEE 28th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, pages 107–110, July 2017.

[142] David L Weaver and Tom Gremond. *The SPARC architecture manual (Version 9)*. PTR Prentice Hall Englewood Cliffs, NJ 07632, 1994.

[143] Thomas F Wenisch, Anastasia Ailamaki, Babak Falsafi, and Andreas Moshovos. Mechanisms for store-wait-free multiprocessors. In *ACM SIGARCH Computer Architecture News*, volume 35, pages 266–277. ACM, 2007.

[144] Kenneth C Yeager. The mips r10000 superscalar microprocessor. *Micro, IEEE*, 16(2):28–41, 1996.

[145] Xiangyao Yu and Srinivas Devadas. Tardis: Time traveling coherence algorithm for distributed shared memory. In *2015 International Conference on Parallel Architecture and Compilation, PACT 2015, San Francisco, CA, USA, October 18-21, 2015*, pages 227–240, 2015.

[146] Guowei Zhang, Webb Horn, and Daniel Sanchez. Exploiting commutativity to reduce the cost of updates to shared data in cache-coherent systems. In *Proceedings of the 48th International Symposium on Microarchitecture*, MICRO-48, pages 13–25, New York, NY, USA, 2015. ACM.

[147] Sizhuo Zhang, Muralidaran Vijayaraghavan, and Arvind. Weak memory models: Balancing definitional simplicity and implementation flexibility. In *Proceedings of the 2017 International Conference on Parallel Architectures and Compilation*, Portland, OR, USA, 2017.

[148] Sizhuo Zhang, Muralidaran Vijayaraghavan, and Arvind. Weak memory models: Balancing definitional simplicity and implementation flexibility. *arXiv preprint arXiv:1707.05923*, 2017.

[149] Sizhuo Zhang, Muralidaran Vijayaraghavan, Dan Lustig, and Arvind. Weak memory models with matching axiomatic and operational definitions. *arXiv preprint arXiv:1710.04259*, 2017.

[150] Sizhuo Zhang, Muralidaran Vijayaraghavan, Andrew Wright, Mehdi Alipour, and Arvind. Constructing a weak memory model. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, pages 124–137, June 2018.

[151] Sizhuo Zhang, Andrew Wright, Thomas Bourgeat, and Arvind. Composable building blocks to open up processor design. In *The 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, October 2018.

[152] B. Zimmer, P. F. Chiu, B. Nikolić, and K. Asanović. Reprogrammable redundancy for cache Vmin reduction in a 28nm RISC-V processor. In *2016 IEEE Asian Solid-State Circuits Conference (A-SSCC)*, pages 121–124, Nov 2016.

[153] B. Zimmer, Y. Lee, A. Puggelli, J. Kwak, R. Jevtić, B. Keller, S. Bailey, M. Blagojević, P. F. Chiu, H. P. Le, P. H. Chen, N. Sutardja, R. Avizienis, A. Waterman, B. Richards, P. Flatresse, E. Alon, K. Asanović, and B. Nikolić. A RISC-V vector processor with simultaneous-switching switched-capacitor DC converters in 28 nm FDSOI. *IEEE Journal of Solid-State Circuits*, 51(4):930–942, April 2016.